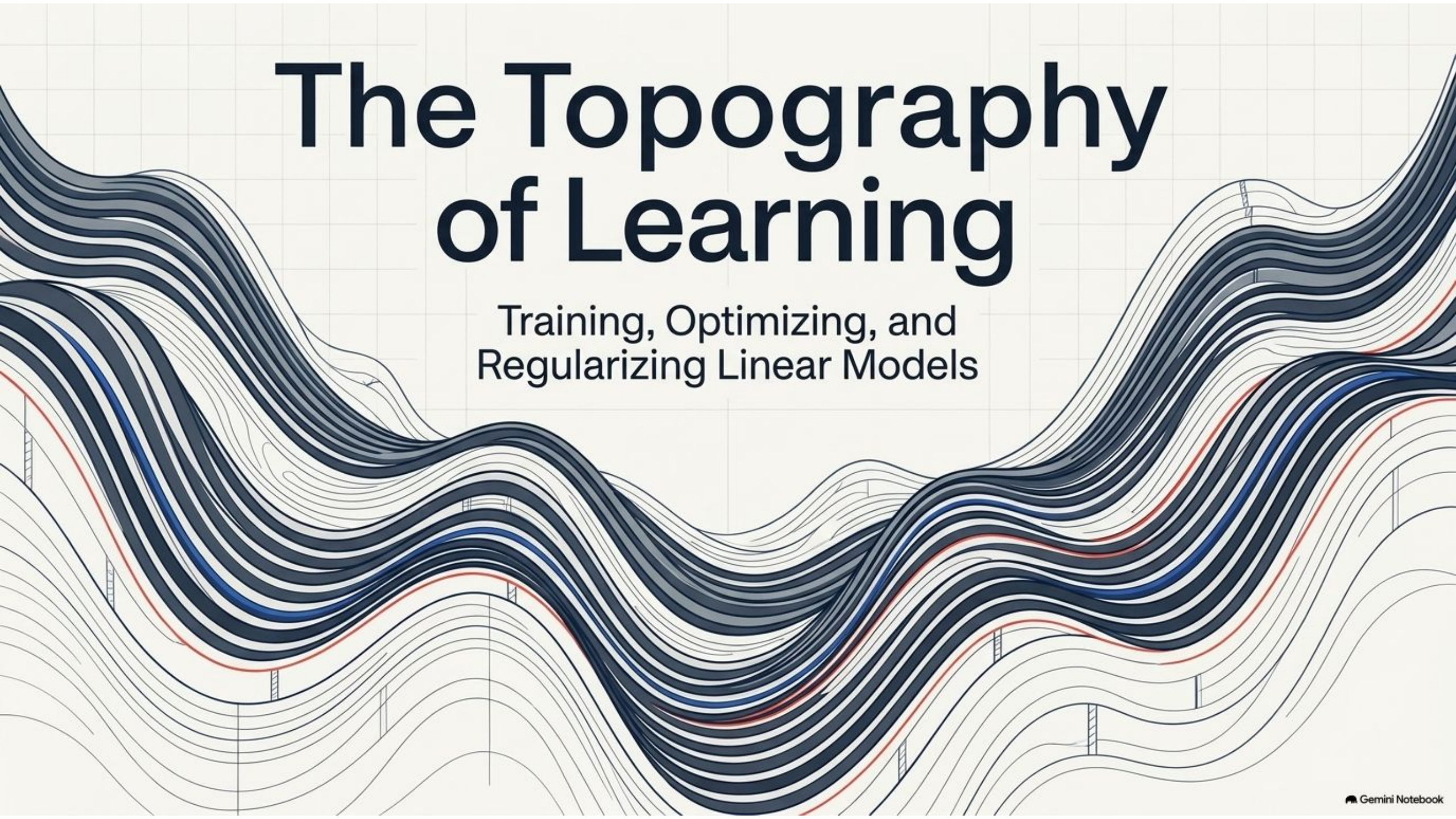
# The Topography of Learning

## Training, Optimizing, and Regularizing Linear Models



Gemini Notebook

# The Foundation: Linear Regression

## The Objective

A linear model predicts a value by computing a weighted sum of input features, plus a bias term. The training objective is to find the exact parameter vector that minimises the Mean Squared Error (MSE) across the dataset.

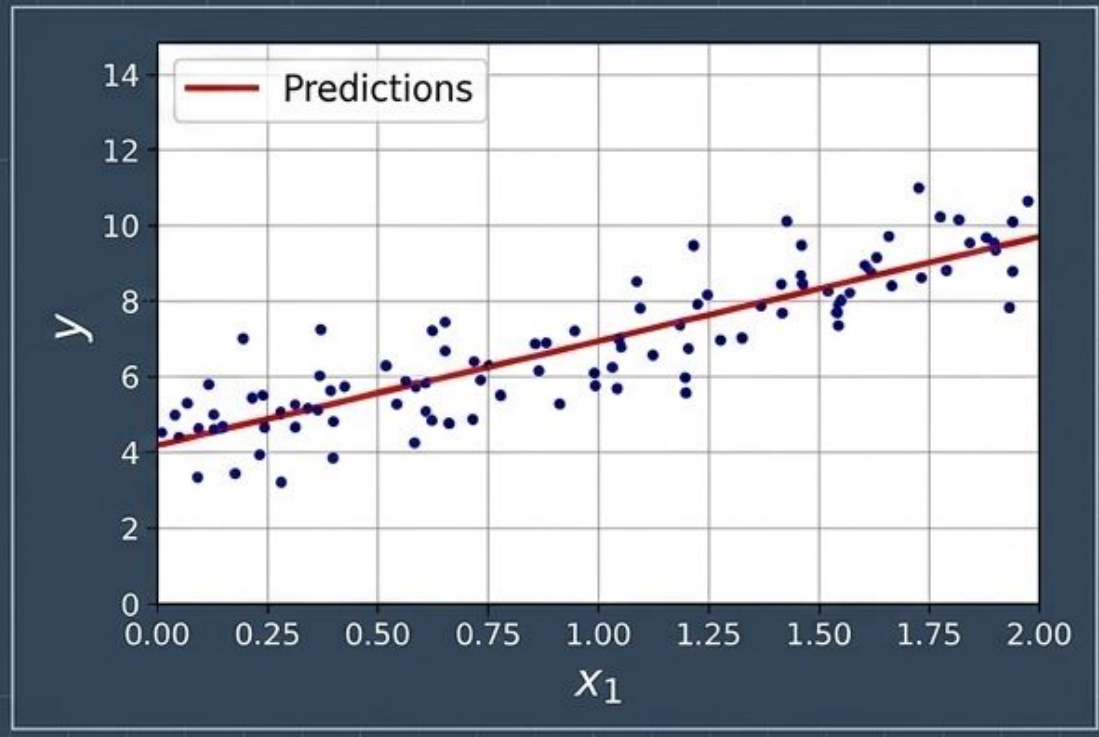
The Prediction

Parameter Vector (includes bias $\theta_0$ & weights)

$$\hat{y} = h_\theta(\mathbf{x}) = \theta \cdot \mathbf{x}$$

Parameter Vector (includes bias $\theta_0$ & weights)

Feature Vector

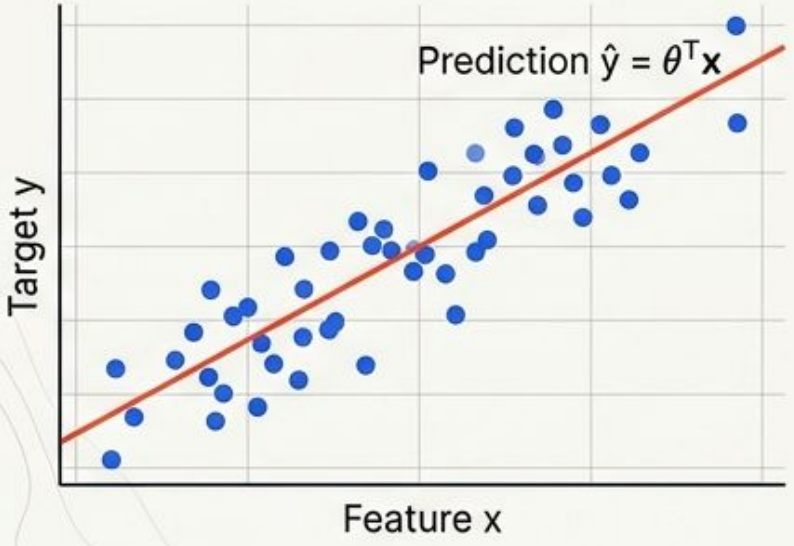



Gemini Notebook

# The Core Mechanism of Prediction

## The Model

$$\hat{y} = \theta^T \mathbf{x}$$



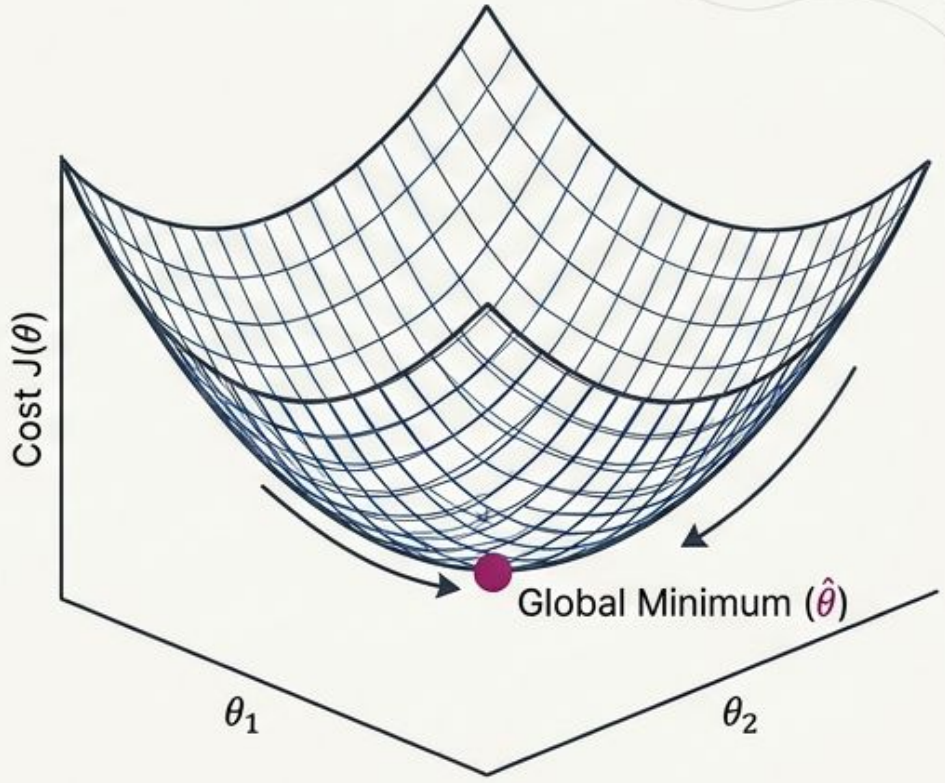

### The Normal Equation

A closed-form solution exists to directly calculate the optimal parameters:

$$\hat{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

## The Measurement

$$\text{MSE} = \frac{1}{m}\sum\left(\theta^T\mathbf{x}^{(i)} - y^{(i)}\right)^2$$




Gemini Notebook

# The Closed-Form Solutions

## A. The Normal Equation

**Equation:** $\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

**Mechanism:** Computes the model parameters that minimise the cost function directly.

**The Flaw:** Computes the inverse of an (n+1) × (n+1) matrix. Computational complexity is $O(n^{2.4})$ to $O(n^3)$. Doubling the features yields **up to 8x the computation time**.

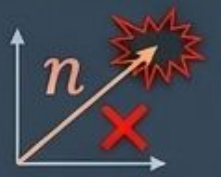
## B. Singular Value Decomposition (SVD)

**Mechanism:**
Computes the Moore-Penrose pseudoinverse ($\theta = \mathbf{X}^+\mathbf{y}$). Used by Scikit-Learn's default `LinearRegression`.

**The Advantage:**
Handles **non-invertible matrices perfectly matrices perfectly** and is **significantly faster** at $O(n^2)$.

## The Bottom Line

Both methods scale exceptionally well for millions of instances (O(m)), but choke catastrophically on large numbers of features (e.g., 100,000+).

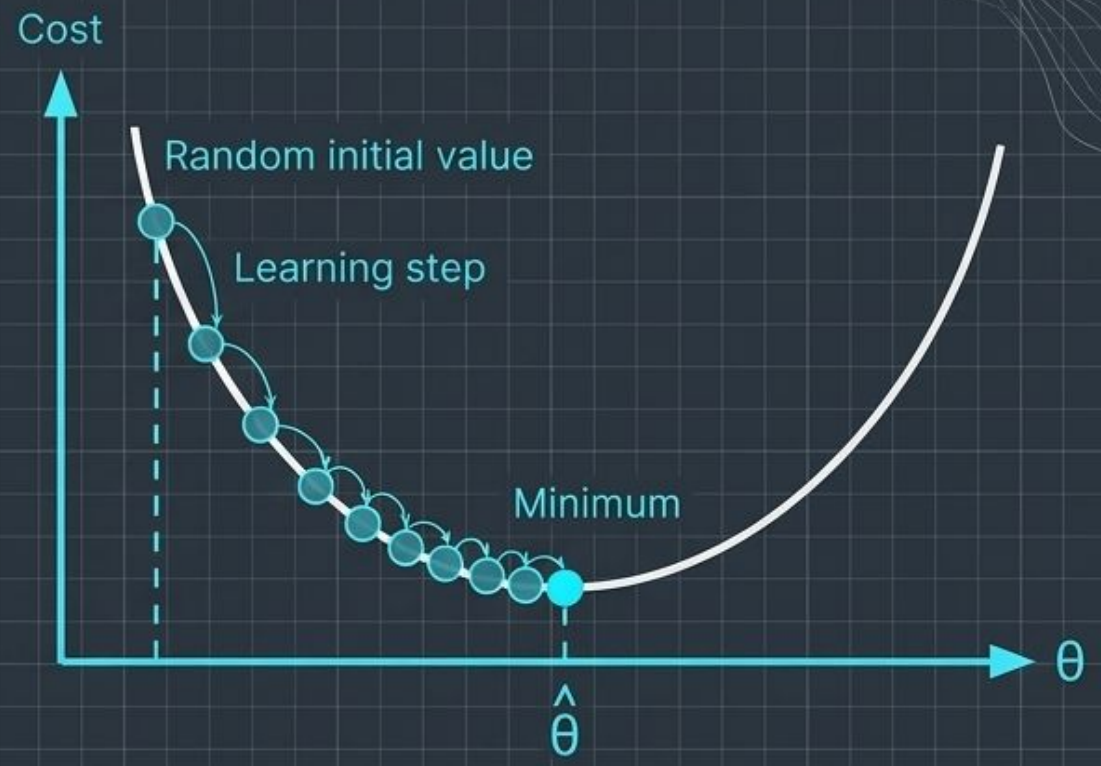



Gemini Notebook

# Enter Gradient Descent

**The Problem:** Closed-form mathematical solutions fail at high dimensions.

**The Solution:** Gradient Descent (GD). Measure the local gradient of the error function with respect to parameter θ, and take a step in the direction of the descending gradient.




**CRITICAL CONSTRAINT**

Feature scaling (e.g., StandardScaler) is absolutely mandatory. Without it, the cost function becomes an elongated bowl, and Gradient Descent will take agonizingly long to converge.

Gemini Notebook

# Navigating the Cost Function via Gradient Descent

1

**The Compass: Gradient Vector**

Calculates the steepest slope of the cost function.

$$\nabla_\theta \mathrm{MSE}(\theta) = \frac{2}{m} \mathrm{X}^T (\mathrm{X}\theta - \mathrm{y})$$

2

**The Stride: Learning Rate ($\eta$)**

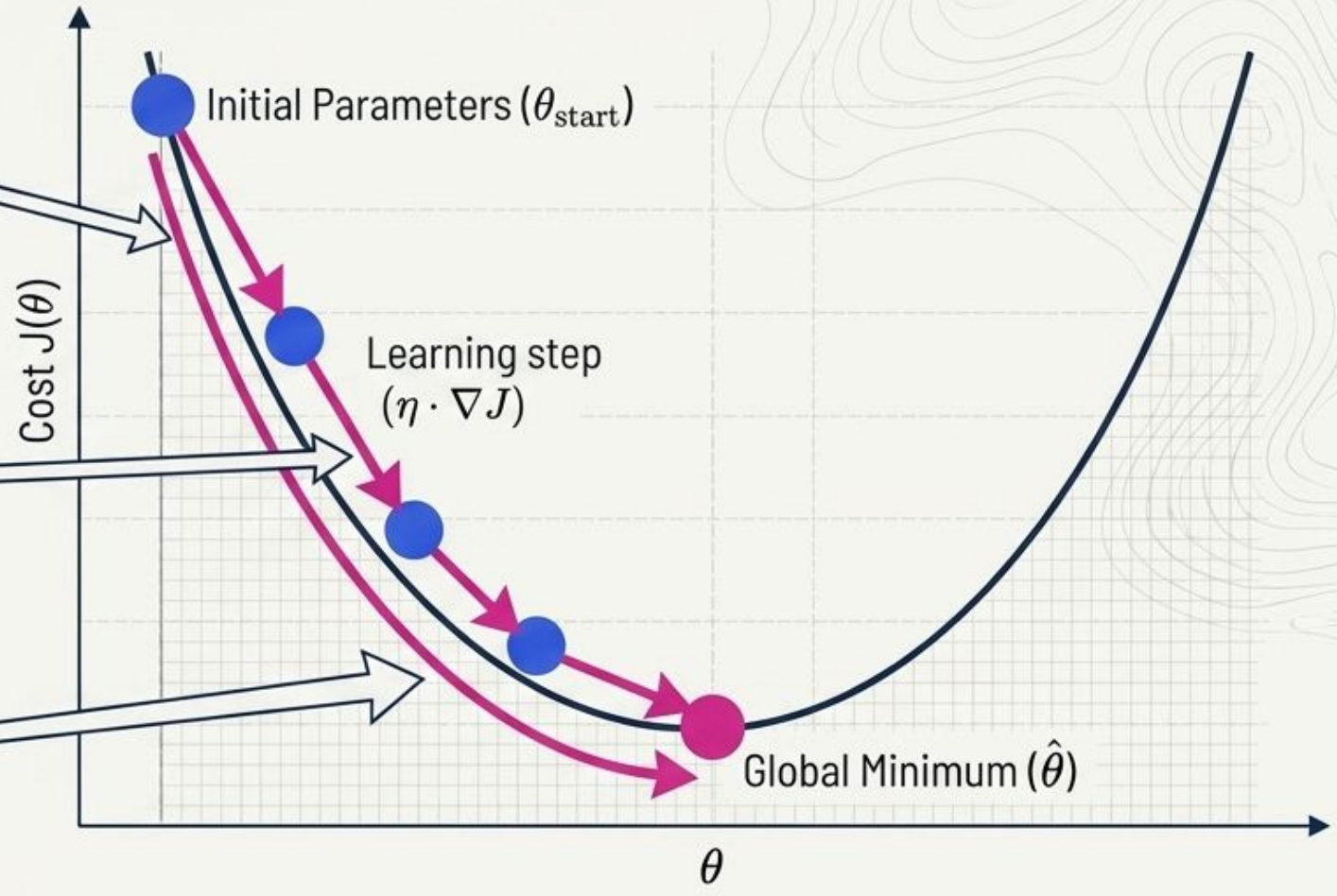
Determines the size of the step taken in the direction of the gradient.

3

**The Engine: Update Rule**

Iteratively adjusts parameters until convergence.

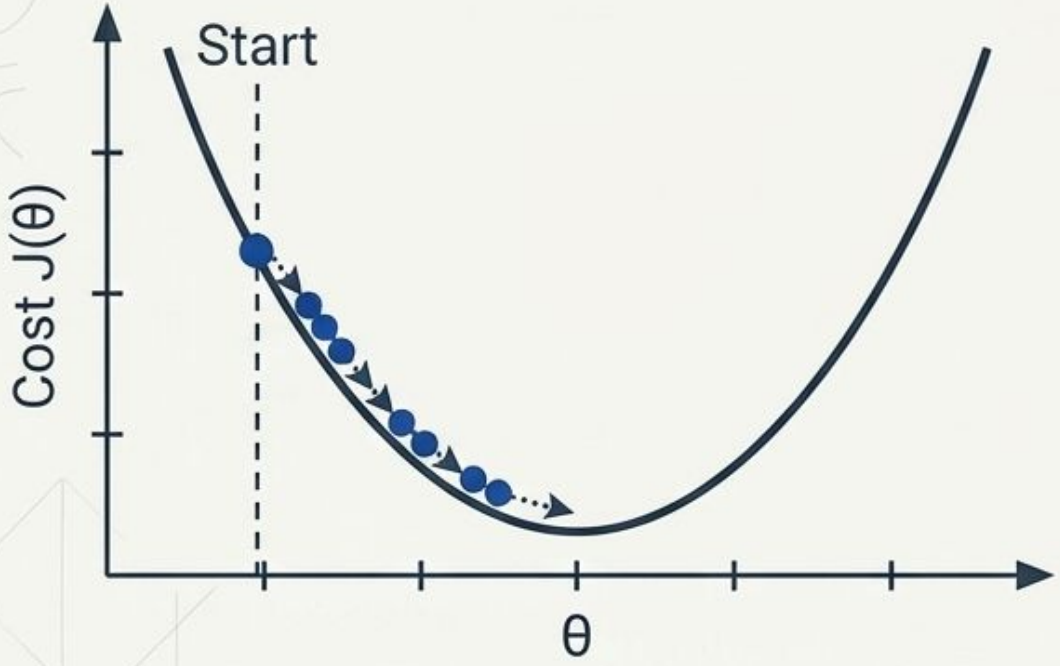
$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathrm{MSE}(\theta)$$



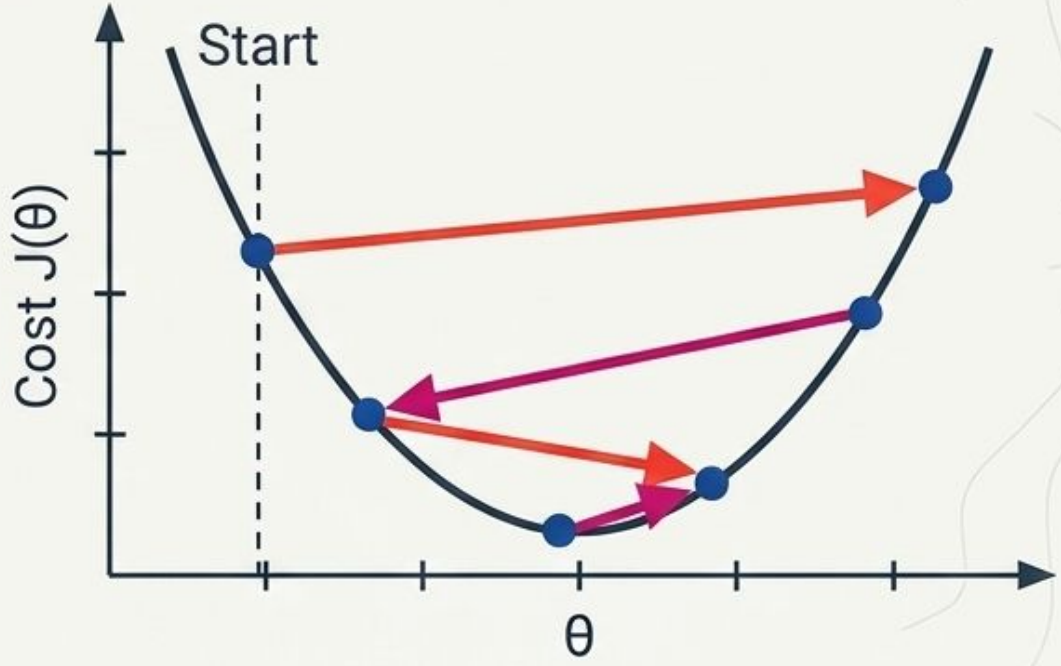

Gemini Notebook

# The Pitfalls of Pace and Calibration

## Learning Rate Too Small




## Learning Rate Too Large



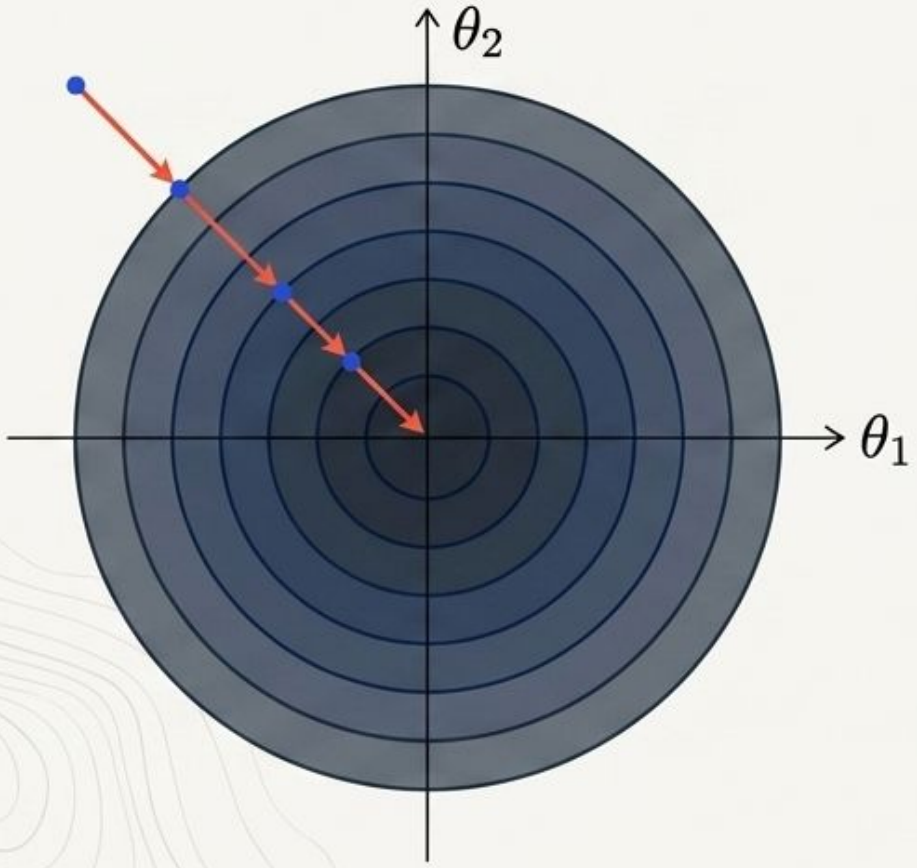

The learning rate (η) dictates whether a model converges efficiently, stalls indefinitely, or diverges entirely.

Gemini Notebook

# The Topography of Unscaled Data

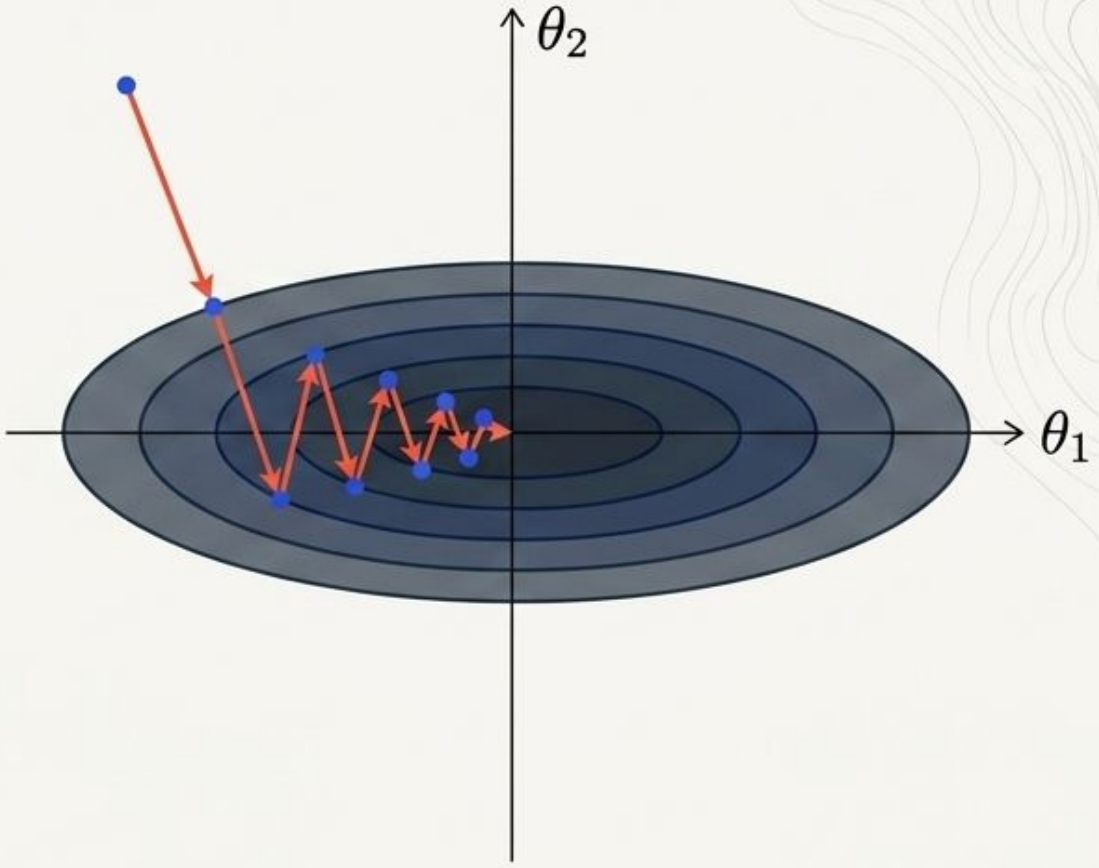
## Feature Scaling Applied

Standardized features create a symmetrical terrain.




## Without Scaling

Disparate feature scales distort the loss surface.




Gemini Notebook

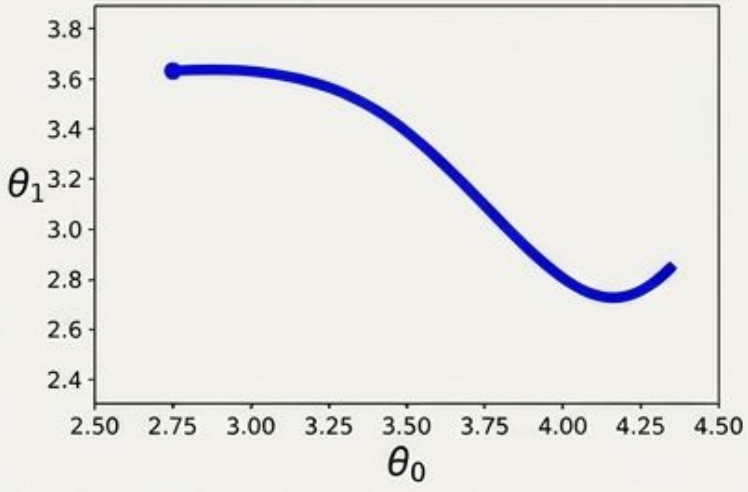
# The Gradient Descent Triad

## Batch Gradient Descent

**Mechanism:** Uses the entire training set to compute gradients.

**Characteristics:** Smooth, direct path. Highly stable.

**Drawback:** Terribly slow on large datasets.

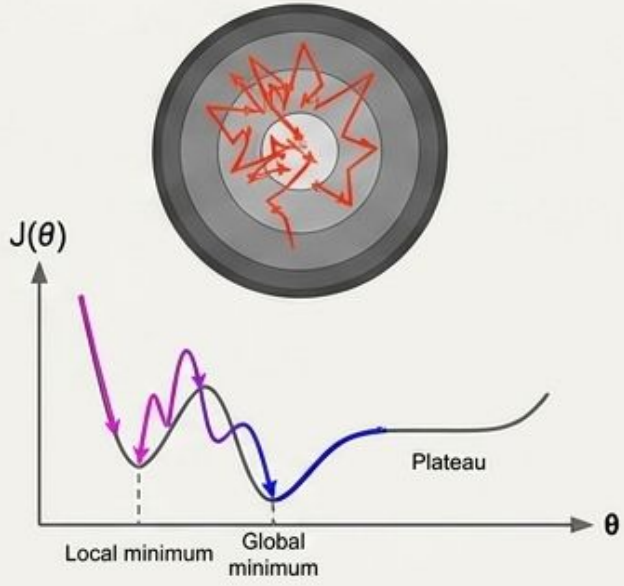



## Stochastic Gradient Descent (SGD)

**Mechanism:** Picks a single random instance per step.

**Characteristics:** Erratic and highly oscillatory. Requires a Learning Schedule to settle.

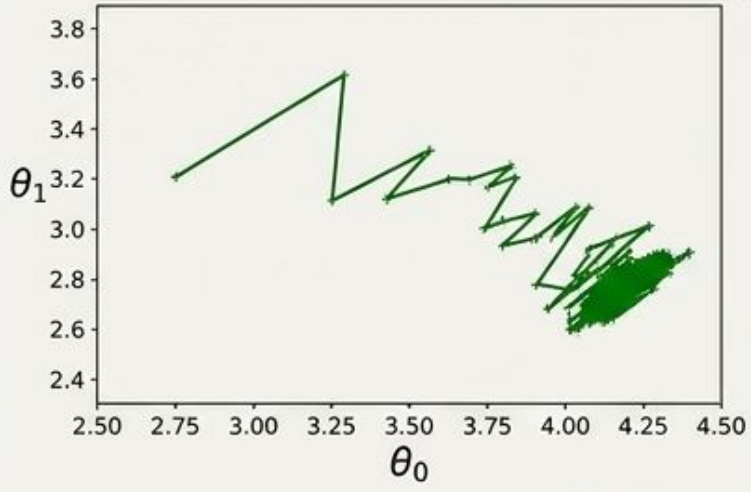
**Advantage:** Randomness allows it to jump out of local minima traps.




## Mini-batch Gradient Descent

**Mechanism:** Computes gradients on small random sets of instances.

**Characteristics:** The balanced middle ground. Faster than Batch, more stable than Stochastic.




Gemini Notebook

# Batch Gradient Descent

**Input:**

- Dataset D = {($x_1$, $y_1$), ($x_2$, $y_2$), ..., ($x_n$, $y_n$)}
- Learning rate $\eta > 0$
- Number of epochs E
- Loss function $L(\theta, X, Y)$ → returns average loss for dataset
- Gradient function $\nabla L(\theta, X, Y)$ → returns gradient over the whole dataset

**Initialize** $\theta$ randomly

**For** epoch = 1 to E do:

$g \leftarrow \nabla L(\theta, X, Y)$ // Compute gradient for the entire dataset

$\theta \leftarrow \theta - \eta * g$ // Update parameters using the average gradient

**End For**

**Output**: Optimized parameters $\theta$

- **Partial derivative of the cost function:**

$$\frac{\partial}{\partial \theta_j} MSE(\theta) = \frac{2}{m} \sum_{i=1}^{m} (\theta^T x^i - y^i) x_j^i$$

- **Gradient vector of the cost function**

$$\nabla_\theta MSE(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\theta) \\ \frac{\partial}{\partial \theta_1} MSE(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} MSE(\theta) \end{pmatrix} = \frac{2}{m} X^T (X\theta - y)$$

- **Gradient descent step**

$$\theta^{new} = \theta^{old} - \eta \nabla_\theta MSE(\theta)$$

# Stochastic Gradient Descent

**Input:**

- Dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
- Learning rate $\eta > 0$
- Number of epochs $E$
- Loss function $L(\theta, x, y)$ → returns loss for parameters $\theta$
- Gradient function $\nabla L(\theta, x, y)$ → returns gradient w.r.t. $\theta$

**Initialize** $\theta$ randomly

**For** epoch = 1 to E do:

    Shuffle dataset D

    **For** each $(x_i, y_i)$ in D do:

        $g \leftarrow \nabla L(\theta, x_i, y_i)$  // Compute gradient for a single sample

        $\theta \leftarrow \theta - \eta * g$  // Update parameters

    **End For**

**End For**

**Output:** Optimized parameters $\theta$

# Mini-batch Gradient Descent

**Input:**

- Dataset $D = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$
- Learning rate $\eta > 0$
- Number of epochs $E$
- Mini-batch size $m$ $(1 < m < n)$
- Loss function $L(\theta, X_batch, Y_batch)$
- Gradient function $\nabla L(\theta, X_batch, Y_batch)$

**Initialize** $\theta$ randomly

**For** epoch = 1 to E do:

    Shuffle dataset D

    Partition D into batches of size m

    **For** each batch B in D do:

        $g \leftarrow \nabla L(\theta, X_B, Y_B)$ // Compute gradient
        $\theta \leftarrow \theta - \eta * g$ // Update parameters

    **End For**

**End For**

**Output**: Optimized parameters $\theta$

# Going Non-Linear: Polynomial Regression

## The Concept

Linear models can fit non-linear data by artificially adding powers of each feature as entirely new features.

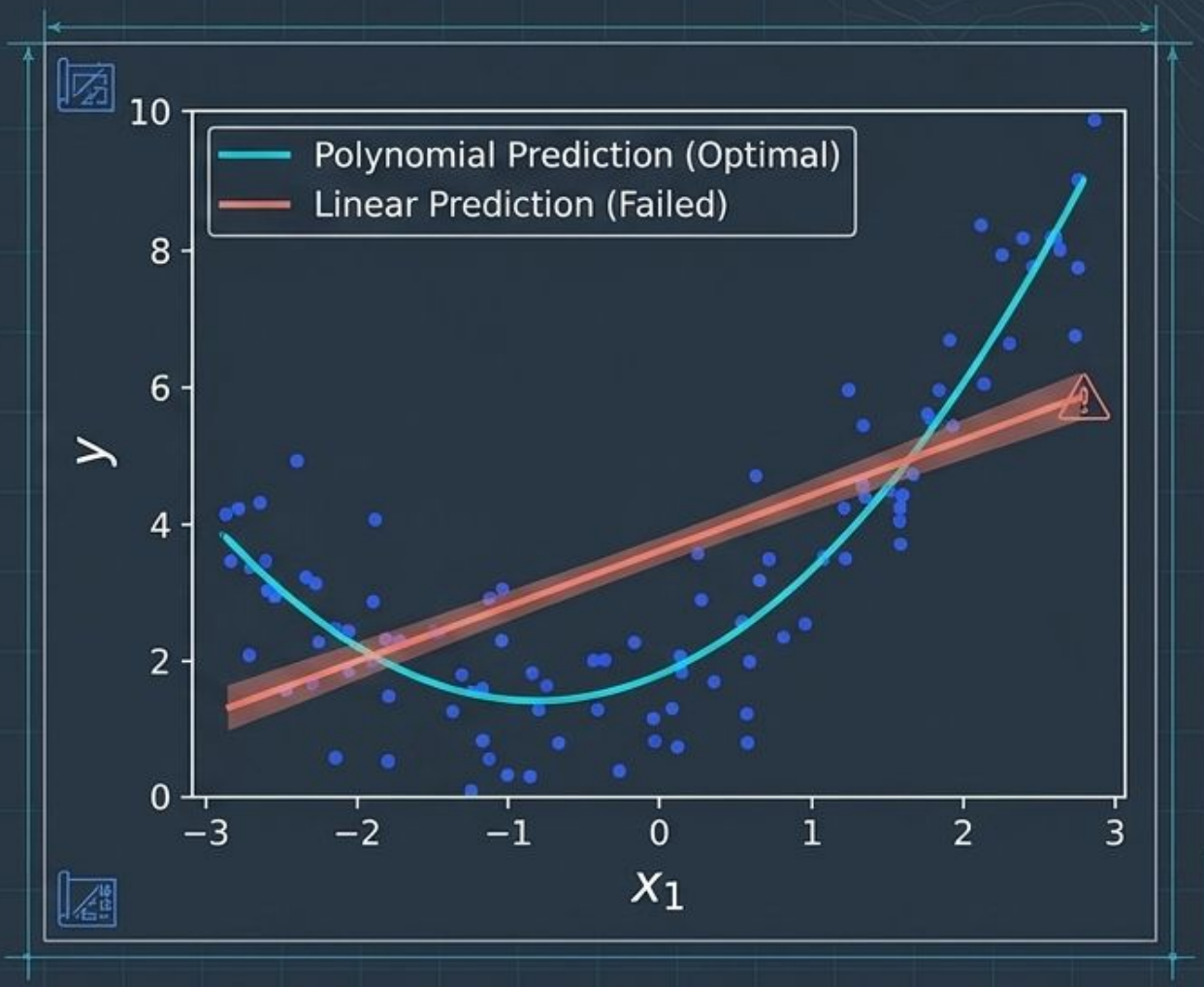
## The Mechanism

If a feature is $x_1$, add $x_1^2$. The linear model becomes:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2.$$

## The Hidden Power

Unlike pure Linear Regression, Polynomial Regression can find synergistic relationships between features (by adding feature combinations like $a \times b$).



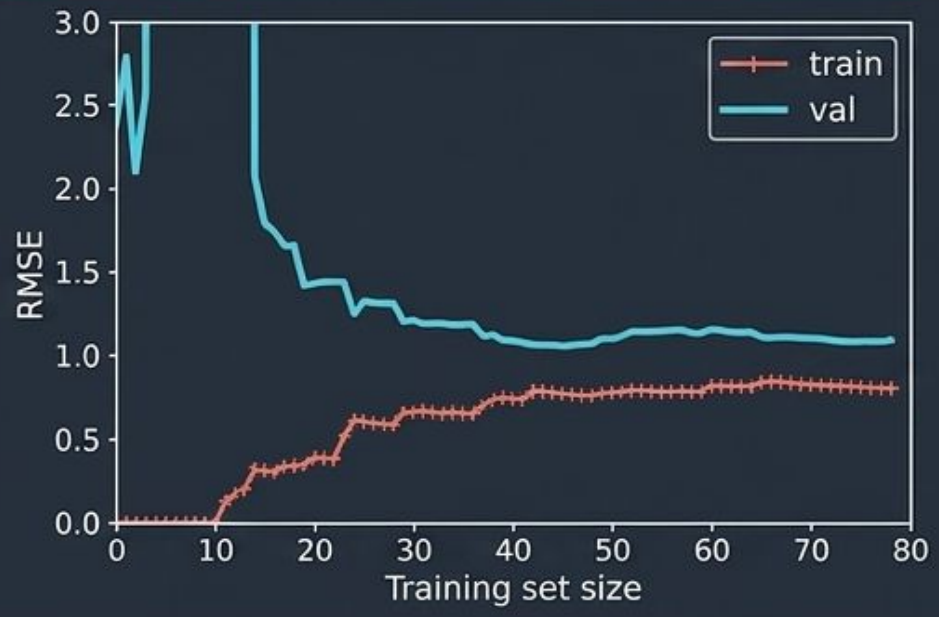

Gemini Notebook

# Diagnosing Models: Learning Curves

High-degree polynomials memorize noise. We use Learning Curves to detect this.

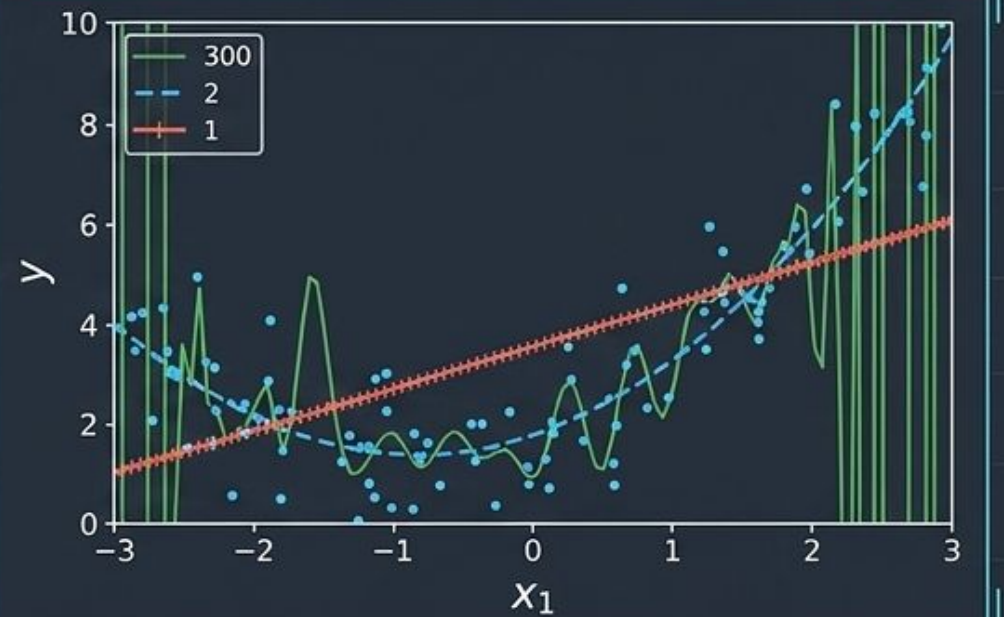
**Diagnosis 1: Underfitting (Linear Model)**

- Symptom: Both training and validation errors reach a high plateau and are very close together.
- Fix: Use a more complex model or engineer better features.




**Diagnosis 2: Overfitting (High-Degree Polynomial)**

- Symptom: Training error is very low, but there is a significant, persistent gap between training and validation curves.
- Fix: Gather more training data or regularise (constrain) the model.

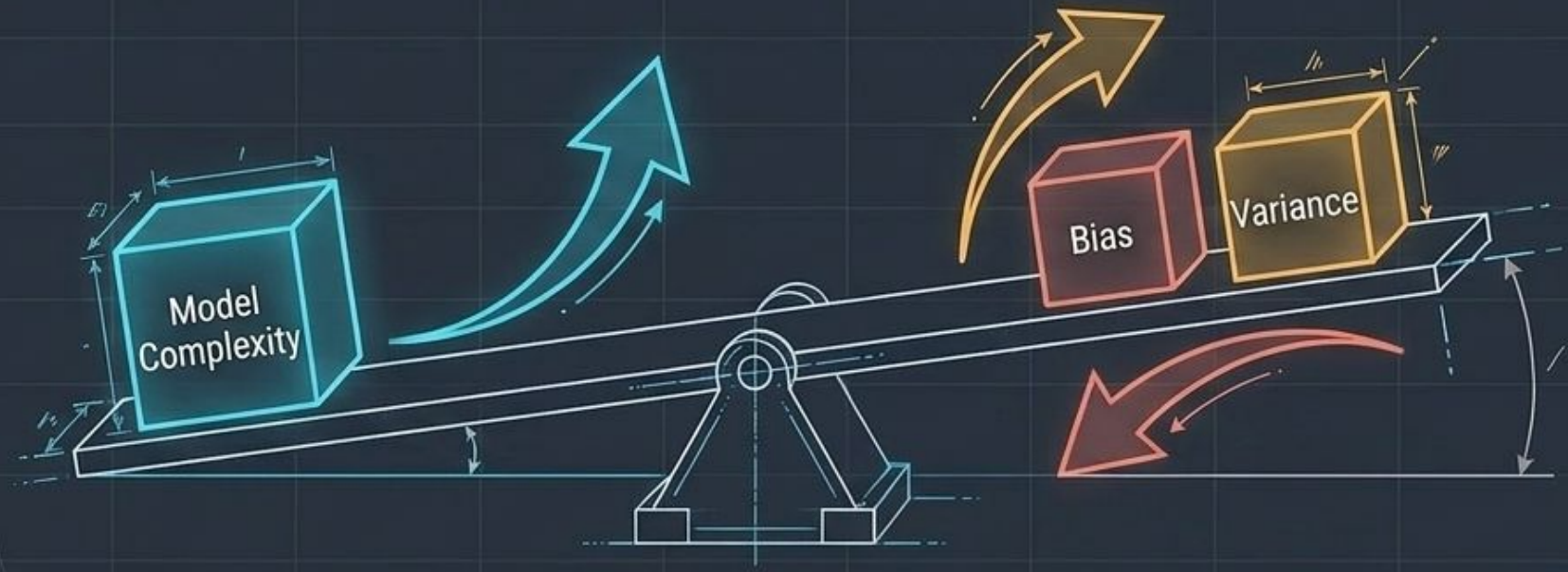



Gemini Notebook

# The Bias/Variance Trade-Off

A model's generalisation error is the sum of three distinct components:

1. **Bias:** Error from wrong assumptions (e.g., assuming quadratic data is linear). High bias = Underfitting.
2. **Variance:** Error from excessive sensitivity to tiny variations in training data. High variance = Overfitting.
3. **Irreducible Error:** The noisiness of the data itself. Only fixed by data cleaning.




**The Rule:** Increasing model complexity naturally increases variance and reduces bias. Reducing complexity increases bias and reduces variance.

Gemini Notebook

# The Regularization Toolkit

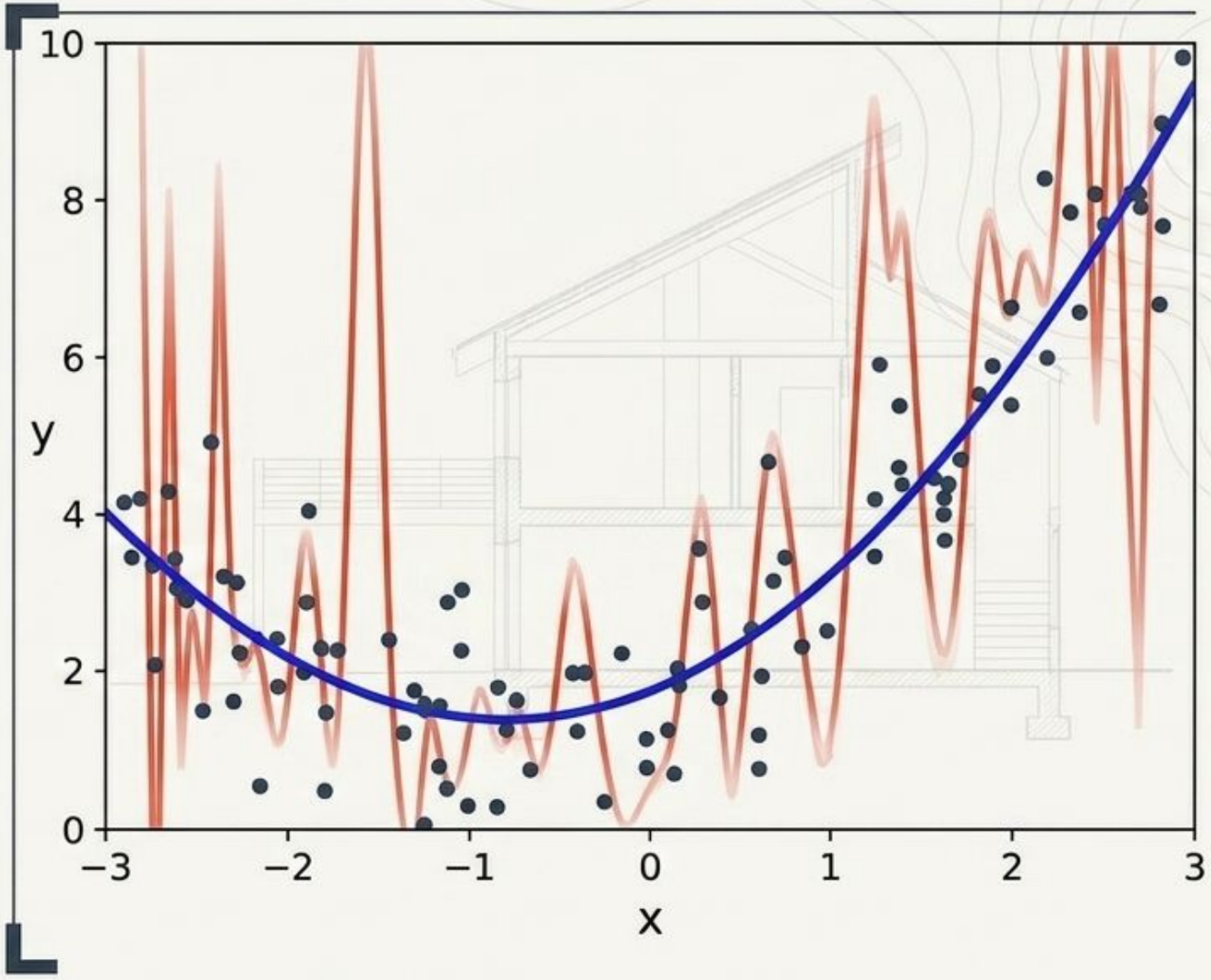
## Core Concept

Regularization forces the learning algorithm to keep its weights (coefficients) as small as possible. The bias term ($\theta_0$) is intentionally left unregularized.

## Mechanism

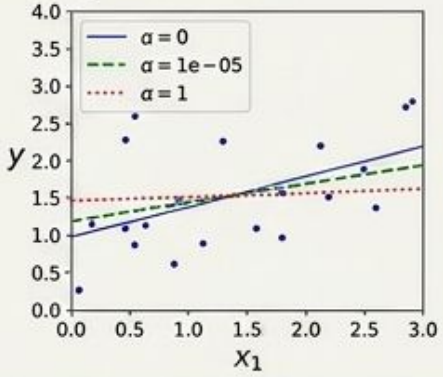
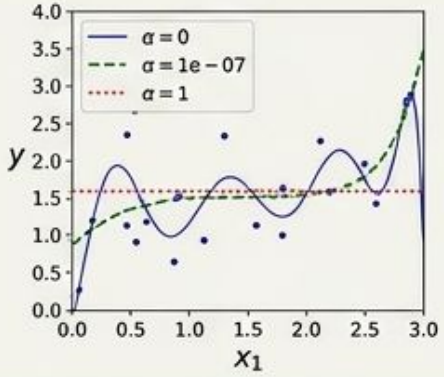
$$J(\theta) = \text{MSE}(\theta) + \text{Penalty Term}$$

We add a mathematical penalty to the cost function so the model pays a price for using large, volatile weights.



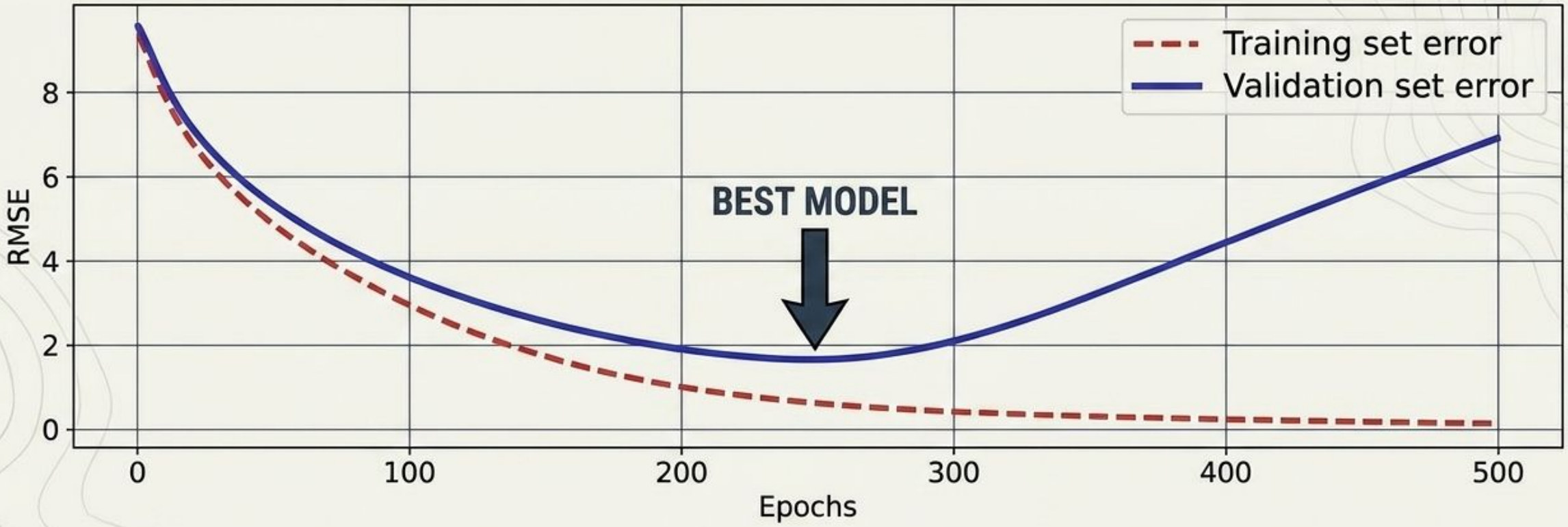

Gemini Notebook

# Regularization Showdown: Ridge, Lasso, and Elastic Net

| Ridge (L2 Regularization) | Lasso (L1 Regularization) | Elastic Net |
|---|---|---|
| **Penalty Equation:** $\alpha \frac{1}{2} \sum (\theta_i)^2$ | **Penalty Equation:** $\alpha \sum \lvert\theta_i\rvert$ | **Penalty Equation:** $r\,\alpha \sum \lvert\theta_i\rvert + \left(\frac{1-r}{2}\right) \alpha \sum (\theta_i)^2$ |
| **Behavior:** Shrinks coefficients smoothly, but never exactly to zero.   | **Behavior:** Can shrink coefficients entirely to zero, acting as automatic feature selection.   | **Behavior:** A hybrid controlled by ratio $r$. Preferred over Lasso when features outnumber instances or are strongly correlated. |
| **Solution:** Has a closed-form solution: $\hat{\theta} = (\mathbf{X}^T\mathbf{X} + \alpha\mathbf{A})^{-1}\mathbf{X}^T\mathbf{y}$ | | |

Gemini Notebook

# Early Stopping: The Beautiful Free Lunch




**Takeaway:** Stop training the exact moment validation error reaches its minimum. This provides regularization without mathematical penalties.

Geoffrey Hinton called it a “beautiful free lunch”.”

Gemini Notebook

# Shifting to Classification: Logistic Regression

**The Pivot**

Logistic Regression is actually a regression algorithm used for classification. It estimates the precise probability that an instance belongs to a specific class.

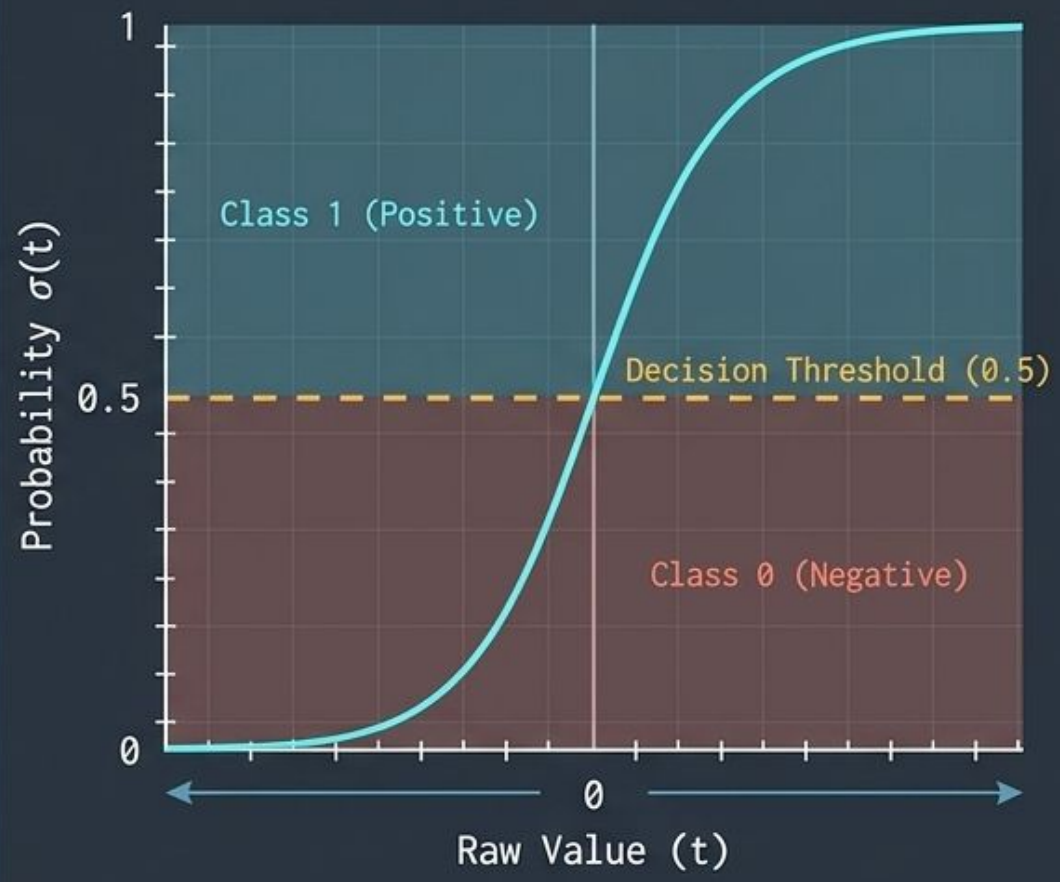
**The Math**

It computes a weighted sum of inputs (just like Linear Regression), but passes the result through the logistic function:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

**The Output Mechanism**

If the estimated probability is ≥ 50%, it predicts Class 1 (positive class). Otherwise, it predicts Class 0 (negative class).




Gemini Notebook

# Measuring Classification Error (Log Loss)

$$J(\theta) = -\frac{1}{m}\sum \left[ y^{(i)} \log(p^{(i)}) + (1 - y^{(i)}) \log(1 - p^{(i)}) \right]$$

Penalizes the model if it predicts near 0 when the true class is 1.

Penalizes the model if it predicts near 1 when the true class is 0.

### Topographical Reassurance

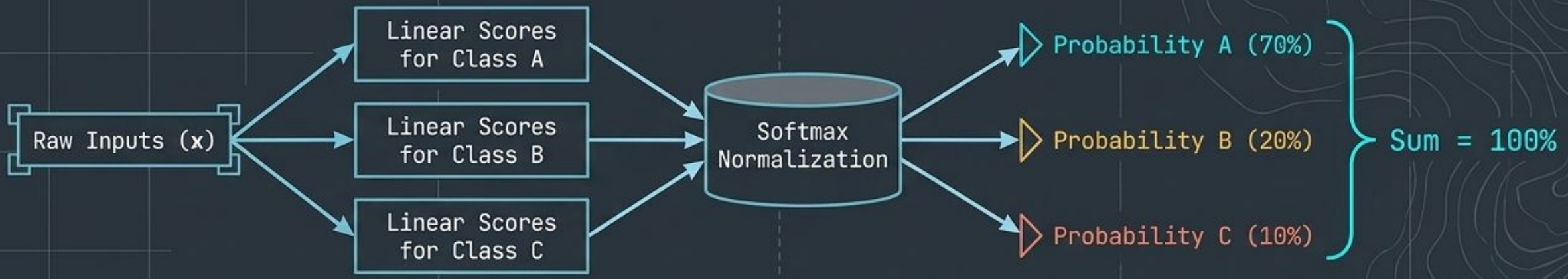
Despite lacking a closed-form solution, this cost function is strictly convex.

Gradient Descent is still mathematically guaranteed to find the global minimum.

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m}\sum (\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}_j^{(i)}$$

Gemini Notebook

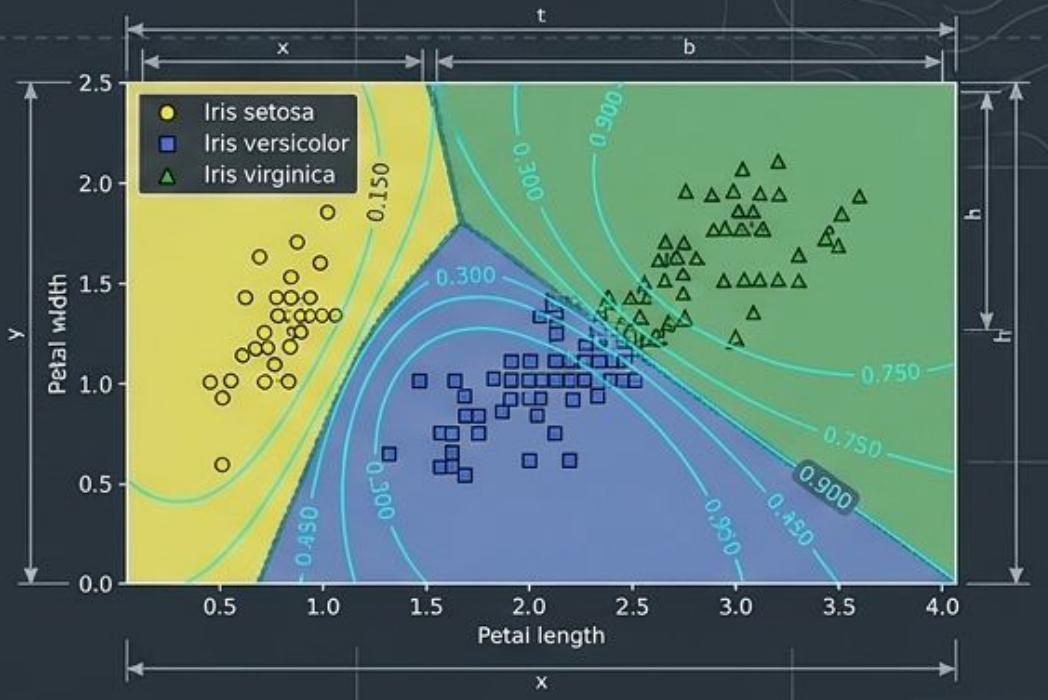
# Multi-Class via Softmax Regression




**The Concept:**
Softmax Regression generalizes logistic regression to support multiple classes directly, without combining multiple binary models.

**The Mechanism:**
1. Computes a raw score $s_k(x)$ for each class $k$.
2. Applies the Softmax Function to normalize these scores into estimated probabilities.

**The Training Objective:** Uses Cross-Entropy Loss, a cost function that specifically penalises models for estimating a low probability for the true target class.




Gemini Notebook

# Softmax Regression

- Softmax score for class $k$

$$s_k(x) = x^T \theta^k$$

- Each class has its own dedicated parameter vector $\theta^k$. All these vectors are typically stored as rows in a $parameter\ matrix\ \Theta$.
- Estimate the probability $\hat{p}^k$ that the instance belongs to class $k$ by running the scores (logits or log-odds) through the Softmax function:

$$\hat{p}^k = \sigma\big(s(x)\big)_k = \frac{e^{s_k(x)}}{\sum_{j=1}^{K} e^{s_j(x)}}$$

- Softmax Regression classifier prediction

$$\hat{y} = \underset{k}{argmax}\ \sigma\big(s(x)\big)_k$$

# Softmax Regression

- **Cross entropy** cost function is used to measure how well a set of estimated class probabilities matches the target classes.

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^i \log\left(\hat{p}_k^i\right)$$

- $y_k^i$ is the target probability that the $i$-th instance belongs to class $k$. In general, it is either equal to 1 or 0, depending on whether the instance belongs to the class or not.
- Cross entropy gradient vector for class $k$

$$\frac{\partial}{\partial \theta^k} J(\Theta) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^i - y_k^i\right)x^i$$

# KL Divergence (Kullback-Leibler Divergence)

- What is KL Divergence?
  - A measure of how one probability distribution differs from a second, reference probability distribution.
- Given two probability distributions P (true) and Q (approximation):

$$D_{KL}(P||Q) = \sum_{x} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

# KL Divergence in Terms of Cross-Entropy

- **Entropy** of a distribution $P$: This quantifies the inherent uncertainty in your true distribution $P$.

$$H(P) = -\sum_{x} P(x)\log P(x)$$

- **Cross-Entropy** between distributions $P$ and $Q$: This measures how far your predicted distribution $Q$ is from the true distribution $P$.

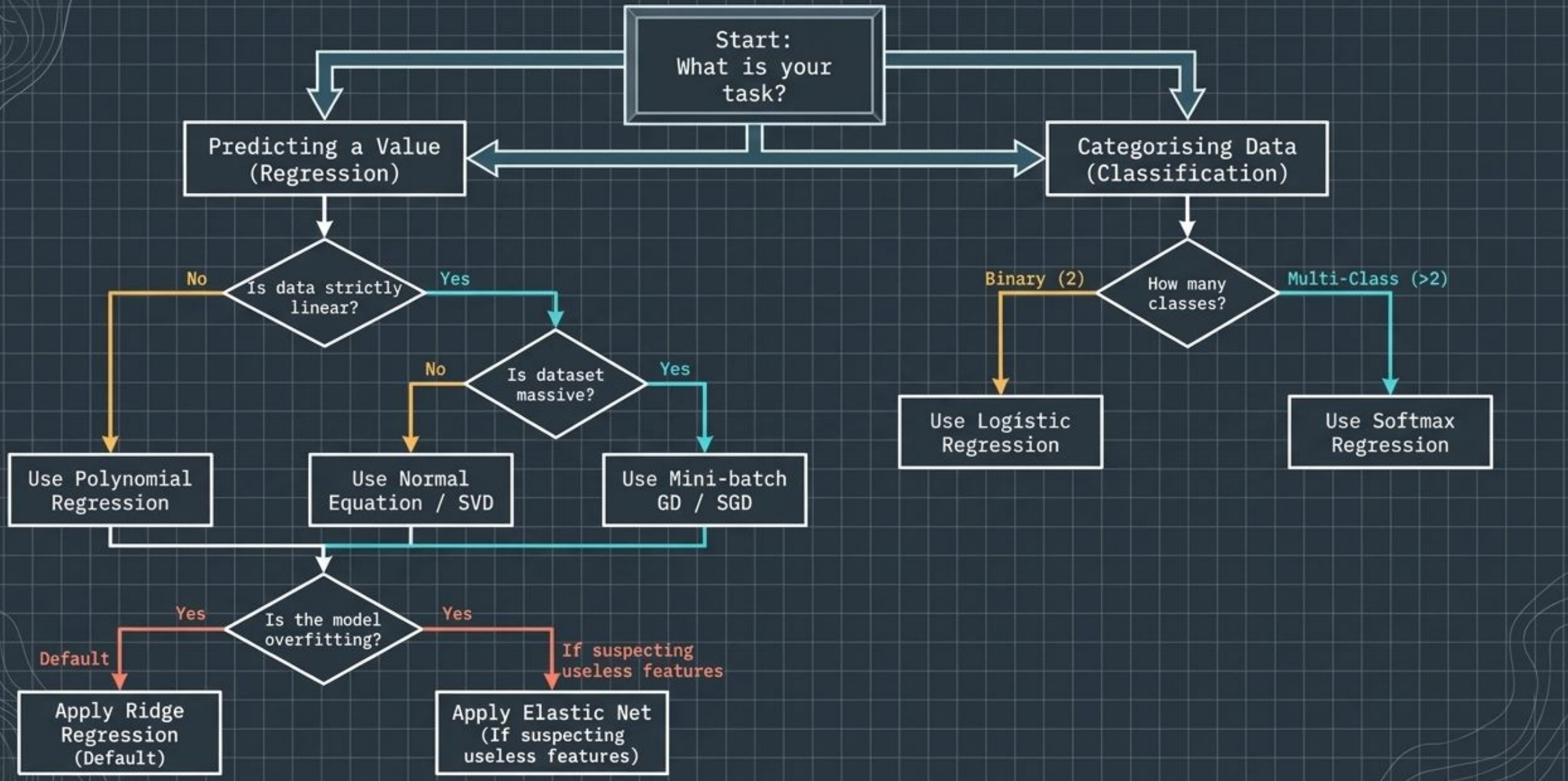
$$H(P,Q) = -\sum_{x} P(x)\log Q(x)$$

- **KL Divergence** as a Difference:

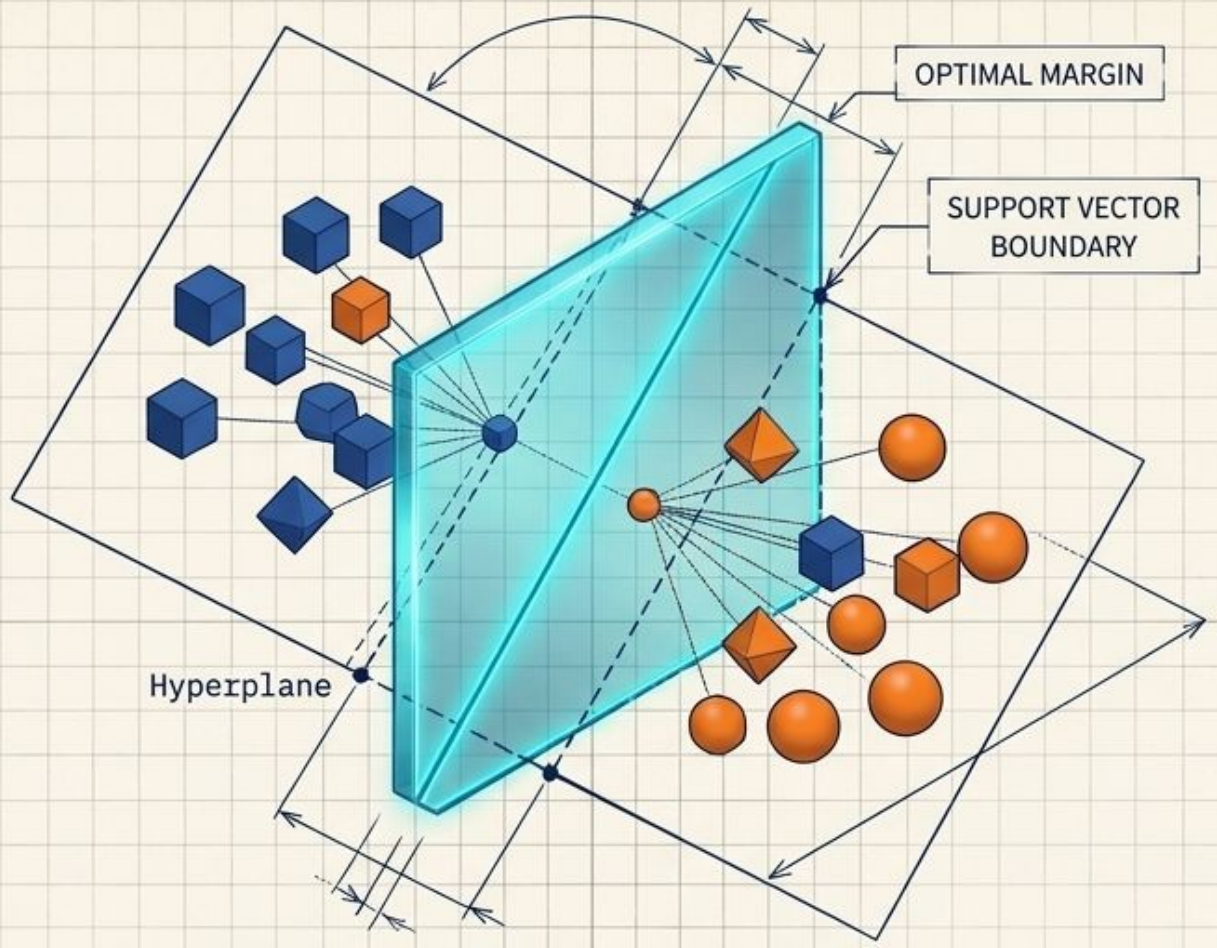
$$D_{\mathrm{KL}}(P||Q) = H(P,Q) - H(P)$$

  - If $Q$ approximates $P$ well, cross-entropy is close to entropy ⇒ small KL divergence.
  - If $Q$ is far from $P$, cross-entropy is much larger ⇒ large KL divergence.

# Master Synthesis: The Optimization Decision Tree




Gemini Notebook



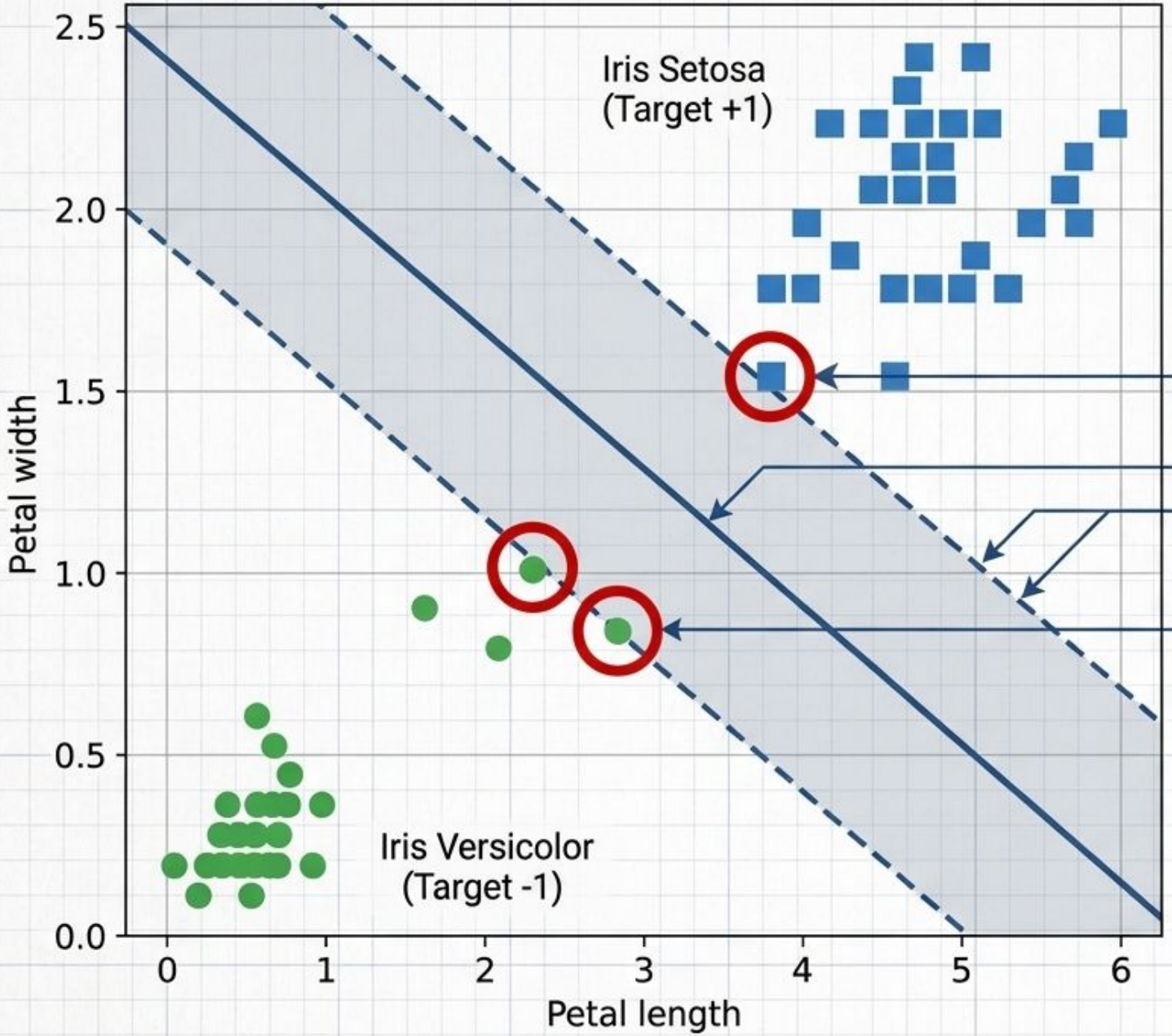

# Demystifying Support Vector Machines (SVMs)

## A Visual & Intuitive Guide to Margin Architecture

Gemini Notebook

# What is a Support Vector Machine?

- Supervised learning model for classification and regression
- Finds the optimal hyperplane that separates classes with the maximum margin
- Effective in high-dimensional spaces
- SVMs are well suited for classification of complex small- or medium-sized datasets
- Robust to overfitting, especially in high-dimensional spaces




## The Core Intuition: Fitting the Widest Highway

Support Vector Machines act as urban planners seeking to build the widest possible street between two classes.

The Decision Boundary (Centerline)

The Margins (Curbs)

### The Pillars of the Street

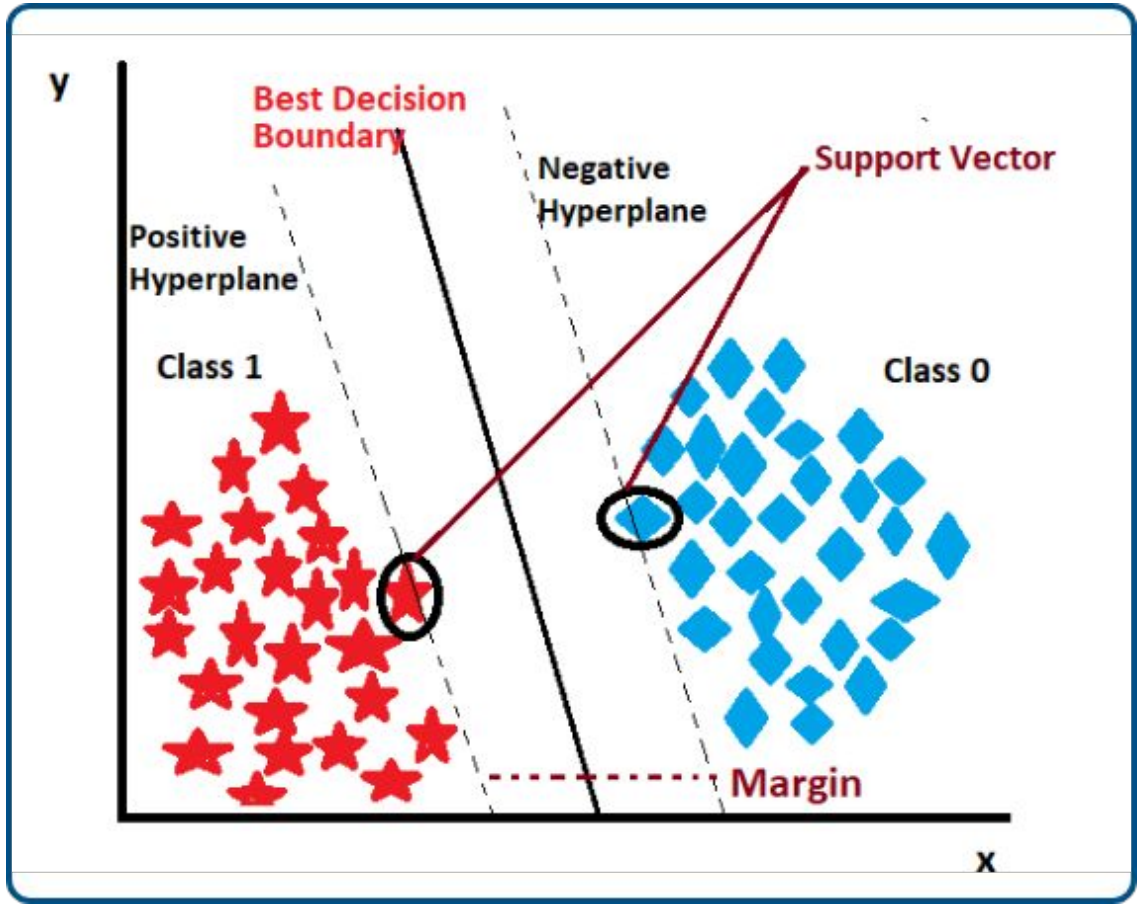
Support vectors are the foundational pillars resting exactly on the curbs. They alone hold up the highway.

Deleting all other non-support vectors (ghosting them out) leaves the highway perfectly unchanged.

Gemini Notebook

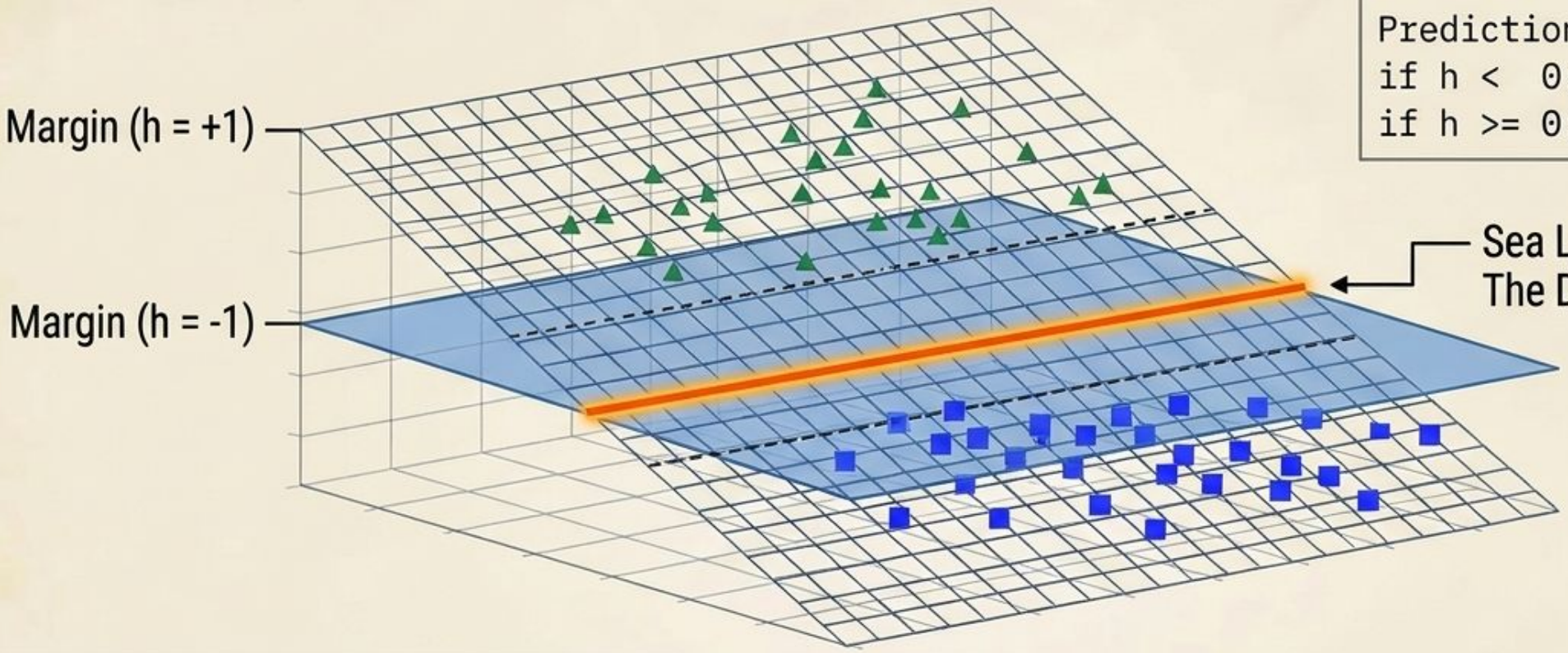
# Hard Margin Classification

**Assumes perfect linear separability without any boundary violations.**



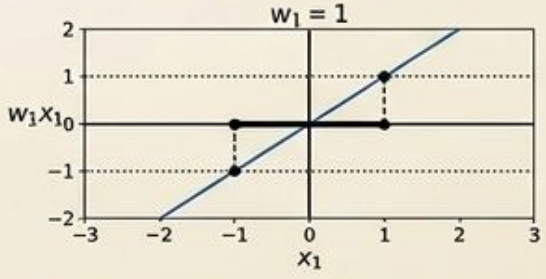
# Under the Hood: The Decision Function ($h = w^T x + b$)

```
Prediction Logic:
if h <  0: Predict Class 0
if h >= 0: Predict Class 1
```

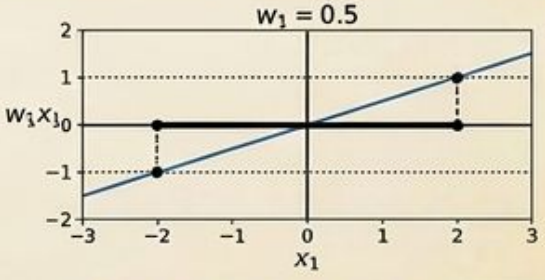



**Training Objective:**
To maximise the margin, we must minimise the weight vector slope.
**Minimise:** $^1/_2\, w^T w$

Large Weight ($w_1$) = Steep Slope = Narrow Margin




Small Weight ($w_2$) = Shallow Slope = Wide Margin




Gemini Notebook

## System Parameters

**Weights (w) = [-1.2, -3.0]^T**

**Bias (b) = 3.6**

**Formula: h = (w_1 * x_1) + (w_2 * x_2) + b**

## Worked Example: Calculating Altitude on the Grid

**Point A (Setosa, +1): x_1 = [1.5, 0.2]^T**
**h = (-1.2 * 1.5) + (-3.0 * 0.2) + 3.6 = +1.2**

**Result:** Since h >= 1, it is correctly classified and safely outside the margin.

**Point B (Versicolor, -1): x_2 = [3.5, 1.0]^T**
**h = (-1.2 * 3.5) + (-3.0 * 1.0) + 3.6 = -3.6**

**Result:** Since h <= -1, it is correctly classified and safely outside the margin.

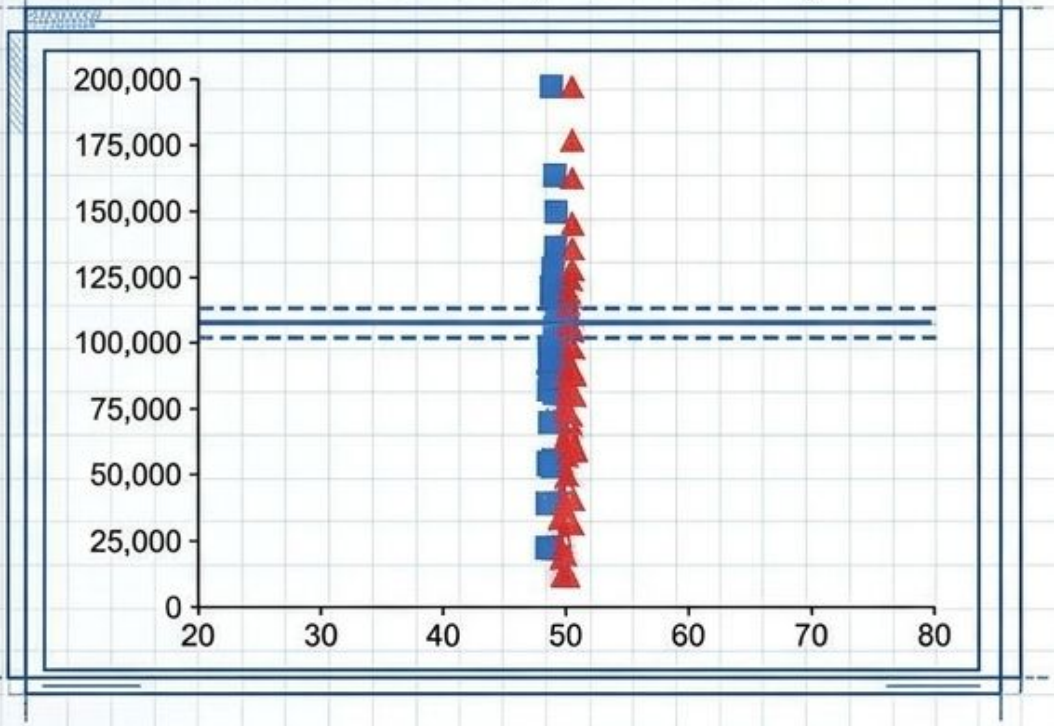
**Point D (Ambiguous): x_3 = [2.0, 0.5]^T**
**h = (-1.2 * 2.0) + (-3.0 * 0.5) + 3.6 = -0.3**

**Result:** Since h < 0, we predict Versicolor (-1). However, because -1 < h < 1, the point sits dangerously inside the street.

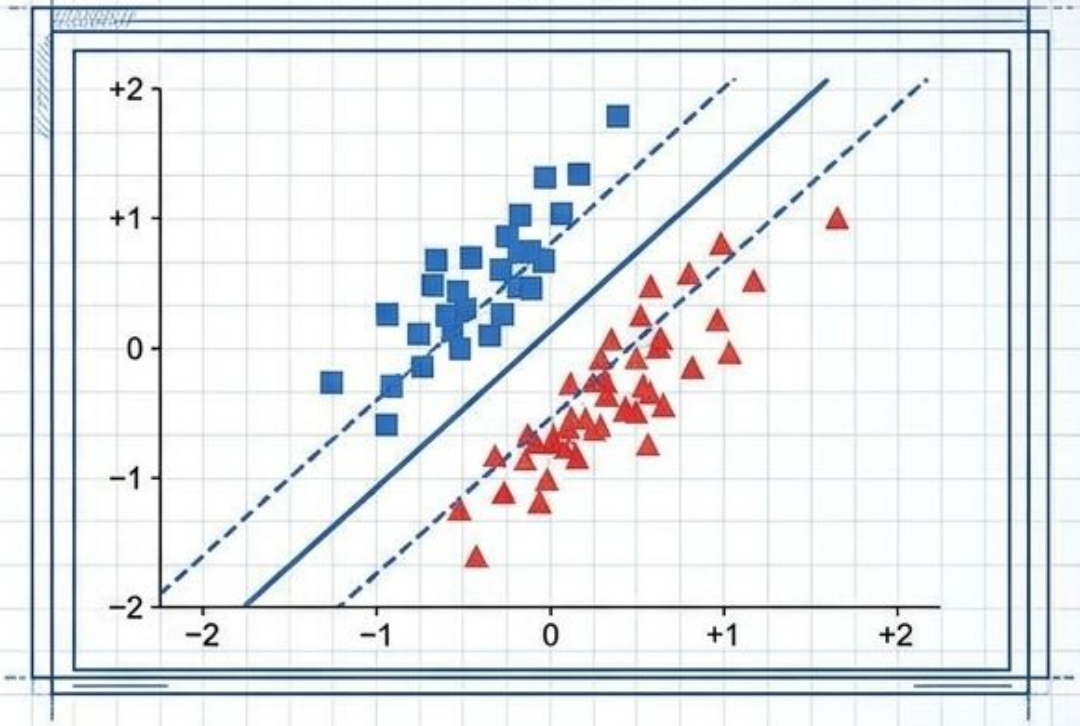
Gemini Notebook

# Geometrical Sensitivity Demands Strict Feature Scaling

### Unscaled Data (The Problem)




### Scaled Data (The Solution)




## The Housing Example

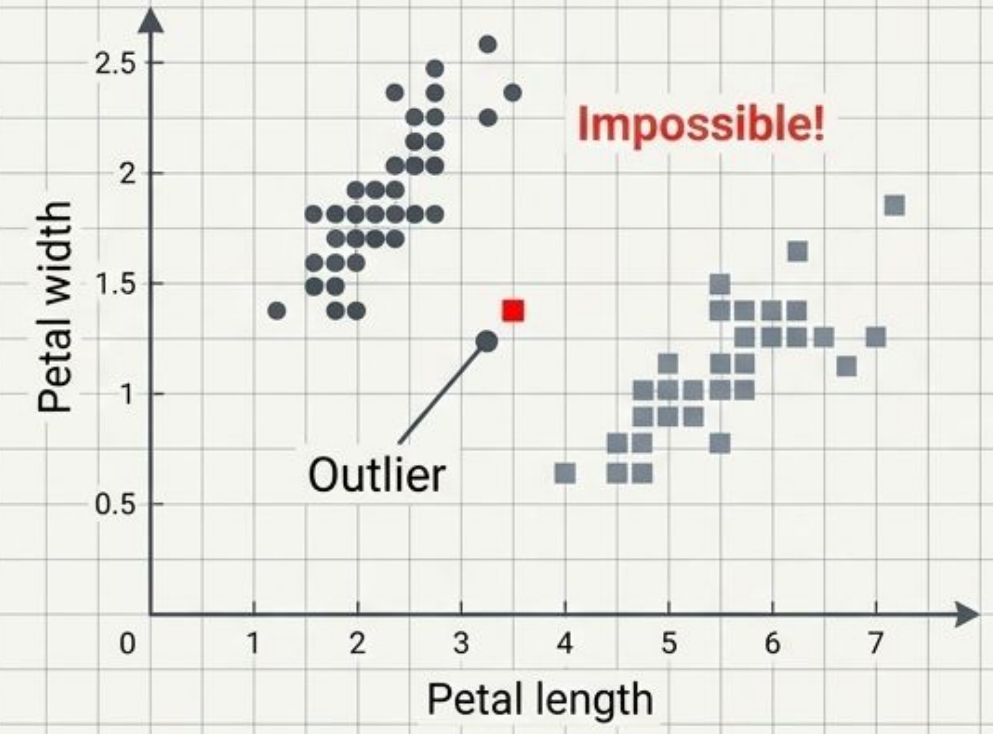
If mapping Age (20-80) vs Income ($20k-$200k), the Income axis is mathematically thousands of times wider. Because SVMs measure raw geometric distance, the algorithm physically ignores Age entirely, drawing a vertical boundary.

## The Mandatory Fix

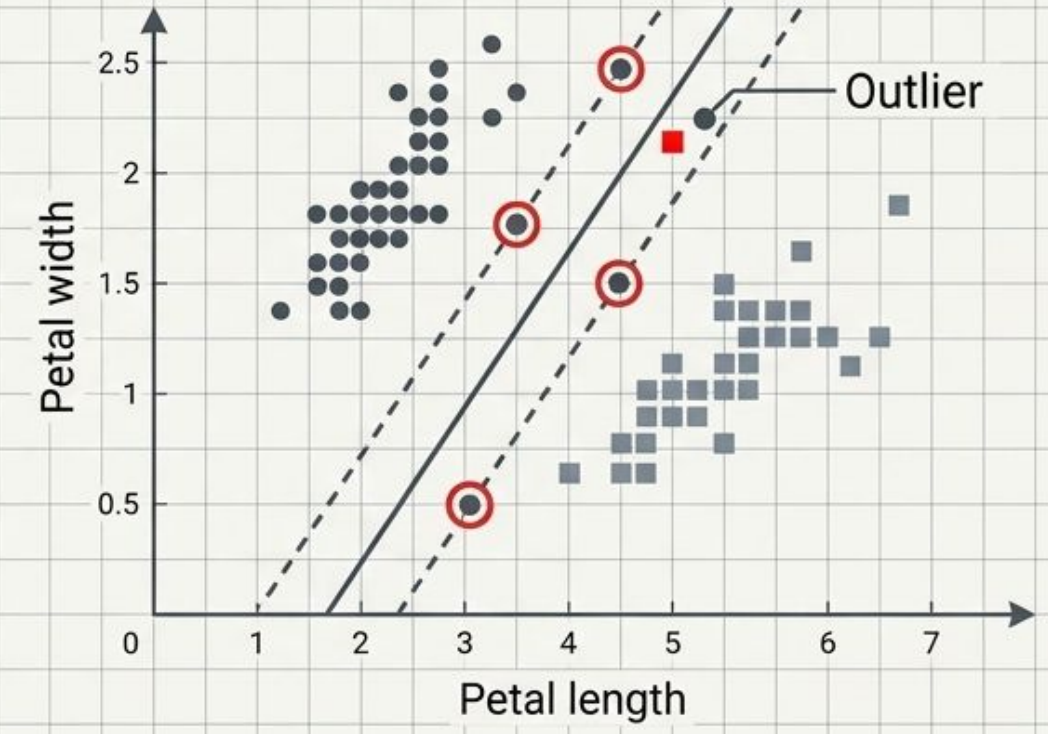
StandardScaler standardises features to a mean of 0 and variance of 1, forcing the algorithm to evaluate all dimensions equally.

Gemini Notebook

**Failure State 1: Mathematically Impossible.** A single outlier breaks linear separability.




**Failure State 2: Structural Compromise.** A **single outlier** forces an unacceptably narrow margin, guaranteeing poor generalization.

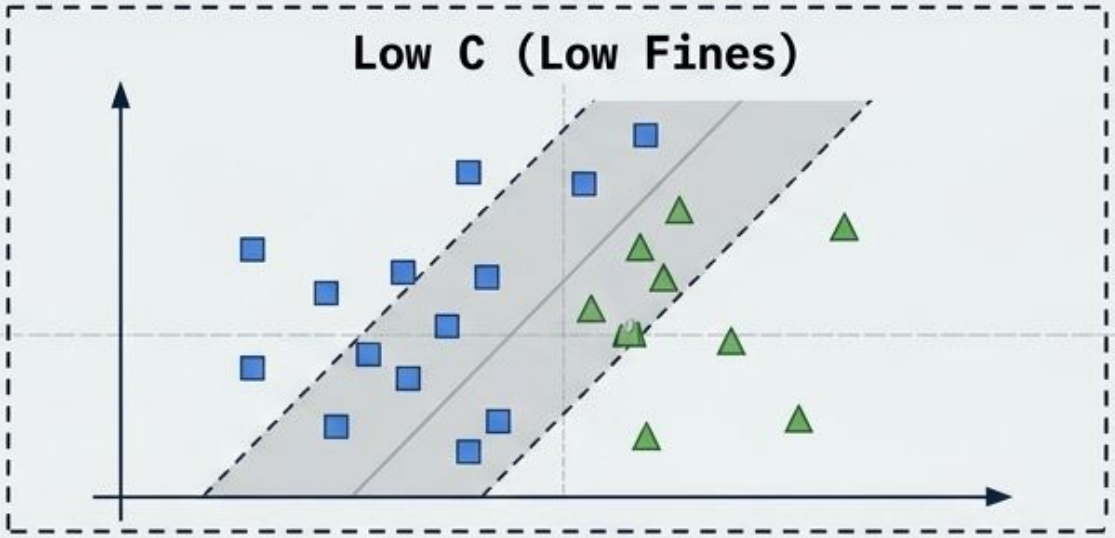


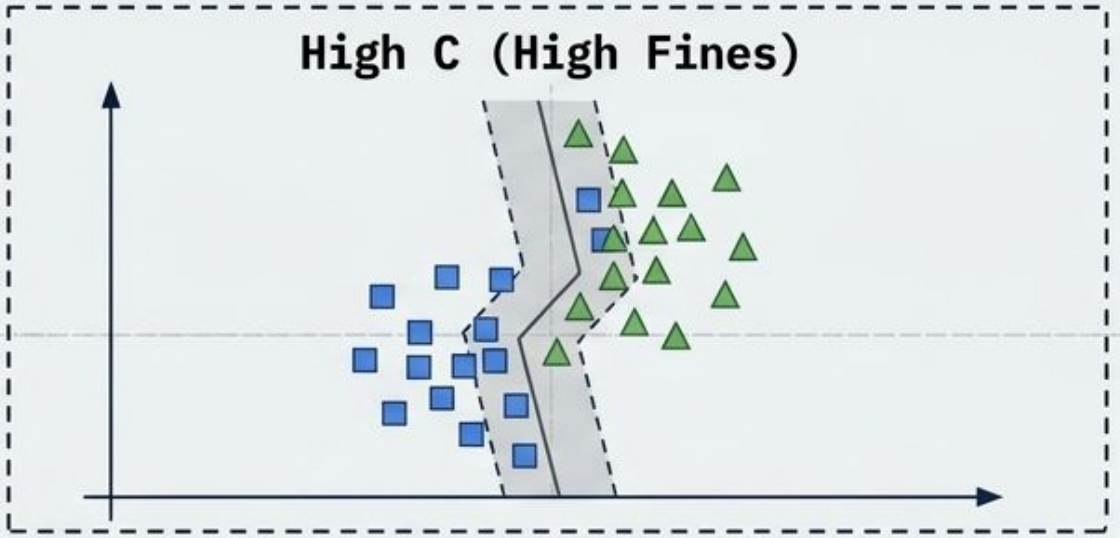

**Hard margin SVMs demand perfect data. Real-world data is never perfect.**

Gemini Notebook

# Engineering the Soft Margin with "Parking Ticket Fines"

Real data has outliers. The C Hyperparameter acts as “parking ticket fines” for points violating the margin, regulating the strictness of the street.




| | Low C Architecture | High C Architecture |
|---|---|---|
| **(Visual)** | Wide, lenient highway | Narrow, strict highway |
| **(Tolerance)** | High tolerance for margin violations | Low tolerance for margin violations |
| **(Risk Profile)** | Risk of Underfitting (Too generic) | Risk of Overfitting (Capturing noise) |

Gemini Notebook

# The Optimization Objectives

**Hard Margin**

Maximize the margin width by strictly minimizing the weight vector.

$$\text{minimize: } \frac{1}{2}w^{T}w$$

**Soft Margin (Slack)**

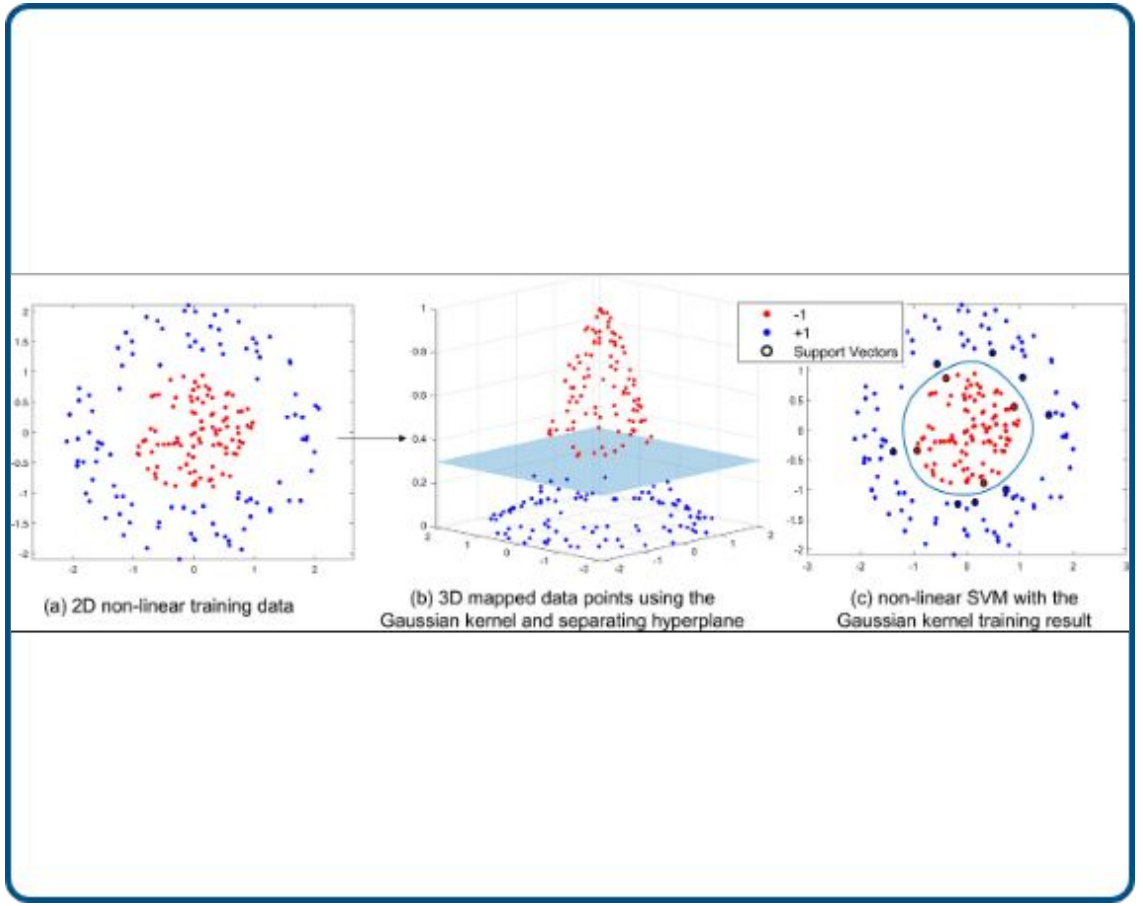
Introduce Slack $\zeta$ and hyperparameter $C$ to balance the tradeoff.

$$\text{minimize: } \frac{1}{2}w^{T}w + C\sum\zeta^{i}$$

# Lifting the Dimensionality

Many datasets are **Linearly Inseparable** in their original, raw feature space.

- **The Solution:** Physically add polynomial features (like squaring a value) to elevate the data into a higher dimension.
- **The Result:** In this elevated 2D or 3D space, a simple, flat linear hyperplane can now successfully slice between the classes.
- This is the foundation of tackling complex, real-world patterns.



# Worked Example: Polynomial Mapping

### 1D Iris Problem

Suppose Versicolors are grouped tightly between Petal Lengths $1 < x < 3$. Setosas are $x < 1$ and Virginicas are $x > 3$. You cannot draw a single straight line (OD point) to isolate Versicolor.

### Mapping to 2D

We apply a simple polynomial transformation, squaring the feature.

$$x \rightarrow (x, x^2)$$

### The Math

**Pt A (Versi):**

$$x = 2 \rightarrow (2,4)$$

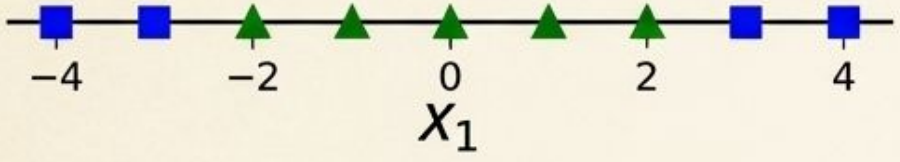
**Pt B (Setosa):**

$$x = 0.5 \rightarrow (0.5,0.25)$$

A linear line in this new 2D space easily separates the parabola's center!

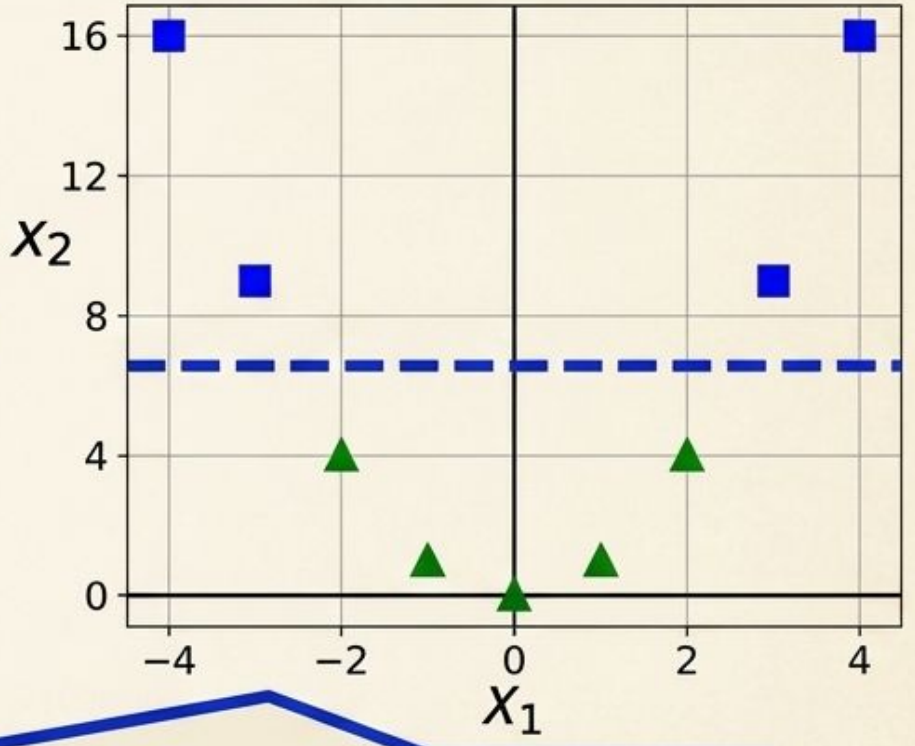
# The Nonlinear Problem: Lifting the Dimensionality

1D Space: Linearly Inseparable




Add polynomial feature:
$X_2 = (X_1)^2$

2D Space: Linearly Separable

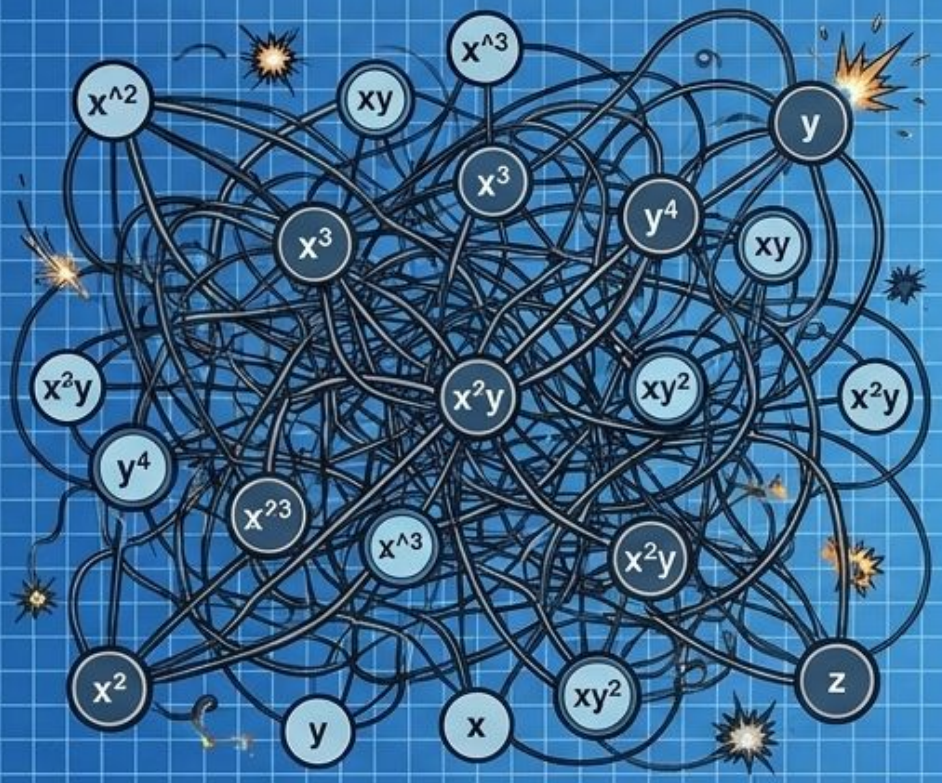



Adding polynomial features physically elevates data into higher dimensions where a simple linear hyperplane can successfully separate it.

Gemini Notebook

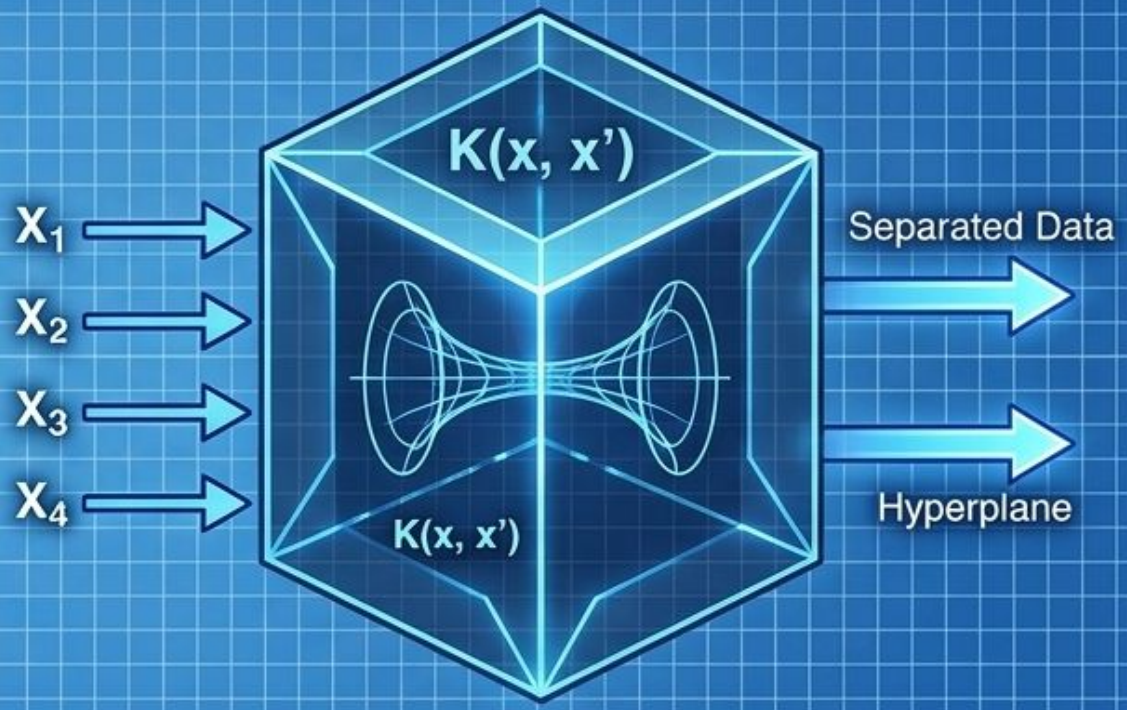
# Bypassing the Combinatorial Explosion: The Kernel Trick

## The Brute Force Path (Failure)




Actually computing polynomial features (x^3, x^4) causes a combinatorial explosion. The CPU crashes attempting to map infinite permutations.

## The Kernel Wormhole (Magic)




The Kernel trick mathematically replaces the high-dimensional mapping entirely. It computes the exact geometric distance between points in the higher dimension without ever actually transforming the data.

Gemini Notebook

# Demystifying the Kernel Trick

The Dual Equation requires computing the dot product of transformed high-dimensional features:

$$\varphi(\mathbf{x}^{(i)})^T \varphi(\mathbf{x}^{(j)})$$

**The Sleight of Hand**

We mathematically replace the transformation entirely with the Polynomial Kernel:

$$K(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^T \mathbf{b})^2$$

**The Result**

We never compute the high-dimensional mapping $\varphi$. We simply replace the dot product with its square.

$$\varphi(\mathbf{x}^{(i)})^1 = (\mathbf{a}^T \mathbf{b})^2$$

The mathematical result is strictly identical to transforming the data, but computationally trivial.

Gemini Notebook

# WORKED EXAMPLE: THE SHORTCUT IN ACTION

Two points, $\mathbf{a} = [1.5, 0.2]^T$ and $\mathbf{b} = [2.0, 0.5]^T$.

START

## THE LONG WAY (EXPLICIT 3D MAPPING)

1. Transform to $\phi(\mathbf{x}) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$

2. $\phi(\mathbf{a}) = [2.25, 0.4243, 0.04]^T$
   $\phi(\mathbf{b}) = [4.0, 1.4142, 0.25]^T$

3. Dot Product:
   $9.0 + 0.60 + 0.01 = 9.61$

## THE KERNEL SHORTCUT

1. Equation:
   $(\mathbf{a}^T * \mathbf{b})^2$

2. Calculation:
   $(1.5 * 2.0 + 0.2 * 0.5)^2$
   $= (3.1)^2 = 9.61$

**9.61**

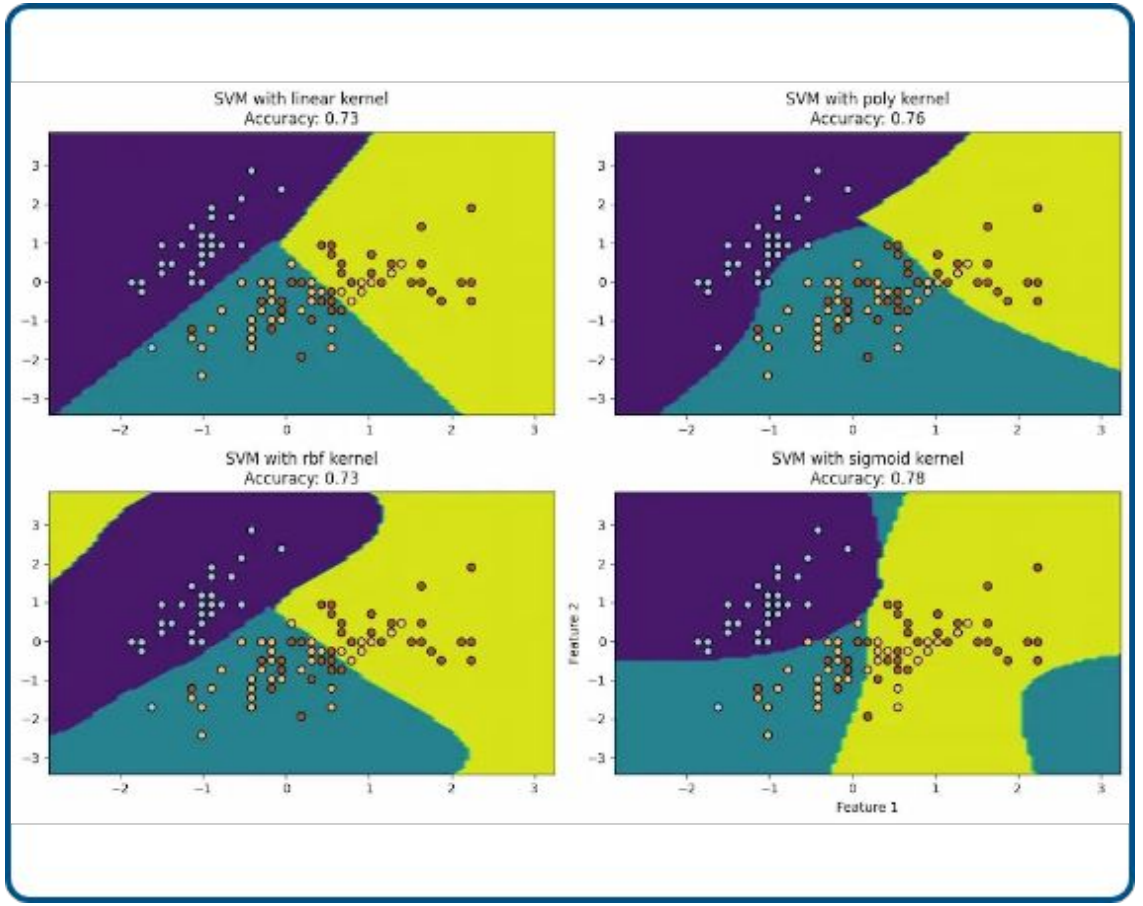
**CONCLUSION:** Both yield exactly 9.61. The shortcut arrives instantly; the long way exhausts memory at scale.

Gemini Notebook

# Similarity & The Gaussian RBF

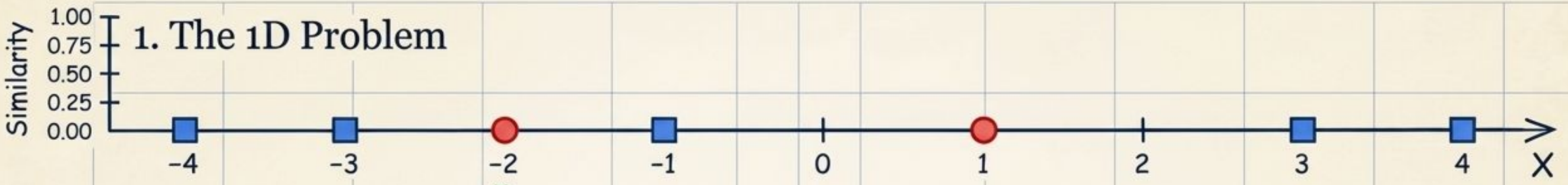
Another way to tackle nonlinearity is by generating **Similarity Features**.

- Instead of polynomial math, we measure how much each instance resembles a specific "landmark".
- The **Gaussian RBF** transforms spatial distance into a smooth similarity score ranging from 0 (distant) to 1 (identical).
- The RBF Kernel acts mathematically as if we placed a landmark at *every single* training instance, creating infinite dimensions!

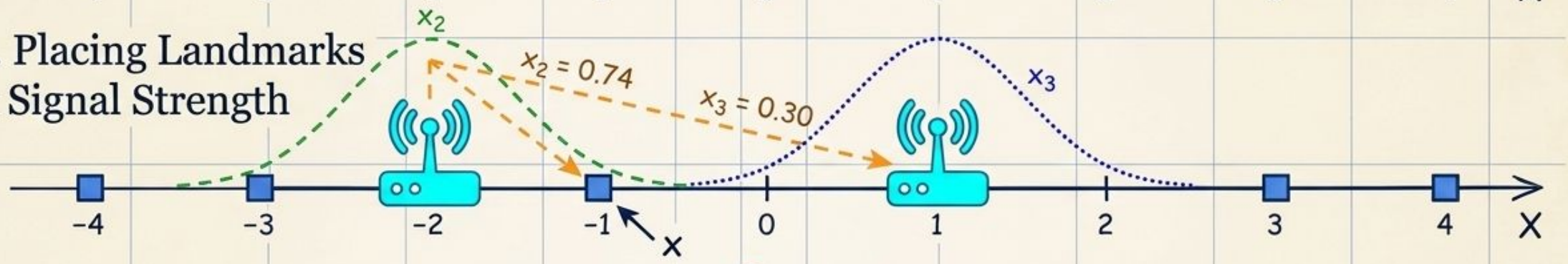


# Feature Similarity: Elevating Data with Gaussian RBF Landmarks

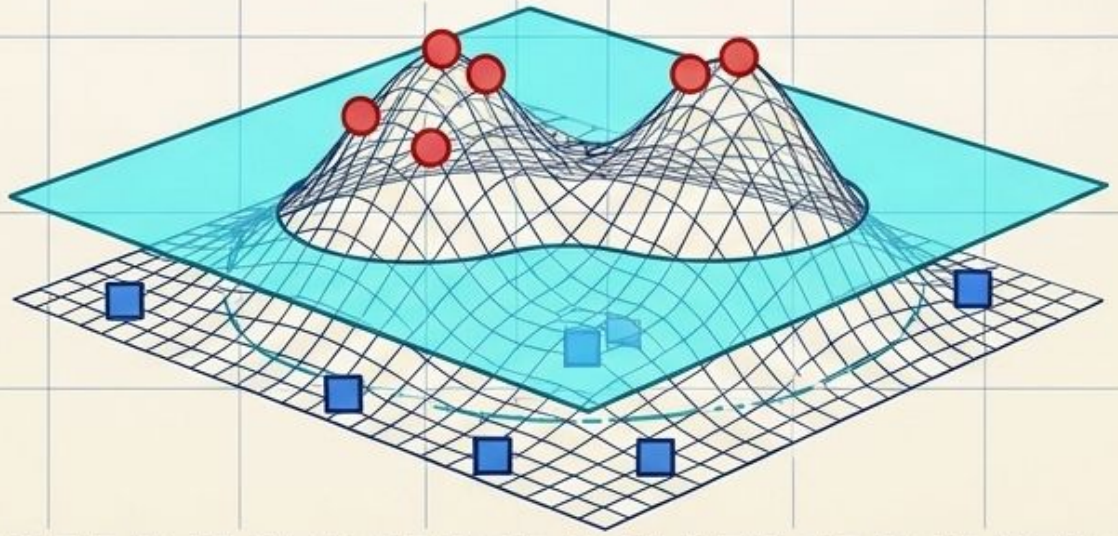
## 1. The 1D Problem




## 2. Placing Landmarks & Signal Strength




## 3. The 2D Elevation



Gemini Notebook

# Worked Example: Mapping RBF Similarity Topology

## System Coordinates

Landmark 1 ($L_1$) = $[1.5, 0.2]^T$

Landmark 2 ($L_2$) = $[3.5, 1.0]^T$

Our Point ($x$) = $[2.5, 0.6]^T$

Signal drop-off rate (gamma) = 1.0

## Calculating Similarity (Signal Strength)

Signal 1 ($f_1$):

Squared distance to $L_1$: $1.0^2 + 0.4^2$ = 1.16

Similarity $f_1$ = exp(-1.0 * 1.16) = 0.3135

Signal 2 ($f_2$):

Squared distance to $L_2$: $(-1.0)^2 + (-0.4)^2$ = 1.16

Similarity $f_2$ = exp(-1.0 * 1.16) = 0.3135

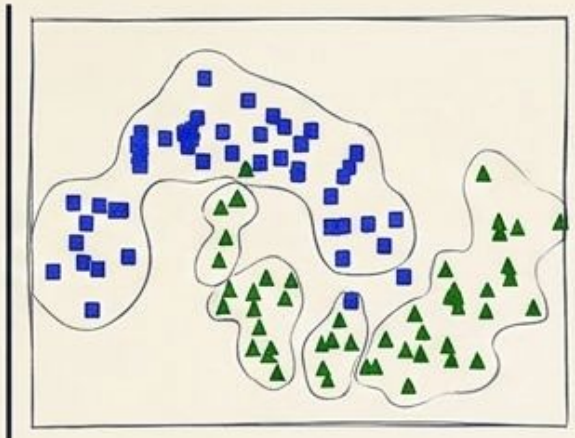
**Result: The original point [2.5, 0.6] is now represented in a brand new, linearly separable similarity space as coordinates [0.3135, 0.3135].**

Gemini Notebook

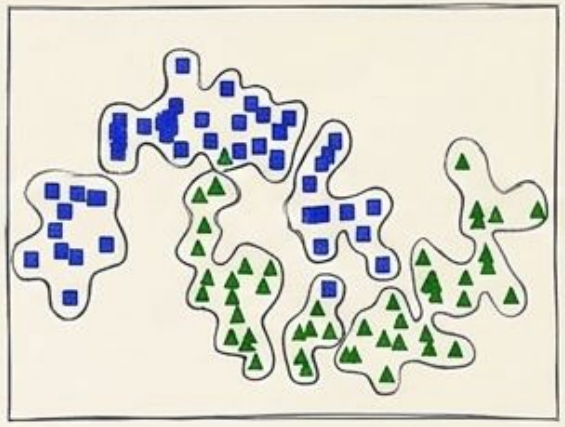
# Tuning the RBF Engine: C and Gamma

Increasing Gamma (Range of Influence)

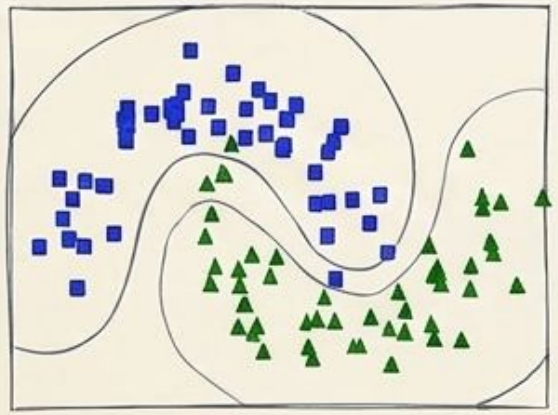
Narrow bell curves, but softened margin enforcement.



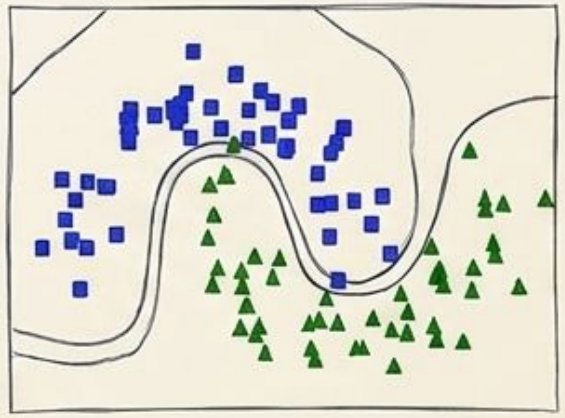
Severe Overfitting: Tight, irregular boundaries wiggling around individual points.



**Underfitting Risk:** Smooth, highly generalised sweeping boundaries.



Wide range of influence, but strict margin adherence.



Increasing C (Tolerance for Violations)

Gemini Notebook

# Kernelized SVMs

- 
- The function $K(a,b) = (a^T b)^2$ is a second-degree polynomial kernel.
- In Machine Learning, a kernel is a function capable of computing the dot product $\phi(a)^T \phi(b)$, based only on the original vectors $a$ and $b$, without having to compute the transformation $\phi$.
- **Common kernels**
  - Linear: $K(a,b) = a^T b$
  - Polynomial: $K(a,b) = \gamma(a^T b + r\ )^d$
  - Gaussian RBF: $K(a,b) = \exp(-\gamma \parallel a - b \parallel^2)$
  - Sigmoid: $K(a,b) = \tanh(\gamma a^T b + r)$

# Mercer's Theorem: The License to Kernelize

## Mercer's Theorem Statement

If a function $K$ respects Mercer's conditions (***continuous***, ***symmetric***), a ***high-dimensional*** **mapping** $\phi$ is mathematically guaranteed to exist, even if we never know what it is.

## Lightning Fast Inference

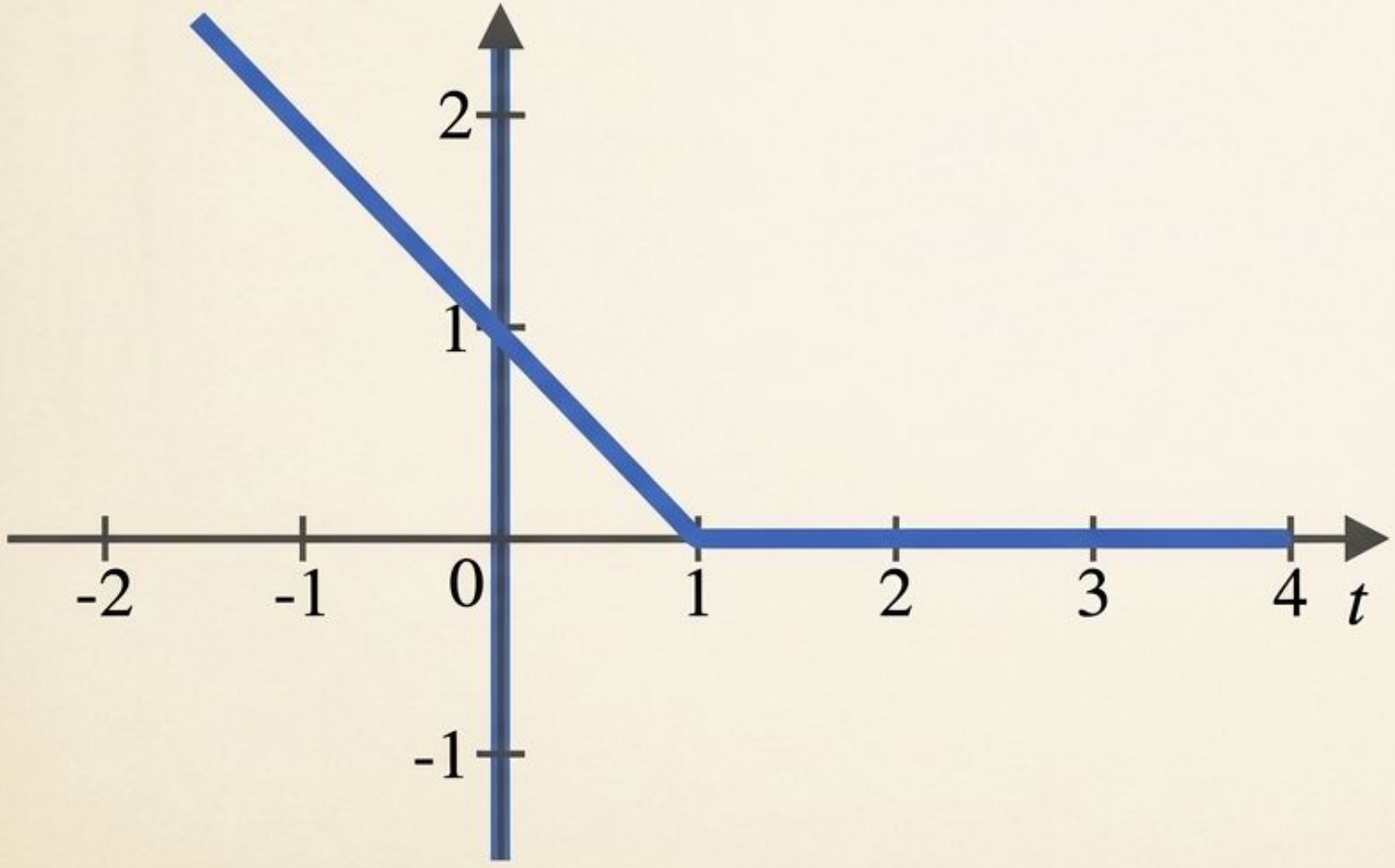
$$h = \sum (\alpha^{(i)} t^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(\text{new})})) + b$$

Because $\alpha = 0$ for all points except Support Vectors, a new prediction only requires calculating **dot products** against the Support Vectors, not the entire dataset.

Gemini Notebook

# Scaling to Infinity: Online SVMs & Hinge Loss

The Hinge Loss Function: $\max(0, 1 - t)$




For massive out-of-core datasets that do not fit in memory, QP solvers fail.

Instead, we use Stochastic Gradient Descent (`SGDClassifier`) to iteratively minimise the Hinge Loss function.

The mathematically flat zero section ensures that instances correctly classified outside the margin contribute exactly zero to the cost function—retaining the defining characteristic of an SVM.

Gemini Notebook

| Class | Time Complexity | Out-of-core Support | Kernel Trick |
| --- | --- | --- | --- |
| LinearSVC | $O(m \times n)$ | No | No (Fastest for linear) |
| SGDClassifier | $O(m \times n)$ | Yes (Best for massive datasets) | Yes (Powerful, but chokes on huge datasets) |
| SVC | $O(m^2 \times n)$ to $O(m^3 \times n)$ | No | |

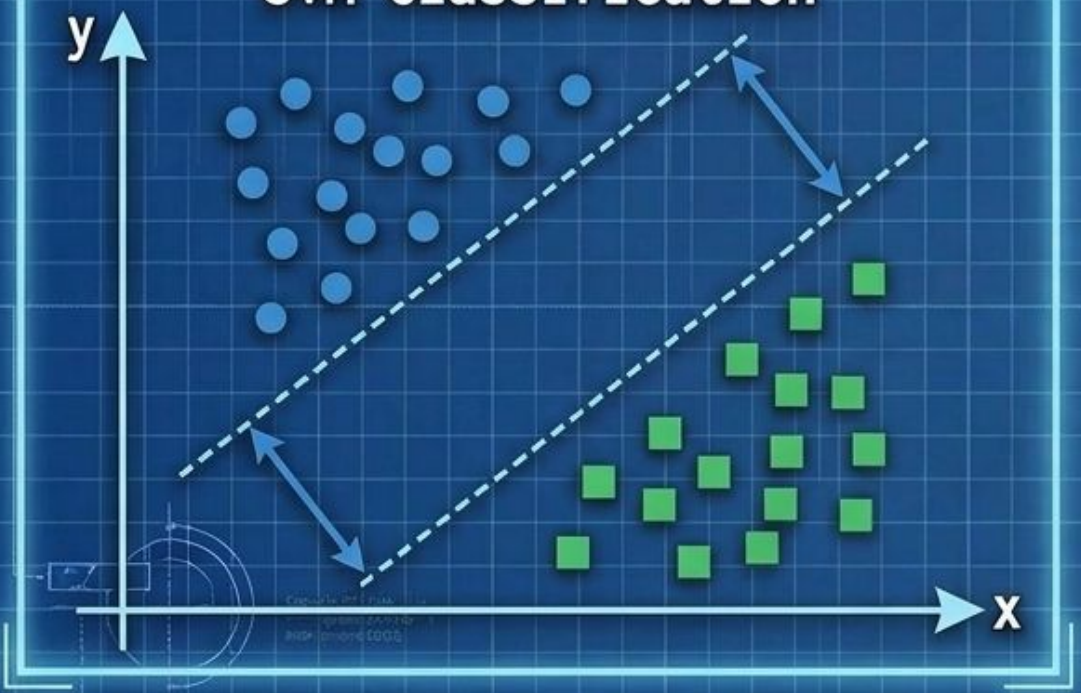
**Scaling is mandatory:** Always standardize features (e.g., StandardScaler) before training any SVM architecture.

Gemini Notebook

# Reversing the Paradigm for SVM Regression (SVR)

The mathematical engine remains identical, but the geometric objective is perfectly inverted.

## SVM Classification




Goal: Fit the widest empty highway between classes, strictly limiting instances inside the street.

## SVM Regression

Goal: Build a crowded street. Contain as many points as possible inside a margin of fixed width (epsilon).

**Epsilon Insensitivity:** Adding more data points strictly inside the margin does not affect the model's trajectory at all.

Gemini Notebook

# The Architect's Blueprint: Inference Logic

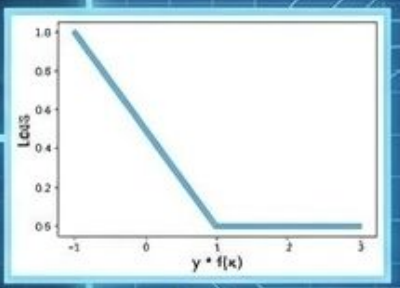
```
Code Block 1: Preprocessing
FUNCTION preprocess_data(features):
    // Mandatory geometrical standardization
    RETURN StandardScaler(features)

Code Block 2: Linear Inference (Primal)
FUNCTION predict_linear(x, w, b):
    h = dot_product(w, x) + b
    IF h < 0:
        RETURN Class 0
    ELSE:
        RETURN Class 1

Code Block 3: Kernelized Inference (Dual)
FUNCTION  predict_kernelized(x_new, support_vectors, alphas, targets, b):
    h = 0
    // Iterate only through Support Vectors
    FOR i IN support_vectors:
        K = kernel_function(x[i], x_new) // Dot product proxy
        h += alpha[i] * targets[i] * K

    RETURN Class 1 IF (h + b) ≥ 0 ELSE Class 0
```

Gemini Notebook

# The Architect's Blueprint: Online Training Engine

Code Block: Hinge Loss Training

```
FUNCTION train_online_svm(X, y, C):
    w = initialize_weights(); b = initialize_bias()
    FOR epoch IN epochs:
        FOR instance (x, t) in (X, y):
            // Calculate Hinge Loss: flatlines at zero for correct predictions
            loss = max(0, 1 - t * (dot_product(w, x) + b))
            IF loss > 0:
                // Penalty applies. Iteratively minimise using SGD.
                w, b = update_weights_SGD(w, b, loss, C)
            ELSE:
                // Instance safely outside margin. Costs exactly zero.
                CONTINUE
    RETURN w, b
```




Math Matches Geometry

## Closing Takeaway

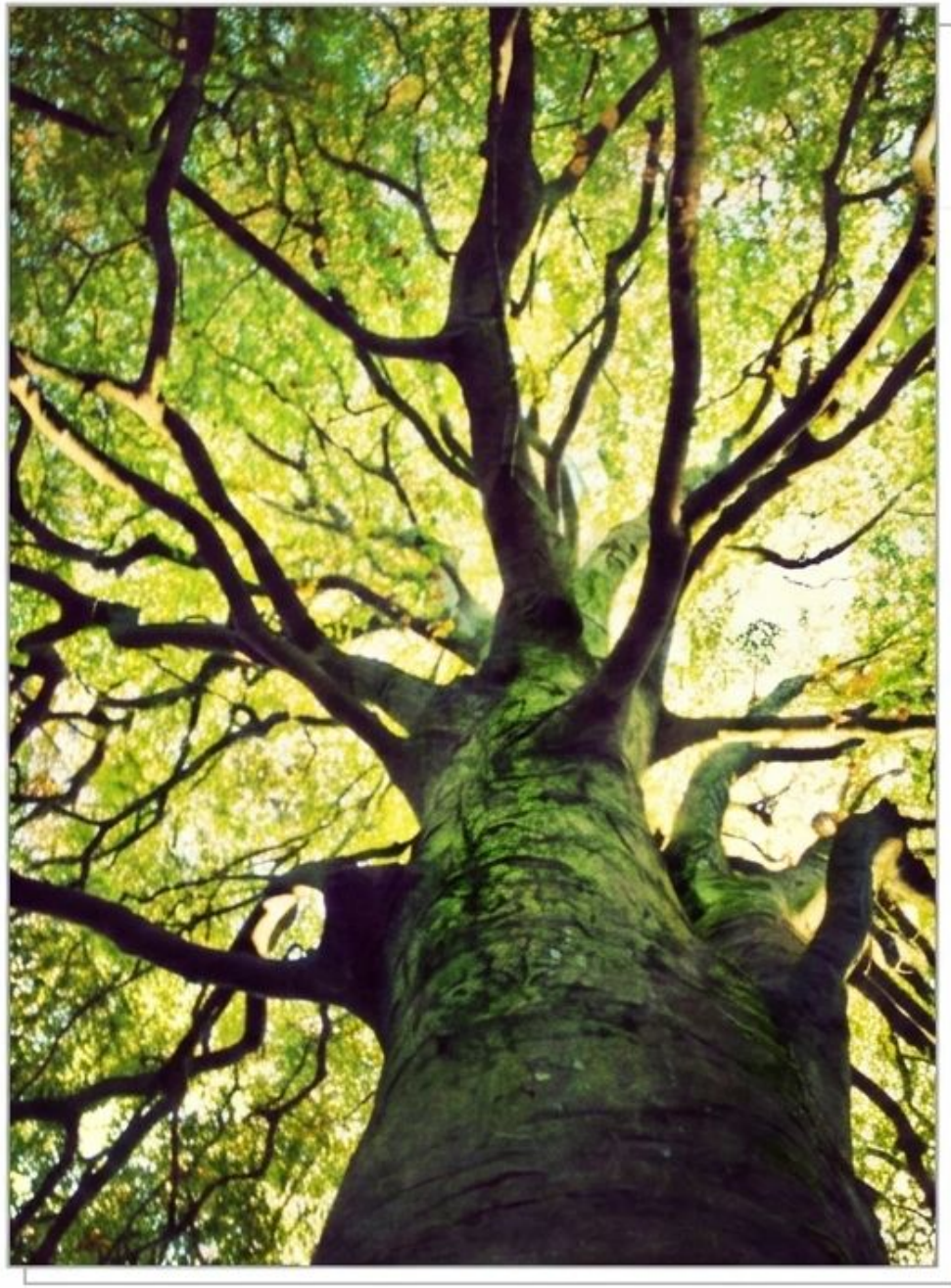
The elegance of SVM lies in its selective memory. By ignoring the vast majority of safely classified data (Hinge Loss = 0), the algorithm re-relies exclusively on the support vectors—the structural pillars—to generalize complex spatial boundaries.

Gemini Notebook

# The Anatomy of a Decision Tree

Mechanics, Mathematics, and Optimization for Classification and Regression

Gemini Notebook



# The White Box Model

**A Decision Tree is a flowchart-like structure that mimics human decision-making. Unlike opaque "black box" algorithms, its logic is fully transparent, intuitive,  and easy to interpret.**

### Dual-Use

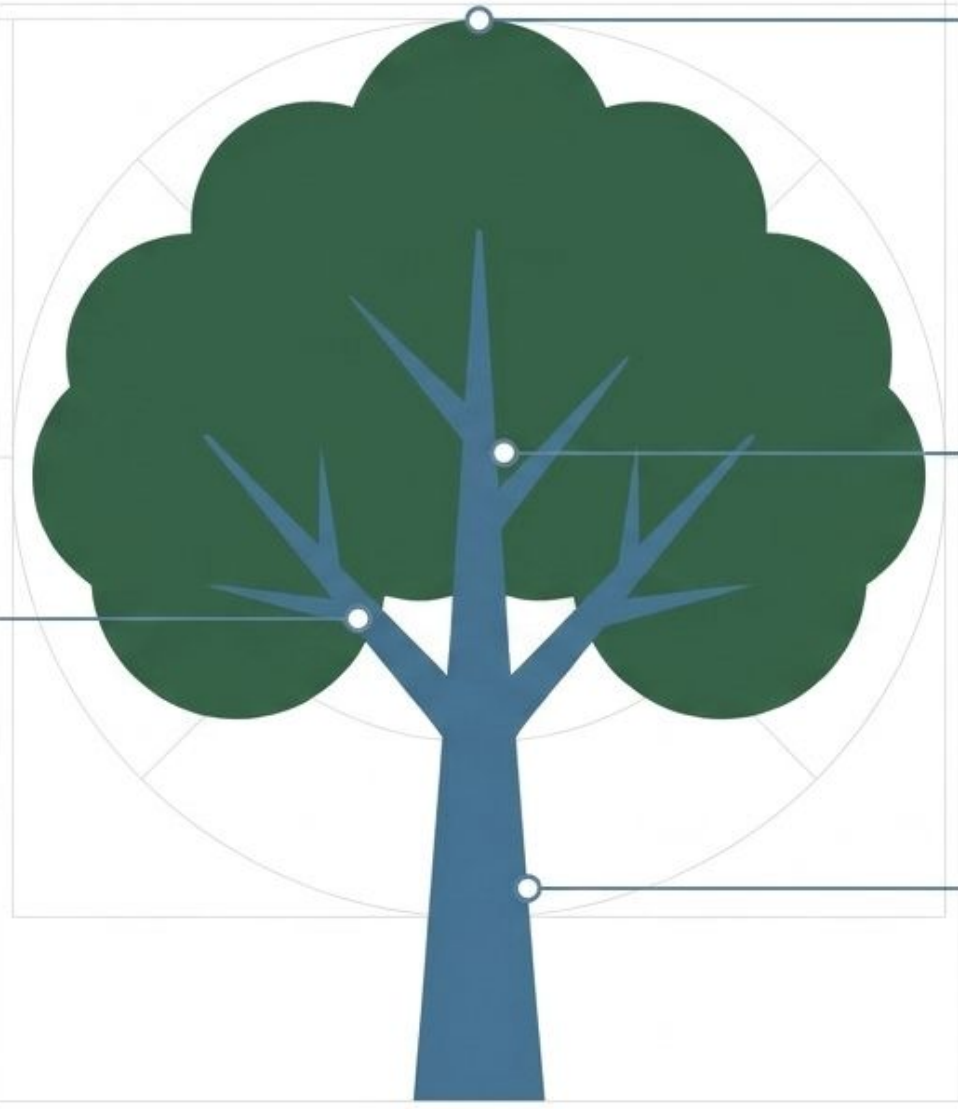
Capable of handling both classification (categorizing data) and regression (predicting numerical values).

### Traceability

Every prediction follows a distinct, easily audited path from root to final decision.

Gemini Notebook

# Structural Anatomy



## Root

The very top node, representing the entire dataset before the first split.

## Internal Node

Represents a specific test on an attribute (e.g., 'Is age > 30?').

## Branch

Represents the definitive outcome of the test, directing the path downward.

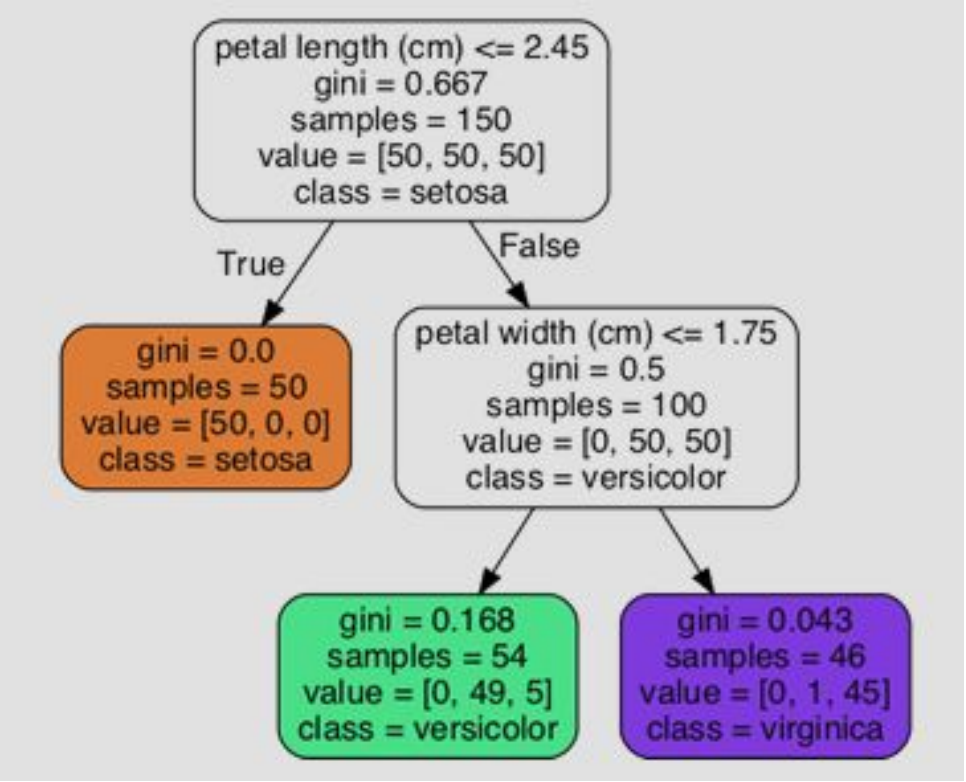
## Leaf Node

The terminal end of a branch, representing a final class label or numerical decision.

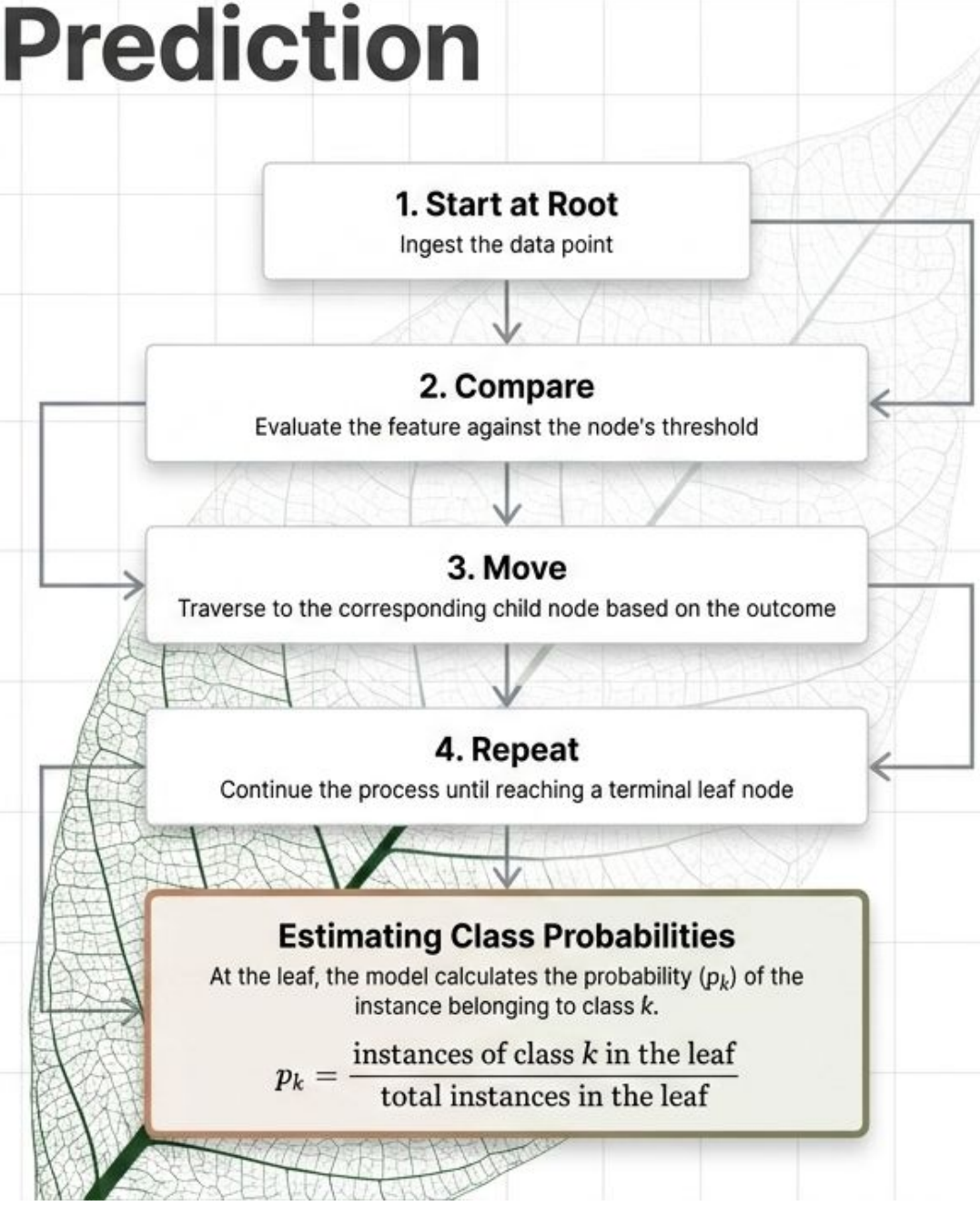
Gemini Notebook

# Training and Visualizing a Decision Tree

- https://drive.google.com/file/d/1IRHHx6kV-NpytBlxGwovEVLRVYRSPg5-/view?usp=sharing



# The Path to Prediction




Gemini Notebook

# Example: *AllElectronics* Customer Database

| RID | age | income | student | credit_rating | buys_computer |
|---|---|---|---|---|---|
| 1 | youth | high | no | fair | no |
| 2 | youth | high | no | excellent | no |
| 3 | middle_aged | high | no | fair | yes |
| 4 | senior | medium | no | fair | yes |
| 5 | senior | low | yes | fair | yes |
| 6 | senior | low | yes | excellent | no |
| 7 | middle_aged | low | yes | excellent | yes |
| 8 | youth | medium | no | fair | no |
| 9 | youth | low | yes | fair | yes |
| 10 | senior | medium | yes | fair | yes |
| 11 | youth | medium | yes | excellent | yes |
| 12 | middle_aged | medium | no | excellent | yes |
| 13 | middle_aged | high | yes | fair | yes |
| 14 | senior | medium | no | excellent | no |

# The Challenge of Construction

## AllElectronics Customer Database

| age | income | student | credit_rating | buys_computer |
|---|---|---|---|---|
| youth | high | no | fair | **no** |
| youth | high | no | excellent | **no** |
| middle_aged | high | no | fair | **yes** |
| senior | medium | no | fair | **yes** |
| senior | low | yes | fair | **yes** |
| senior | low | yes | excellent | **no** |

## Problem Statement

**The Objective:**
Construct the most efficient tree to predict if a customer buys a computer.

**The Core Question:**
Out of all available features (age, income, student, credit), **which one should we split on first to isolate the target classes** most effectively?

Gemini Notebook

# Splitting Criteria

# Information Gain

Based on Information theory, it measures the **impurity** of the dataset (D)

- **Entropy Formula**:

$$Entropy(D) = -\sum_{i=1}^{m} p_i \log_2 p_i$$

- **Entropy for Attribute:**

$$Entropy_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Entropy(D_j)$$

- **Information Gain**:

$$Gain(A) = Entropy(D) - Entropy_A(D)$$

# Example: Entropy Calculation

- 
- For the *AllElectronics* dataset:

$$Entropy(D) = -\left(\frac{9}{14}\right)\log^2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log^2\left(\frac{5}{14}\right) \approx 0.940$$

# Example: Information Gain for *age*

- **Values:** youth, middle_aged, senior

$$Gain(age) = Entropy(D) - \left[\left(\frac{5}{14}\right) \times \left(-\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}\right) + \left(\frac{4}{14}\right) \times \left(-\frac{4}{4}\log_2\frac{4}{4}\right) + \left(\frac{5}{14}\right) \times \left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}\right)\right] \approx 0.940 - 0.694 \approx 0.246$$

- Similarly,
  - $Gain(income) = 0.029$
  - $Gain(student) = 0.151$
  - $Gain(credit\ rating) = 0.048$
- Because ***age*** has the highest information gain among the attributes, it is selected as the splitting attribute.

# Gain Ratio

- **The Flaw:** Information Gain is biased towards multi-valued attributes (e.g., unique IDs).
- **The Fix:** Gain Ratio normalizes the score by dividing by Split Information.
- The attribute with the **maximum gain ratio** is selected as the **splitting attribute**.
- **Split Information:**

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

# Computation of Gain Ratio for the Attribute $Income$

- $Splitinfo_{income}(D)$

$$= -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right) = 1.557$$

- $Gain(income) = 0.029$

- $GainRatio(income) = \frac{0.029}{1.557} = 0.019$

# Gini Index (CART)

**The Alternative:** Measures impurity via probability of misclassification

- **Gini Impurity:**

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

$$\boldsymbol{Gini(Split) = \sum_{j=1}^{v} \frac{N_j}{N} Gini(Node_j)}$$

- $p_i$ is the probability that a tuple in $D$ belongs to class $C_i$ and is estimated by $\frac{|C_{i,D}|}{D}$.
- The sum is computed over $m$ classes.
- **Splitting Criterion**: Choose attribute with **smallest Gini index** after split.

# Induction of a Decision Tree Using The Gini Index

- Gini index to compute the impurity of $D$:

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2$$

- Split on ***age***
  - $youth$: $5\ (yes = 2, no = 3) \rightarrow\ G = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$
  - $middle - aged$: $4\ (yes = 4, no = 0) \rightarrow\ G = 0$
  - $senior$: $5\ (yes = 3, no = 2) \rightarrow\ G = 0.48$
- Weighted:
  - $\frac{5}{14} \cdot 0.48 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.48 = \frac{10}{14} \cdot 0.48 \approx \mathbf{0.3429}$

# Continue…

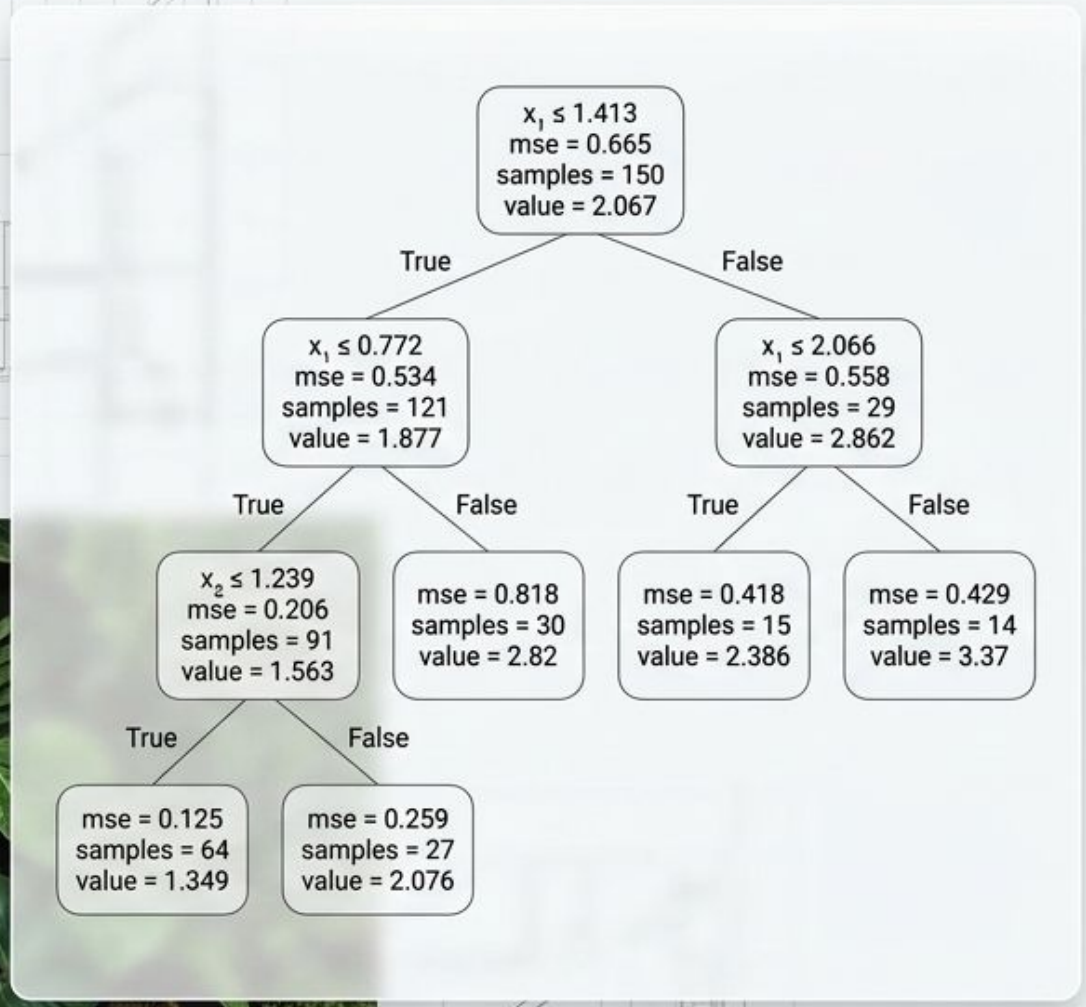
- Split on ***student***
  - $no$: $7\ (yes = 3, no = 4) \rightarrow\ G = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = \frac{24}{49} \approx 0.4898$
  - $yes$: $7\ (yes = 6, no = 1) \rightarrow\ G = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = \frac{12}{49} \approx 0.2449$
- Weighted:
  - $\frac{7}{14} \cdot 0.4898 + \frac{7}{14} \cdot 0.2449 \approx \mathbf{0.3673}$
- Similarly,
  - Split on ***income*** **= 0.4405**
  - Split on ***credit_rating*** **= 0.4286**
- Best feature to split node is ***age***.

# Synthesis: Splitting Criteria Matrix

| CRITERION | CORE FORMULA | SELECTION LOGIC | KEY DRAWBACK/BIAS |
|---|---|---|---|
| Information Gain | $Entropy(D) - Entropy_A(D)$ | Maximize Gain | Biased toward attributes with many levels. |
| Gain Ratio | $\frac{Gain(A)}{SplitInfo_A(D)}$ | Maximize Ratio | Can overcompensate, ignoring otherwise good splits. |
| Gini Index (CART) | $1 - \sum p_i^2$ | Minimize Impurity Index | Favors larger partitions; standard in modern implementations. |

Gemini Notebook

# Regression Trees (CART Architecture)




**CART** (Classification and Regression Trees) splits training sets into subsets using a feature *k* and threshold $t_k$ to minimize a specific cost function.

## Regression Cost Function

The model searches for a split that minimizes the Mean Squared Error (MSE) of the resulting child nodes, weighted by the number of instances (m).

$$J(k, t_k) = \frac{m_{\text{left}}}{m} MSE_{\text{left}} + \frac{m_{\text{right}}}{m} MSE_{\text{right}}$$

Where:

$$MSE_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i=1}^{m_{\text{node}}} (y_i - \bar{y}_{\text{node}})^2,$$

Gemini Notebook

# Example: Predicting House Prices Based on Square Footage

| Square Feet (x) | Price (y) in $1000s |
|---|---|
| 1000 | 200 |
| 1500 | 250 |
| 2000 | 300 |
| 2500 | 350 |
| 3000 | 400 |
| 3500 | 450 |

# Step 1: Calculate Possible Splits

- First, we need to find all possible split points between our data points:
- Possible splits: 1250, 1750, 2250, 2750, 3250

# Step 2: Calculate MSE for Each Split

**Split at 1250:**

- Left node (≤1250): [1000]
  - Mean = 200
  - MSE = $(200-200)^2 = 0$
- Right node (>1250): [1500,2000,2500,3000,3500]
  - Mean = (250+300+350+400+450)/5 = 350
  - MSE = $(250-350)^2 + (300-350)^2 + (350-350)^2 + (400-350)^2 + (450-350)^2$
  - MSE = 10000 + 2500 + 0 + 2500 + 10000 = 25000
- Total MSE = 0 + 25000 = 25000

**Split at 1750:**

- Left node (≤1750): [1000,1500]
  - Mean = (200+250)/2 = 225
  - MSE = $(200-225)^2 + (250-225)^2 = 625 + 625 = 1250$
- Right node (>1750): [2000,2500,3000,3500]
  - Mean = (300+350+400+450)/4 = 375
  - MSE = $(300-375)^2 + (350-375)^2 + (400-375)^2 + (450-375)^2$
  - MSE = 5625 + 625 + 625 + 5625 = 12500
- Total MSE = 1250 + 12500 = 13750

# Continue…

**Split at 2250:**

- Left node (≤2250): [1000,1500,2000]
  - Mean = (200+250+300)/3 = 250
  - MSE = (200-250)² + (250-250)² + (300-250)² = 2500 + 0 + 2500 = 5000
- Right node (>2250): [2500,3000,3500]
  - Mean = (350+400+450)/3 = 400
  - MSE = (350-400)² + (400-400)² + (450-400)² = 2500 + 0 + 2500 = 5000
- Total MSE = 5000 + 5000 = 10000

**Split at 2750:**

- Left node (≤2750): [1000,1500,2000,2500]
  - Mean = (200+250+300+350)/4 = 275
  - MSE = (200-275)² + (250-275)² + (300-275)² + (350-275)²
  - MSE = 5625 + 625 + 625 + 5625 = 12500
- Right node (>2750): [3000,3500]
  - Mean = (400+450)/2 = 425
  - MSE = (400-425)² + (450-425)² = 625 + 625 = 1250
- Total MSE = 12500 + 1250 = 13750

# Continue…

**Split at 3250:**

- Left node (≤3250): [1000,1500,2000,2500,3000]
  - Mean = (200+250+300+350+400)/5 = 300
  - MSE = $(200-300)^2 + (250-300)^2 + (300-300)^2 + (350-300)^2 + (400-300)^2$
  - MSE = 10000 + 2500 + 0 + 2500 + 10000 = 25000
- Right node (>3250): [3500]
  - Mean = 450
  - MSE = $(450-450)^2 = 0$
- Total MSE = 25000 + 0 = 25000

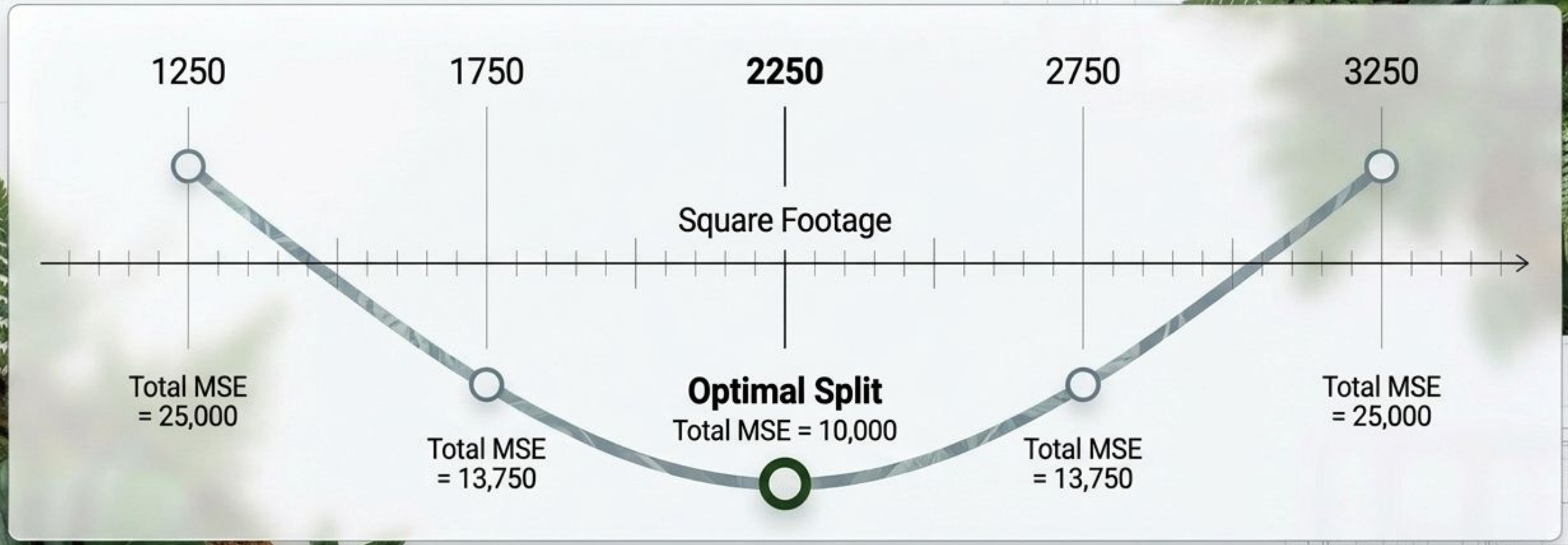
# Step 3: Choose the Best Split

- **Best split is at 2250 with MSE = 10000**

| Split Point | Total MSE |
|---|---|
| 1250 | 25000 |
| 1750 | 13750 |
| **2250** | **10000** |
| 2750 | 13750 |
| 3250 | 25000 |

# Visualizing the Split (MSE Minimization)

Predicting housing prices based on square footage.




**Takeaway:** The algorithm literally maps every potential division between data points and selects 2250 sq ft because it yields the lowest possible combined prediction error.

Gemini Notebook

# Computational Complexity

## Prediction (Inference)

**Fast and efficient.** Traversing the tree from root to leaf is logarithmically fast, making it excellent for real-time predictions.

$$O(\log_2 m)$$

Latency: Low

Time: 10ms

Leafs: 16

## Training (Construction)

Latency: High

$$O(n \times m \log_2 m)$$

Iterations: 1,000,000+

Features: 500

Progress: 45%

⚠ High Computational Cost

The **Bottleneck**. Searching every feature and every threshold for the optimal split becomes **computationally expensive**. Without tuning, decision trees are not suitable for very large datasets.

Gemini Notebook

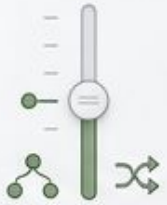
# Regularization: The Control Panel

Left unchecked, a decision tree will organically grow until every leaf contains a single data point—massive overfitting. We use hyperparameters to constrain this growth.

## Control Board

### max_depth

Limits the maximum vertical levels of the tree.



### min_samples_split

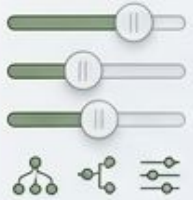
The minimum number of samples a node must possess to allow another split.

### min_samples_leaf

The minimum number of samples required to form a terminal leaf.

### max_leaf_nodes

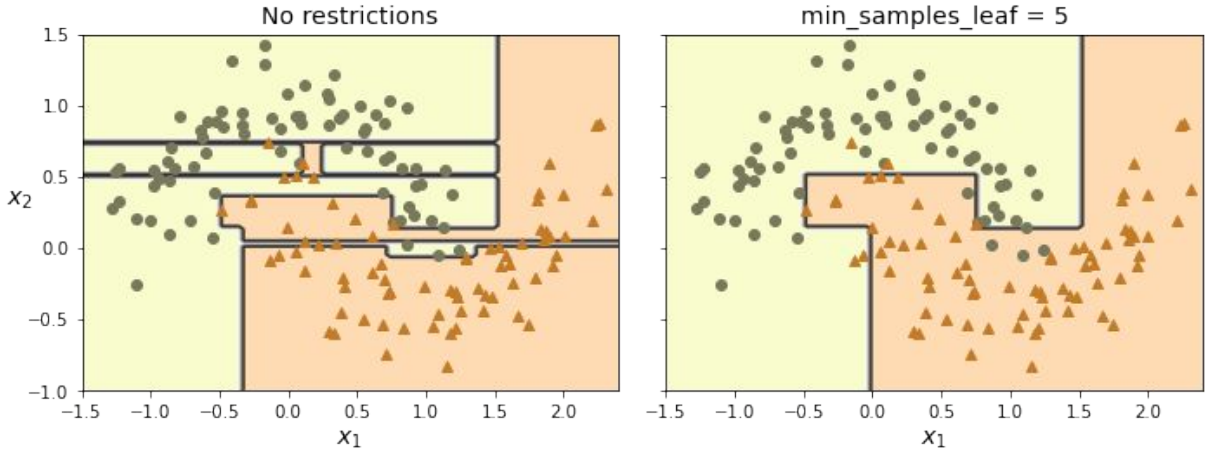
A hard cap on the total number of terminal leaves allowed.

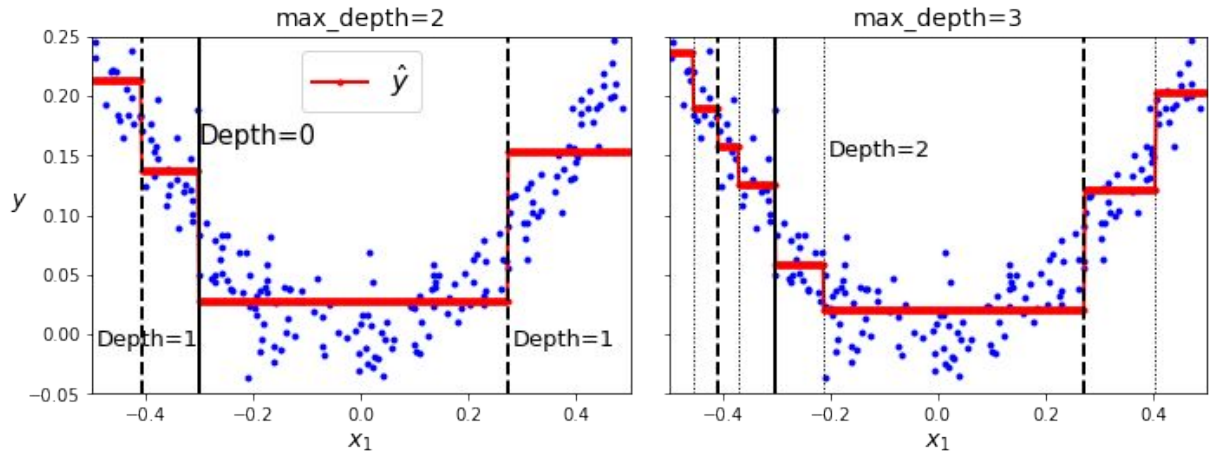


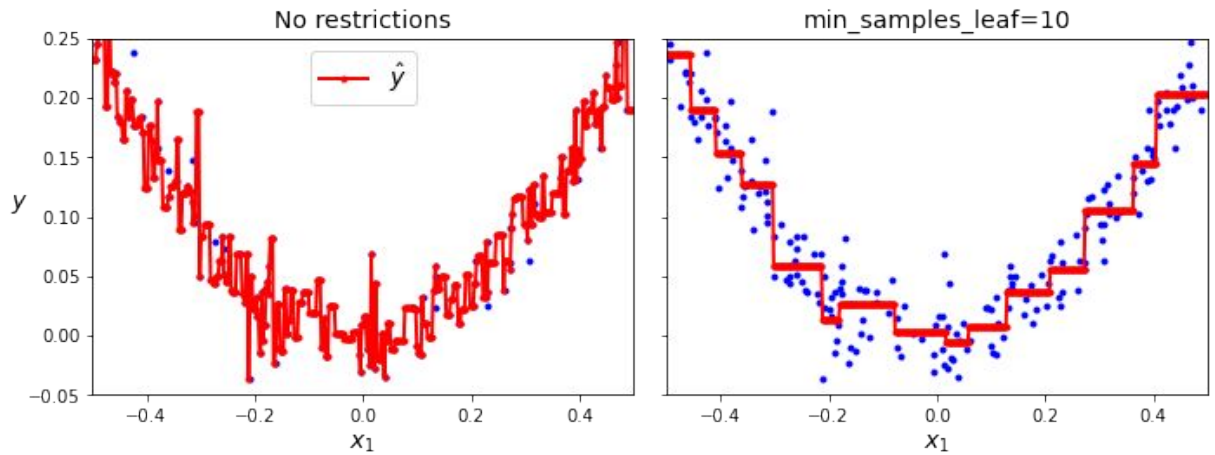
### max_features

The maximum number of features evaluated for splitting at any single node.

Gemini Notebook


No restrictions
min_samples_leaf = 5
$x_1$
$x_2$


max_depth=2
max_depth=3
$\hat{y}$
Depth=0
Depth=1
Depth=1
Depth=2
$x_1$
$y$


No restrictions
min_samples_leaf=10
$\hat{y}$
$x_1$
$y$

# Maintenance: Pruning Strategies

**Core Concept:** Removing unnecessary branches to improve model generalization. A node whose children are all leaf nodes is removed if the purity improvement it provides is not statistically significant.

Overgrown Tree

Branch Removal

Complexity Reduction

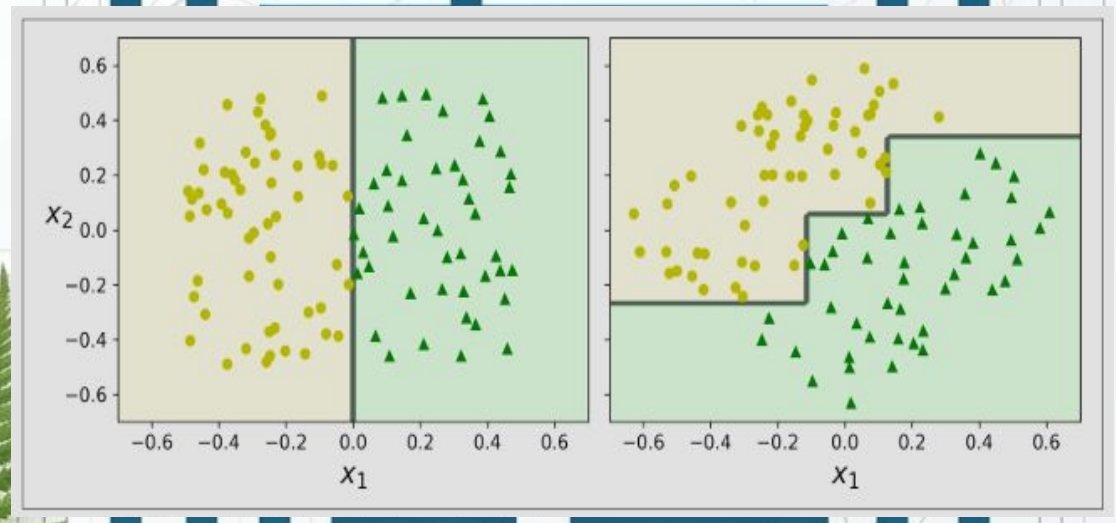
Pruned Tree

**1. Pre-pruning**: Halt the tree's construction early, stopping growth before it reaches perfect purity on the training set.

**2. Post-pruning:** Allow the tree to grow entirely, then aggressively evaluate and remove statistically insignificant branches retroactively.

Gemini Notebook

# Instability & The Maze




## The Vulnerability

Decision trees map orthogonal boundaries (straight lines). Because of this rigid structure, they are highly sensitive to even minor variations in training data.

## The Rotation Problem

Simply rotating the dataset mathematically can completely ruin the model's performance, turning a simple split into a complex maze of stair-step boundaries.

## The Escape Route

To overcome this inherent instability, practitioners rarely use single trees in isolation. They rely on PCA (Principal Component Analysis) to orient data or deploy ensemble methods like Random Forests to average out the variance.

Gemini Notebook

# The Final Verdict

## The Strengths (Why we use them)

- **Interpretability:** True "white box" models that mimic human decision-making.
- **Flexibility:** Natively handles both classification and regression tasks.
- **Simplicity:** Requires minimal data preparation (though computationally heavy to train on massive datasets).

## The Flaws (What to watch out for)

- **Overfitting:** Inherently prone to mapping noise without strict regularization hyperparameters.
- **Instability:** Extremely sensitive to small data variations and dataset rotations.
- **Splitting Biases:** Information gain criteria inherently favor attributes with more levels, requiring metrics like the Gini Index or Gain Ratio to correct.

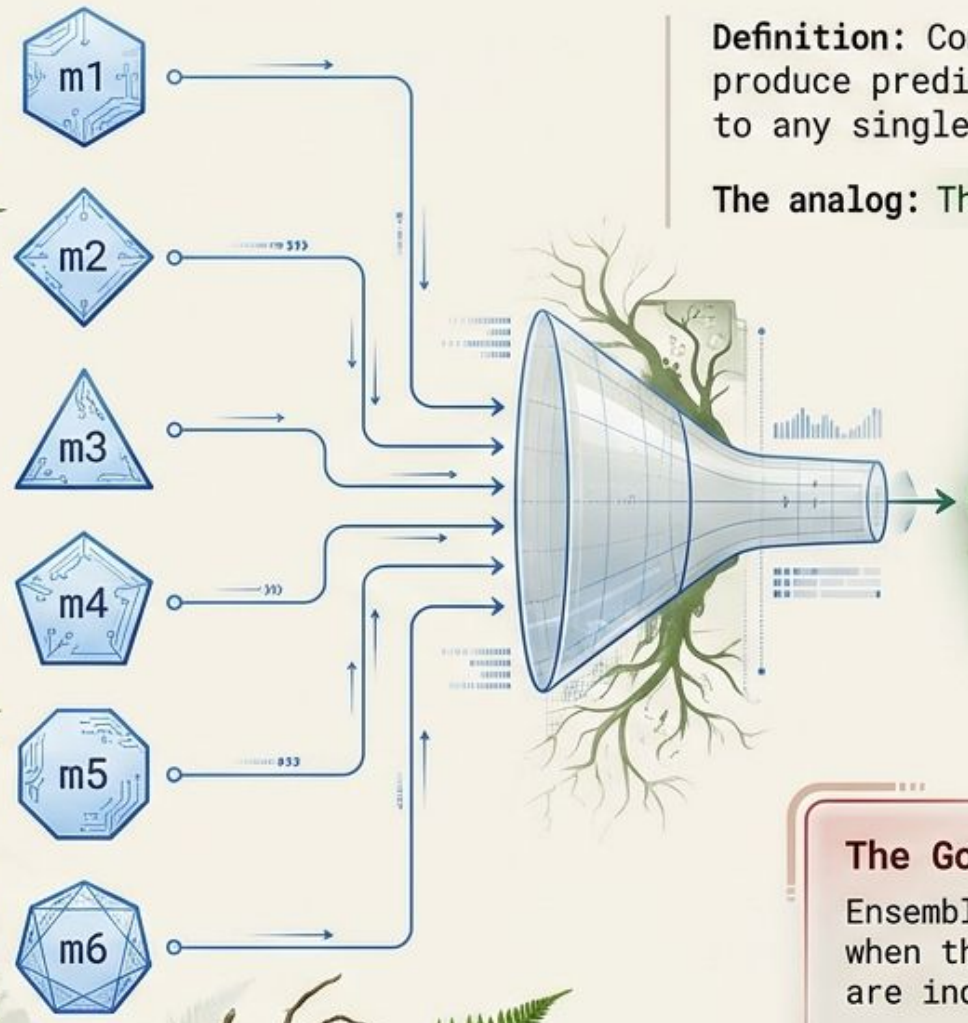
Gemini Notebook

# Ensemble Learning and Random Forests

# The Predictive Power of the Wisdom of the Crowd

- **The flaw of the individual:**
  - A single decision tree is highly prone to overfitting. It memorizes the training data, capturing the underlying noise alongside the true signal, resulting in high variance.
- **The ensemble solution:**
  - By aggregating the output of multiple diverse models, we dramatically reduce variance. The ensemble thrives on independent, uncorrelated errors cancelling each other out.

# The Core Philosophy




**Definition:** Combine multiple models to produce predictive performance superior to any single model.

**The analog:** The Wisdom of the Crowd.

**The Golden Rule:**

Ensemble methods work best when the underlying predictors are independent.

# The 4 Pillars

1. Voting / Averaging
2. Bagging & Pasting
3. Boosting
4. Stacking

Gemini Notebook

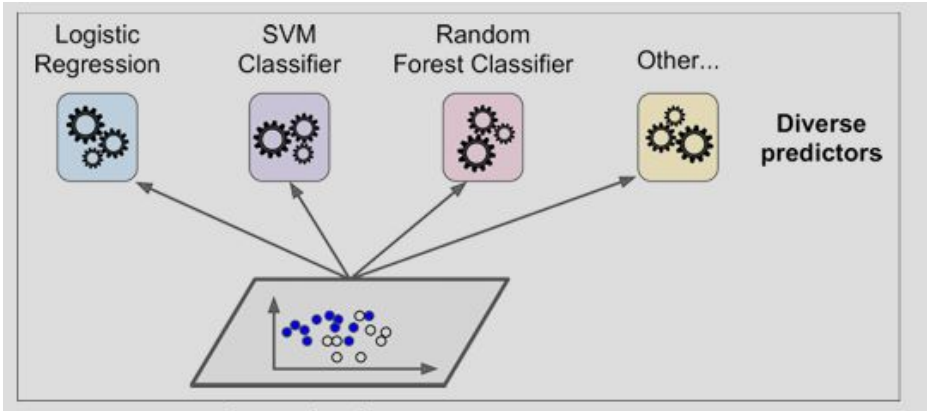
# Voting Classifiers

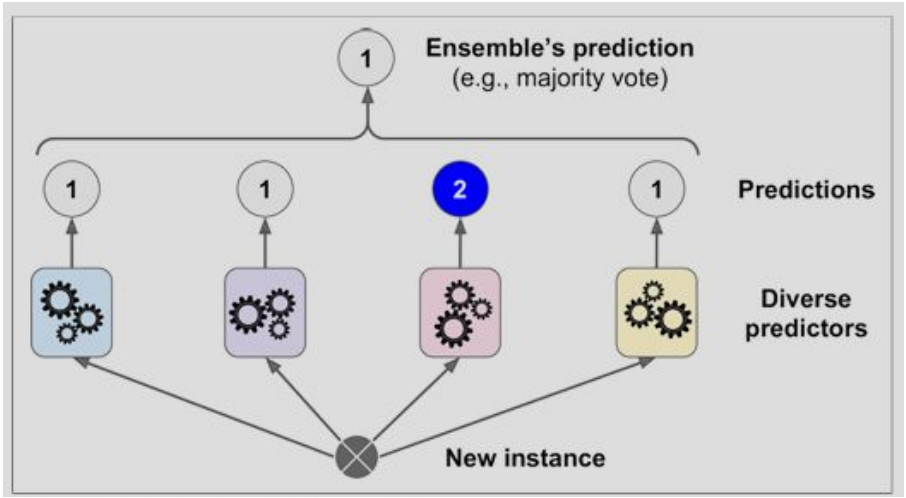
- **Idea:** Aggregate predictions from multiple classifiers and predict the class that gets the most votes.
- **Hard Voting (Majority Rules):** Each classifier casts a discrete class prediction, and the ensemble outputs the mode of these predictions.

$$\hat{y} = mode\{C_1(x), C_2(x), \dots, C_n(x)\}$$

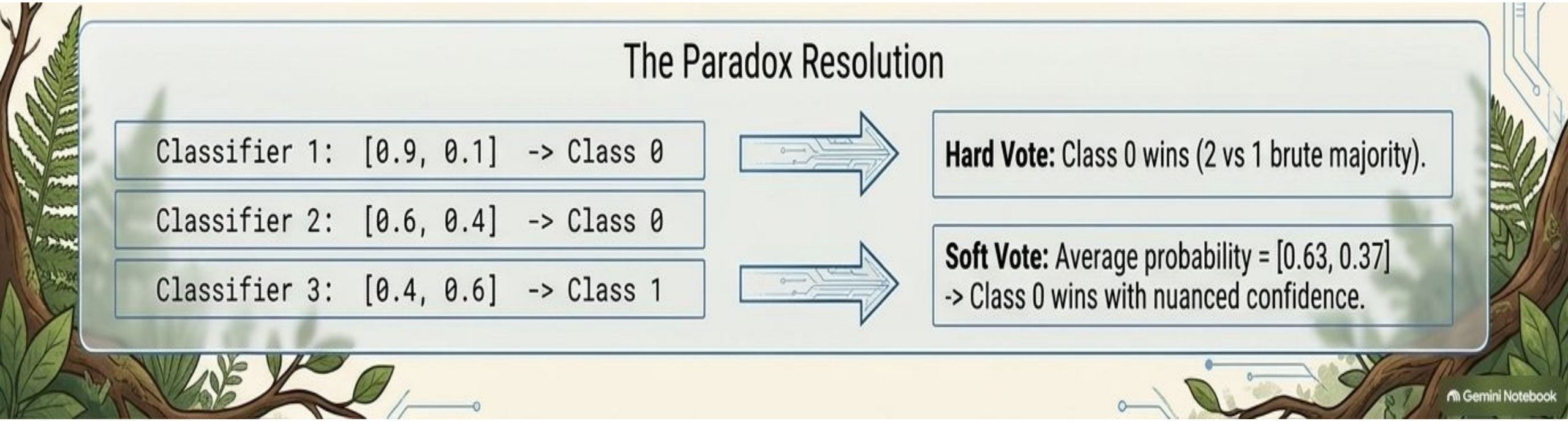
- **Soft Voting (Confidence-Weighted Averaging):** If all classifiers can predict class probabilities, the ensemble averages the individual probabilities and predicts the class with the highest average probability.

$$\hat{y} = \underset{k}{argmax} \frac{1}{n} \sum_{i=1}^{n} P(y = k | C_i, k)$$



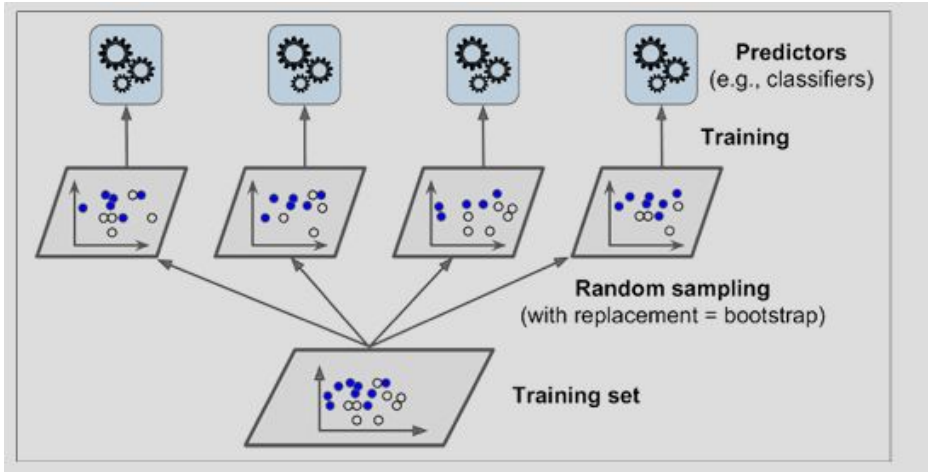



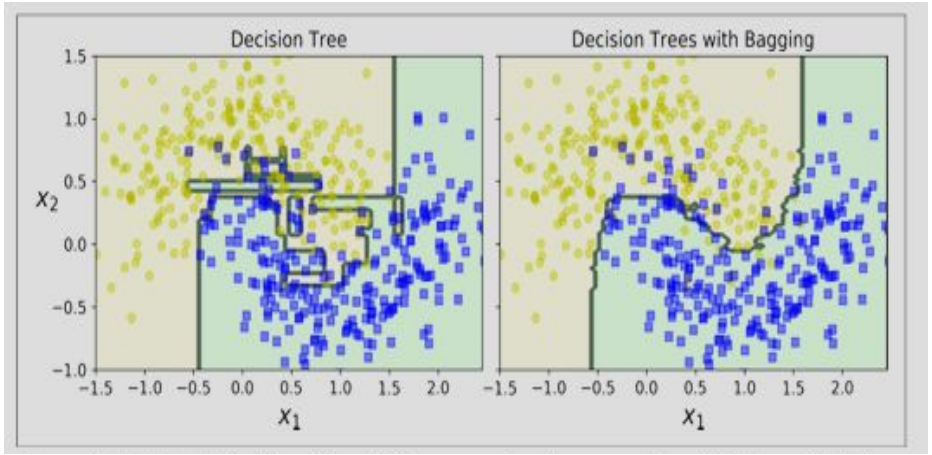
# Example: Hard Vs Soft Voting



# Bagging and Pasting
## Tackling Variance

- **Bagging (Bootstrap Aggregating)**
  - **Sampling with replacement**
  - Creates highly diverse training subsets, dramatically reduces variance while maintaining strong generalization
- **Pasting**
  - **Sampling without replacement**
  - Data points are partitioned into distinct, non-overlapping subsets. Often used for massive datasets.



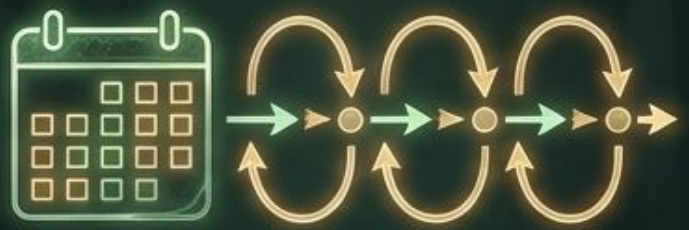
# The Out-of-Bag (OOB) Guarantee: A Built-In Validation Set

## Step 1: Single Draw Miss

For a dataset of size $m$, the probability of not selecting a given instance in a single draw (with replacement) is:

$$1 - 1/m$$

## Step 2: Across $m$ Draws



The probability of an instance never being chosen across an entire bootstrap sample is:

$$(1 - 1/m)^m$$

## Step 3: The Infinite Limit

As the dataset grows large ($m \to \infty$), this converges to a mathematical constant:

$$\lim_{m \to \infty} \left(1 - \frac{1}{m}\right)^m = e^{-1} \approx 0.368$$

Insight: ~36.8% of instances are systematically left out. Because the model never sees them during training, they serve as an automatic, mathematically guaranteed validation set.

Gemini Notebook

# Random Patches and Random Subspaces

**Random Patches:** Sample both instances and features

**Random Subspaces:** Sample only features (keep all instances)

Useful for high-dimensional data

# Random Forests: The Ultimate Bagging Ensemble

## The Architecture

An ensemble of **Decision Trees** trained via **Bagging**.

## The Secret Sauce

**Extra Randomness**.

Instead of searching all features for the best split, the algorithm searches for the best feature among a **random subset** of features.

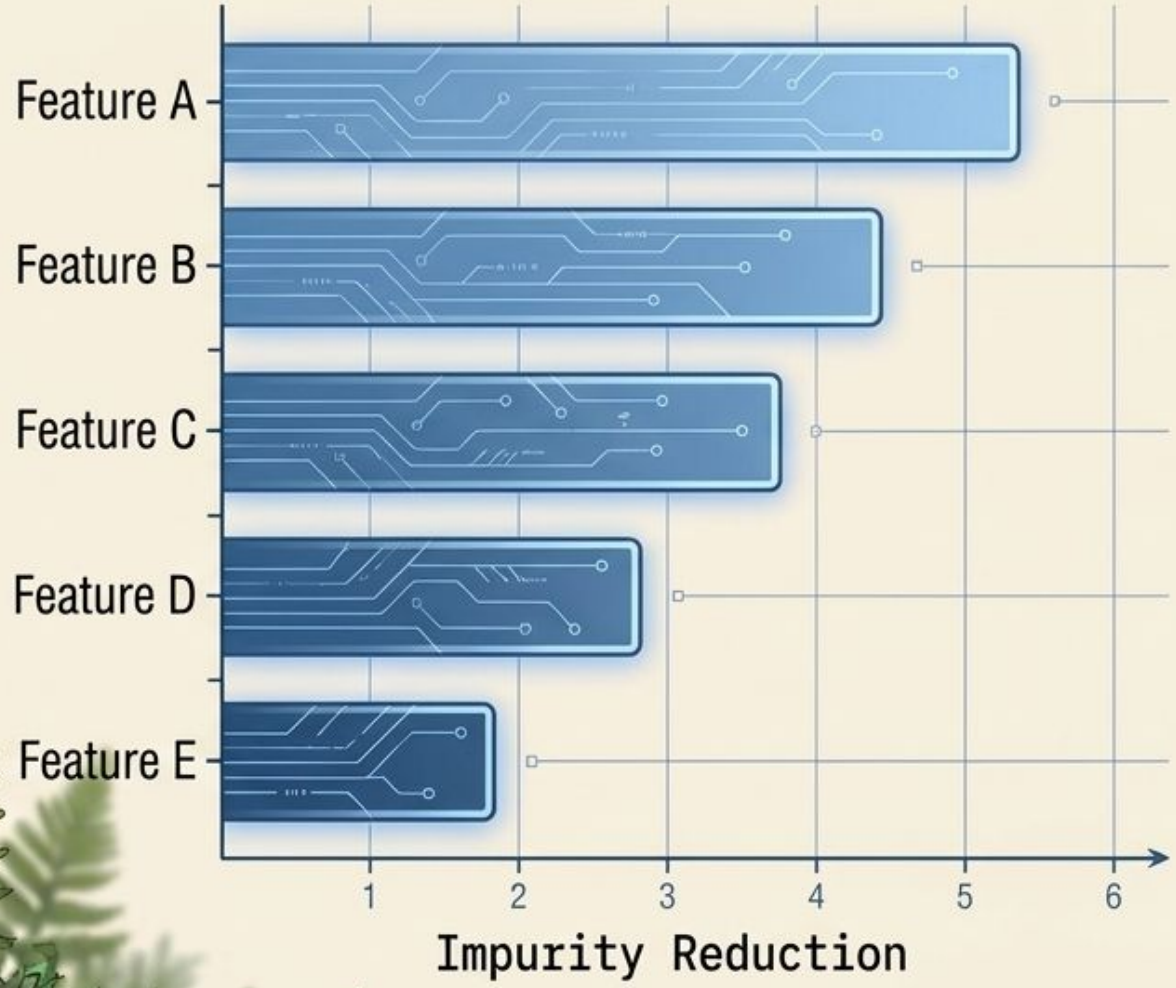
## The Result

More diverse trees lead to superior generalization.

Variant: **Extra-Trees (Extremely Randomized Trees)** introduce even more randomness by using random thresholds for feature splits.

Gemini Notebook

# Decoding the Forest: Feature Importance




## Measuring What Matters.

### The Mechanism

Random Forests measure feature importance by calculating exactly how much a specific feature **reduces impurity** across all trees in the entire forest.

### The Value

Transforms an opaque, complex ensemble into an **interpretable tool** for **feature selection** and deep data understanding.

Gemini Notebook

# Quantifying Influence: Mean Decrease in Impurity (MDI)

## Single Tree Feature Importance ($TFI_j$)

$$TFI_j = \frac{\sum_{t \in T:\ \text{split on } j} N_t \left( I_t - \frac{N_{t\text{left}}}{N_t} I_{t\text{left}} - \frac{N_{t\text{right}}}{N_t} I_{t\text{right}} \right)}{\sum_{t \in T\ (\text{all interior nodes})} N_t \left( I_t - \frac{N_{t\text{left}}}{N_t} I_{t\text{left}} - \frac{N_{t\text{right}}}{N_t} I_{t\text{right}} \right)}$$

Measures how much nodes splitting on feature $j$ reduce the weighted impurity ($I$) relative to all nodes in the specific tree.

## Forest-Wide Feature Importance ($MDI_j$)

$$MDI_j = \frac{1}{N_T} \sum_{k=1}^{N_T} TFI_{j,k}$$

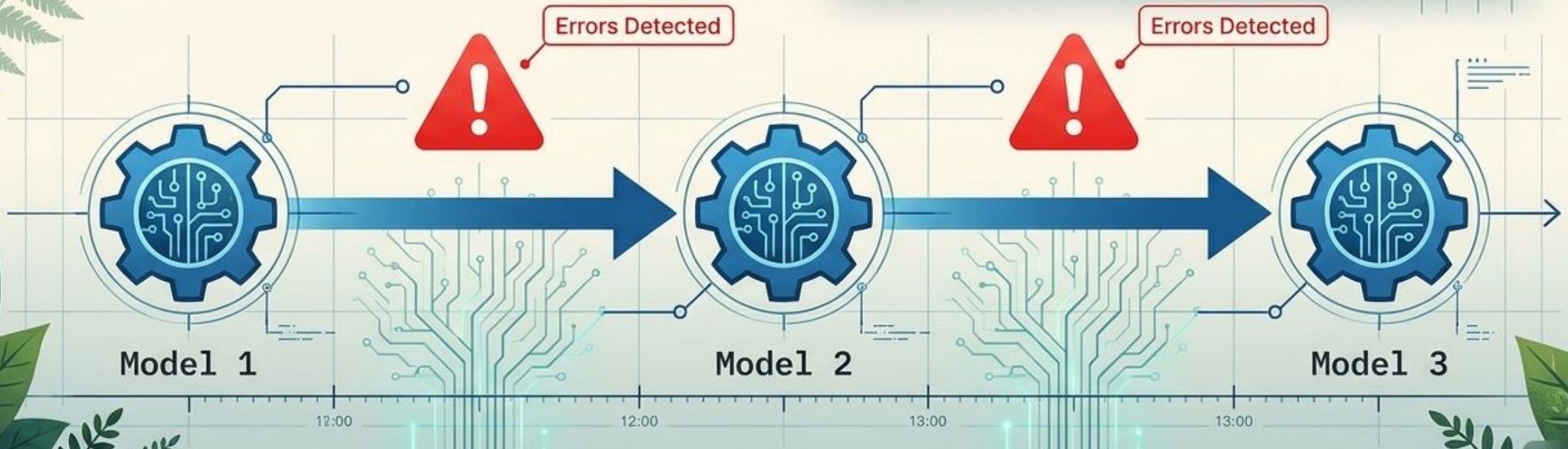
Averages the $TFI$ across the entire forest of $N_T$ trees, yielding a highly stable, generalised metric of predictive influence.

Gemini Notebook

# Tackling Bias:
# The Paradigm Shift to Boosting

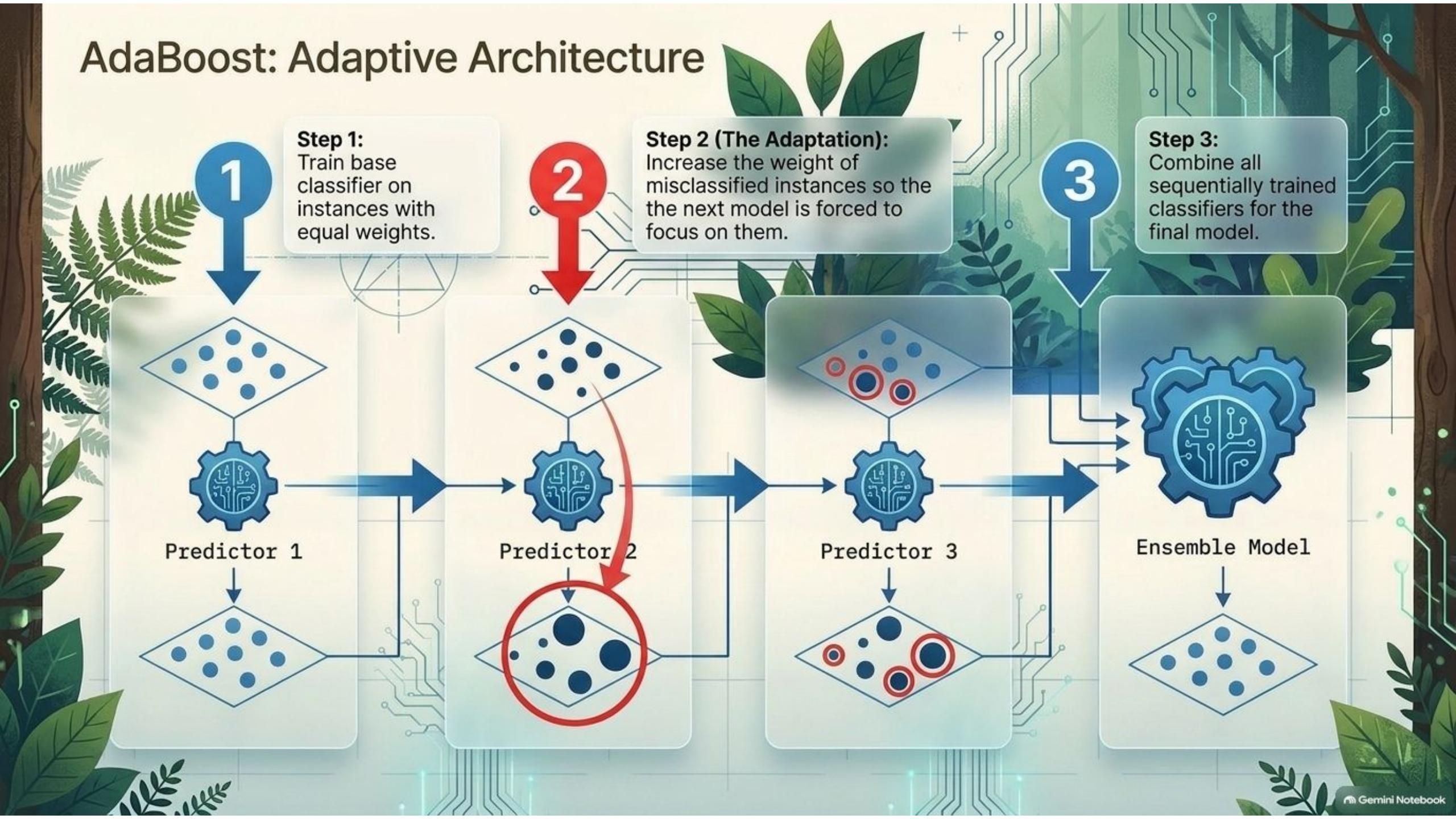
**The Shift:**
Unlike Bagging (parallel independence), Boosting is strictly sequential.

**The Logic:**
Each new predictor is added explicitly to correct the specific errors and blind spots of the previous predictor.




Gemini Notebook

# AdaBoost: Adaptive Architecture




Gemini Notebook

# AdaBoost: The Mathematics of Weighting

1. **Initial weight $w^i$** of each instance:

$$w^i = \frac{1}{m}$$

2. **Weighted error rate of the $j$-th predictor**

$$r_j = \frac{\left(\sum_{\substack{i=1 \\ \hat{y}_j^i \neq y^i}}^{m} w^i\right)}{\left(\sum_{i=1}^{m} w^i\right)}$$

Where $\hat{y}_j^i$ is the $j$-th predictor's prediction for the $i$-th instance

3. **Predictor weight ($\alpha_j$)**

$$\alpha_j = \eta \times \log\left(\frac{1 - r_j}{r_j}\right)$$

4. **Weight update rule** boosts the weights of the misclassified instances.

$$for\ i = 1,2, \dots, m$$

$$w^i = \begin{cases} w^i, & if\ \hat{y}_j^i = y^i \\ w^i \exp(\alpha_j), & if\ \hat{y}_j^i \neq y^i \end{cases}$$

Weights are normalized by dividing each $w^i$ by $\sum_{k=1}^{m} w^k$

5. **Ensemble prediction**

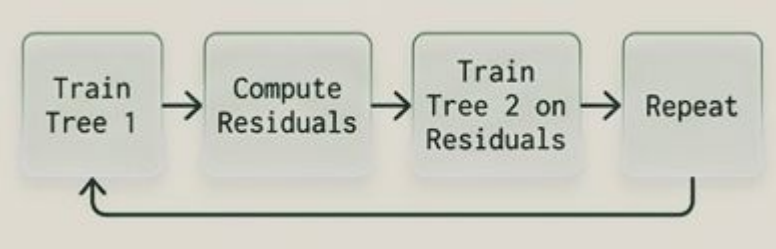
$$\hat{y}_x = \underset{k}{argmax} \sum_{\substack{j=1 \\ \hat{y}_j(x)=k}}^{N} \alpha_j$$

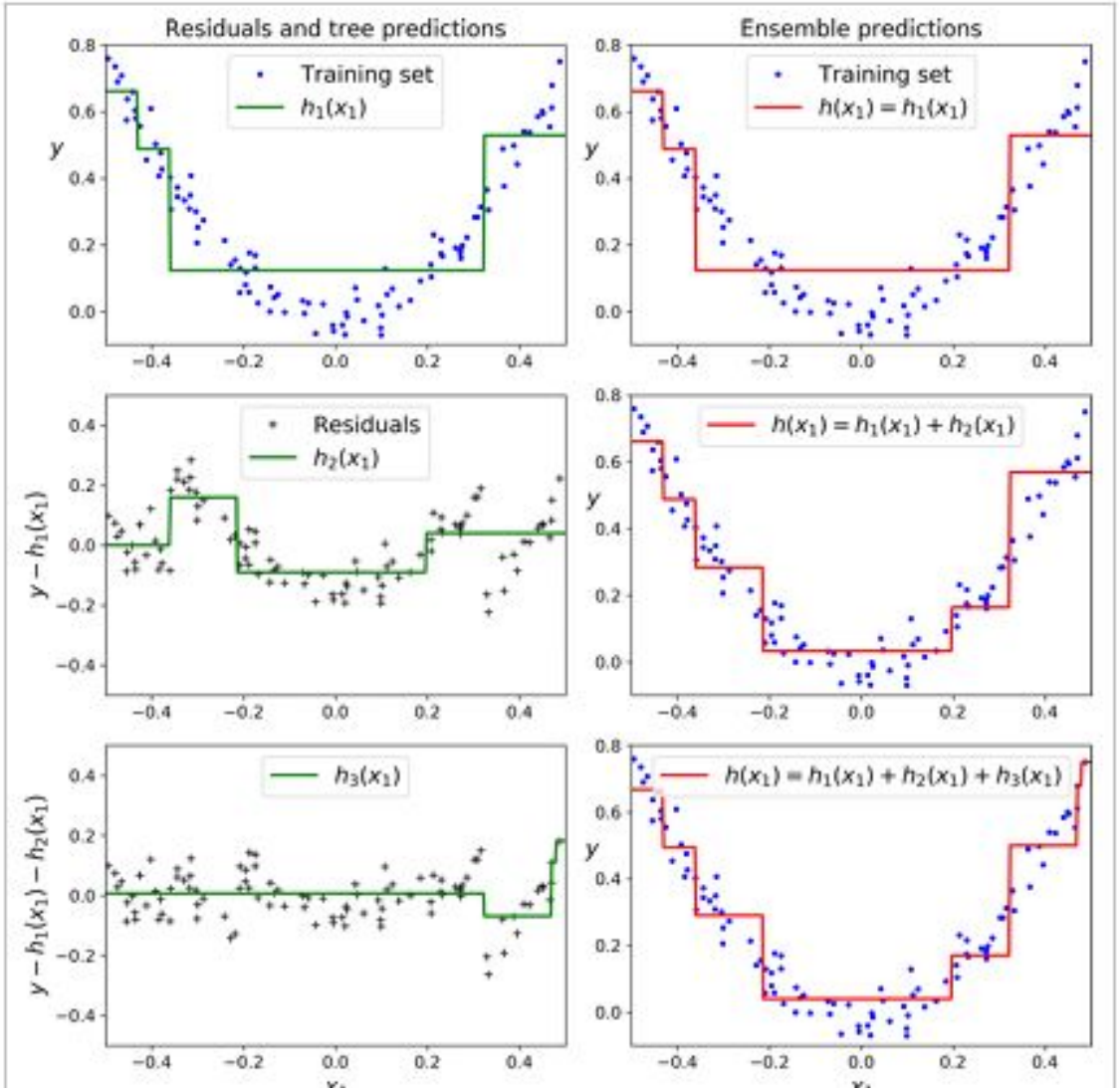
The final output is determined by a weighted majority vote of all $N$ predictors

# Gradient Boosting

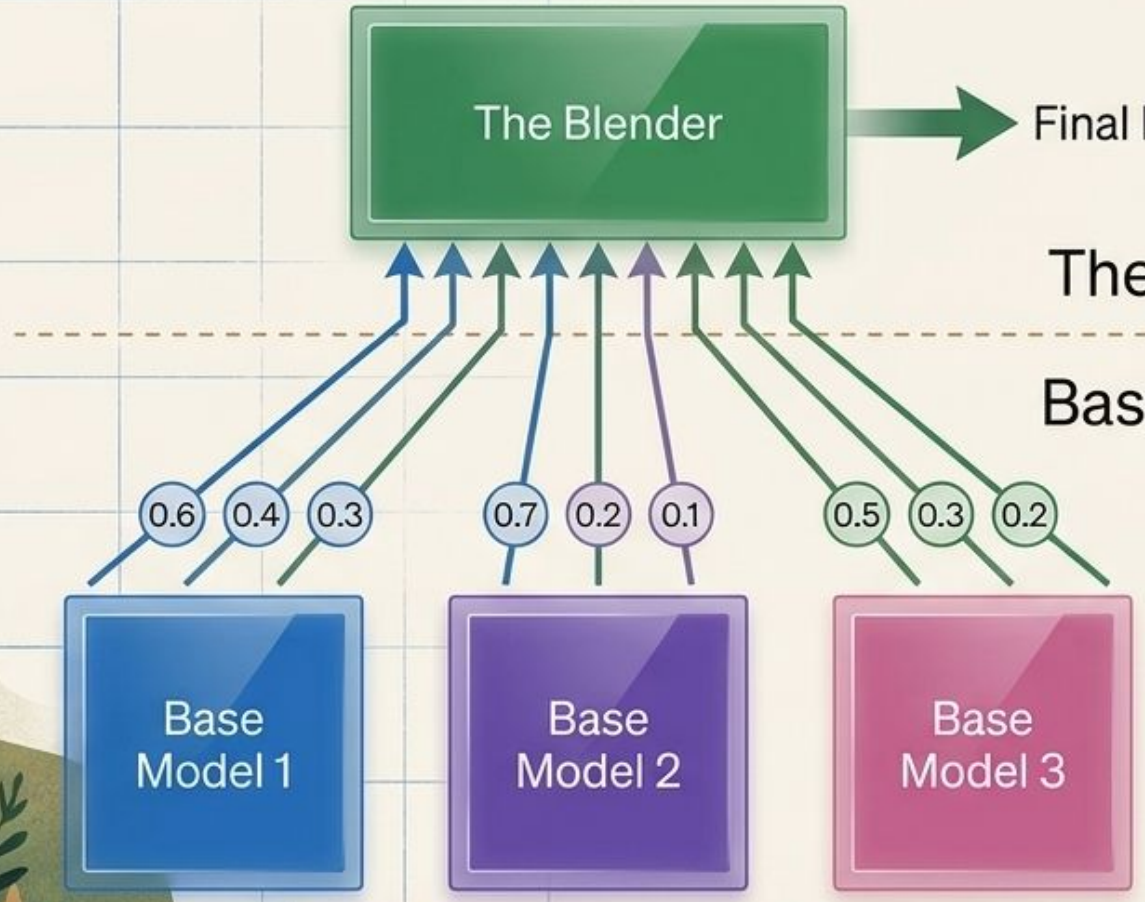
## Learning from Residuals

- Instead of tweaking instance weights (like AdaBoost), it fits the new model directly to the residual errors of its predecessor.
- **Example:** Gradient Tree Boosting, or Gradient Boosted Regression Trees (GBRT).



# Stacking (Stacked Generalization): Meta-Learning Architectures




## Why just vote when you can learn how to combine?

Train a meta-model to intelligently weigh and combine the predictions of several base models.

**The Blender Definition:**

The final, overarching meta-learner that takes the predictions of Layer 1 models as its input features.

Gemini Notebook

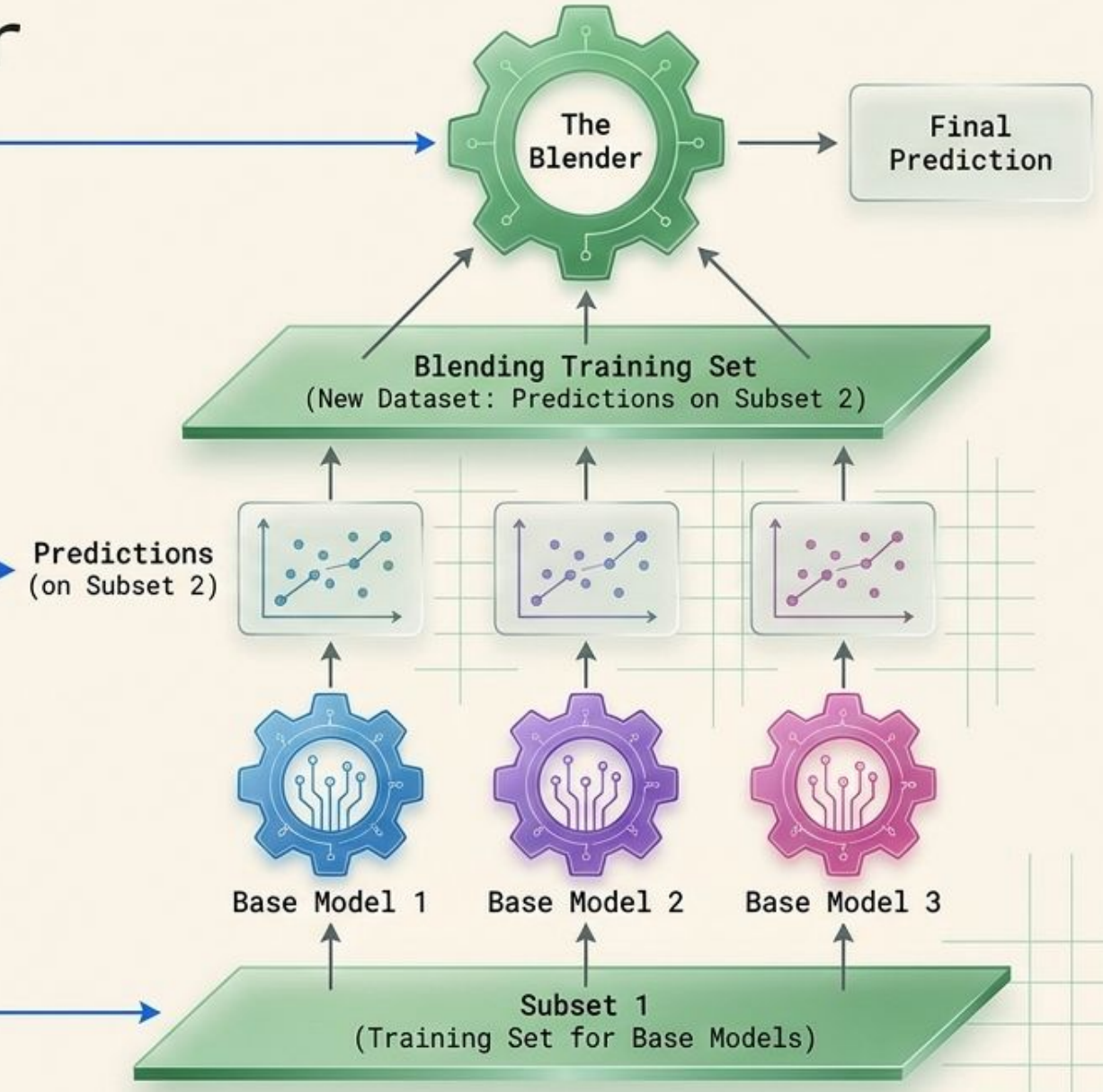
# Training the Blender

**The Challenge:** How do you train the Blender without leaking data?

**Step 3:** Train the Blender on this new dataset to map the predictions to the final target value.

**Step 2:** Use those trained base models to make predictions on Subset 2 (creating a clean Blending training set).

**Step 1:** Train base models strictly on Subset 1.




Gemini Notebook

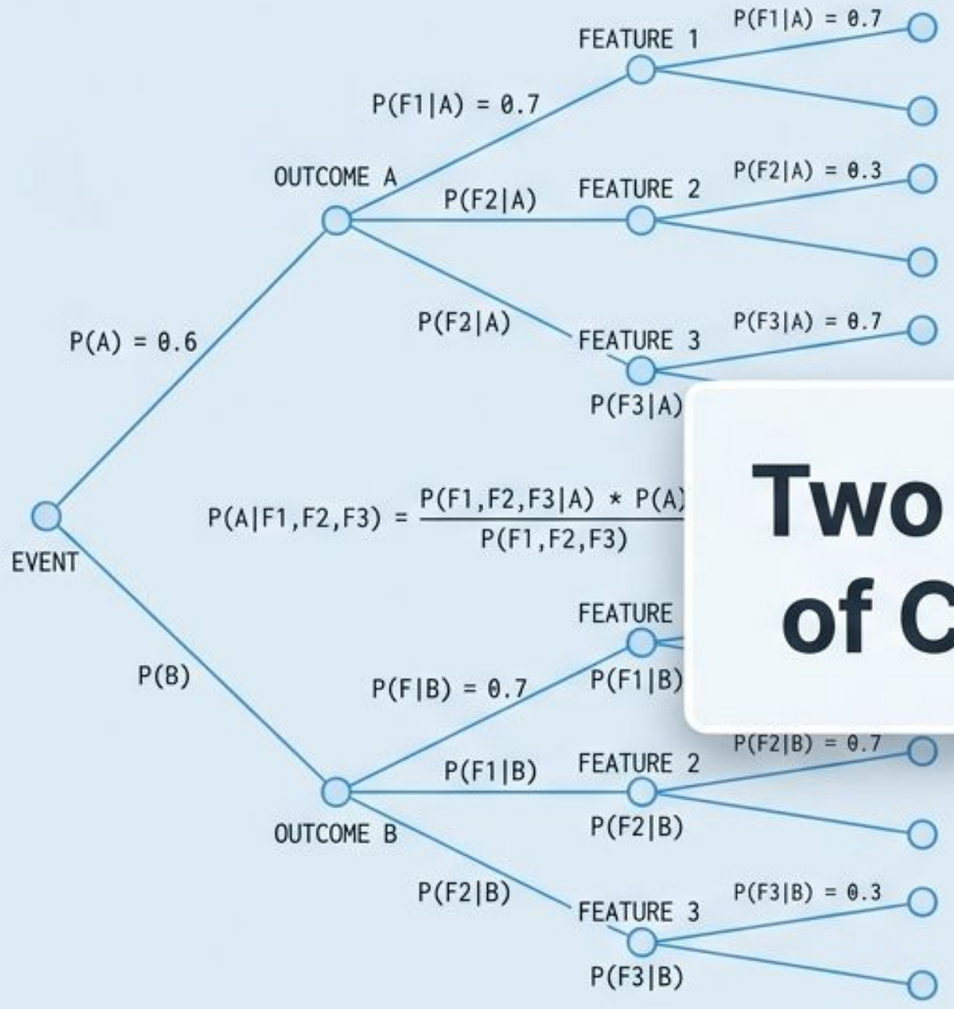
# Synthesis: The Ensemble Architecture Matrix

| | Bagging & Random Forests | Boosting (AdaBoost/GBRT) | Stacking |
|---|---|---|---|
| **Primary Goal** | Reduce Variance & Overfitting | Reduce Bias & Underfitting | Meta-Optimization & Consensus |
| **Execution Style** | Parallel / Independent | Sequential / Dependent | Hierarchical / Layered |
| **Core Mechanism** | Data sampling & Feature subsets | Error correction & Residual fitting | Meta-learner predicting on predictions |

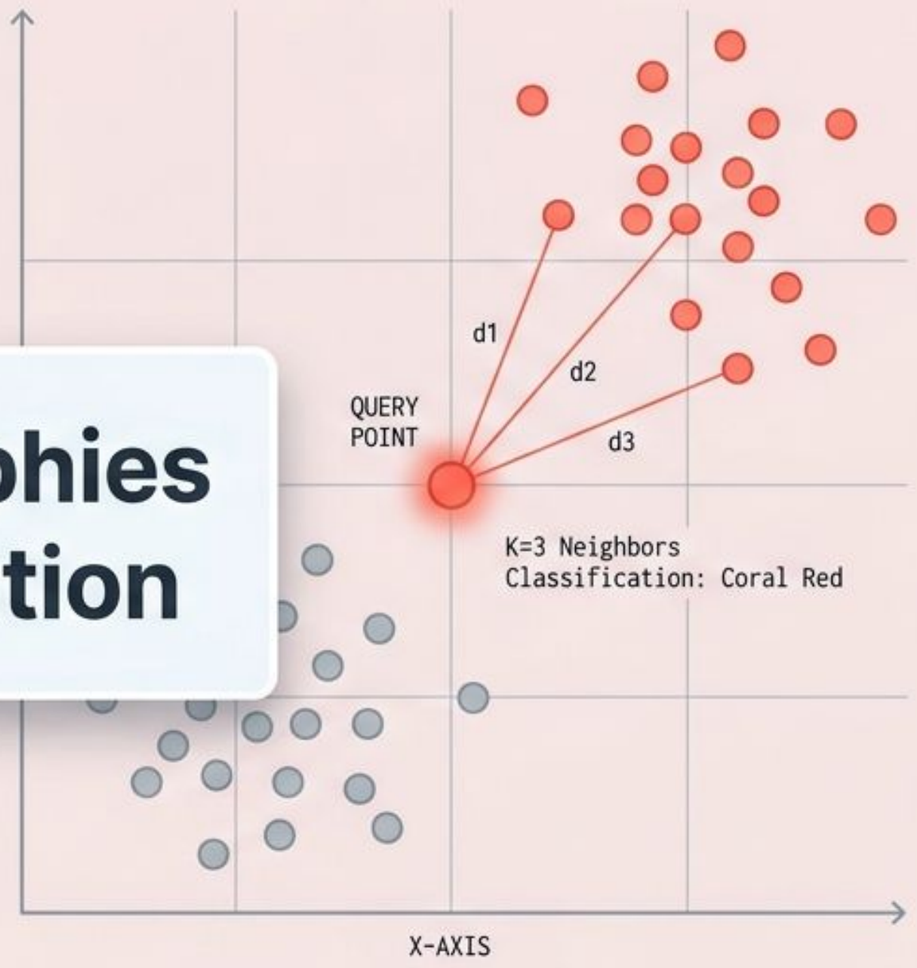
The true Wisdom of the Crowd requires choosing the right architecture for the right data ecosystem.

Gemini Notebook

# Two Philosophies of Classification




## Naïve Bayes: The Probability Engine




## K-Nearest Neighbors: The Proximity Engine

Gemini Notebook

# The Ground Truth: The AllElectronics Database

Our goal is to predict if a new person will buy a computer. We will solve this exact problem twice, using two entirely different mechanical philosophies.

| RID | age | income | student | credit_rating | buys_computer |
|---|---|---|---|---|---|
| 1 | youth | high | no | fair | no |
| 2 | youth | high | no | excellent | no |
| 3 | middle_aged | high | no | fair | yes |
| 4 | senior | medium | no | fair | yes |
| 5 | senior | low | yes | fair | yes |
| 6 | senior | low | yes | excellent | no |
| 7 | middle_aged | low | yes | excellent | yes |
| 8 | youth | medium | no | fair | no |
| 9 | youth | low | yes | fair | yes |
| 10 | senior | medium | yes | fair | yes |
| 11 | youth | medium | yes | excellent | yes |
| 12 | middle_aged | medium | no | excellent | yes |
| 13 | middle_aged | | | | yes |
| 14 | senior | | | | no |

## The Test Subject

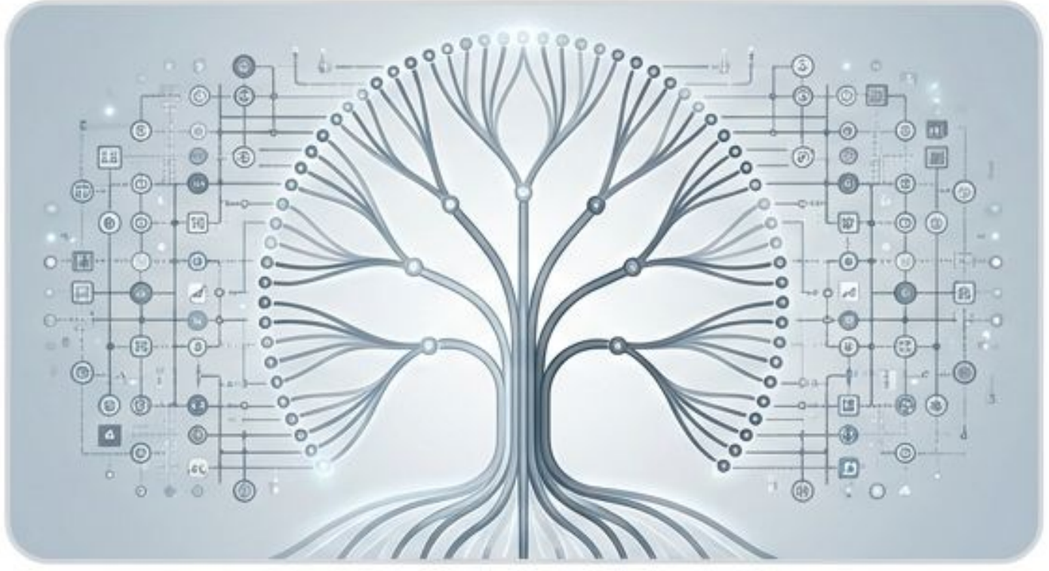
`Age = youth` `Income = medium`

`Student = yes` `Credit Rating = fair`

Gemini Notebook

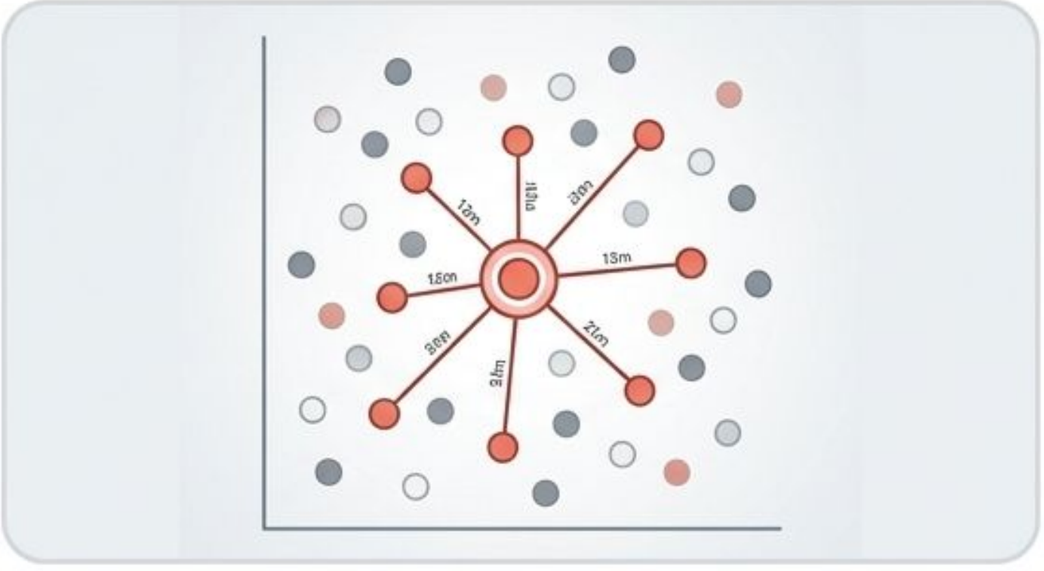
# Machines solve problems by looking through different lenses

## The Global Lens



Builds a mathematical model of historical trends from the entire dataset before making a prediction.

## The Local Lens



Builds no prior model. Waits for a specific test case, then looks only at its immediate physical neighbors.

Gemini Notebook

# The Anatomy of a Probabilistic Mind

**Posterior Probability**
(What we want to find out)

**Likelihood**
(How often this specific data occurs in a class)

$$P(H|X) = \frac{P(X|H) * P(H)}{P(X)}$$

**Prior Probability**
(The baseline chance of a class occurring)

**Evidence**
(A constant we can ignore for comparison)

Key Takeaway: The algorithm assumes all attributes are **conditionally independent**—the "Naïve" assumption that no attribute influences another.

Gemini Notebook

# Phase 1: Calculating Global Priors and Likelihoods

| | RID | age | income | student | credit_rating | buys_computer |
|---|---|---|---|---|---|---|
| 1 | 20721 | 19 | $10,000 | no | fair | yes |
| 2 | 20722 | 29 | $10,000 | yes | medium | yes |
| 3 | 20723 | 28 | $10,000 | no | fair | no |
| 4 | 20724 | 32 | $10,000 | yes | medium | yes |
| 5 | 20735 | 24 | $10,000 | no | fair | yes |
| 6 | 20736 | 26 | $10,000 | yes | fair | no |
| 7 | 20737 | 27 | $25,000 | yes | fair | yes |
| 8 | 20738 | 28 | $10,000 | no | fair | no |
| 9 | 20739 | 31 | $10,000 | yes | medium | no |
| 10 | 20737 | 31 | $10,000 | no | medium | yes |
| 11 | 20732 | 29 | $20,000 | no | fair | yes |
| | RID | age | income | student | credit_rating | buys_computer |

## Step 1: Class Priors

P(buys_computer=yes) = 9/14 = 0.643
P(buys_computer=no) = 5/14 = 0.357

## Step 2: Attribute Likelihoods
(for our test subject)

Age (youth|yes): 2/9 = 0.222 | Age (youth|no): 3/5 = 0.60

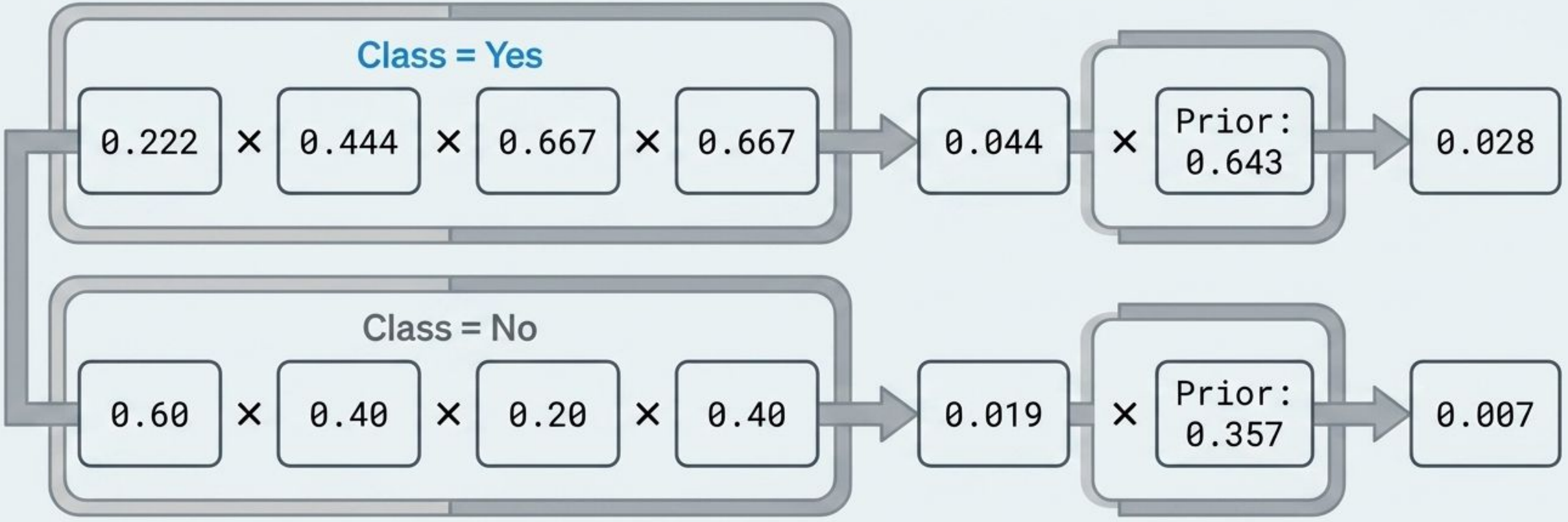
Income (medium|yes): 4/9 = 0.444 | Income (medium|no): 2/5 = 0.40

Student (yes|yes): 6/9 = 0.667 | Student (yes|no): 1/5 = 0.20

Credit (fair|yes): 6/9 = 0.667 | Credit (fair|no): 2/5 = 0.40

Gemini Notebook

# Phase 2: The Multiplication Engine




`argmax(0.028, 0.007)` -> **PREDICTION: BUYS COMPUTER = YES**

Gemini Notebook

# Diagnosing the Naïve Bayes Classifier

## ↗ Strengths

**Speed & Simplicity**
Highly computationally efficient.

**Scale**
Excels with high-dimensional data.

**Resilience**
Inherently robust to noise in the dataset.

**Flexibility**
Handles both continuous and discrete data seamlessly.

## ⚠ Vulnerabilities

### The Independence Flaw

The core assumption (that attributes don't affect each other) is often violated in real-world scenarios.

Gemini Notebook

# The Anatomy of a Geometric Mind

*Principle of Similarity: Birds of a feather flock together.*

## Core Concepts

### Instance-Based Learning

Also known as a "Lazy Learner". It builds zero prior models of the historical data.

### The Mechanism

It waits for a new data point, plots it in a geometric space alongside historical data, and calculates physical distance. It then assigns a class based on a majority vote of the nearest neighbors.

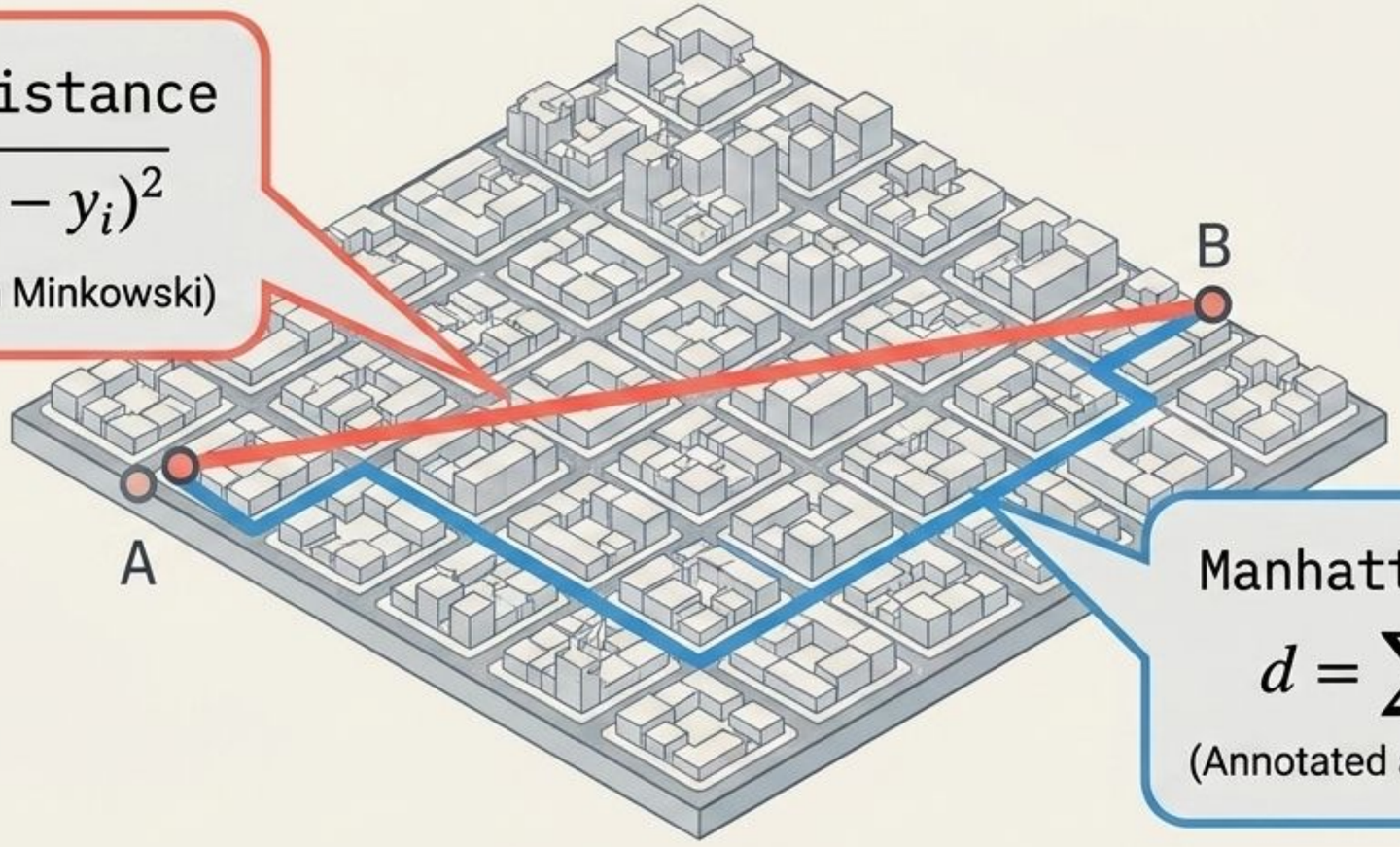
Gemini Notebook

# The Geometry of Distance Metrics

**Euclidean Distance**

$$d = \sqrt{\sum (x_i - y_i)^2}$$

(Annotated as p=2 in Minkowski)




**Manhattan Distance**

$$d = \sum |x_i - y_i|$$

(Annotated as p=1 in Minkowski)

The choice of metric fundamentally changes how the algorithm perceives “closeness”.

Gemini Notebook

# Test Case

- **New Person:**
  - Age = youth
  - Income = medium
  - Student = yes
  - Credit Rating = fair
- **Goal**: Predict if they will buy a computer.

# Encoding Dataset

## Age:

- youth=0, middle_aged=1, senior=2

## Income:

- low=0, medium=1, high=2

## Student:

- no=0, yes=1

## Credit:

- fair=0, excellent=1

## Buys:

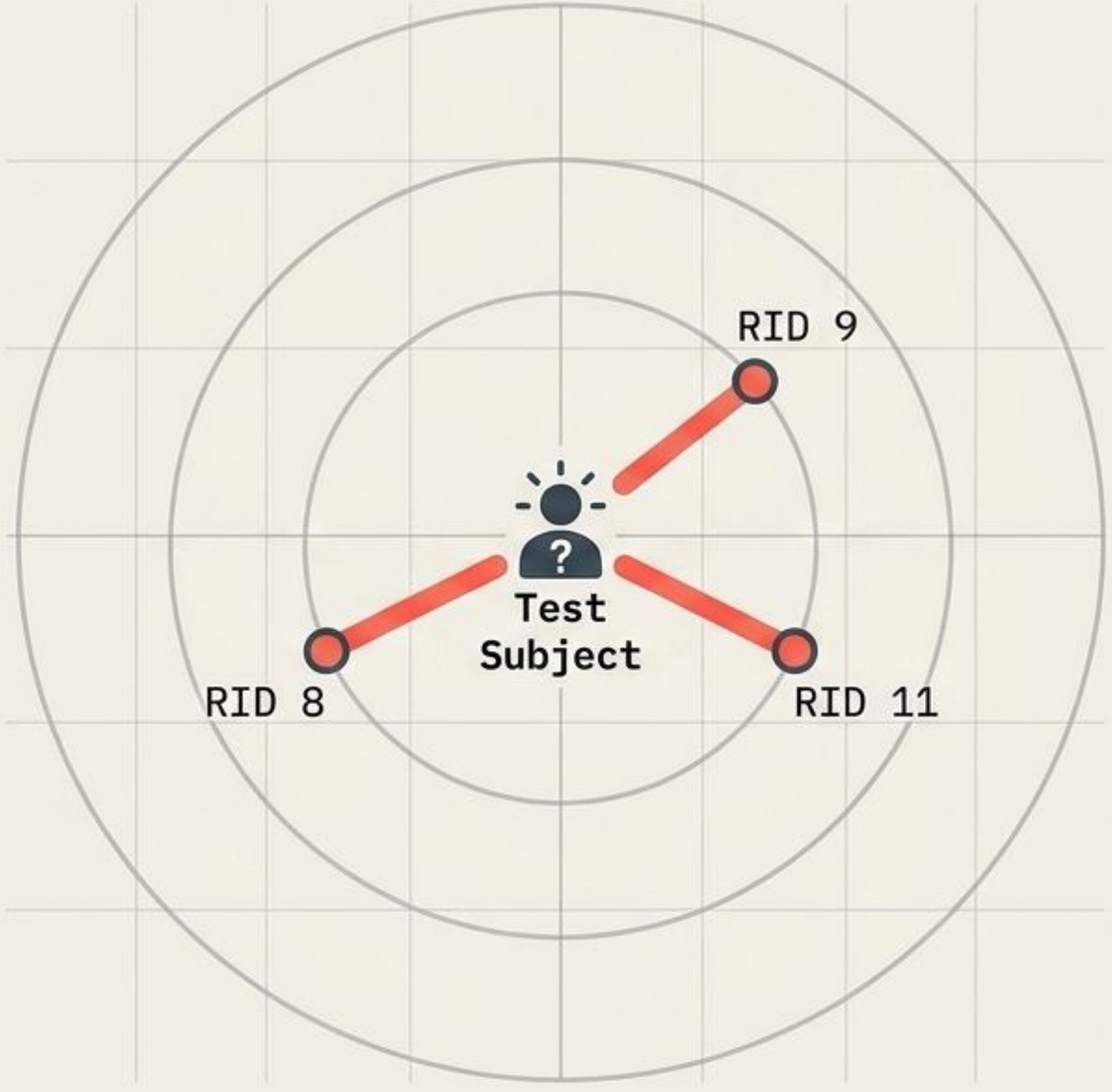
- no=0, yes=1

Test case = (0,1,1,0)

# Phase 1: The Local Proximity Search

- RID 1: $\sqrt{((0-0)^2+(1-2)^2+(1-0)^2+(0-0)^2)} = \sqrt{(0+1+1+0)} = \sqrt{2} \approx 1.41$
- RID 2: $\sqrt{((0-0)^2+(1-2)^2+(1-0)^2+(0-1)^2)} = \sqrt{(0+1+1+1)} = \sqrt{3} \approx 1.73$
- RID 3: $\sqrt{((0-1)^2+(1-2)^2+(1-0)^2+(0-0)^2)} = \sqrt{(1+1+1+0)} = \sqrt{3} \approx 1.73$
- RID 4: $\sqrt{((0-2)^2+(1-1)^2+(1-0)^2+(0-0)^2)} = \sqrt{(4+0+1+0)} = \sqrt{5} \approx 2.24$
- RID 5: $\sqrt{((0-2)^2+(1-0)^2+(1-1)^2+(0-0)^2)} = \sqrt{(4+1+0+0)} = \sqrt{5} \approx 2.24$
- RID 6: $\sqrt{((0-2)^2+(1-0)^2+(1-1)^2+(0-1)^2)} = \sqrt{(4+1+0+1)} = \sqrt{6} \approx 2.45$
- RID 7: $\sqrt{((0-1)^2+(1-0)^2+(1-1)^2+(0-1)^2)} = \sqrt{(1+1+0+1)} = \sqrt{3} \approx 1.73$
- RID 8: $\sqrt{((0-0)^2+(1-1)^2+(1-0)^2+(0-0)^2)} = \sqrt{(0+0+1+0)} = \sqrt{1} = 1.00$ ✅
- RID 9: $\sqrt{((0-0)^2+(1-0)^2+(1-1)^2+(0-0)^2)} = \sqrt{(0+1+0+0)} = \sqrt{1} = 1.00$ ✅
- RID 10: $\sqrt{((0-2)^2+(1-1)^2+(1-1)^2+(0-0)^2)} = \sqrt{(4+0+0+0)} = \sqrt{4} = 2.00$
- RID 11: $\sqrt{((0-0)^2+(1-1)^2+(1-1)^2+(0-1)^2)} = \sqrt{(0+0+0+1)} = \sqrt{1} = 1.00$ ✅
- RID 12: $\sqrt{((0-1)^2+(1-1)^2+(1-0)^2+(0-1)^2)} = \sqrt{(1+0+1+1)} = \sqrt{3} \approx 1.73$
- RID 13: $\sqrt{((0-1)^2+(1-2)^2+(1-1)^2+(0-0)^2)} = \sqrt{(1+1+0+0)} = \sqrt{2} \approx 1.41$
- RID 14: $\sqrt{((0-2)^2+(1-1)^2+(1-0)^2+(0-1)^2)} = \sqrt{(4+0+1+1)} = \sqrt{6} \approx 2.45$

# Phase 2: The K=3 Voting Ring




**Voting Tally Box**

RID 8 (Distance 1.00)
**-> Votes No**

RID 9 (Distance 1.00)
**-> Votes Yes**

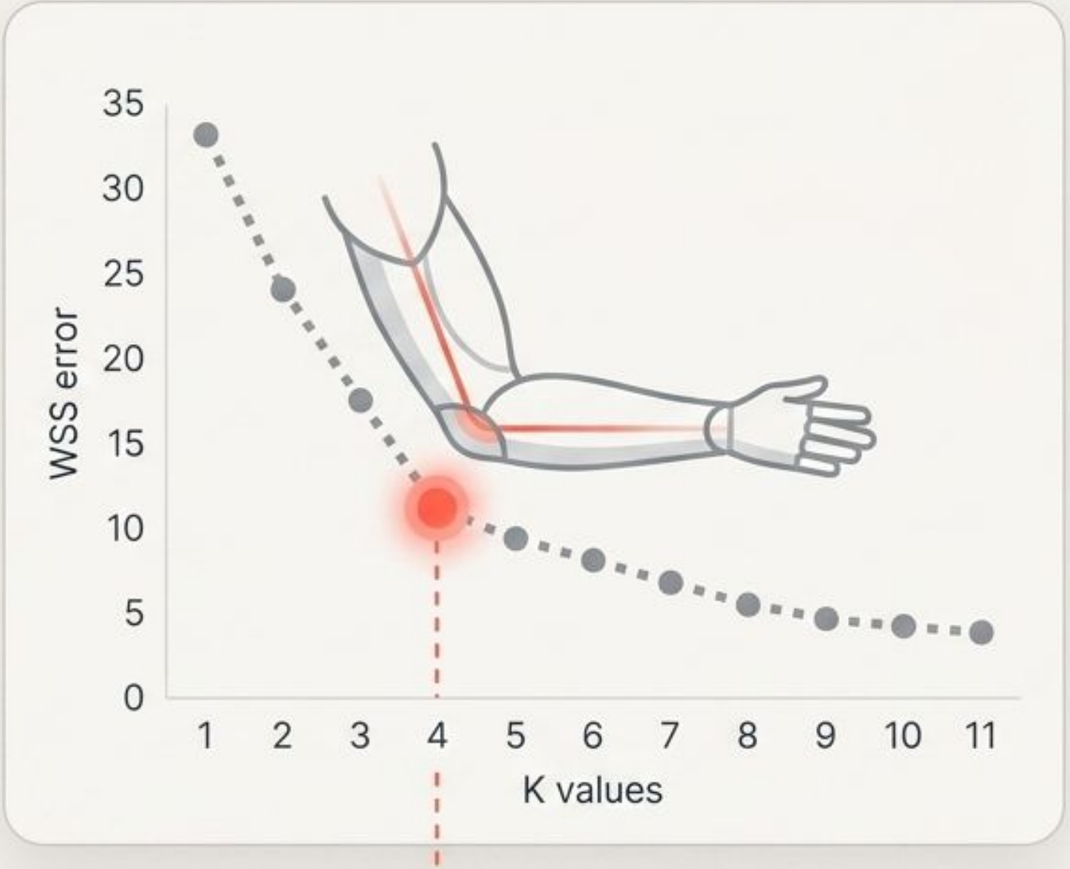
RID 11 (Distance 1.00)
**-> Votes Yes**

**MAJORITY VOTE (2 to 1)**
**-> PREDICTION:**
**BUYS COMPUTER = YES**

Gemini Notebook

# Calibrating K: The Elbow Method

**Small K:** Too small. Model is hyper-sensitive to localized noise, causing severe overfitting.




**Large K:** Too large. Model oversimplifies the boundaries, causing underfitting.

Gemini Notebook

# Diagnosing K-Nearest Neighbors

## Strengths

**Intuitiveness:** Exceptionally simple to explain and understand.

**Elegance:** Zero training phase required; handles multi-class problems natively.

**Small Data:** Performs exceptionally well with small datasets.

## Vulnerabilities

**Computational Weight:** Exhaustively expensive for large datasets (must calculate distance to every point).

**Feature Sensitivity:** Easily thrown off by irrelevant features.

**Maintenance:** Strictly requires feature scaling/normalization to work properly.

Gemini Notebook

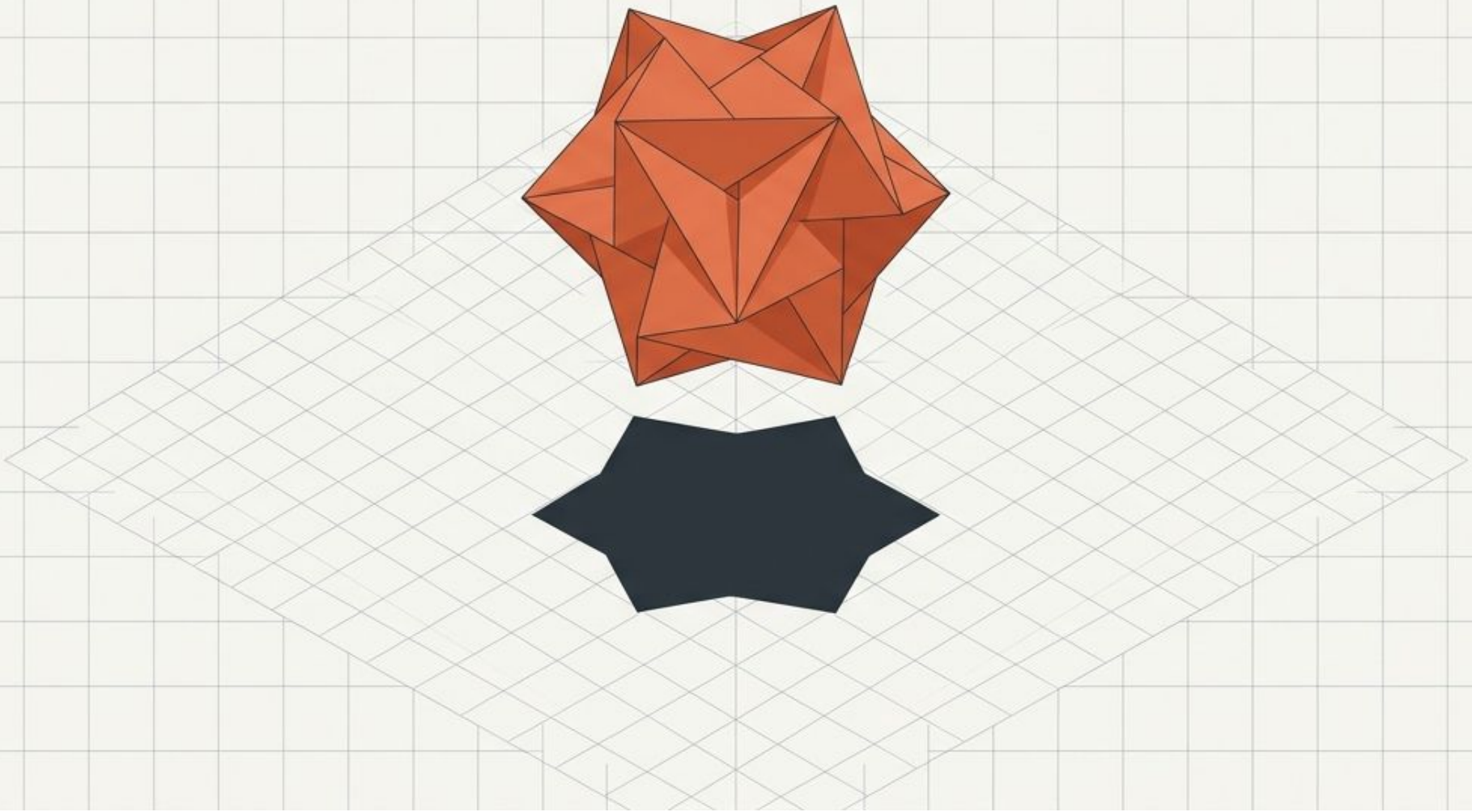
# The Algorithm Selection Matrix

| | Naïve Bayes | K-Nearest Neighbors |
|---|---|---|
| **Core Philosophy** | Probability (Global Trends) | Geometry (Local Proximity) |
| **Learning Style** | Eager (Builds model first) | Lazy (Waits for test case) |
| **High-Dimensional Data** | Excellent (Scales well) | Poor (Curse of Dimensionality) |
| **Training Speed** | Fast | Instant (Zero training) |
| **Prediction Speed** | Instant | Very Slow (Calculates all distances) |
| **Fatal Vulnerability** | Broken Independence Assumption | Unscaled Irrelevant Features |

Gemini Notebook

# Dimensionality Reduction

## Extracting Essence from High-Dimensional Space



Gemini Notebook

# THE BURDEN OF THE UNREFINED BLOCK

## Our goal as data architects: Maximising signal while minimising noise.

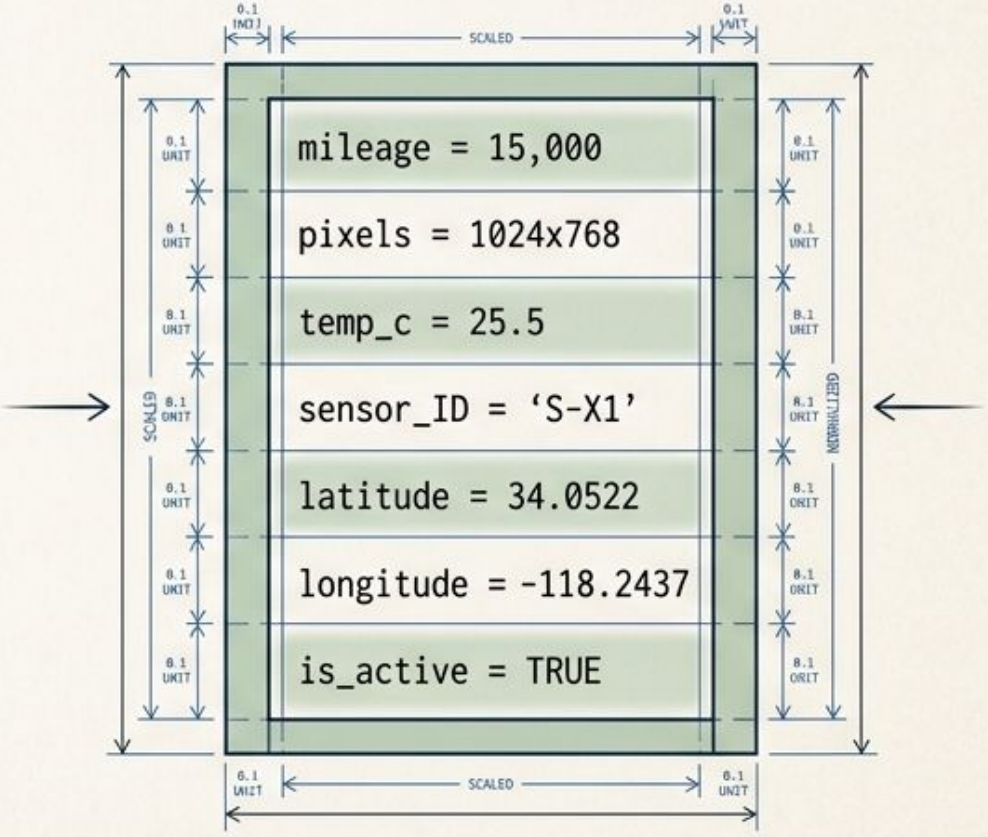
- In real-world problems, datasets contain thousands or millions of features.
- Feeding this directly into a model guarantees slow training, storage bloat, and the inability to find a good solution.

### UNSTRUCTURED RAW DATA (NOISE)

| | | | |
|---|---|---|---|
| mileage | mileage | 15,000 | null |
| pixels | pixels | 1024x768 | null |
| date_acquired | date_acquired | null | N/A |
| temp_c | temp_c | 25.5 | TBD |
| sensor_ID | sensor_ID | ‘S-X1’ | null |
| sensor_ID | latitude | 34.0522 | null |
| error_log | error_log | null | N/A |
| user_comment_14 | longitude | -118.2437 | null |
| latitude | latitude | -118.2437 | null |
| longitude | longitude | TRUE | N/A |
| null | null | N/A | TBD |
| N/A | N/A | N/A | N/A |
| TBD | TBD | TBD | TBD |

Attribute: A raw data type (e.g., ‘mileage’, ‘pixels’).

### CURATED FEATURE VECTOR (SIGNAL)




Feature: A curated, predictive attribute + its value (e.g., ‘mileage = 15,000’).

Gemini Notebook

# What is Dimensionality Reduction?

**Definition:** The process of reducing the number of features (dimensions) in a dataset.

**Goal:** Simplify data while preserving as much meaningful information as possible.

**Benefits:**

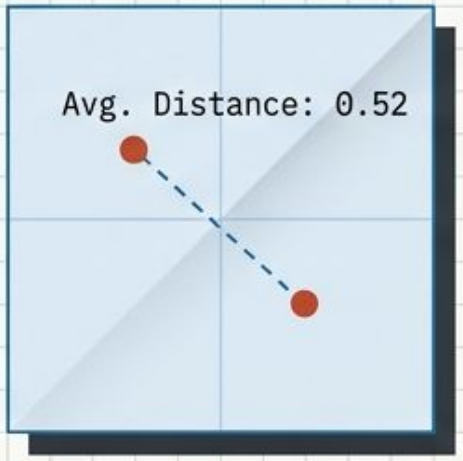
- Reduces computational cost
- Visualizes high-dimensional data
- Avoids overfitting (curse of dimensionality)
- Removes redundant or noisy features

# The Curse of Dimensionality

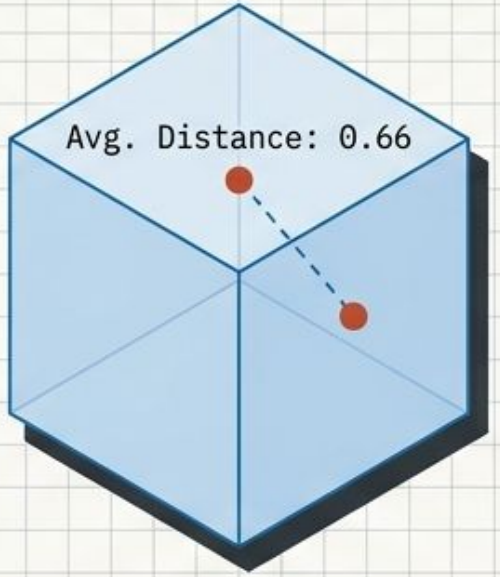
As features grow, data space explodes exponentially. Visualization becomes impossible, models overfit, and distance metrics lose all meaning.

- In a 10,000-dimensional unit hypercube, the probability that a data point is located less than 0.001 from a border is >99.999999%.
- In high dimensions, every instance is an outlier sitting on the border of the space.

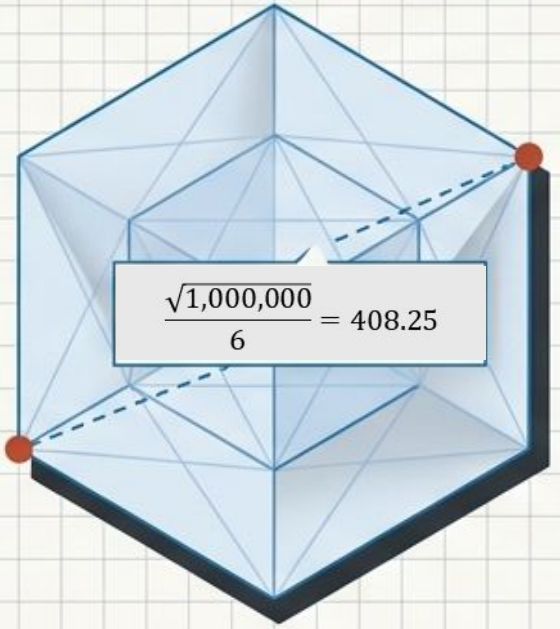
## 2D Space




## 3D Space




## 1,000,000D Space

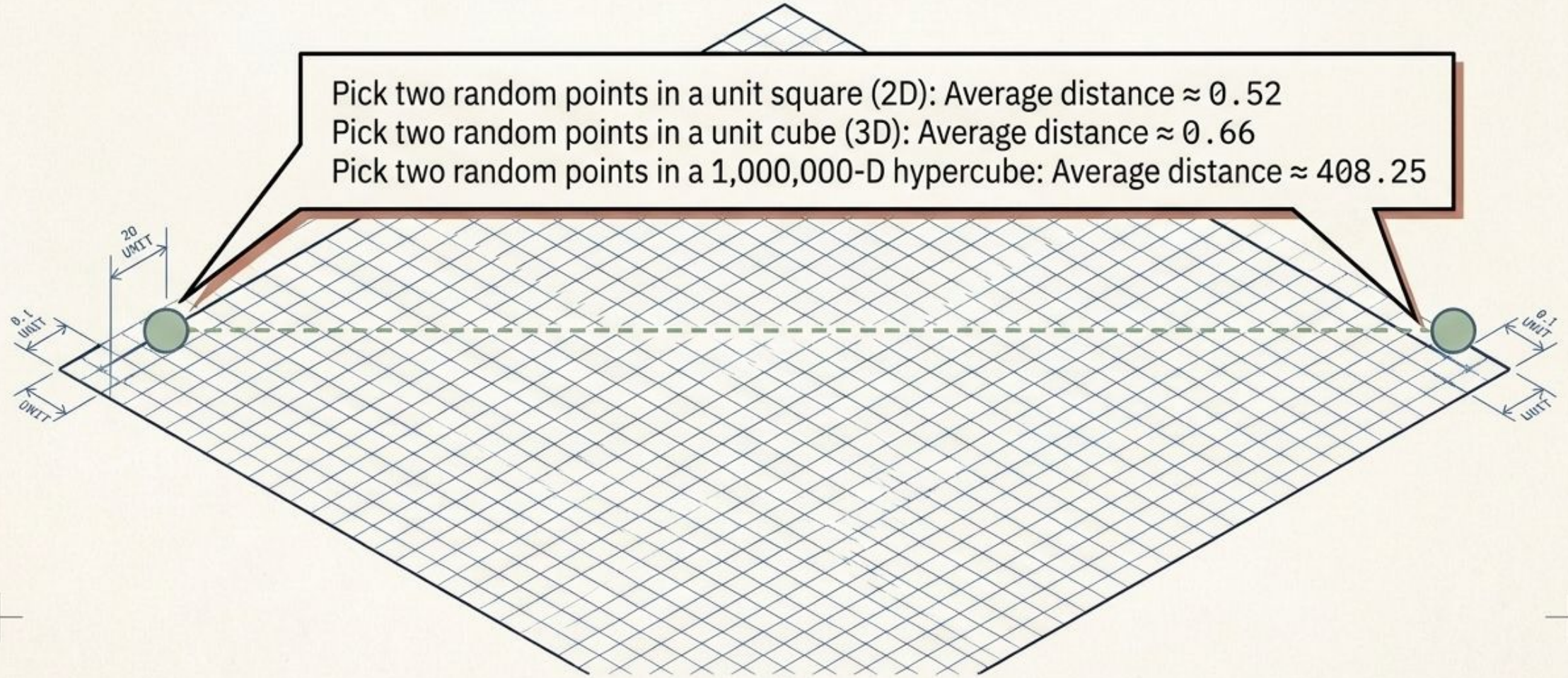



**Takeaway: In high-dimensional spaces, almost all data points are isolated outliers.**

Gemini Notebook

# When Distance Loses Its Meaning

How can two points within a single unit hypercube be so far apart? There is simply too much empty space. Predictions become massive extrapolations. To compensate, we would need more training instances instances than atoms in the observable universe. Overfitting becomes inevitable.

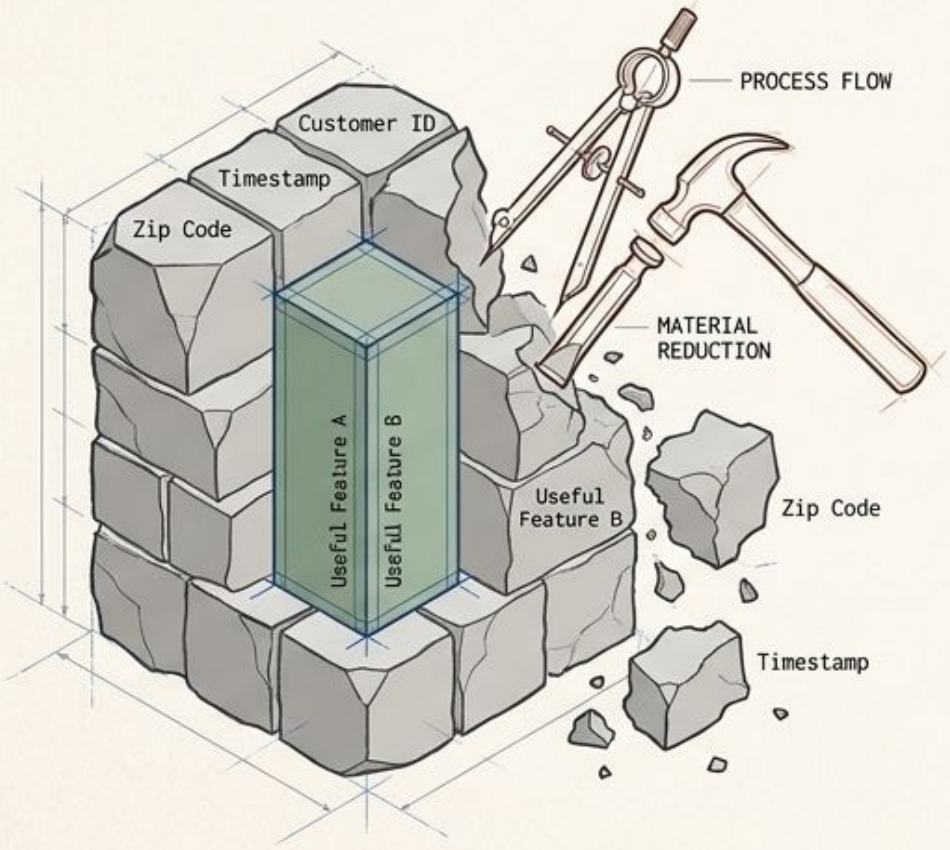



Gemini Notebook

# Two Tools for the Master Craftsman

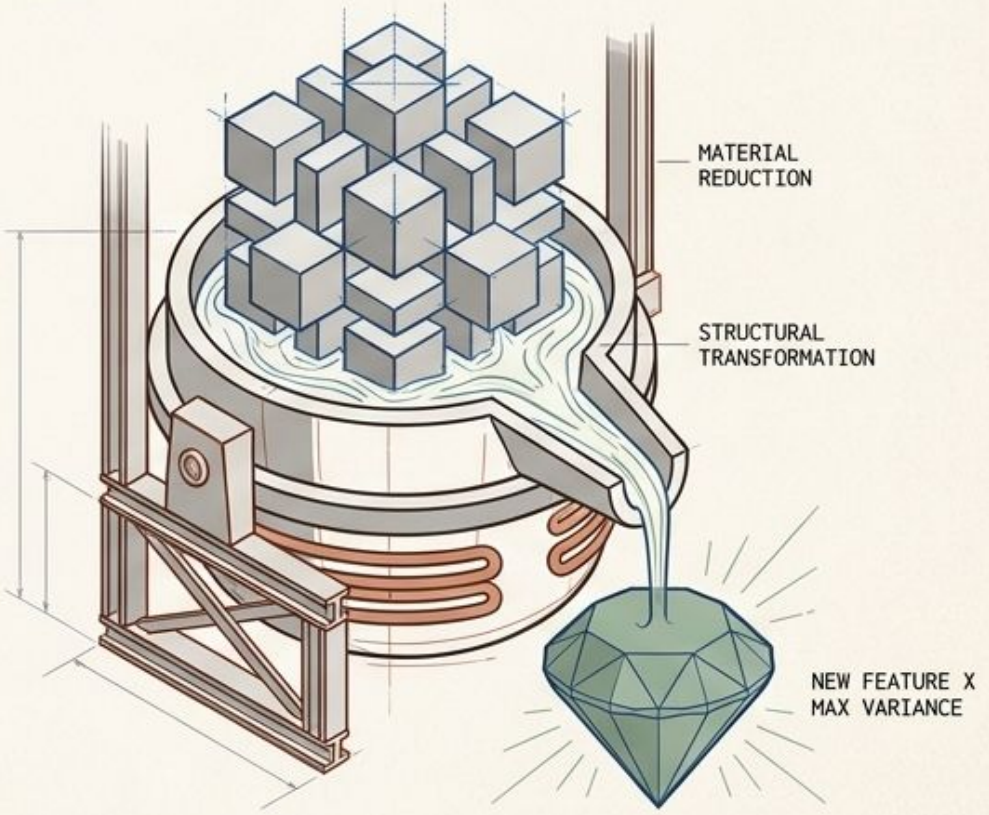
## Feature Selection

Keeping the most useful existing features and discarding the rest.
(Retains original interpretability).




## Feature Extraction

Mathematically compressing and combining existing features to produce entirely new, denser features.
(Loses original interpretability, captures maximum variance).



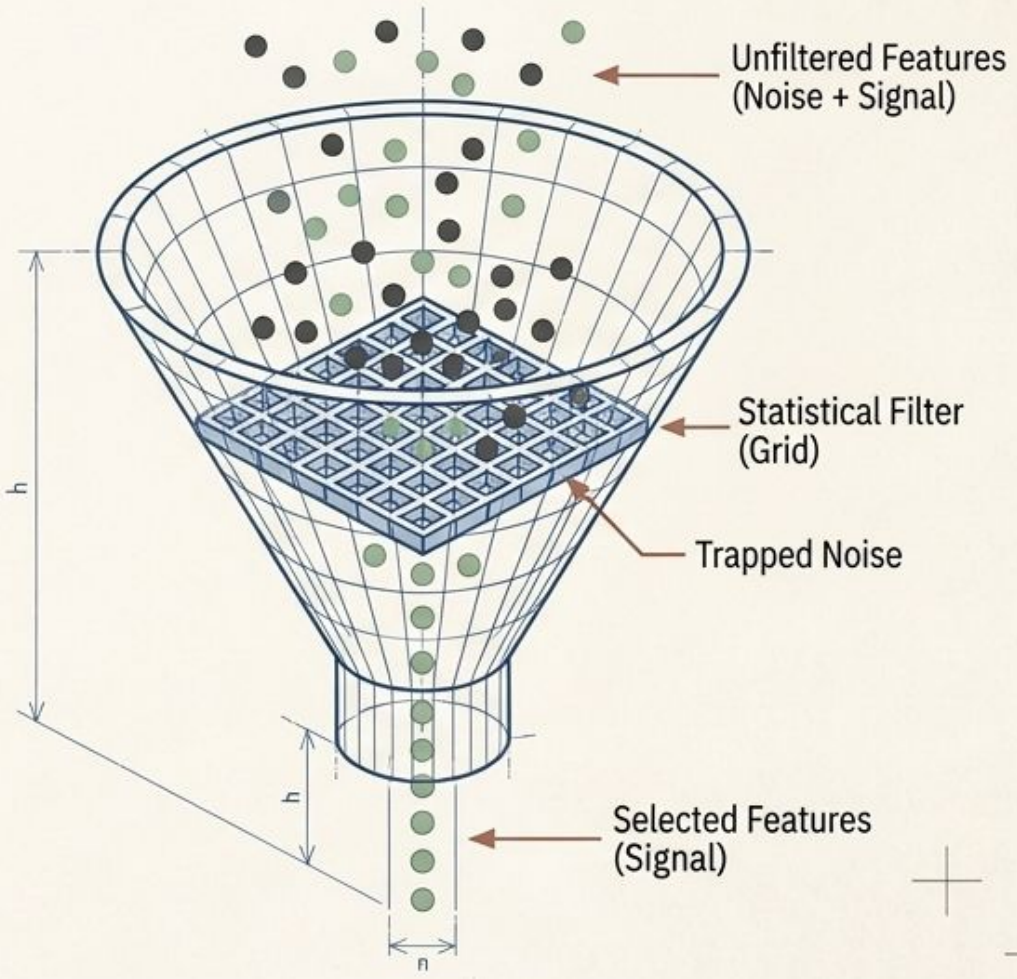

Gemini Notebook

# Filter Methods: The Statistical Scalpel

Evaluates the intrinsic properties of features independently of any machine learning model. Fast, computationally cheap, and avoids overfitting.

- **Categorical Targets**: Chi-Square tests if the feature and target are independent. Mutual Information measures entropy reduction.
- **Continuous Targets**: ANOVA F-test (linear dependencies) or Pearson Correlation.

```
p_value = chi2(feature, target) \
if p_value < 0.05: keep_feature()
```



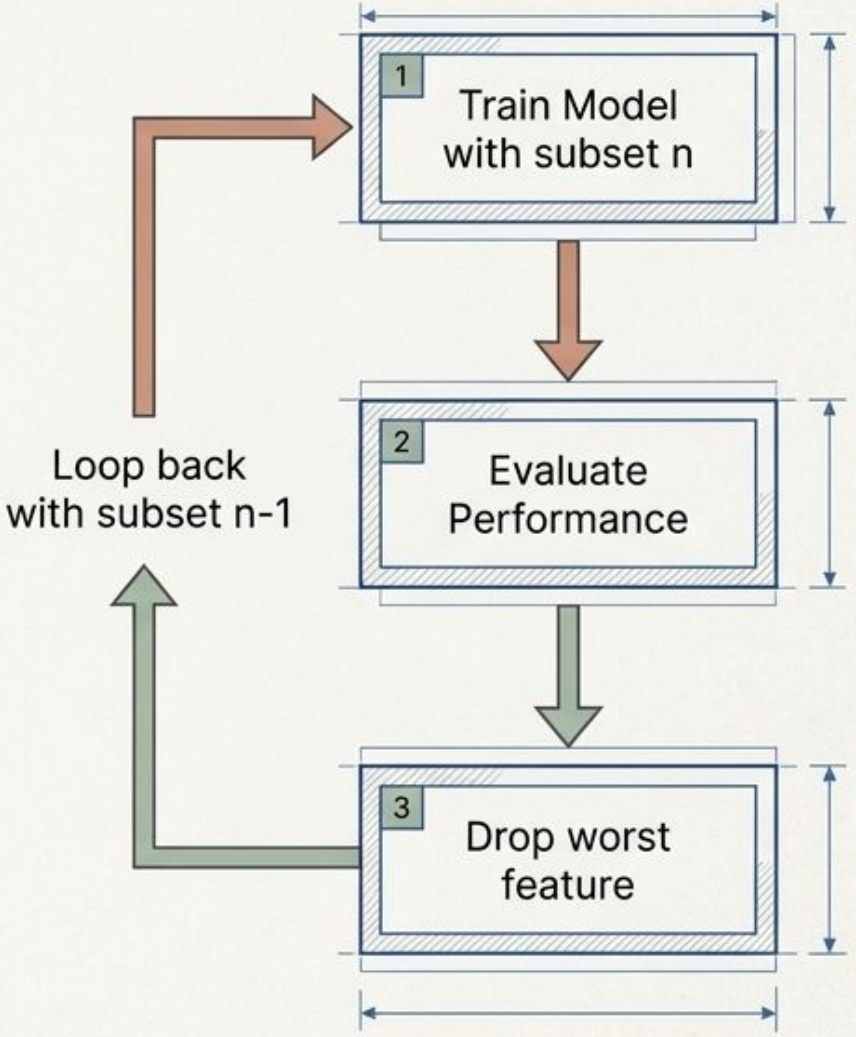

Gemini Notebook

# **Wrapper Methods:** The Iterative Chisel

- Evaluates subsets of features by training an actual model on them.
- **Sequential Forward Selection**: Starts with zero features, adds the best one iteratively.
- **Recursive Feature Elimination (RFE)**: Starts with all features, drops the least important one iteratively.
- Trade-off: Highly accurate, but computationally expensive and prone to overfitting on small datasets.

PSEUDOCODE: RFE LOOP

```
while features.count > target_k:
    model.fit(X, y)
    importances = model.coef_
    worst_feature = argmin(importances)
    X.drop(worst_feature)
```



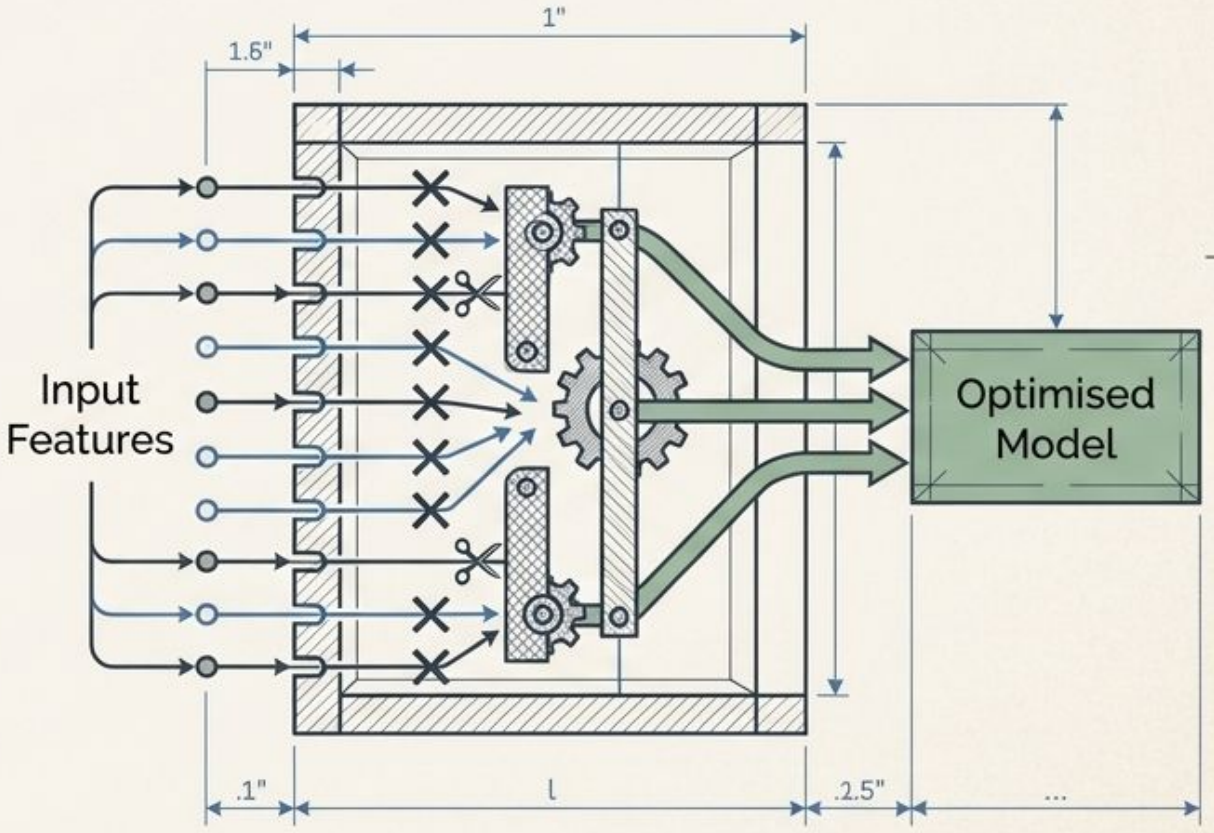

Gemini Notebook

# Embedded Methods: The Algorithm as Architect

Feature selection is deeply integrated into the learning algorithm itself. Faster than Wrapper methods, more accurate than Filter methods.

**Key Techniques:**

- **Tree-based Algorithms**: Random Forests and Gradient Boosting naturally discard useless features while splitting nodes based on `impurity reduction`.
- **L1 Regularisation (LASSO)**: Adds a mathematical penalty that forces the weights of irrelevant features down to `absolute zero`.




GeminiNotebook

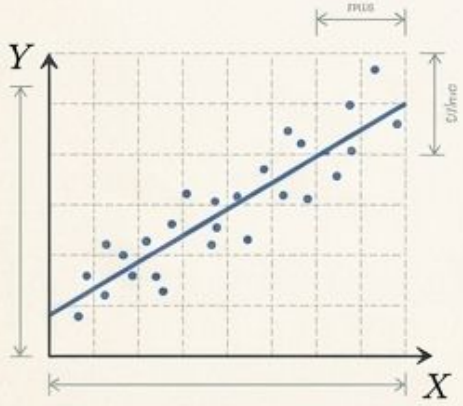
# Forcing Zero: The LASSO L1 Penalty

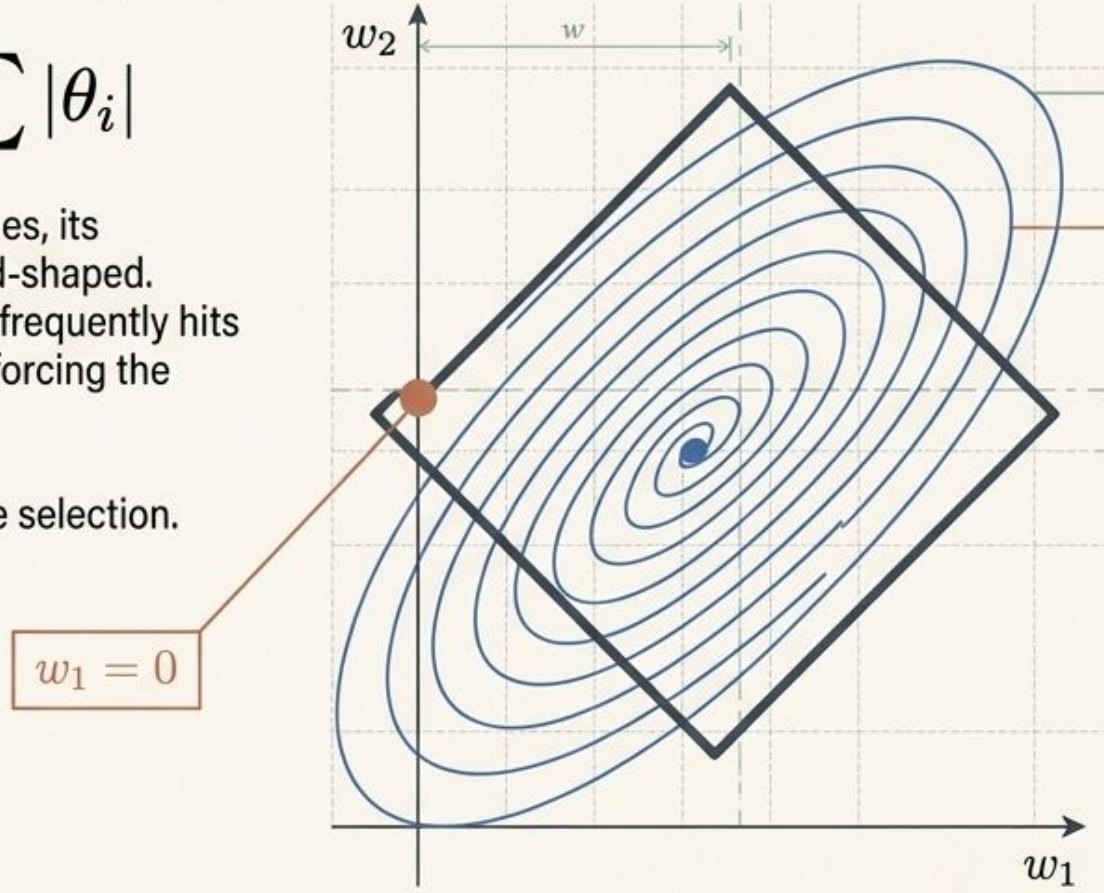
Least Absolute Shrinkage and Selection Operator (LASSO).

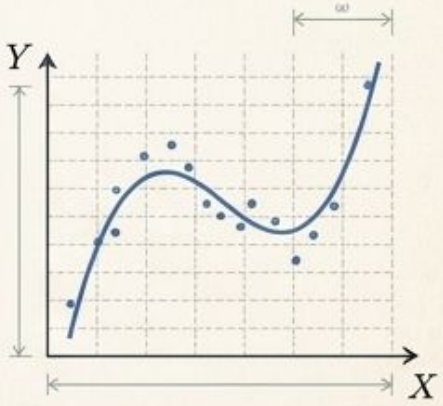
$$J(\theta) = \text{MSE}(\theta) + \alpha \sum |\theta_i|$$

Because the L1 penalty uses absolute values, its mathematical geometric space is diamond-shaped. When the algorithm minimises the cost, it frequently hits the corners of this diamond on the axes—forcing the feature weight to exactly zero.

Result: Automatic, perfectly sparse feature selection.



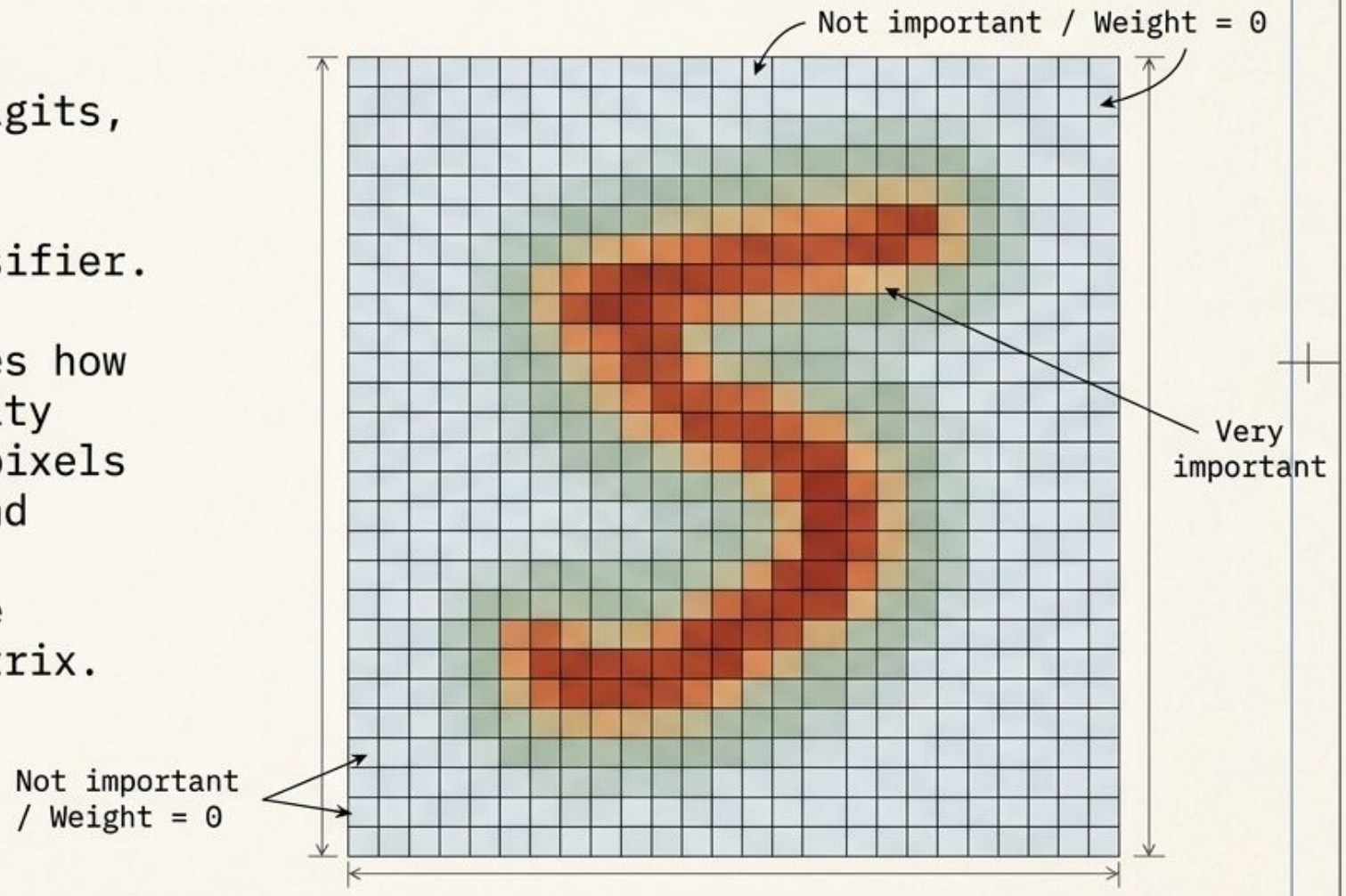

Gemini Notebook

# Real-World Embedded Selection: MNIST

Dataset: 70,000 handwritten digits, 28x28 pixels (784 features).

The Model: Random Forest Classifier.

The Insight: The model measures how much each pixel reduces impurity across all trees. The border pixels offer zero predictive value and are automatically dropped, dynamically performing feature selection on the raw image matrix.




Gemini Notebook

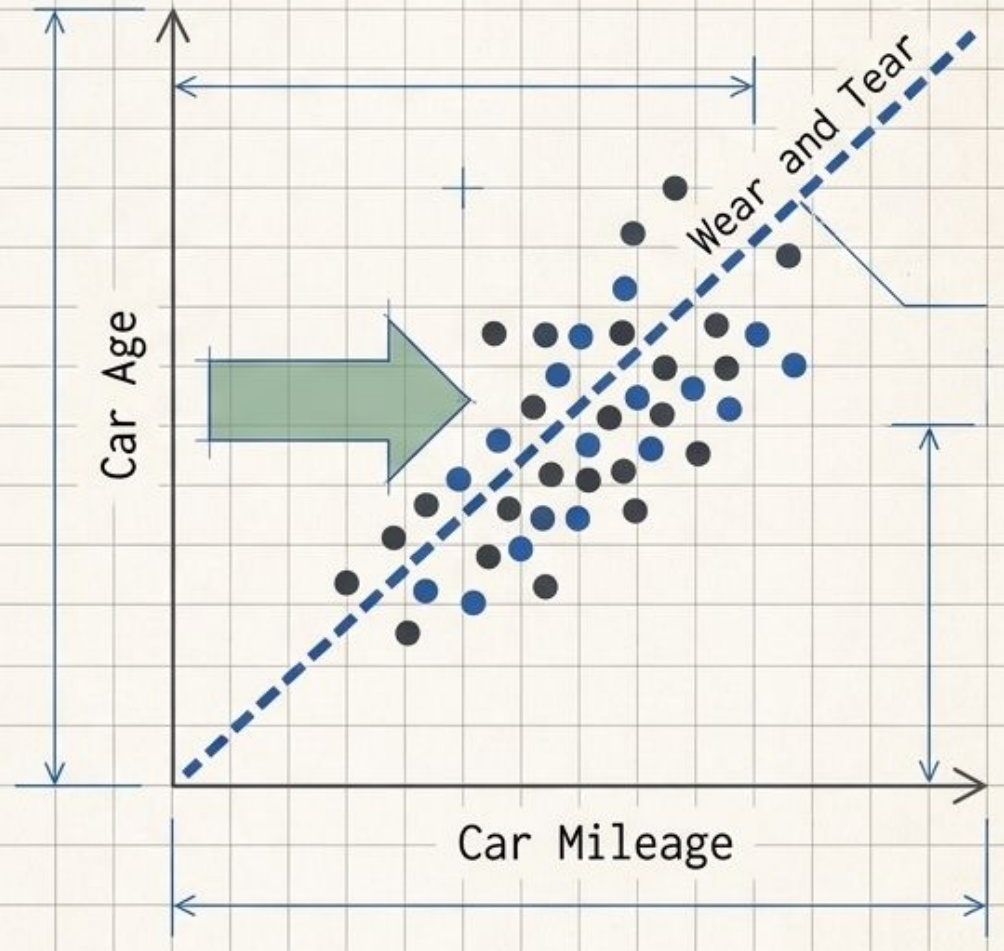
# The Architect's Cheat Sheet: Feature Selection

| Method Family | Computational Speed | Risk of Overfitting | Model Independence | Best Use Case |
|---|---|---|---|---|
| Filter Methods | Fast | Low | High | Initial dimensionality reduction on massive datasets. |
| Wrapper Methods | Slow | High | Low | Maximising accuracy when computational resources allow. |
| Embedded Methods | Medium | Medium | Low | High-dimensional datasets (e.g., text, genomics) using regularised models. |

Gemini Notebook

# When Chipping Fails: Recasting the Form

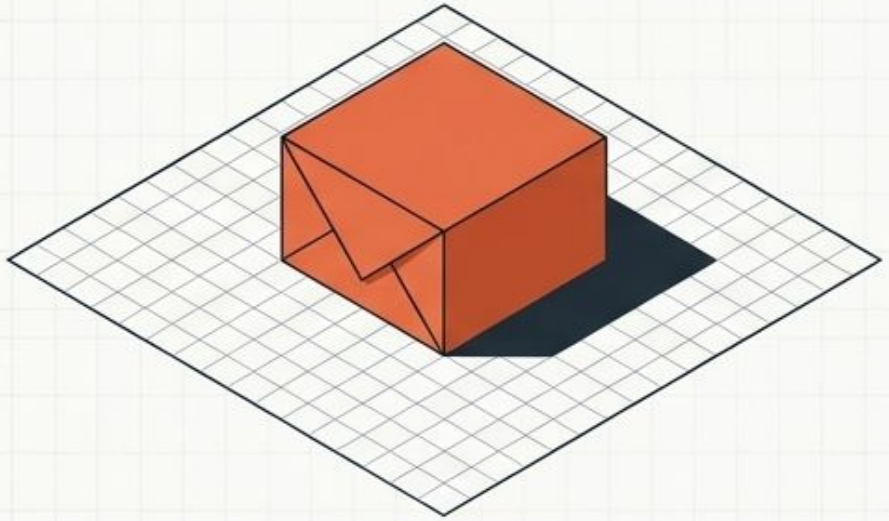
- Feature Selection fails when valuable information is spread thinly across hundreds of highly correlated features. Dropping them loses information.
- Feature Extraction (Dimensionality Reduction): Merges features by projecting the dataset onto a lower-dimensional subspace while preserving the maximum amount of variance.
- The Goal: Simplify the data, compress storage space, and enable 2D/3D DataViz, even if we lose original interpretability.




Gemini Notebook

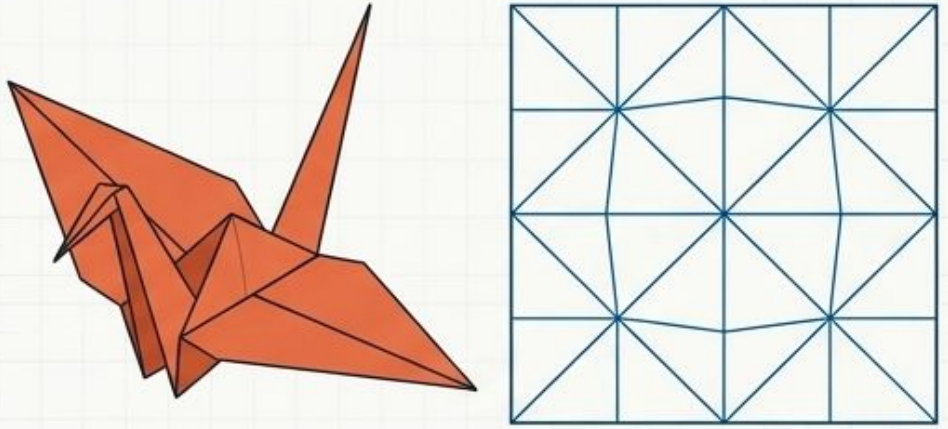
# The Grand Divide: Two Paths to Simplicity

## Path 1: Projection



| | |
|---|---|
| **Visual Metaphor:** | The Shadow |
| **Core Philosophy:** | Project data onto a lower-dimensional subspace. |
| **Goal:** | Find the axes of maximum global variance. |
| **Flagship Algorithm:** | PCA (Principal Component Analysis) |

## Path 2: Manifold Learning

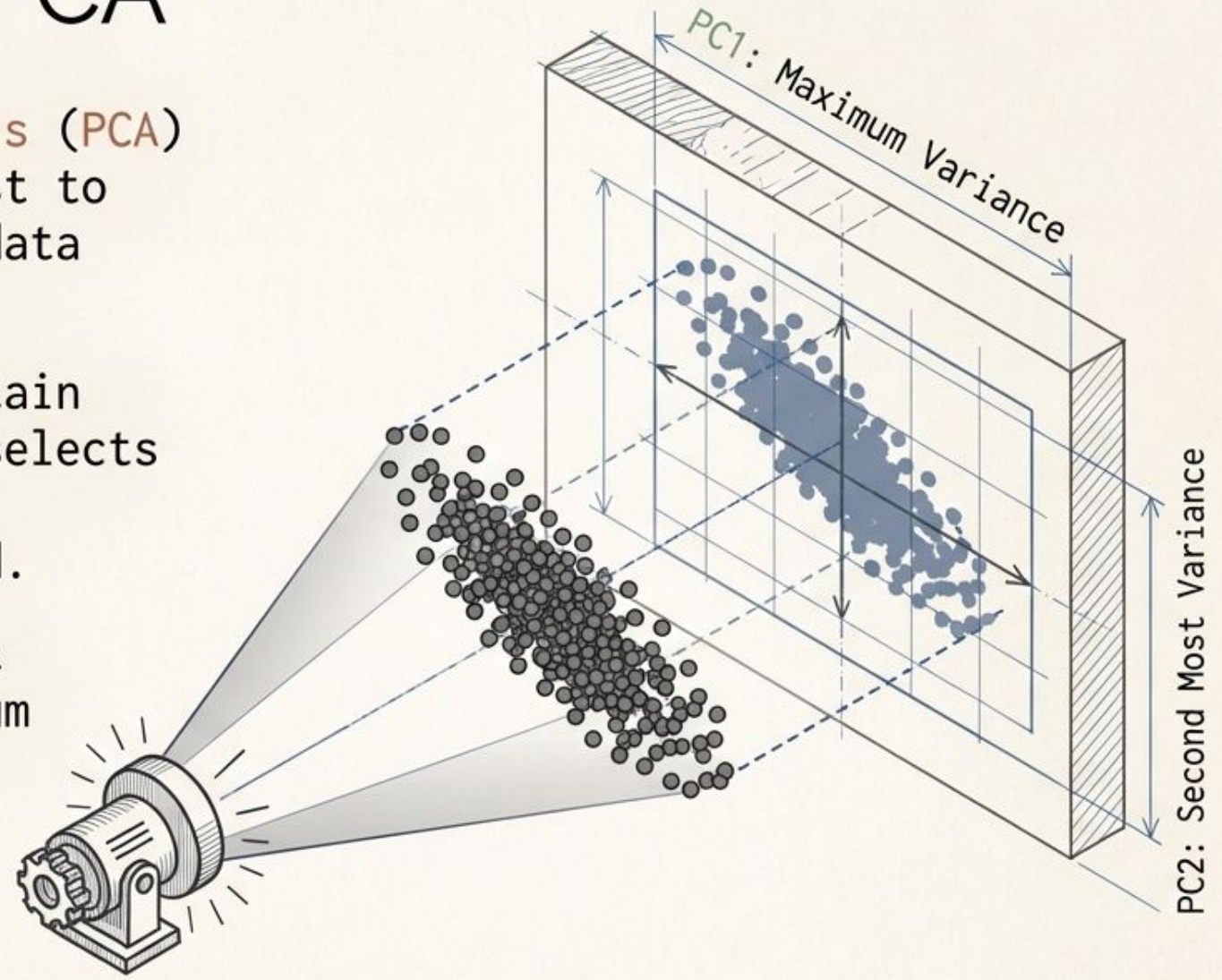


| | |
|---|---|
| **Visual Metaphor:** | The Unfolding |
| **Core Philosophy:** | Twisted high-dimensional data lies close to a simpler lower-dimensional shape. |
| **Goal:** | Unroll the data while preserving local neighbor relationships. |
| **Flagship Algorithm:** | LLE (Locally Linear Embedding) |

Gemini Notebook

# The Shadow of PCA

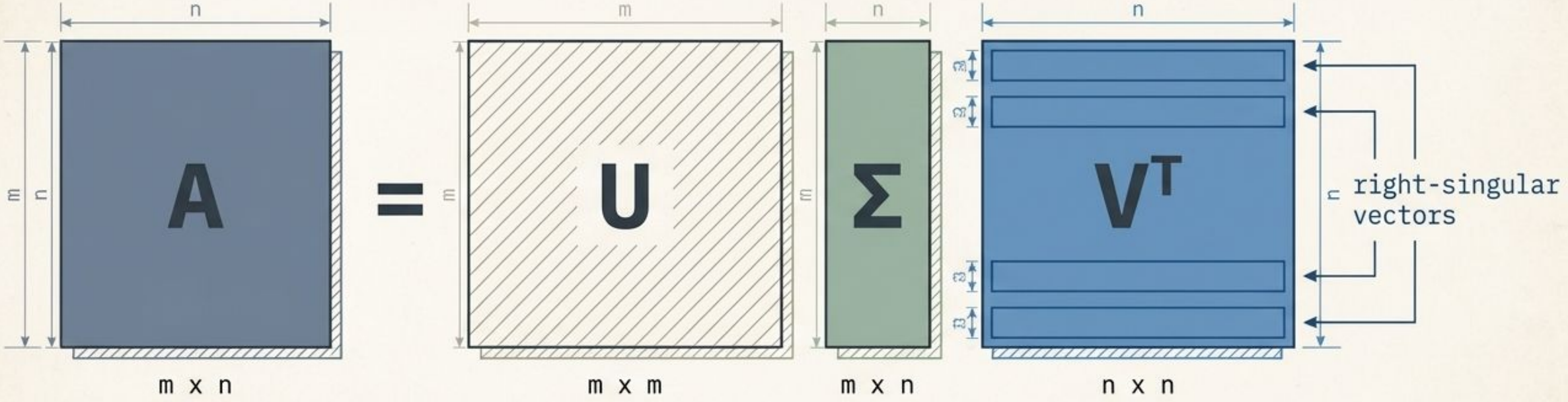
- Principal Component Analysis (PCA) finds the hyperplane closest to the data and projects the data onto it.
- Maximising Variance: To retain the most information, PCA selects the axis that creates the shadow with the most spread.
- The 1st Principal Component (PC1) is the axis of maximum variance. PC2 is the orthogonal axis with the second most variance.




Gemini Notebook

# Anatomy of an Equation: SVD

To find the Principal Components, PCA relies on a matrix factorisation technique called Singular Value Decomposition (SVD).

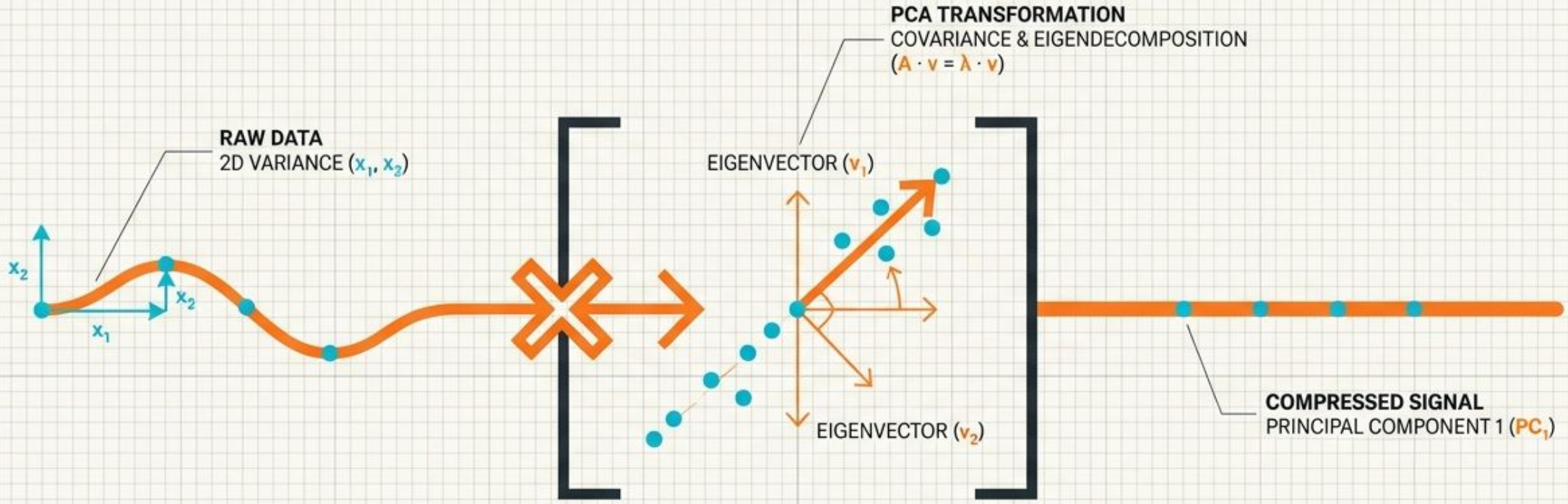



**A (Raw Data):** The centred training set matrix.

**U & Σ:** Left singular vectors and Singular values.

**V^T (The Blueprint):** The orthogonal matrix whose rows contain the right singular vectors—these are the unit vectors that define all our Principal Components.

Gemini Notebook




# The Mathematics of Compression

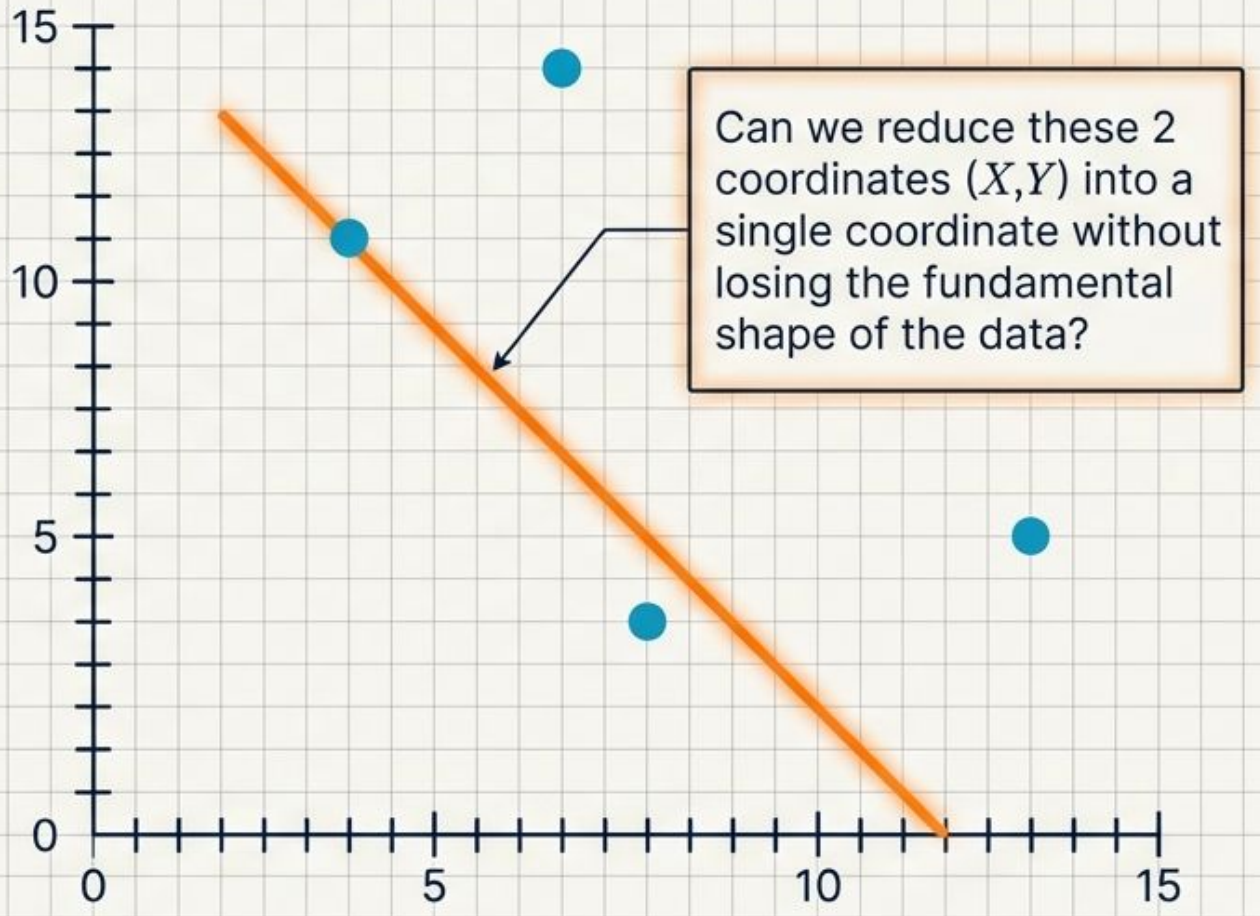
A visual teardown of Principal Component Analysis (PCA) from 2D raw data to 1D signal.

Gemini Notebook

# **The Objective:** Compress without collapsing.

## The Raw Input

| | |
|---|---|
| X = 4 | Y = 11 |
| X = 8 | Y = 4 |
| X = 13 | Y = 5 |
| X = 7 | Y = 14 |




Gemini Notebook

# Step 1: Shifting to the Origin

Part A

## Calculate Means

Mean of X = (4+8+13+7) / 4 = 8

Mean of Y = (11+4+5+14) / 4 = 8.5

Subtract Mean

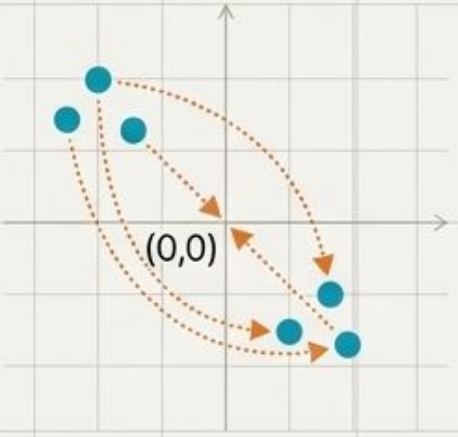
Part B

$(x_i - \bar{x})$ and $(y_i - \bar{y})$

Part C

## Centered Matrix (Xc, Yc)

$$\begin{bmatrix} -4 & 2.5 \\ 0 & -4.5 \\ 5 & -3.5 \\ -1 & 5.5 \end{bmatrix}$$




Gemini Notebook

# Step 2: The Anatomy of Covariance

$$var(x) = \frac{\sum(x - \bar{x})^2}{n - 1}$$

**Variance of X [14]:** How much the data spreads horizontally.

$$\Sigma = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

**Variance of Y [23]:** How much the data spreads vertically.

$$cov(x, y) = \frac{\sum(x - \bar{x})(y - \bar{y})}{n - 1}$$

**Covariance [-11]:** The negative sign indicates an inverse relationship—as X increases, Y tends to decrease.

Gemini Notebook

# Step 3: Extracting the Core Frequencies

**Covariance Matrix**

$$\begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

**Characteristic Equation**

$$\det(\Sigma - \lambda I) = 0$$

$$\lambda^2 - 37\lambda + 201 = 0$$

**Primary Eigenvalue ($\lambda_1$)**

$\lambda_1 = 30.38$

**Secondary Eigenvalue ($\lambda_2$)**

$\lambda_2 = 6.62$

**Key Insight:** Eigenvalues (λ) represent the magnitude of variance captured in a specific structural direction.

Gemini Notebook

# Step 3 (cont): Mapping Magnitude to Direction

## Primary Frequency

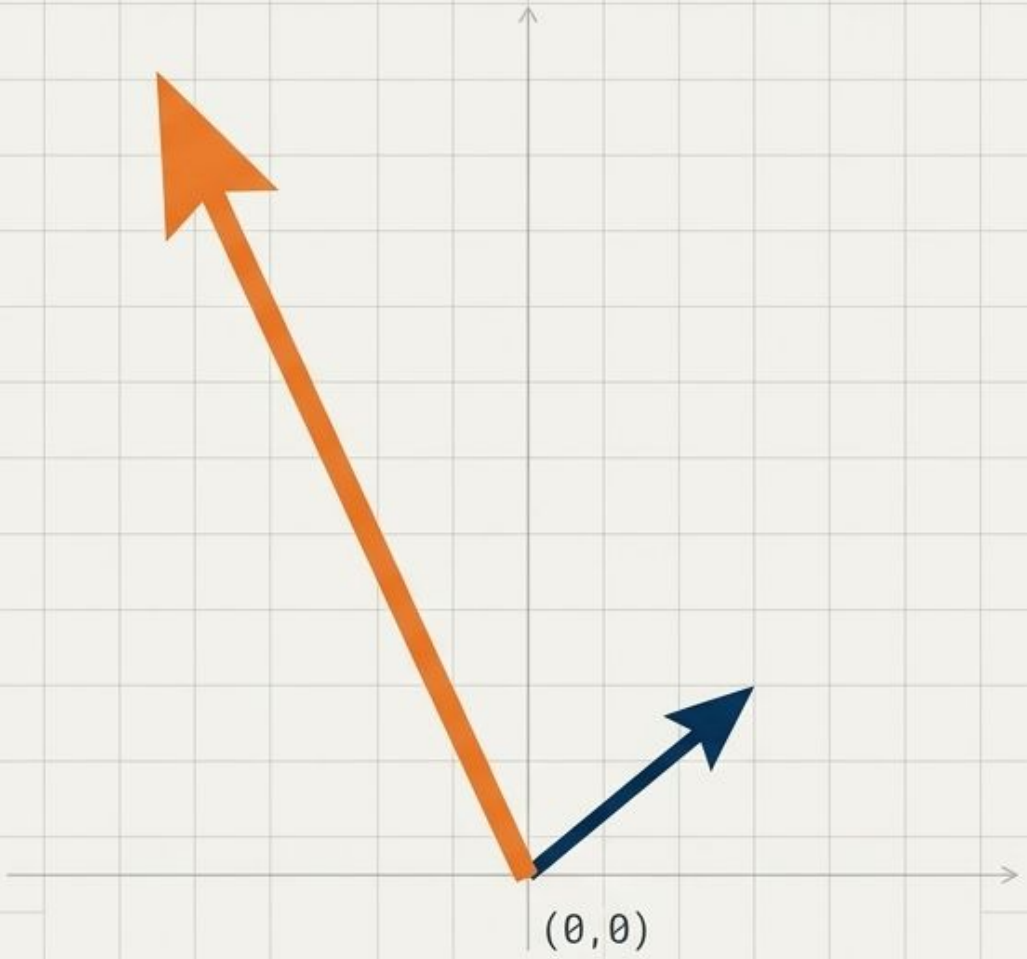
Magnitude: $\lambda_1 = 30.38$

Direction: $v_1 = (-0.557, 0.830)$

## Secondary Frequency

Magnitude: $\lambda_2 = 6.62$

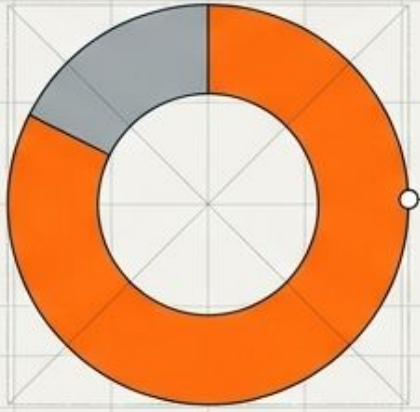
Direction: $v_2 = (0.830, 0.557)$




Gemini Notebook

# Step 4: The Dimensional Diagnostic

| Principal Component 1 | Principal Component 2 |
| --- | --- |
| Variance Captured (λ): 30.38 | Variance Captured (λ): 6.62 |
| Signal Strength: High (Primary Signal) | Signal Strength: Low (Noise/Secondary) |
| **Action: KEEP. This vector becomes our new 1D axis.** | **Action: ~~DISCARD.~~ By dropping this, we achieve ~~dimensionality reduction.~~** |

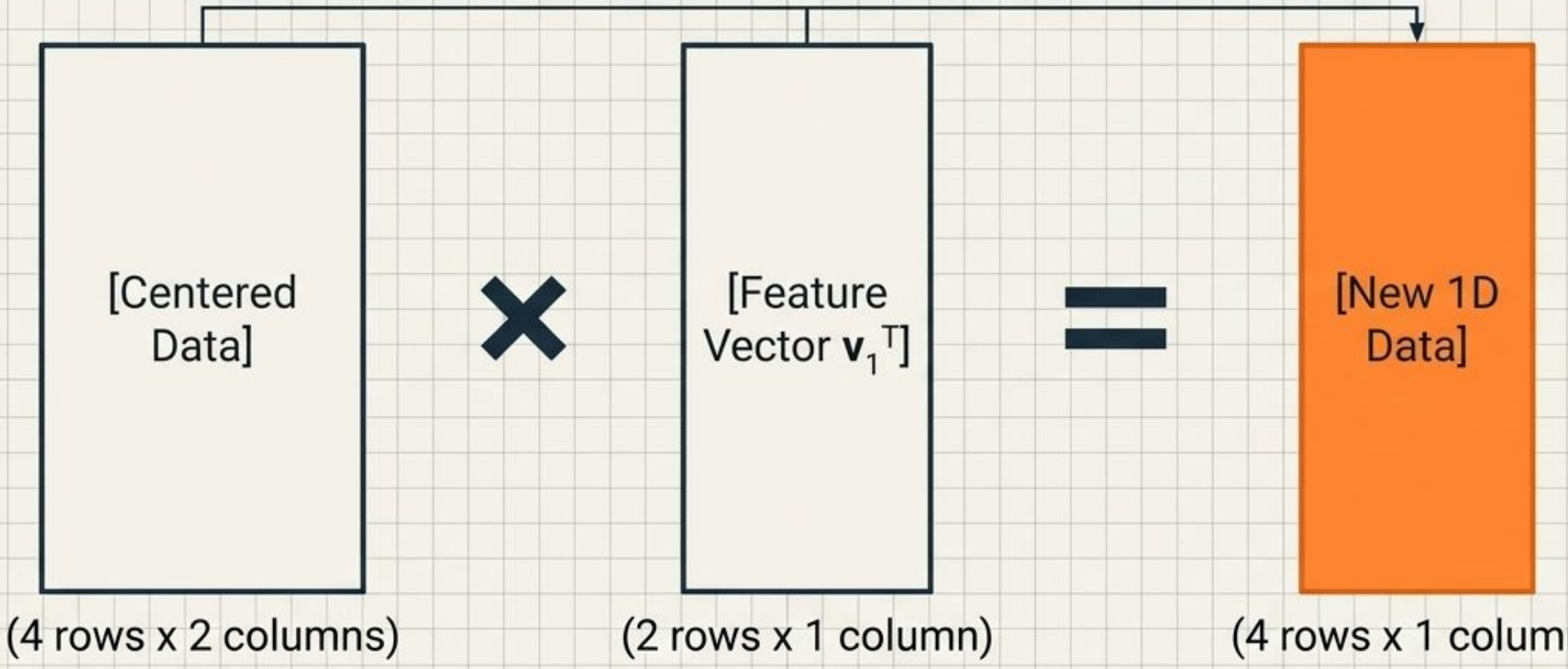


$$\frac{30.38}{(30.38 + 6.62)} = 0.82$$

~82% of structural variance is retained in a single dimension.

Gemini Notebook

# Step 5: The Projection Mechanism




To compress the data mechanically, we multiply the 2D centered dataset by our single chosen eigenvector. This projects the 2D points flat onto the new 1D axis.

Gemini Notebook

# Step 5 (cont): Executing the Transformation

$$PC1_1 = [-4,\ 2.5] \times \begin{bmatrix} -0.557 \\ 0.830 \end{bmatrix}^T = \mathbf{4.303}$$

$$PC1_2 = [0,\ -4.5] \times \mathbf{v}_1^T = \mathbf{-3.735}$$

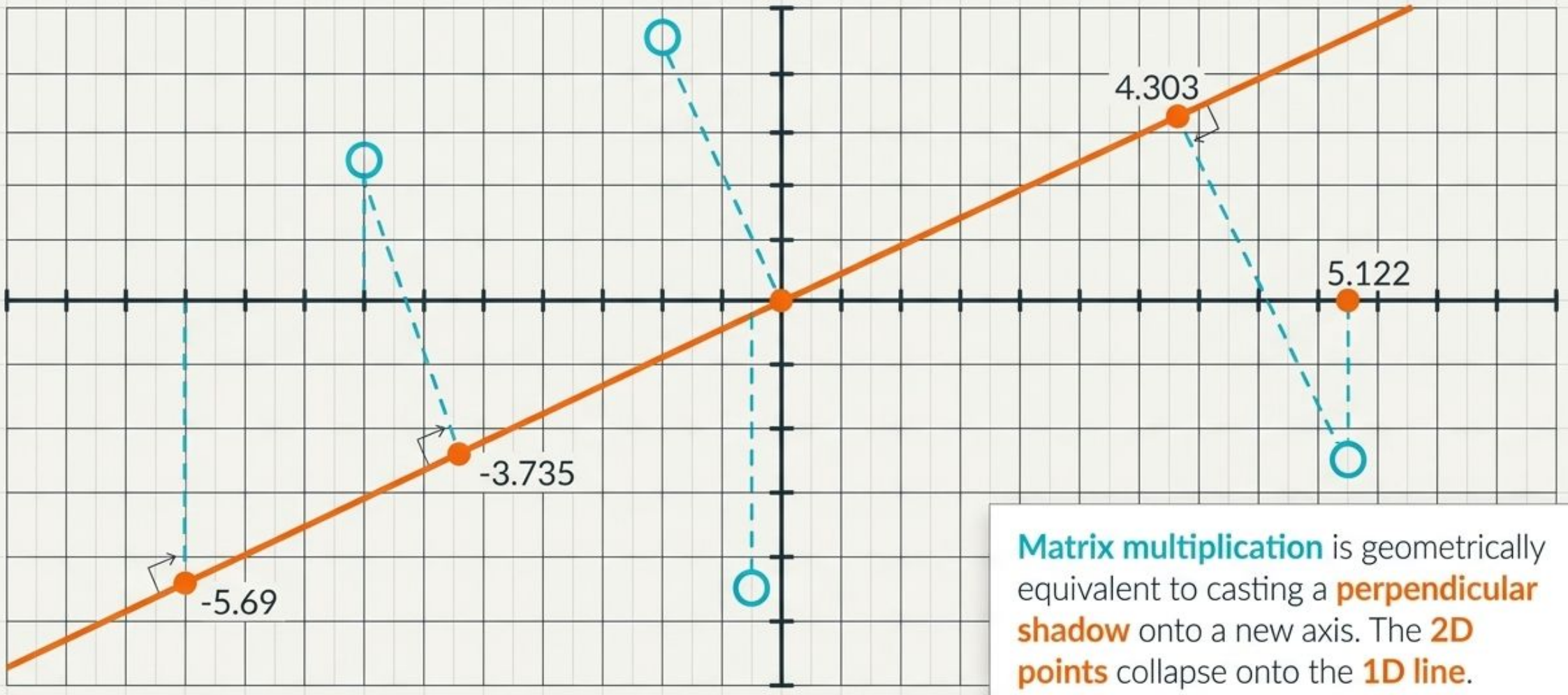
$$PC1_3 = [5,\ -3.5] \times \mathbf{v}_1^T = \mathbf{-5.69}$$

$$PC1_4 = [-1,\ 5.5] \times \mathbf{v}_1^T = \mathbf{5.122}$$

New 1D Dataset = [ **4.303, -3.735, -5.69, 5.122** ]

Gemini Notebook

# The Geometric Drop




**Matrix multiplication** is geometrically equivalent to casting a **perpendicular shadow** onto a new axis. The **2D points** collapse onto the **1D line**.

Gemini Notebook

# Dimensionality Reduction Achieved

Original Raw Data

| X | Y |
|---|---|
| 4 | 11 |
| 8 | 4 |
| 13 | 5 |
| 7 | 14 |

**2 Dimensions.**
**Messy correlations.**

Transformed PC1 Data

| PC1 |
|---|
| 4.303 |
| -3.735 |
| -5.69 |
| 5.122 |

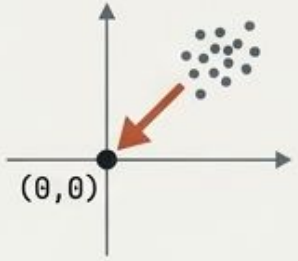
**1 Dimension.**
**Maximum variance retained.**

Conclusion: We successfully compressed two arbitrary coordinates into a single scalar value per data point, mathematically preserving ~82% of the structure.

Gemini Notebook

# The PCA Pipeline: From Raw Data to Compressed Signal

## 1. Center Data



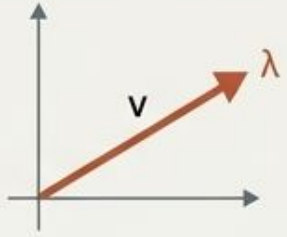

Shift data points so the statistical mean rests exactly at zero.

## 2. Covariance Matrix

$$\begin{bmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) \\ \text{cov}(x_2, x_1) & \text{var}(x_2) \end{bmatrix}$$

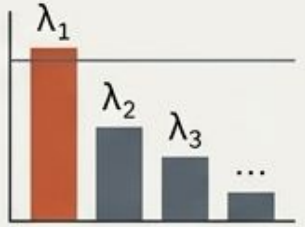
Calculate variance and the numerical relationships between features.

## 3. Eigen-decomposition



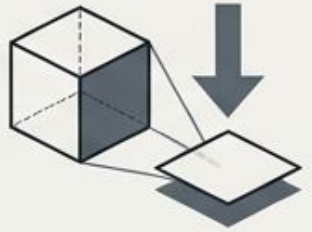

Calculate eigenvalues and eigenvectors of the covariance matrix.

## 4. PC Selection




Sort by highest eigenvalues. Select the top k components to retain ~95% variance.

## 5. Transformation

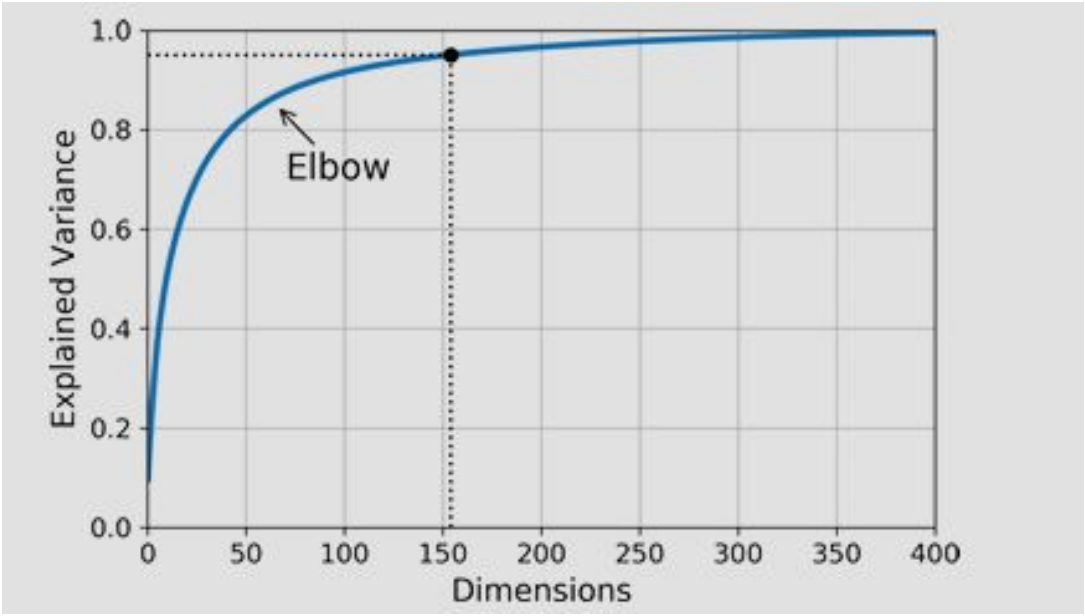


Multiply centered data by selected eigenvectors ($\mathbf{X}_{\text{d-proj}} = \mathbf{X} \cdot \mathbf{W}_d$).

Gemini Notebook

# Choosing the Right Number of Dimensions

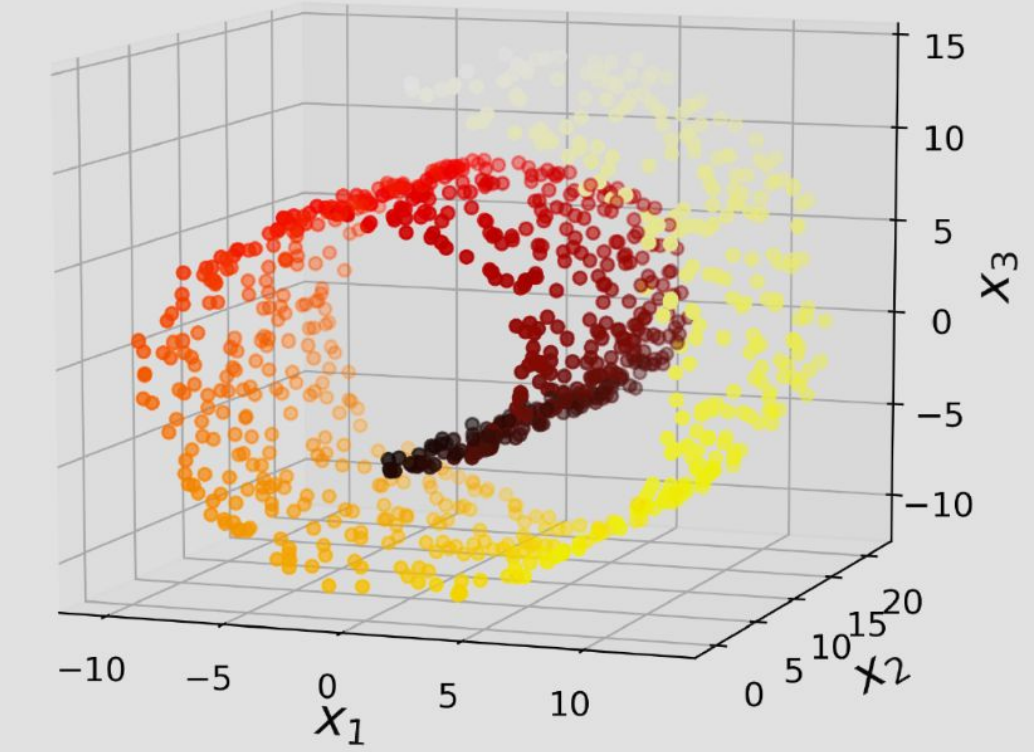
- 
- Choose $k$ such that ~95% of variance is retained.



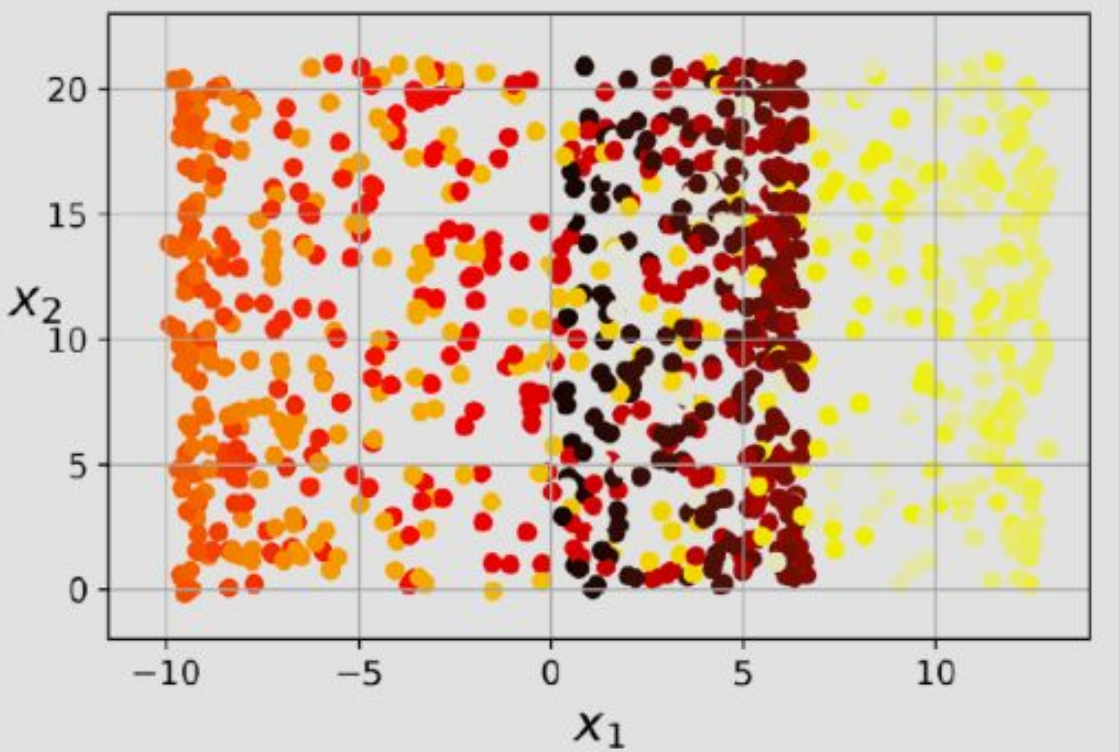
# The Limits of Linear Projection

PCA assumes data lies on a flat hyperplane. When high-dimensional data is twisted, linear projection crushes distant points on top of each other, destroying the structure.

## The Swiss Roll Manifold




## The Projection Failure



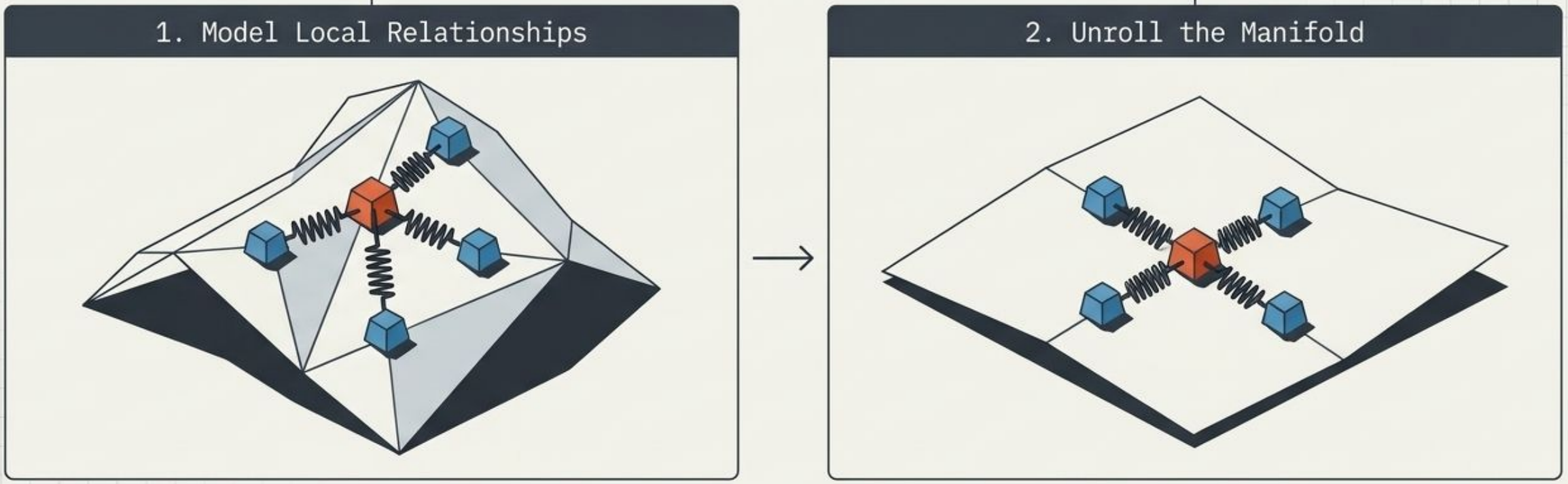

Takeaway: A simpler decision boundary is not always found by simply flattening the space globally.

Gemini Notebook

# Manifold Learning: Locally Linear Embedding (LLE)

Instead of preserving global distances, LLE focuses solely on preserving local neighborhood structures to unroll the manifold.

Origami Unfolding




**x_i** is modeled as a linear function of its neighbors.

**Takeaway:** LLE unrolls twisted data by ensuring that immediate neighbors remain exactly the same during the flattening process.

Gemini Notebook

# How LLE Works?

**For** each training instance :

The algorithm identifies its closest neighbors;

Reconstruct as a linear function of these neighbors;

- **Step 1:** Linearly modeling local relationships

$$\widehat{W} = \underset{W}{argmin} \sum_{i=1}^{m} \left( x^i - \sum_{j=1}^{m} w_{i,j} x^j \right)^2$$

$$subject\ to \begin{cases} w_{i,j} = 0, if\ x^j\ is\ not\ neighbor\ of\ x^i \\ \sum_{j=1}^{m} w_{i,j} = 1, \qquad for\ i = 1,2,\dots,m \end{cases}$$

- **Step 2:** Reducing dimensionality while preserving relationships

$$\hat{Z} = \underset{Z}{argmin} \sum_{i=1}^{m} \left( z^i - \sum_{j=1}^{m} w_{i,j} z^j \right)^2$$

# The Structural Trade-Off: Global vs. Local

You cannot perfectly preserve both the macro-shape of the data and its micro-neighborhoods when compressing dimensions. You must choose what matters.

## Global Variance

Prioritizes macro-distances and total system compression.

Supported Algorithms:

PCA, MDS

## Local Structure

Prioritizes immediate neighbors and unrolling non-linear manifolds.

Supported Algorithms:

LLE, t-SNE

Gemini Notebook

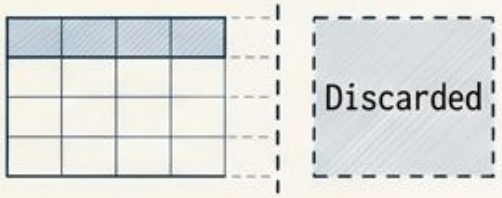
# The Master's Dilemma: Selection vs. Extraction

## Feature Selection

- **Interpretability: High**

| | Age | Income |
|---|---|---|
| ... | ... | ... |
| ... | ... | ... |

- **Information Loss: Abrupt**



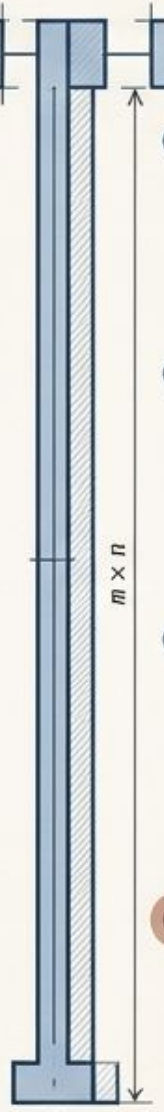

Drops entire columns of data

- **Data Transformation: None**

Original Values → Retained

Original values retained

- **Best for:**
  Medical diagnosis, financial auditing, strict regulatory environments

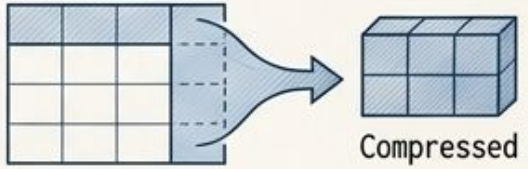



## Feature Extraction (PCA)

- **Interpretability: Zero**

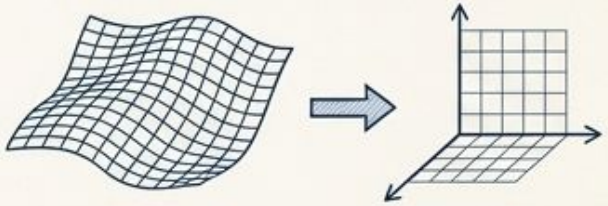
$$V_{\cdot}[\nu_{\iota}, \psi] = \left[\frac{1}{m}\sum_{i=1}^{N} W_i\right] \Rightarrow \begin{bmatrix} PC1 \\ PC2 \end{bmatrix}$$

- **Information Loss: Minimal/Controlled**




Blends data to preserve variance

- **Data Transformation: Total**



Transforms the entire mathematical space

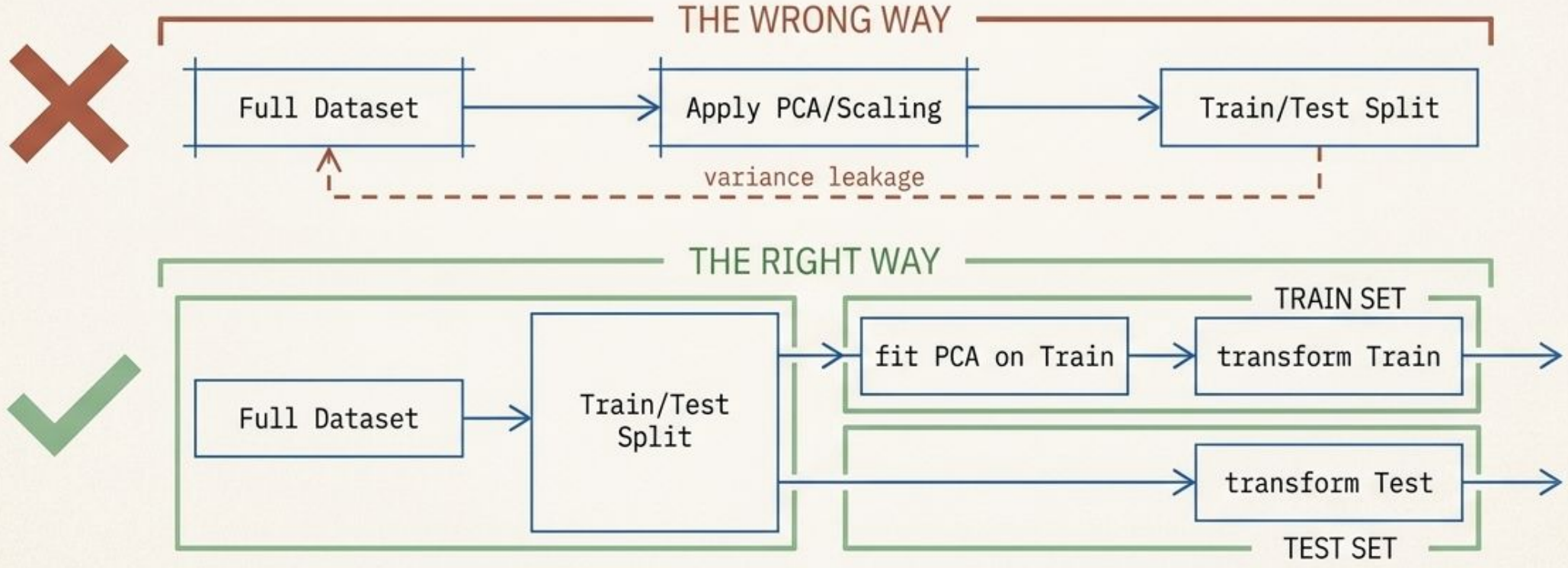
- **Best for:**
  Image compression, signal processing, deep learning pipelines

Gemini Notebook

# The Contamination Trap: Avoiding Data Leakage

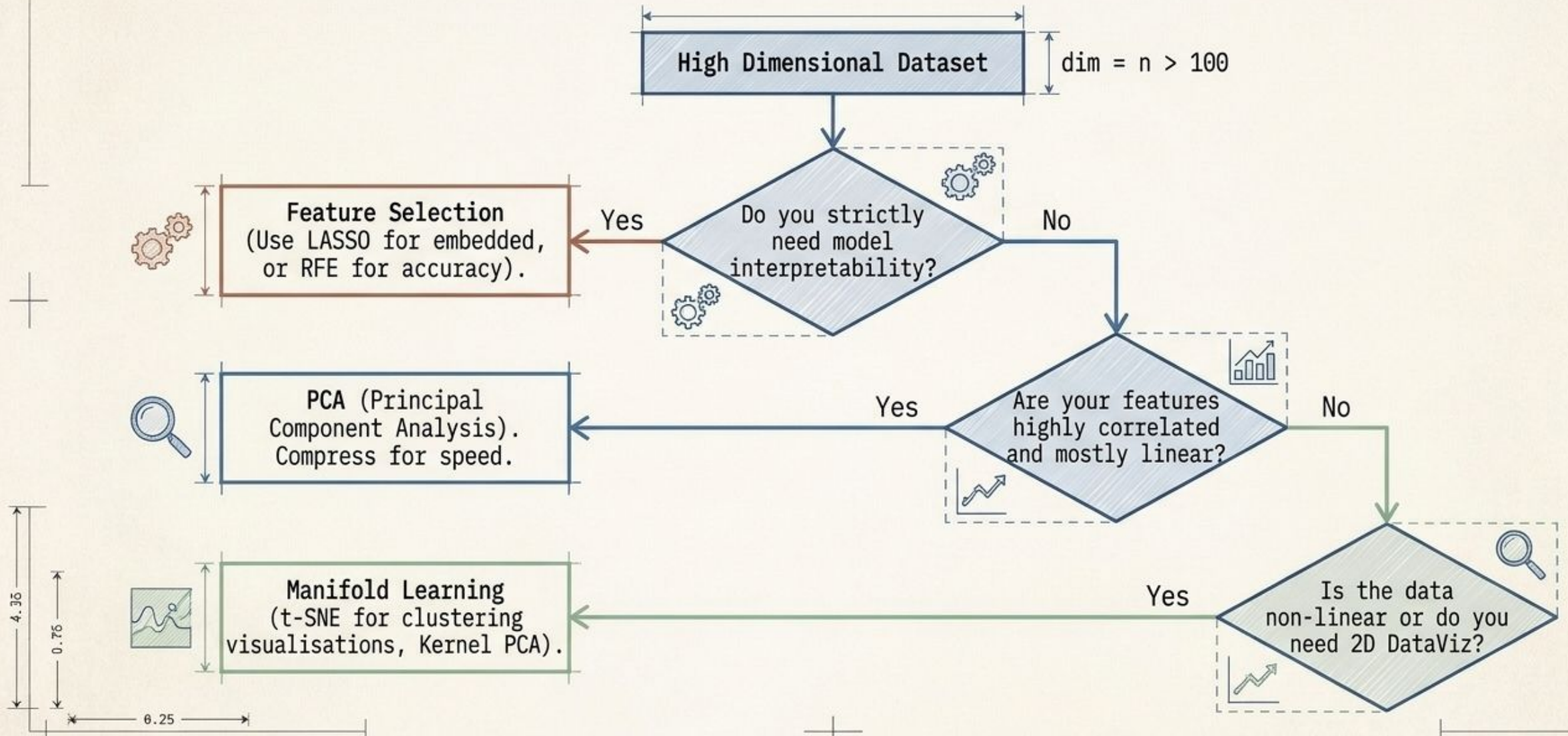
**Golden Rule:** **NEVER** run **Feature Selection** or **Feature Extraction** on your **entire dataset** before splitting.

Doing so allows the model to peek at the test data's distribution, resulting in catastrophic overfitting and false accuracy scores. Always strictly isolate the **Test Set** first.




Gemini Notebook

# The Dimensionality Decision Tree



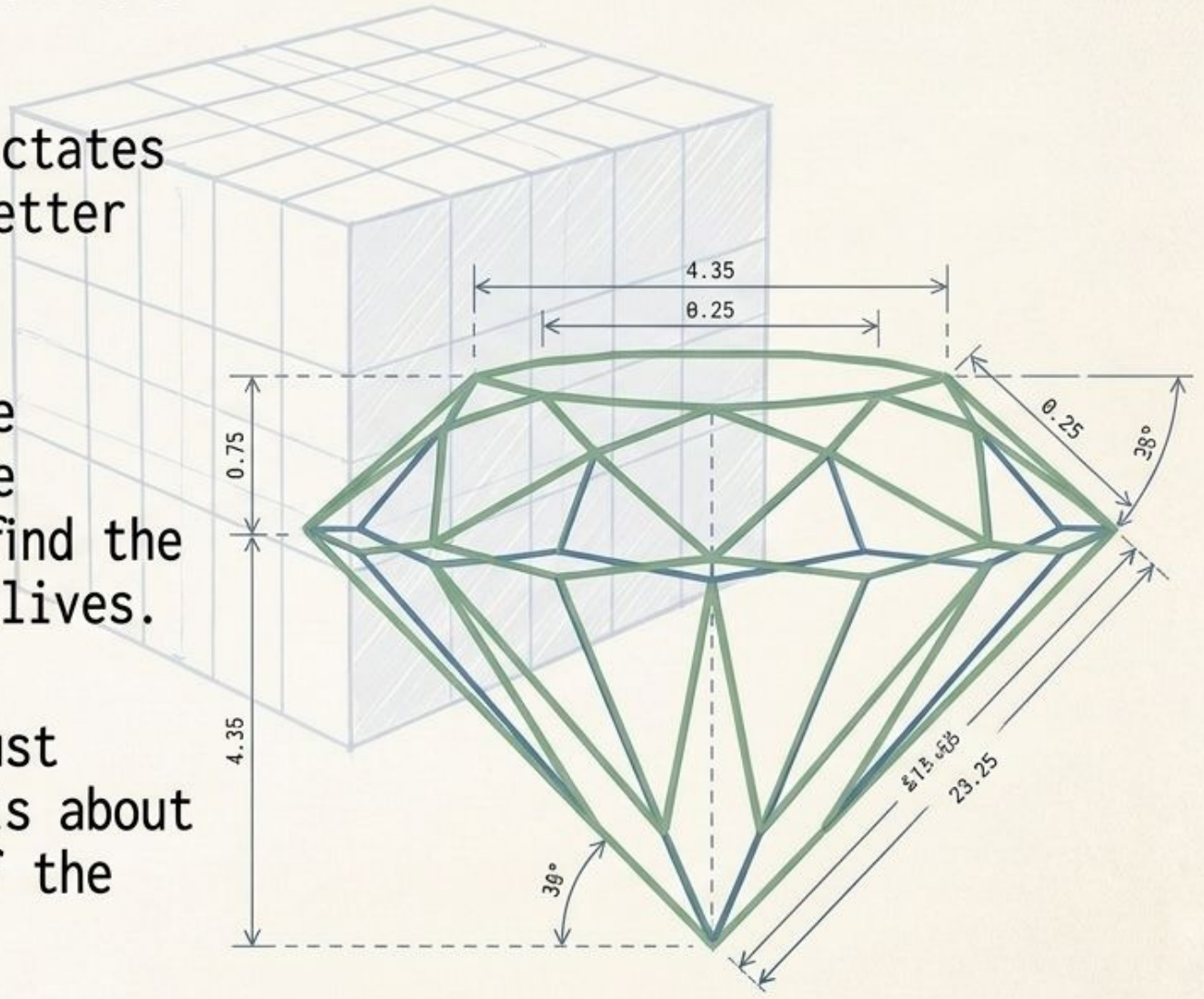

Gemini Notebook

# The Ultimate Essence

The **curse of dimensionality** dictates that more data is not always better data.

Whether you chip away the noise (**Selection**) or recast the space (**Extraction**), your goal is to find the **true manifold** where the **signal** lives.

Great machine learning isn’t just about training algorithms; it is about engineering the **architecture** of the data itself.




Gemini Notebook

# The Dimensionality Reduction Decision Engine

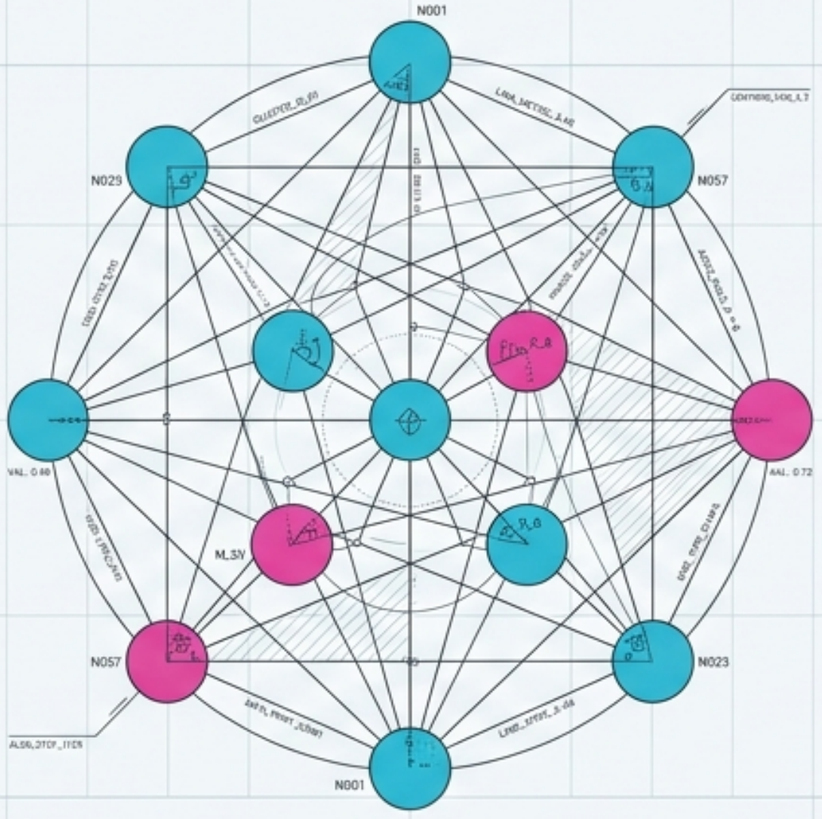
| Algorithm | Approach Type | Primary Strength | Optimal Use Case |
|---|---|---|---|
| PCA | Linear Projection | Global variance & data compression | General purpose feature reduction. |
| LLE | Manifold Learning | Unrolling twisted manifolds | Preserving local relationships. |
| t-SNE | Manifold Learning | Visualizing complex clusters | High-dimensional data visualization. |
| UMAP | Manifold Learning | Fast execution & scalable | Modern alternative to t-SNE. |
| Autoencoders | Neural Networks | Nonlinear dimensionality reduction | Complex image or deep learning data. |
| MDS | Distance Matrix | Preserving exact instance distances | Mapping relative similarities. |
| LDA | Supervised Projection | Class separation during training | Maximizing distance between known classes. |

Select the tool based on the specific geometric assumptions of your dataset.

Gemini Notebook

# Unsupervised Learning Architecture

## A Practitioner’s Playbook for Clustering and Association Mechanics




Gemini Notebook

# Unsupervised Learning

No labels—only features. The goal is to find hidden structures in raw data.

## Clustering

Grouping similar instances.

- **Customer Segmentation** — `technical`
- **Anomaly Detection** — `outlier_data`
- **Image Segmentation** — `maga_segm`

## Association

Finding relationships between variables.

- **Market Basket Analysis** — `show.on_gets`
- **Recommendation Systems** — `recommends`

Gemini Notebook

**Step 1: Initialize**

Randomly drop *K* centroids into the data space.

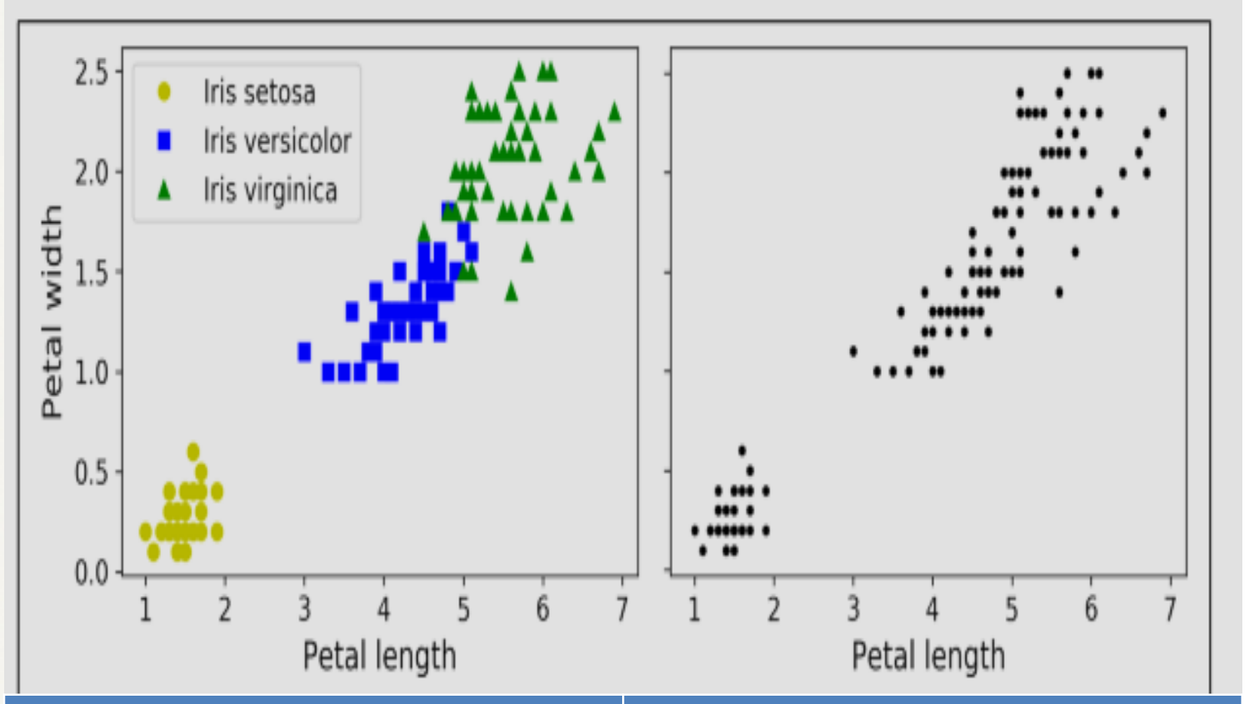
**Step 2: Assign**

Snap each instance to the nearest centroid.

**Step 3: Update**

Move centroids to the mathematical mean of assigned instances.

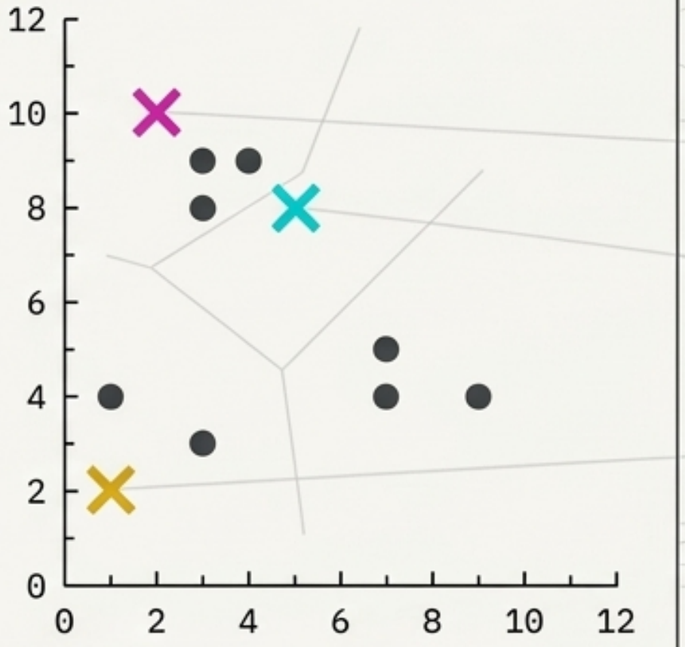
`Repeat until convergence.`




Classification | Clustering

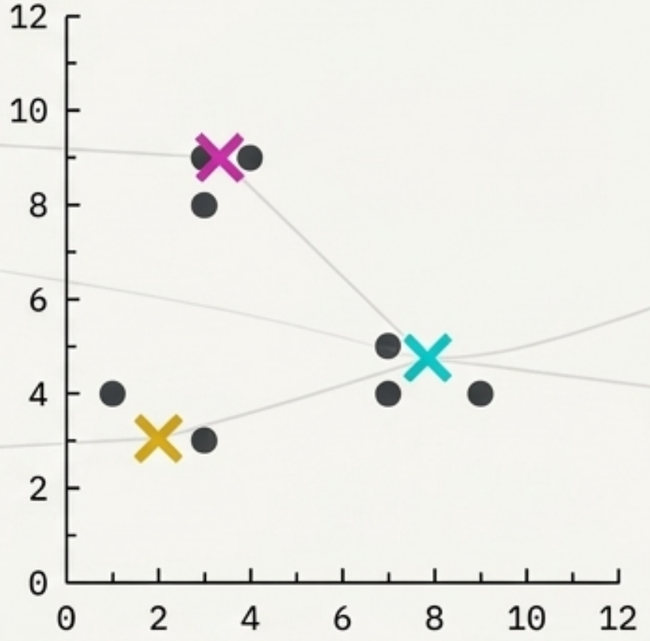
# K-Means Mechanics: The Iteration Tracker

## Iteration 1: Initialization



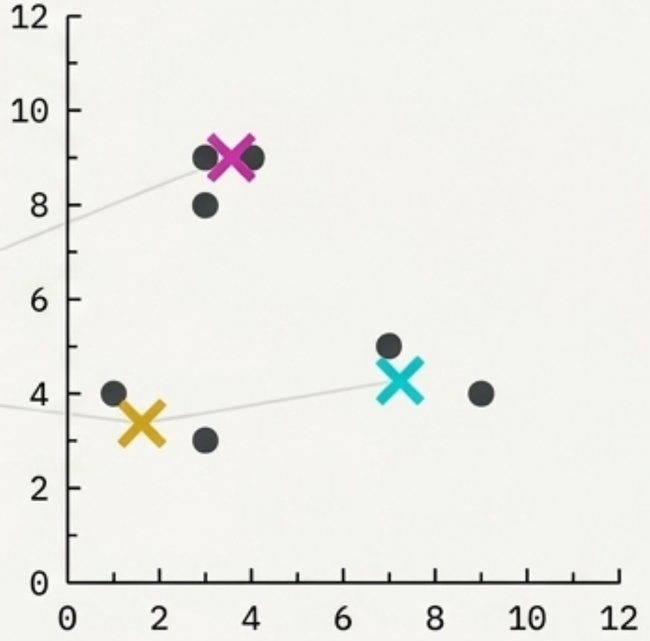

Centroids dropped randomly. Boundaries drawn based on pure proximity.

## Iteration 2: Re-computation




Centroids shift to the mathematical mean of their assigned points.

## Iteration 3: Convergence




Assignments no longer change. Final clusters locked in.

Gemini Notebook

# Centroid Initialization Methods

## 01. Multiple Runs Strategy

### Random Initialization

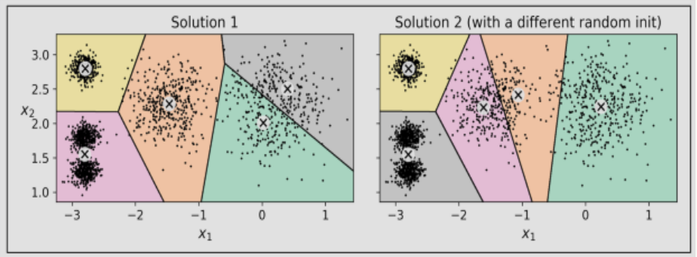
Run the K-Means algorithm multiple times with different random centroid initializations to prevent sub-optimal local convergence.

**Key Challenge**

Standard initialization can lead to poor clusters depending on initial random locations.

**Solution**

Evaluate all generated models and keep only the best overall clustering solution.




## 02. Inertia Metric

### Performance Evaluation

**Use "the model's inertia" as a quantitative performance metric to select the best model.**

**Inertia Definition:**

**Sum of squared distances of samples to their nearest centroid.**

Cost Function Formula

$$J = \sum_{i=1}^{m} \sum_{k=1}^{K} r_{ik} ||x_i - \mu_k||^2$$

where $r_{ik} = 1$ if $x_i$ is in cluster $k$, else 0

## 03. Selection Objective

### Optimization Goal

Inertia acts as a cost function that must be minimized across iterations.

# MINIMIZE

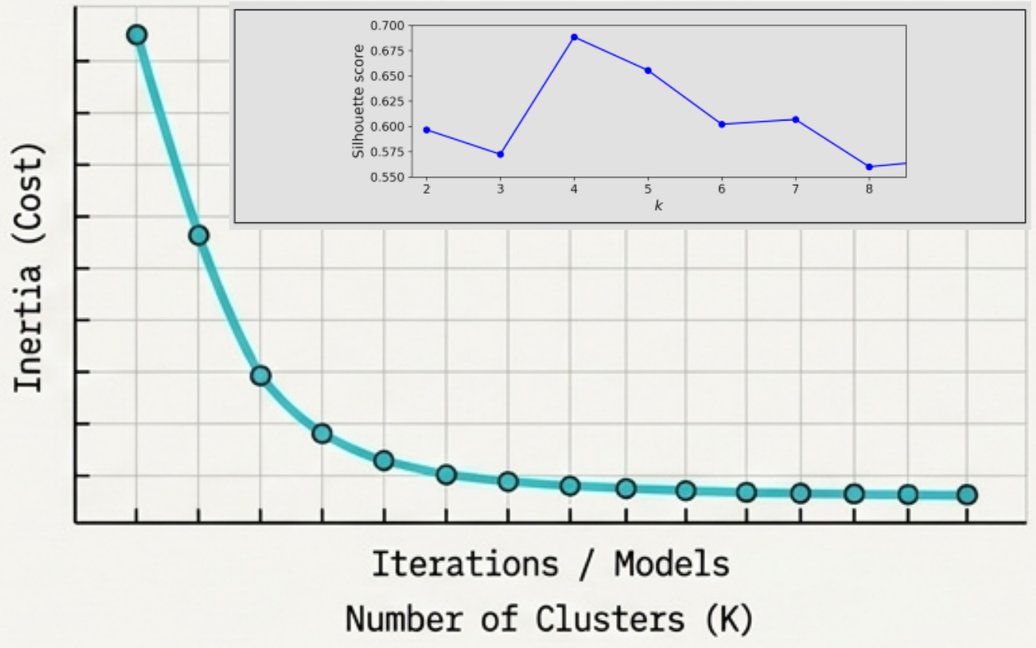
**Lowest Inertia = Best Cluster Quality**

**Final Decision Rule**

Select the model with the lowest inertia out of all random initializations.

# Model Evaluation: Measuring Cluster Quality

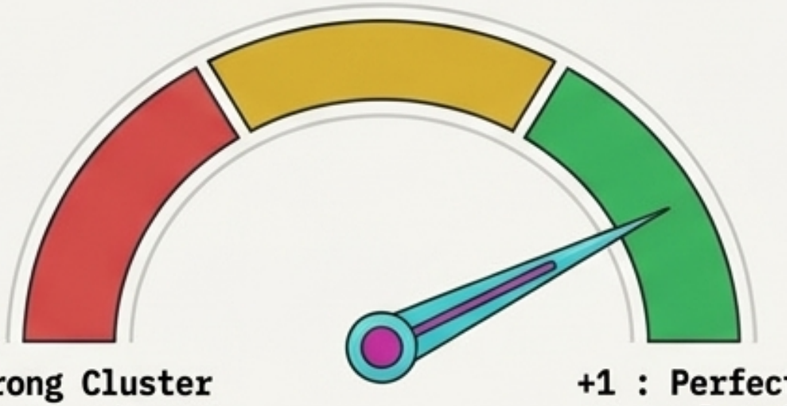
## Inertia Metric




Sum of squared distances to the nearest centroid.
Goal: Select the model with the lowest inertia across multiple random initializations.

## The Silhouette Gauge

**0 : Borderline**
(Instance is close to a boundary)



**-1 : Wrong Cluster**
(Instance assigned incorrectly)

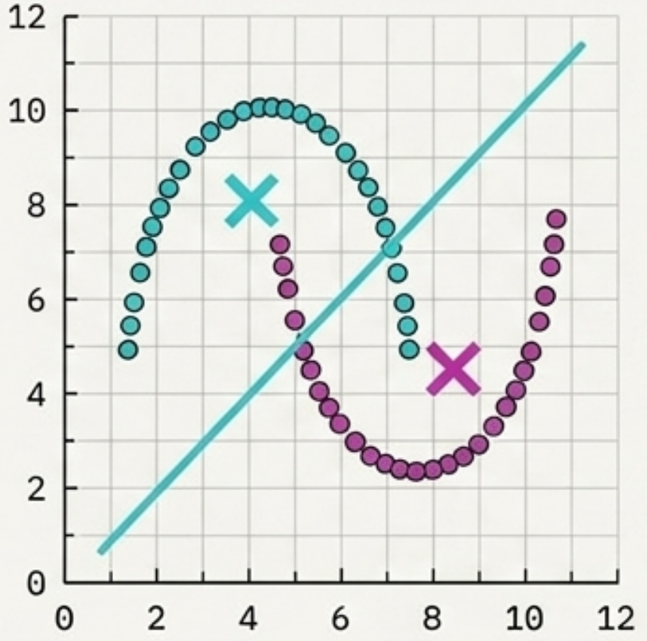
**+1 : Perfectly Isolated**
(Well inside cluster)

**Silhouette = (b - a) / max(a, b)**
where **a** is mean intra-cluster distance and **b** is mean nearest-cluster distance.

Gemini Notebook

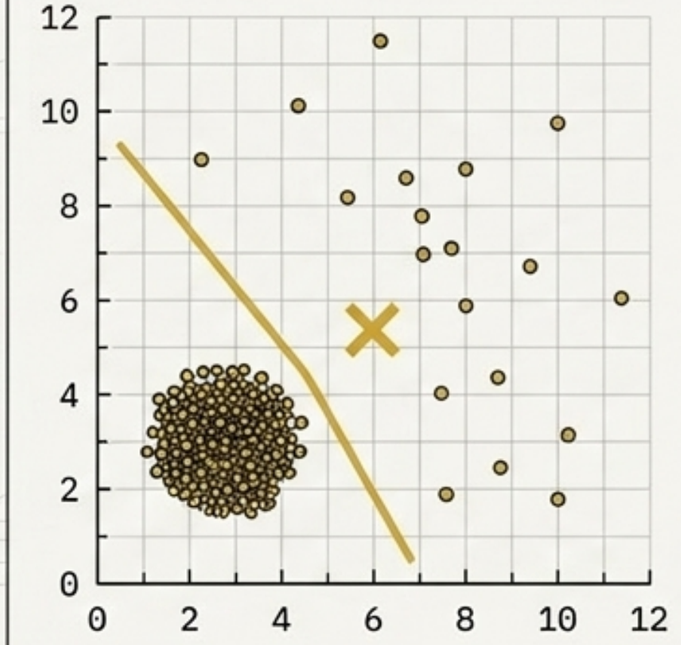
# The Limitations of K-Means

## The Shape Problem




Fails on non-spherical clusters. K-Means assumes spherical groupings.

## The Density Problem




Struggles with varying sizes and densities.

## The K-Value Assumption

Requires predefined K. You must explicitly know the number of clusters before you begin.

Gemini Notebook

# Problem Setup | K-Means Clustering Task

**Objective:**
Cluster the following 8 two-dimensional points into **K = 3 clusters**.

## Input Data Points (x, y)

**A1 (2, 10)**
x = 2, y = 10

**A2 (2, 5)**
x = 2, y = 5

**A3 (8, 4)**
x = 8, y = 4

**A4 (5, 8)**
x = 5, y = 8

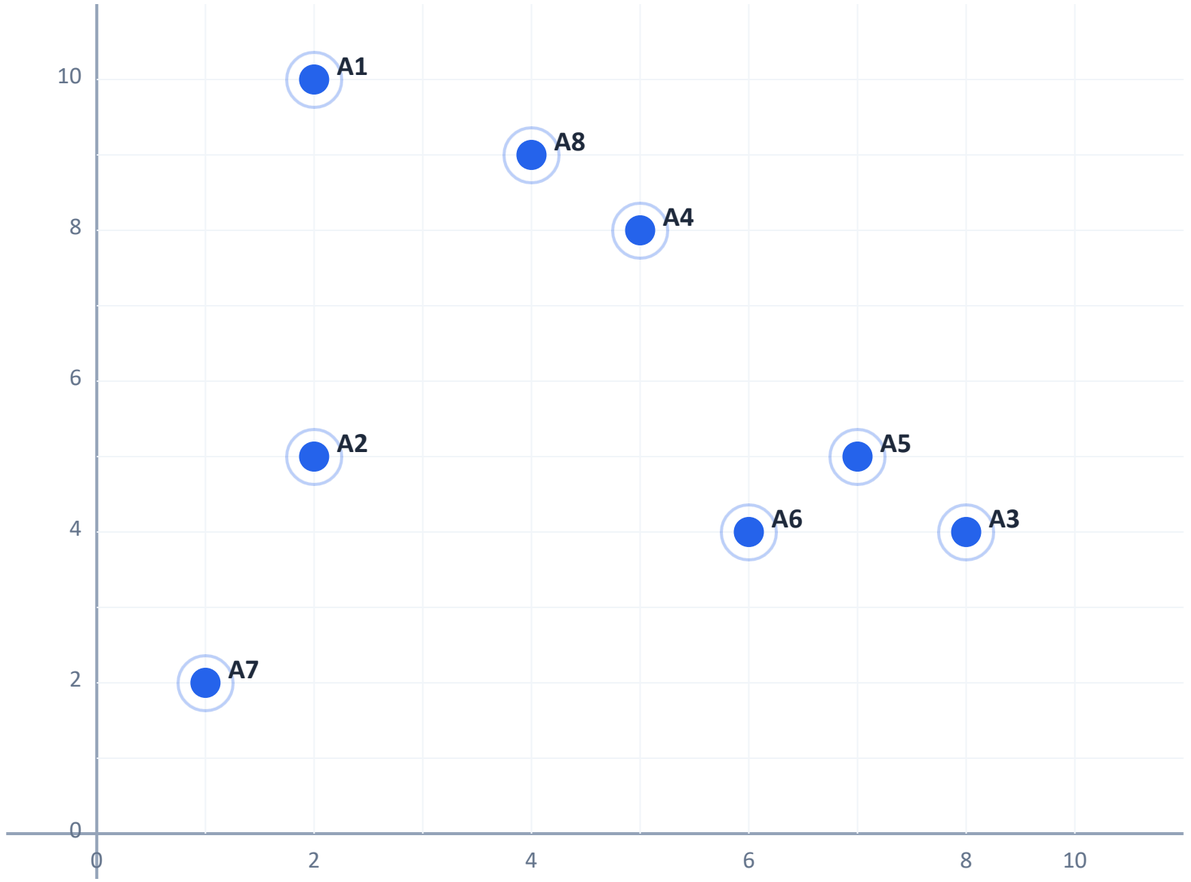
**A5 (7, 5)**
x = 7, y = 5

**A6 (6, 4)**
x = 6, y = 4

**A7 (1, 2)**
x = 1, y = 2

**A8 (4, 9)**
x = 4, y = 9

## Spatial Distribution (2D Coordinate Space)



# Solution | K-Means Initial Setup

**Number of Clusters: $k = 3$**

## Initial Cluster Centers (Centroids)

**Centroid 1**
$m1 = (2, 10)$

**Centroid 2**
$m2 = (5, 8)$

**Centroid 3**
$m3 = (1, 2)$

## Distance Function (Manhattan Distance / L1 Norm)

Defined between two points **a = (x1, y1)** and **b = (x2, y2)**:

$$\rho(a, b) = |x2 - x1| + |y2 - y1|$$

# Iteration 1 – Cluster Assignment | K-Means Algorithm

## Clusters Formed

**CLUSTER 1**

**{ A1 }**

Centered at m1 = (2, 10)

**CLUSTER 2**

**{ A3, A4, A5, A6, A8 }**

Centered at m2 = (5, 8)

**CLUSTER 3**

**{ A2, A7 }**

Centered at m3 = (1, 2)

## Distance Calculations & Assigned Clusters

| Point | Coords | Dist Mean 1 (2, 10) | Dist Mean 2 (5, 8) | Dist Mean 3 (1, 2) | Cluster |
|---|---|---|---|---|---|
| A1 | (2, 10) | 0* | 5 | 9 | 1 |
| A2 | (2, 5) | 5 | 6 | 4* | 3 |
| A3 | (8, 4) | 12 | 7* | 9 | 2 |
| A4 | (5, 8) | 5 | 0* | 10 | 2 |
| A5 | (7, 5) | 10 | 5* | 9 | 2 |
| A6 | (6, 4) | 10 | 5* | 7 | 2 |
| A7 | (1, 2) | 9 | 10 | 0* | 3 |
| A8 | (4, 9) | 3 | 2* | 10 | 2 |

# Iteration 1 – Update Centroids | K-Means Algorithm

**Re-compute cluster centers (means) by calculating the average of all points in each cluster.**

### CLUSTER 1

## { A1 }

Only one point **A1(2, 10)** belongs to Cluster 1. The mean remains unchanged.

**m1 = (2, 10)**

### CLUSTER 2

## { A3, A4, A5, A6, A8 }

x = (8 + 5 + 7 + 6 + 4) / 5 = **6**

y = (4 + 8 + 5 + 4 + 9) / 5 = **6**

**m2 = (6, 6)**

### CLUSTER 3

## { A2, A7 }

x = (2 + 1) / 2 = **1.5**

y = (5 + 2) / 2 = **3.5**

**m3 = (1.5, 3.5)**

## Updated Cluster Centers (Centroids)

**Centroid 1**

**m1 = (2, 10)**

**Centroid 2**

**m2 = (6, 6)**

**Centroid 3**

**m3 = (1.5, 3.5)**

# Iteration 2 – Cluster Assignment | K-Means Algorithm

## Clusters Formed

**CLUSTER 1**

**{ A1, A8 }**

Centered at m1 = (2, 10)

**CLUSTER 2**

**{ A3, A4, A5, A6 }**

Centered at m2 = (6, 6)

**CLUSTER 3**

**{ A2, A7 }**

Centered at m3 = (1.5, 3.5)

## Distance Calculations & Assigned Clusters

| Point | Coords | Dist Mean 1 (2, 10) | Dist Mean 2 (6, 6) | Dist Mean 3 (1.5, 3.5) | Cluster |
|---|---|---|---|---|---|
| A1 | (2, 10) | 0* | 8 | 7 | 1 |
| A2 | (2, 5) | 5 | 5 | 2* | |
| A3 | (8, 4) | 12 | 4* | 7 | 2 |
| A4 | (5, 8) | 5 | 3* | 8 | |
| A5 | (7, 5) | 10 | 2* | 7 | 2 |
| A6 | (6, 4) | 10 | 2* | 5 | |
| A7 | (1, 2) | 9 | 9 | 2* | 3 |
| A8 | (4, 9) | 3* | 5 | 8 | |

# Iteration 2 – Update Centroids | K-Means Algorithm

### CLUSTER 1

Points assigned:

**{ A1, A8 }**

**Mean Calculation:**

```
X: (2 + 4) / 2 = 3.00
Y: (10 + 9) / 2 = 9.50
```

New Centroid:

**m1 = (3, 9.5)**

### CLUSTER 2

Points assigned:

**{ A3, A4, A5, A6 }**

**Mean Calculation:**

```
X: (8 + 5 + 7 + 6) / 4 = 6.50
Y: (4 + 8 + 5 + 4) / 4 = 5.25
```

New Centroid:

**m2 = (6.5, 5.25)**

### CLUSTER 3

Points assigned:

**{ A2, A7 }**

**Mean Calculation:**

```
X: (2 + 1) / 2 = 1.50
Y: (5 + 2) / 2 = 3.50
```

New Centroid:

**m3 = (1.5, 3.5)**

## Updated Cluster Centers Summary Iteration 2 Complete

**m1 = (3, 9.5) | m2 = (6.5, 5.25) | m3 = (1.5, 3.5)**

** Proceed to next iteration for re-assignment based on new centroid positions.*

# Iteration 3 – Cluster Assignment | K-Means Algorithm

## Clusters Formed

**CLUSTER 1**

**{ A1, A4, A8 }**

Centered at m1 = (3, 9.5)

**CLUSTER 2**

**{ A3, A5, A6 }**

Centered at m2 = (6.5, 5.25)

**CLUSTER 3**

**{ A2, A7 }**

Centered at m3 = (1.5, 3.5)

## Distance Calculations & Assigned Clusters

| Point | Coords | Dist Mean 1 (3, 9.5) | Dist Mean 2 (6.5, 5.25) | Dist Mean 3 (1.5, 3.5) | Cluster |
|---|---|---|---|---|---|
| A1 | (2, 10) | **1.5*** | 9.25 | 7 | 1 |
| A2 | (2, 5) | 5.5 | 4.75 | **2*** | 3 |
| A3 | (8, 4) | 10.5 | **2.75*** | 7 | 2 |
| A4 | (5, 8) | **3.5*** | 4.25 | 8 | 1 |
| A5 | (7, 5) | 8.5 | **0.75*** | 7 | 2 |
| A6 | (6, 4) | 8.5 | **1.75*** | 5 | 2 |
| A7 | (1, 2) | 9.5 | 8.75 | **2*** | 3 |
| A8 | (4, 9) | **1.5*** | 6.25 | 8 | 1 |

# Iteration 3 – Update Centroids | K-Means Algorithm

### CLUSTER 1

Points assigned:

**{ A1, A4, A8 }**

**Mean Calculation:**

```
X: (2 + 5 + 4) / 3 = 3.67
Y: (10 + 8 + 9) / 3 = 9.00
```

New Centroid:

**m1 = (3.67, 9)**

### CLUSTER 2

Points assigned:

**{ A3, A5, A6 }**

**Mean Calculation:**

```
X: (8 + 7 + 6) / 3 = 7.00
Y: (4 + 5 + 4) / 3 = 4.33
```

New Centroid:

**m2 = (7, 4.33)**

### CLUSTER 3

Points assigned:

**{ A2, A7 }**

**Mean Calculation:**

```
X: (2 + 1) / 2 = 1.50
Y: (5 + 2) / 2 = 3.50
```

New Centroid:

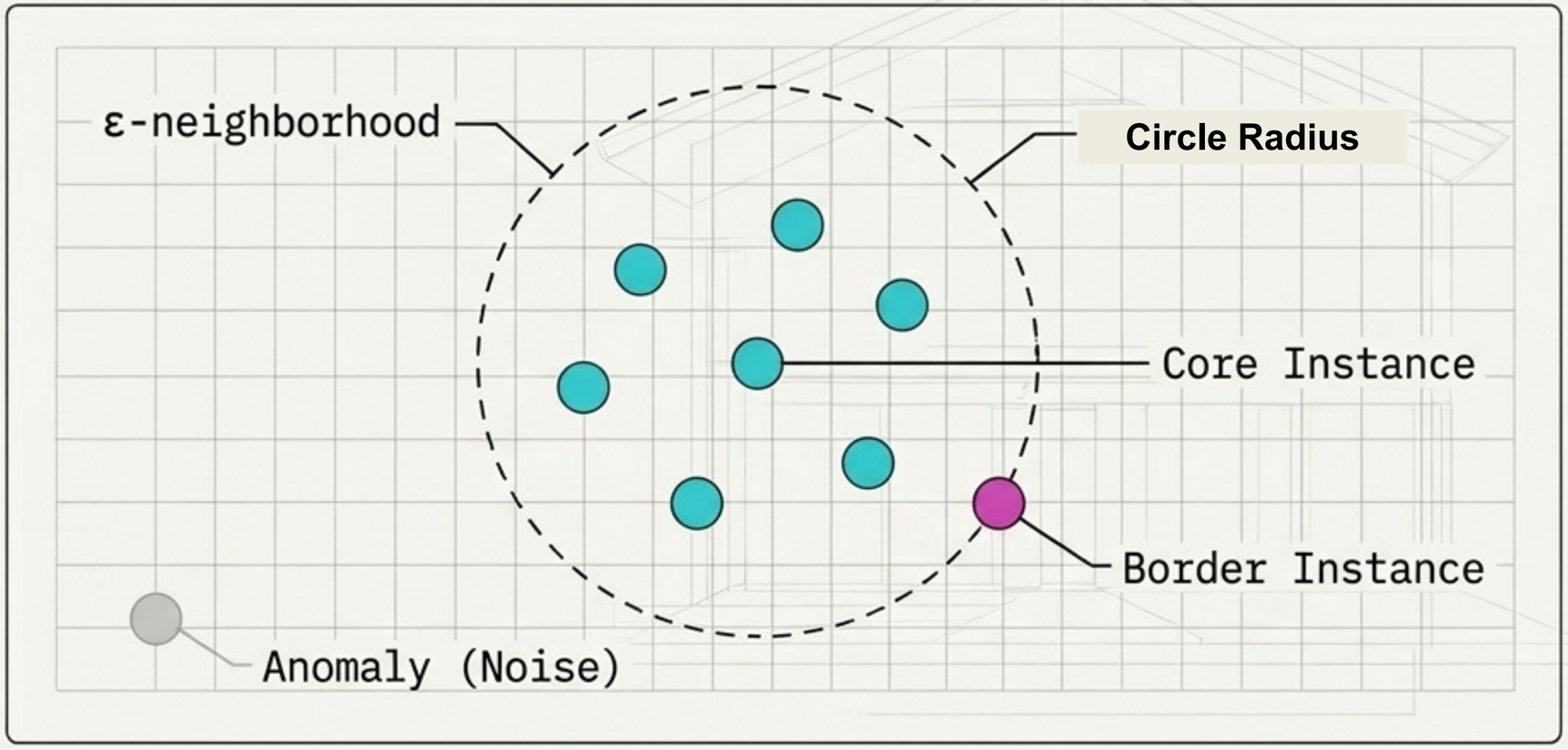
**m3 = (1.5, 3.5)**

## Updated Cluster Centers Summary Iteration 3 Complete

**m1 = (3.67, 9)** | **m2 = (7, 4.33)** | **m3 = (1.5, 3.5)**

** Repeat the process until the cluster assignments no longer change (convergence condition reached).*

# DBSCAN: Density-Based Spatial Clustering




## Hyperparameters

**ε (Epsilon):**
Maximum distance to be considered a neighbor.

**min_samples:**
Minimum points required to form a dense region.

Robust to outliers and discovers arbitrary shapes without a predefined K.
But requires O(m^2) memory if ε is large.

Gemini Notebook

# DBSCAN Algorithm | Density-Based Spatial Clustering

### 1. Neighborhood Definition

The algorithm counts how many instances are within distance **epsilon (ε)**. This region is the instance's **ε-neighborhood**.

### 2. Core Instance Identification

If an instance has at least **min_samples** in its ε-neighborhood (including itself), it is a **core instance**.

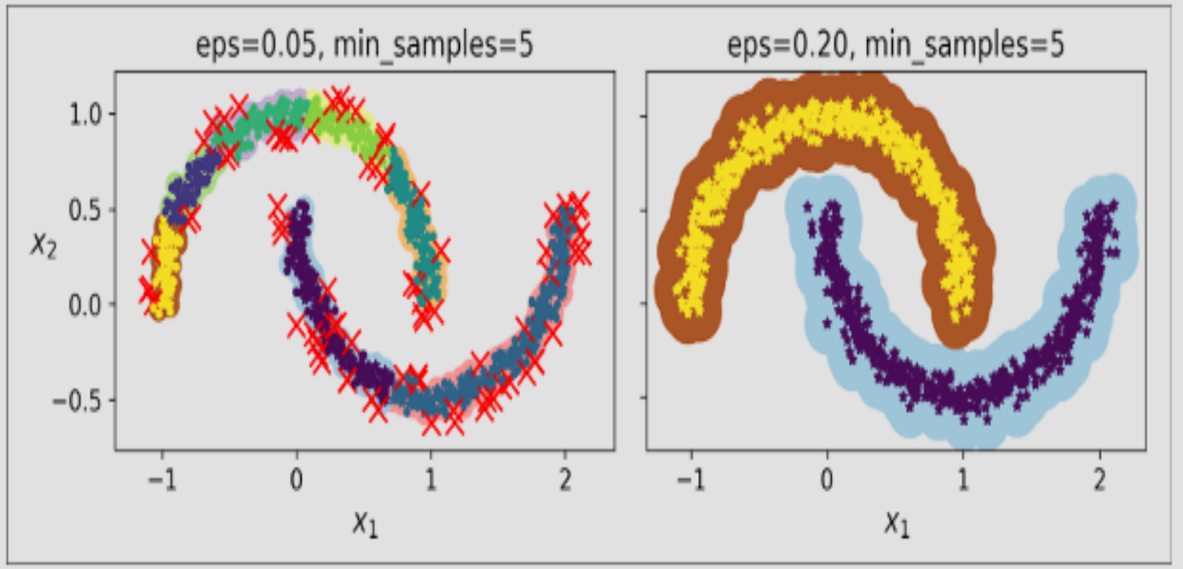
### 3. Cluster Formation

All instances located within the neighborhood of a core instance belong to the **same cluster**.

### 4. Outlier Detection

Any instance that is not a core instance and has no core instance in its neighborhood is designated as an **anomaly**.

**Impact of Hyperparameters (Epsilon & Min Samples)**



# Hierarchical Clustering: Dynamic Granularity

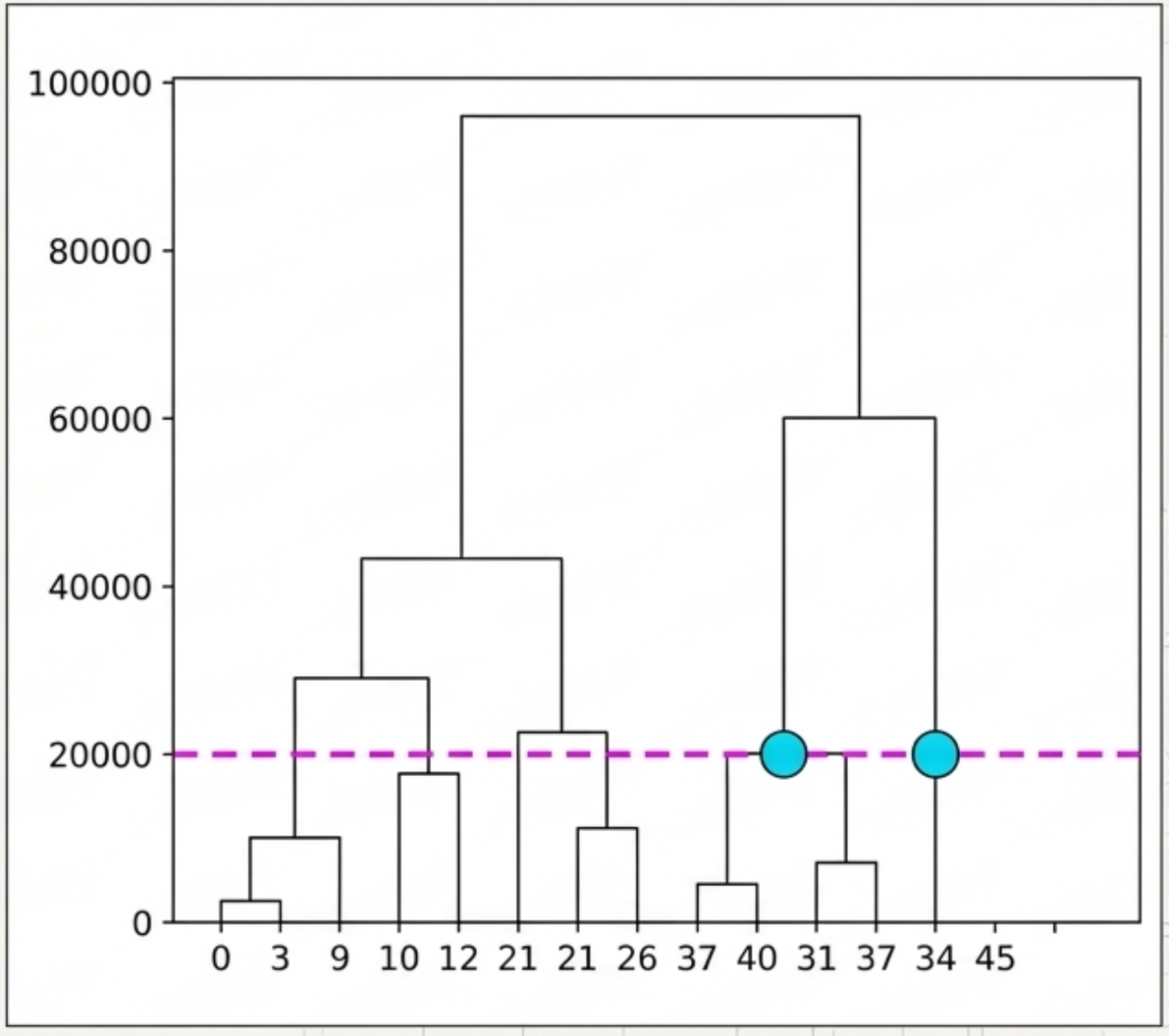
## Agglomerative Mechanics

1. Start with N clusters (every point is its own island).

2. Merge the closest two clusters (Average-linkage: average distance between all respective members).

3. Repeat until a single tree (Dendrogram) is formed.

**Key Insight:** Draw a horizontal threshold line. The number of vertical intersections equals your exact number of clusters (K).

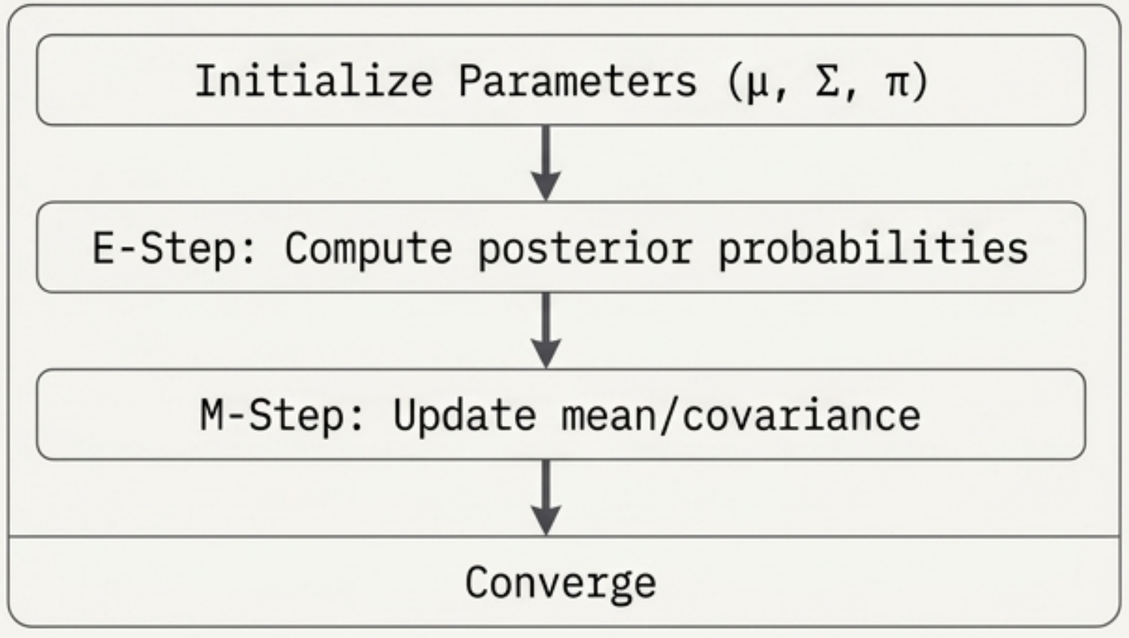



Gemini Notebook

# Gaussian Mixture Models: Probabilistic Soft Boundaries

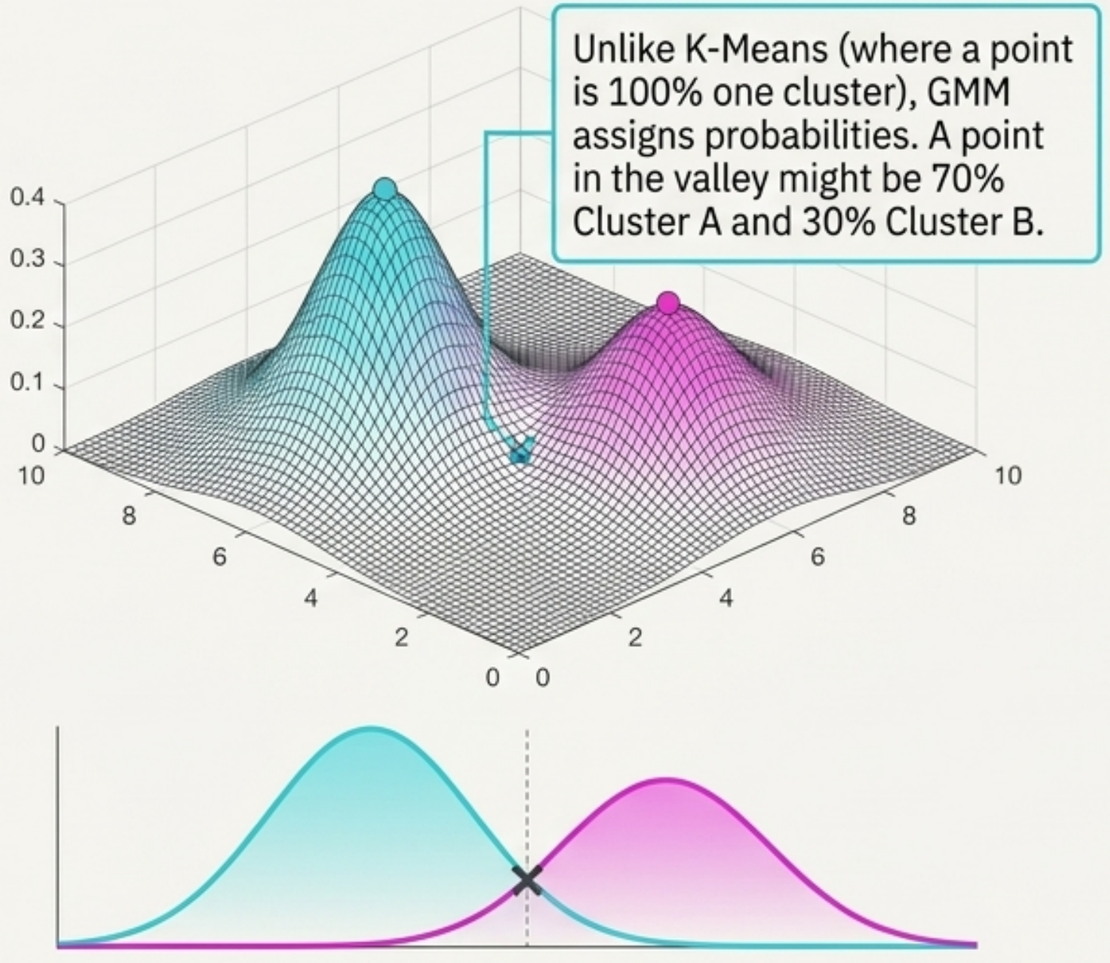
## Soft Clustering & EM

Real-world data is heterogeneous. GMM represents data as a weighted sum of multiple Gaussian distributions.




Mean vector (μ) controls center.
Covariance matrix (Σ) controls elliptical shape.

## The Probability Topography




Gemini Notebook

# Advantages & Limitations

## ADVANTAGES

### Handles Elliptical Clusters

Fits complex, non-spherical data distributions effectively.

### Soft Clustering Probabilities

Assigns probabilistic membership scores to data points.

### Flexible Covariance Structures

Allows full control over cluster shapes, volume, and orientation.

## LIMITATIONS

### Computationally Expensive

Requires iterative Expectation-Maximization (EM) calculations.

### Sensitive to Initialization

Poor initial parameters can cause convergence to local optima.

### High-Dimensional Data Struggles

Suffer from the curse of dimensionality and singular covariances.

Gemini Notebook

# Specialized Niche Algorithms

## BIRCH

**Massive Datasets**

Builds a tree structure to assign new instances without storing all data in memory.

## Mean-Shift

**Shifting Centers**

Places circles on instances, computes mean, shifts circle to mean.

**Drawback:** O(m^2) complexity. Not for large data.

## Affinity Propagation

**Voting System**

Instances vote for similar instances to be representatives.

**Drawback:** O(m^2) complexity.

## Spectral Clustering

**Dimensionality Reduction First**

Creates low-dimensional embedding via similarity matrix, then runs K-Means.

**Drawback:** Struggles with widely varying cluster sizes.

Gemini Notebook

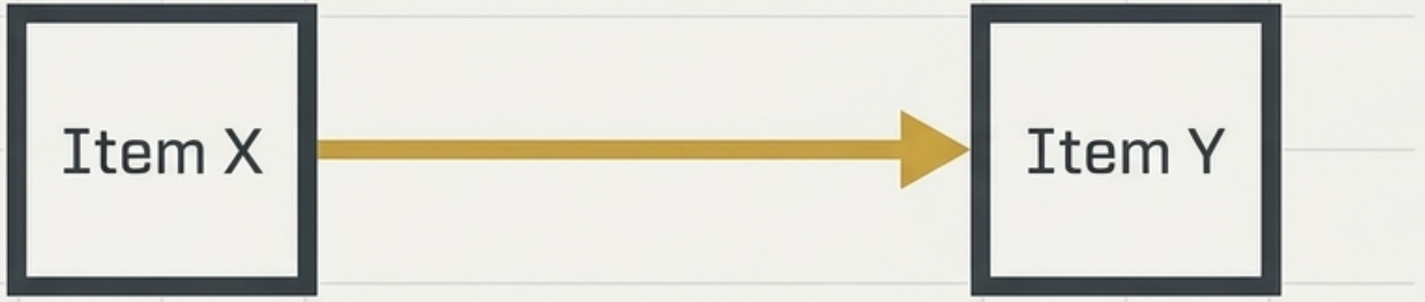
# The Clustering Toolkit Matrix

| | Pre-defined K? | Handles Outliers? | Shape Assumption | Key Strength |
|---|---|---|---|---|
| K-Means | Yes | Poorly | Spherical | Speed & Scale |
| DBSCAN | No | Excellent (Noise) | Arbitrary | Density Variations |
| Hierarchical | No | Fair | Taxonomic | Granularity Control |
| GMM | Yes (AIC/BIC) | Fair | Elliptical | Soft Probabilities |

Gemini Notebook

# Shifting Focus: Association Rule Mining

Moving from grouping instances to finding relationships between variables.




Market Basket Analysis: If a user purchases Item X (e.g., Bread), what is the probability they also purchase Item Y (e.g., Butter)? This technique is the mathematical engine behind targeted marketing and recommendation systems.

Gemini Notebook

Association Rule Mining

# Apriori Algorithm

### Core Concept

The **Apriori Algorithm** is a breadth-first search-based algorithm which calculates the support between items.

### Item Dependency

This support basically maps the dependency of one data item with another which can help us understand what data item influences the possibility of something happening to the other data item.

Example Context:

**Transaction data for an *AllElectronics* database**

| TID | Items |
|---|---|
| **T100** | {I1, I2, I5} |
| **T200** | {I2, I4} |
| **T300** | {I2, I3} |
| **T400** | {I1, I2, I4} |
| **T500** | {I1, I3} |
| **T600** | {I2, I3} |
| **T700** | {I1, I3} |
| **T800** | {I1, I2, I3, I5} |
| **T900** | {I1, I2, I3} |

Association Rule Mining

# Candidate Generation & Itemset Pruning

**Threshold Parameter**

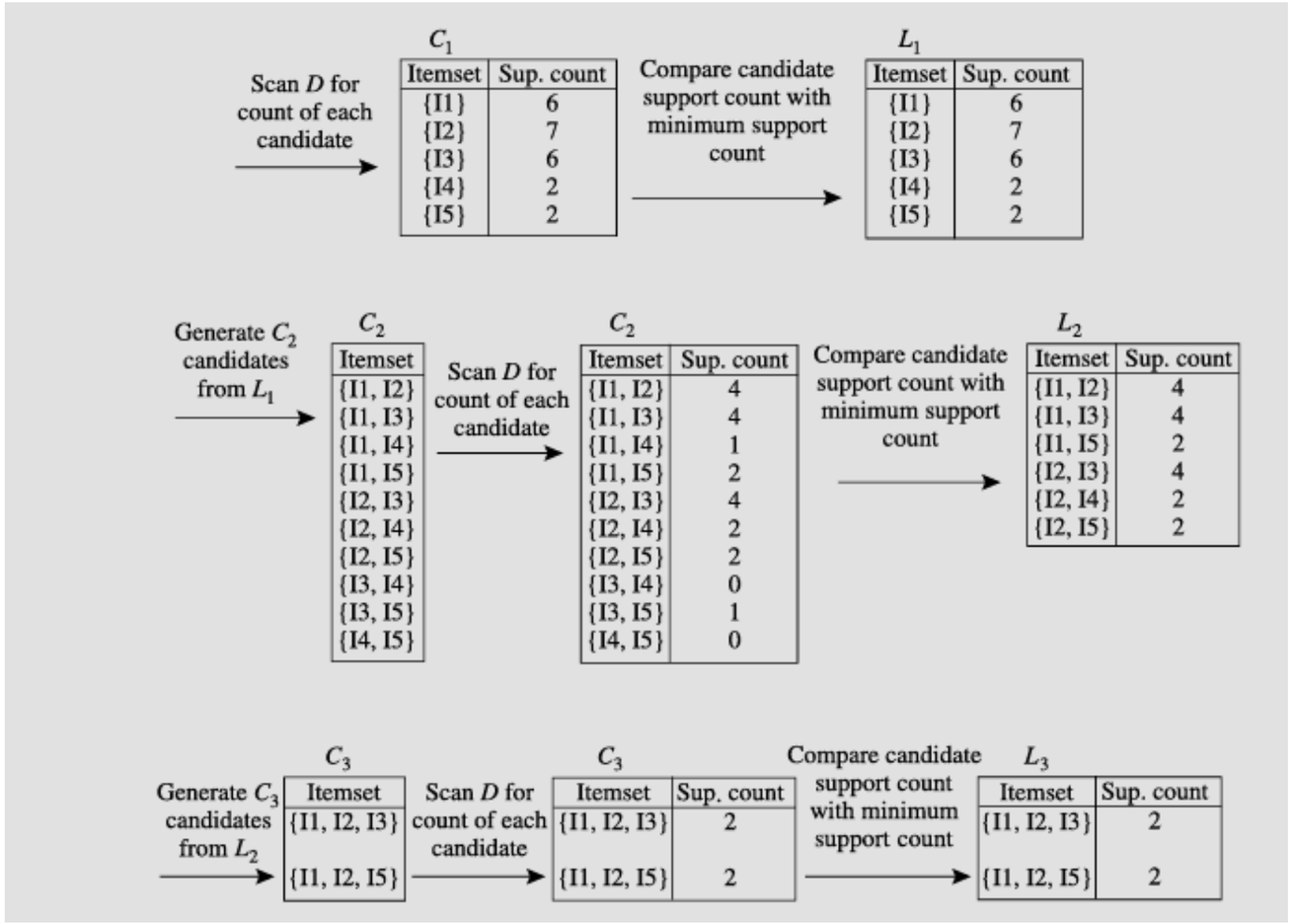
## Support Count = 2

Candidate itemsets with support count below this threshold (**< 2**) are pruned when moving from **Ck** to **Lk**.

**Relative Frequency**

## 2 / 9 = 22%

Calculated over **9 total transactions** (T100–T900). An itemset must appear in at least 22% of all transactions to be considered frequent.



Association Rule Mining

# Generating Association Rules from Frequent Itemsets

## Rule Generation Logic

### 1. Subset Extraction

For each frequent itemset l, generate all nonempty subsets of l.

### 2. Confidence Evaluation

For every nonempty subset s of l, output the rule **s ⇒ (l - s)** if:

#### Confidence Formula

confidence(A ⇒ B) = P(BIA) = $\frac{Support_Support(A \cup B)}{Support_Count(A)}$

Rule is valid if $\mathbf{confidence} \geq \mathbf{min_conf}$ threshold.

### Example Target

Frequent Itemset **X = {I1, I2, I5}**

Goal: Generate and evaluate all candidate association rules from X.

## Worked Example Execution

Nonempty subsets of X: {I1, I2}, {I1, I5}, {I2, I5}, {I1}, {I2}, {I5}

{I1, I2} ⇒ I5 | conf = 2/4 = 50%

{I1, I5} ⇒ I2 | conf = 2/2 = **100%** ✓

{I2, I5} ⇒ I1 | conf = 2/2 = **100%** ✓

I1 ⇒ {I2, I5} | conf = 2/6 = 33%

I2 ⇒ {I1, I5} | conf = 2/7 = 29%

I5 ⇒ {I1, I2} | conf = 2/2 = **100%** ✓

## Threshold Pruning Output

If **min_conf = 70%**, only rules 2, 3, and 6 are output because they meet the minimum confidence threshold to be considered strong.

# FP-Growth: The Tree-Based Alternative

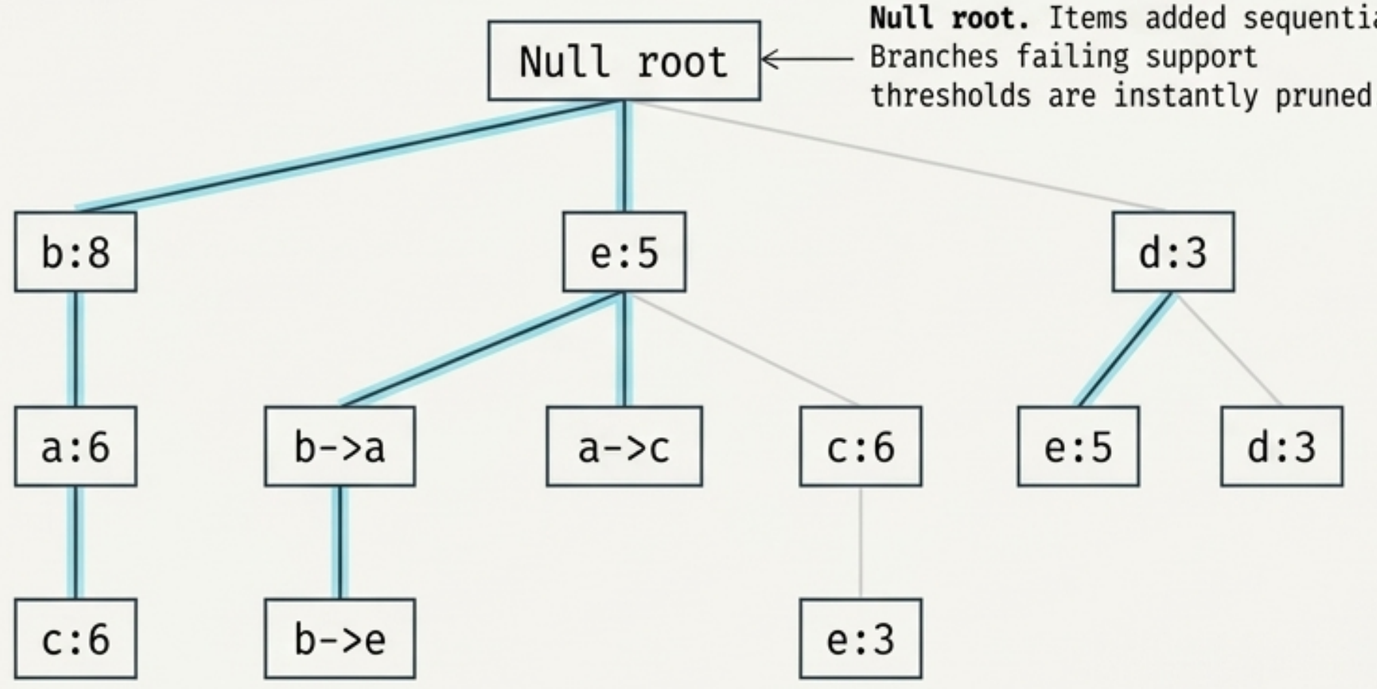
## The Sorting Engine

**Step 1:** Count single-item supports.

**Step 2:** Order frequent items descending: {b:8, a:6, c:6, e:5, d:3}.

**Step 3:** Sort each transaction by this global order.

## The FP-Tree Architecture




By compressing the database into an **FP-Tree**, the algorithm avoids the expensive repeated database scans required by **Apriori**.



Gemini Notebook

Association Rule Mining

# Problem Setup

| Transaction ID | Items |
|---|---|
| T1 | {a, b, c} |
| T2 | {b, c, d, e} |
| T3 | {a, b, d} |
| T4 | {a, c, e} |
| T5 | {a, b, c, e} |
| T6 | {b, e} |
| T7 | {a, b, c, d} |
| T8 | {a, b, e} |
| T9 | {b, c} |

## Configuration Parameters

Threshold Requirement

**Minimum Support (absolute) = 3**

Relative Threshold: min_sup = 33.3% (3 / 9 transactions)

Dataset Overview

- Total Transactions (|D|): **9**
- Unique Items Set: **{a, b, c, d, e}**

## Mining Objective

Identify all frequent itemsets that appear in at least 3 transactions across the database.

ASSOCIATION RULE MINING

# Count Single-Item Supports

| Item | Support Count | Min Support (≥3) Status |
|---|---|---|
| a | 6 | PASS |
| b | 8 | PASS |
| c | 6 | PASS |
| d | 3 | PASS |
| e | 5 | PASS |

## Sorting & Ordering Rules

THRESHOLD CRITERIA

All items meet minimum support threshold **(≥ 3)**.

ORDERING LOGIC

Order frequent items in **descending support**. Break ties lexicographically where needed.

RESULTING FREQUENT ITEM ORDER

Global Order List:

**{b:8, a:6, c:6, e:5, d:3}**

Association Rule Mining

# Sort Each Transaction

| Transaction ID | Original Items | Ordered Items (FP-Order) |
|---|---|---|
| T1 | {a, b, c} | **{b, a, c}** |
| T2 | {b, c, d, e} | **{b, c, e, d}** |
| T3 | {a, b, d} | **{b, a, d}** |
| T4 | {a, c, e} | **{a, c, e}** |
| T5 | {a, b, c, e} | **{b, a, c, e}** |
| T6 | {b, e} | **{b, e}** |
| T7 | {a, b, c, d} | **{b, a, c, d}** |
| T8 | {a, b, e} | **{b, a, e}** |
| T9 | {b, c} | **{b, c}** |

## Ordering Reference

Frequent Pattern Set

**{b:8, a:6, c:6, e:5, d:3}**

Ordering Rules

**1. Frequency Ranking**

Items are sorted in descending order of their support counts across the dataset.

**2. Global Priority Order**

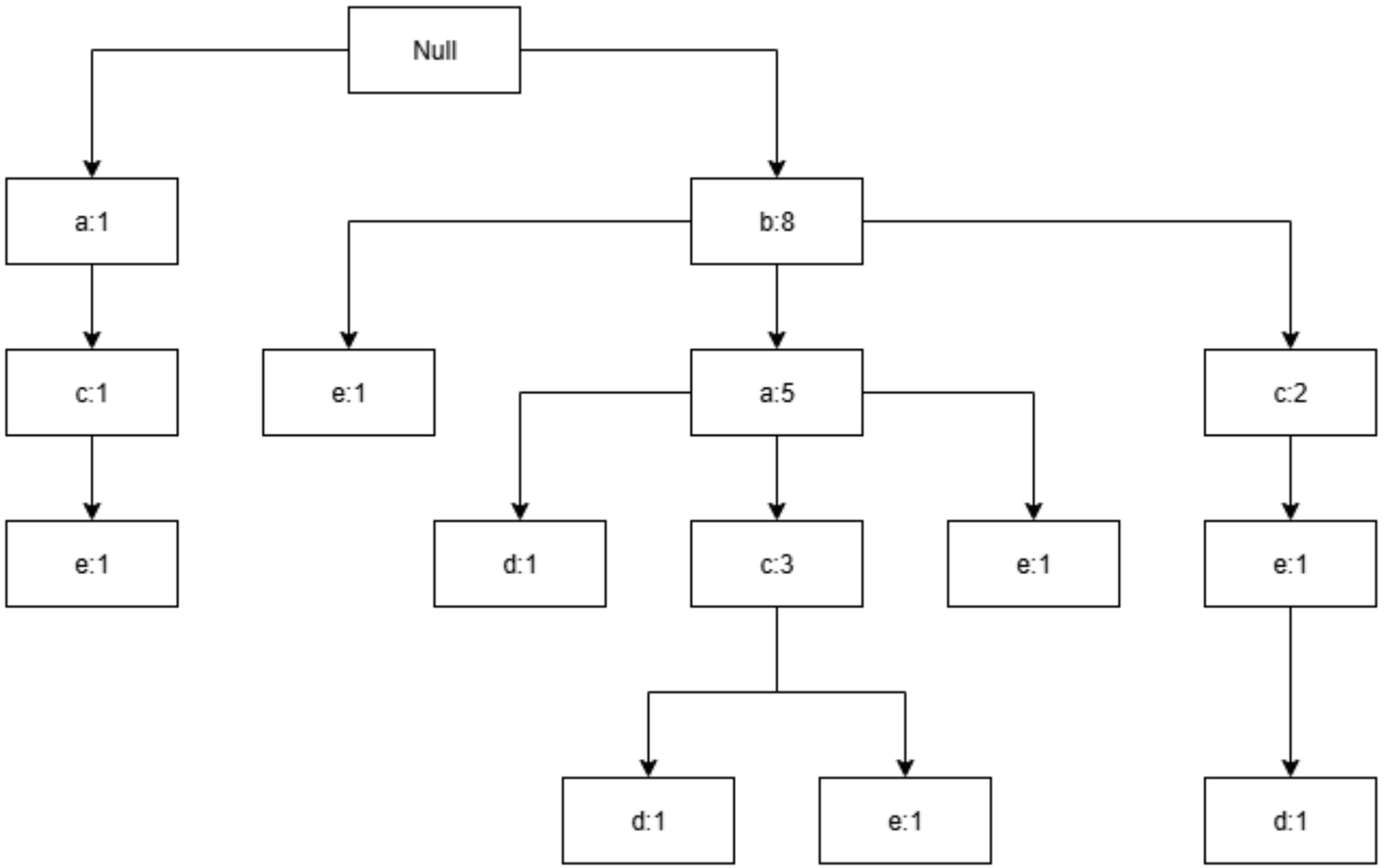
b > a > c > e > d

**3. Consistent Tie-Breaking**

Items with identical counts (a:6, c:6) preserve fixed global priority order in every row.

ASSOCIATION RULE MINING

# Construct FP-Tree




## FP-Tree Structure Rules

CORE ARCHITECTURE

**1. Root Initialization**
Starts with a Null root node as the tree anchor.

**2. Prefix Branch Sharing**
Transactions sharing item prefixes merge along identical paths.

**3. Node Counters**
Each node records item ID and support count (e.g., b:8, a:5).

GLOBAL ITEM ORDERING

**b > a > c > e > d**

ASSOCIATION RULE MINING

# Mine Frequent Pattern Tree (FP-Tree)

| Item | Conditional Pattern Base | Conditional FP-Tree | Frequent Patterns Generated |
|---|---|---|---|
| **d** | {{b,a:1}, {b,a,c:1}, {b,c,e:1}} | <b:3> | {b,d:3} |
| **e** | {{a,c:1}, {b:1}, {b,a,c:1}, {b,a:1}, {b,c:1}} | <b:4, a:3, c:3> | {b,e:4}, {a,e:3}, {c,e:3} |
| **c** | {{a:1}, {b:2}, {b,a:3}} | <b:5, a:4>, <b,a:3> | {b,c:5}, {a,c:4}, {b,a,c:3} |
| **a** | {{b:5}} | <b:5> | {b,a:5} |
| **b** | — | — | — |

## Mining Process Rules

BOTTOM-UP TRAVERSAL

**Process items in ascending order of support (d → e → c → a → b).**

CONDITIONAL PATTERN BASE

**Accumulate all prefix paths in the FP-Tree leading to the target item.**

CONDITIONAL FP-TREE

**Build a sub-tree from prefix paths, retaining nodes that meet minimum support threshold.**

PATTERN GENERATION

**Combine frequent conditional items with the target suffix item.**

ASSOCIATION RULE MINING

# Generating Association Rules from Frequent Itemsets

## Step 1: Formula & Subsets

### Confidence Metric

$$\text{confidence}(A \Rightarrow B) = P(B \mid A) = \frac{Support_Count(A \cup B)}{Support_Count(A)}$$

### Target Frequent Itemset

**X = { b, a, c }**

### Non-empty Subsets of X

To generate rules, all non-empty subsets are evaluated as antecedents:

{ b, a }{ b, c }{ b }{ a }{ c }

## Step 2: Rule Confidence Calculation

| | |
|---|---|
| **{ b, a } ⇒ c** | confidence = 3/5 = **60%** |
| **{ b, c } ⇒ a** | confidence = 3/5 = **60%** |
| **{ a, c } ⇒ b** | confidence = 3/4 = 75% ✓ PASSED |
| **b ⇒ { a, c }** | confidence = 3/8 = **37%** |
| **a ⇒ { b, c }** | confidence = 3/6 = **50%** |
| **c ⇒ { b, a }** | confidence = 3/6 = **50%** |

THRESHOLD EVALUATION

### Minimum Confidence Threshold = 70%

Only the rule **{ a, c } ⇒ b** achieves a confidence score ≥ 70% (75%).

**Final Output Rule: { a, c } ⇒ b**

# Association Strategy: Apriori vs FP-Growth

## Apriori

**Method:** Breadth-First Search

**Mechanic:** Generates candidate itemsets step-by-step and scans the entire database repeatedly.

**Drawback:** High memory and extreme time costs for massive datasets.

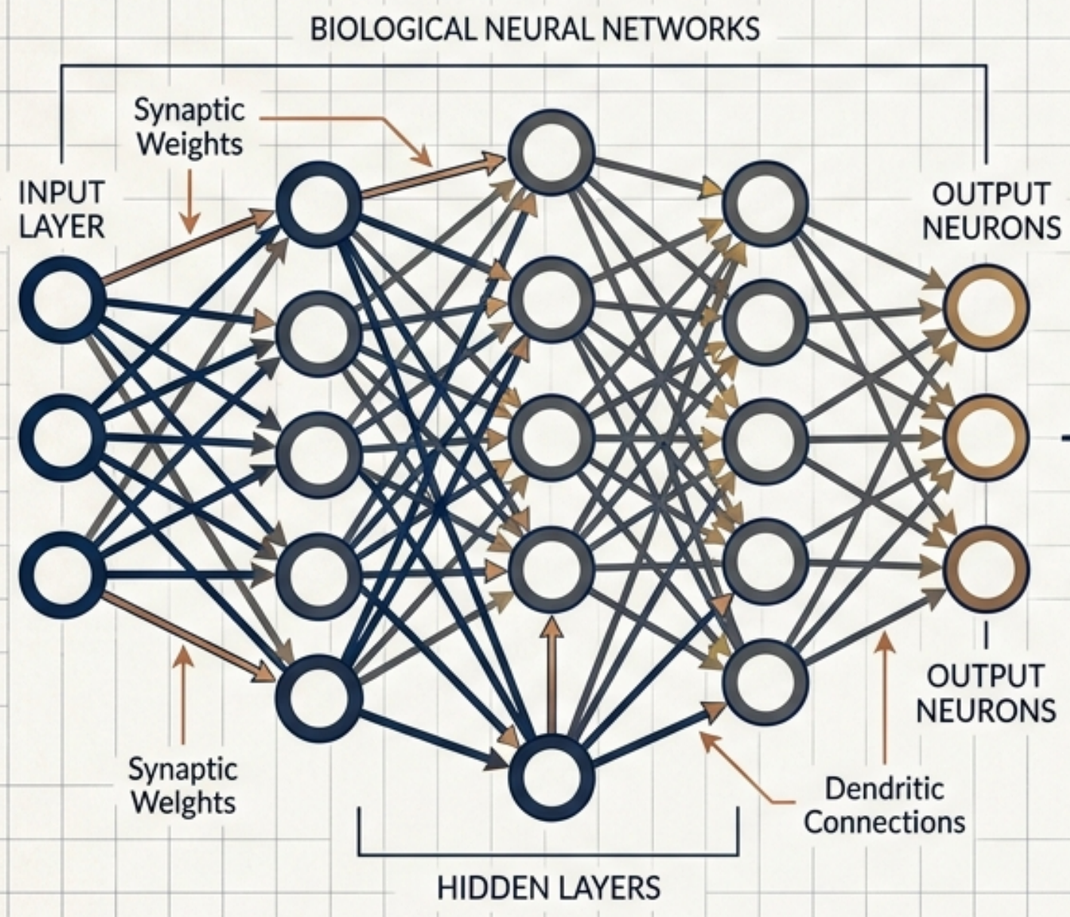
## FP-Growth

**Method:** Depth-First / Tree-Based

**Mechanic:** Compresses database into a Frequent Pattern Tree. Skips candidate generation entirely.

**Advantage:** Significantly faster and vastly more memory-efficient at scale.

# The Blueprint of Intelligence




MATHEMATICAL ABSTRACTION

MATHEMATICAL ABSTRACTION

APPLIED MATHEMATICAL MODELS
DEEP LEARNING FRAMEWORKS

$$\begin{bmatrix} W_{ij} & W_{ij} \\ x_i & f\left(\sum W_{ij}x_j + b_i\right) \\ \vdots & \vdots \\ h_l & h_l = f(W_l h_{l-1} + b_l) \\ \vdots & \vdots \\ y = \text{softmax}(W_{out}h_L + b_{out}) \end{bmatrix}$$

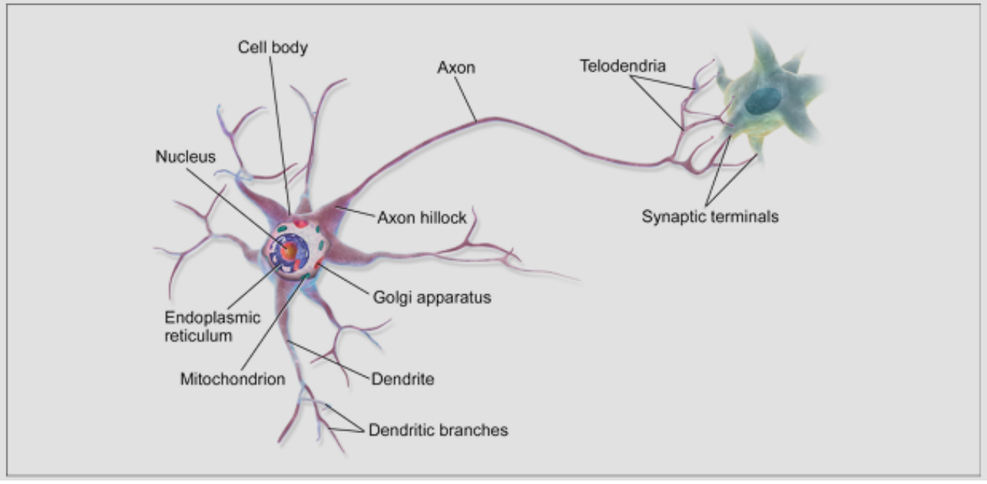
Linear Transformation → Non-Linear Activation → Loss Function Optimization (∇L) → Backpropagation

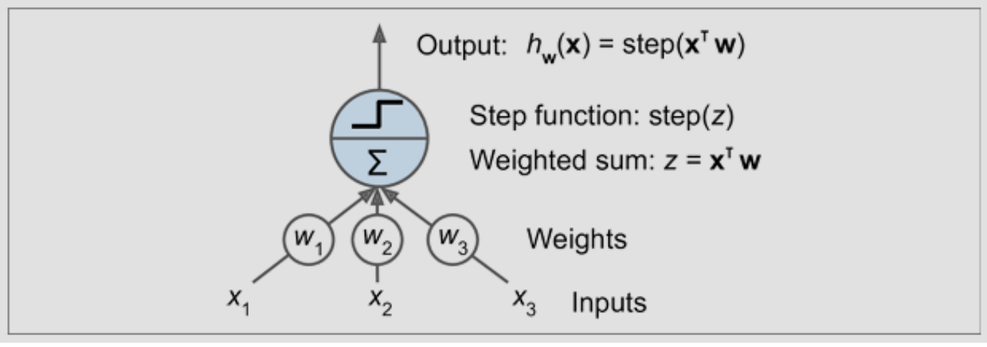
Translating biological neural networks into applied mathematical models and deep learning frameworks.

Gemini Notebook

# Translating Biology into System Architecture

## Biological Wetware



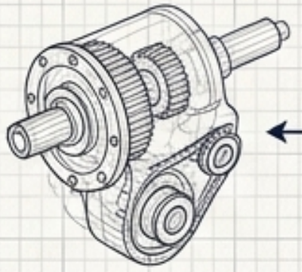

## Mathematical Software

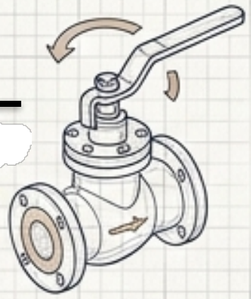
| | | |
|---|---|---|
| Dendrites | → | Inputs $(x_1, x_2, \ldots, x_n)$ |
| Cell Body | → | Processing & Summation |
| Axon | → | Output $(y)$ |

Gemini Notebook

# Deconstructing the Artificial Neuron

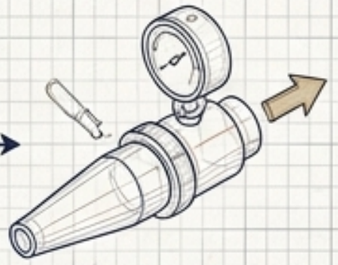
**Weights:**
The connection strength multiplying the input.



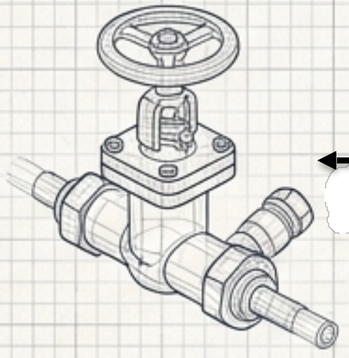


**Activation Function:**
The gatekeeper determining the final state.

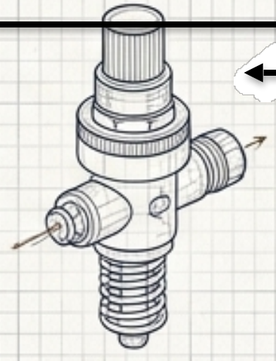
$$y = \phi(w \cdot x + b)$$



**Output:**
The resulting prediction.



**Inputs:**
Data signals entering the system.

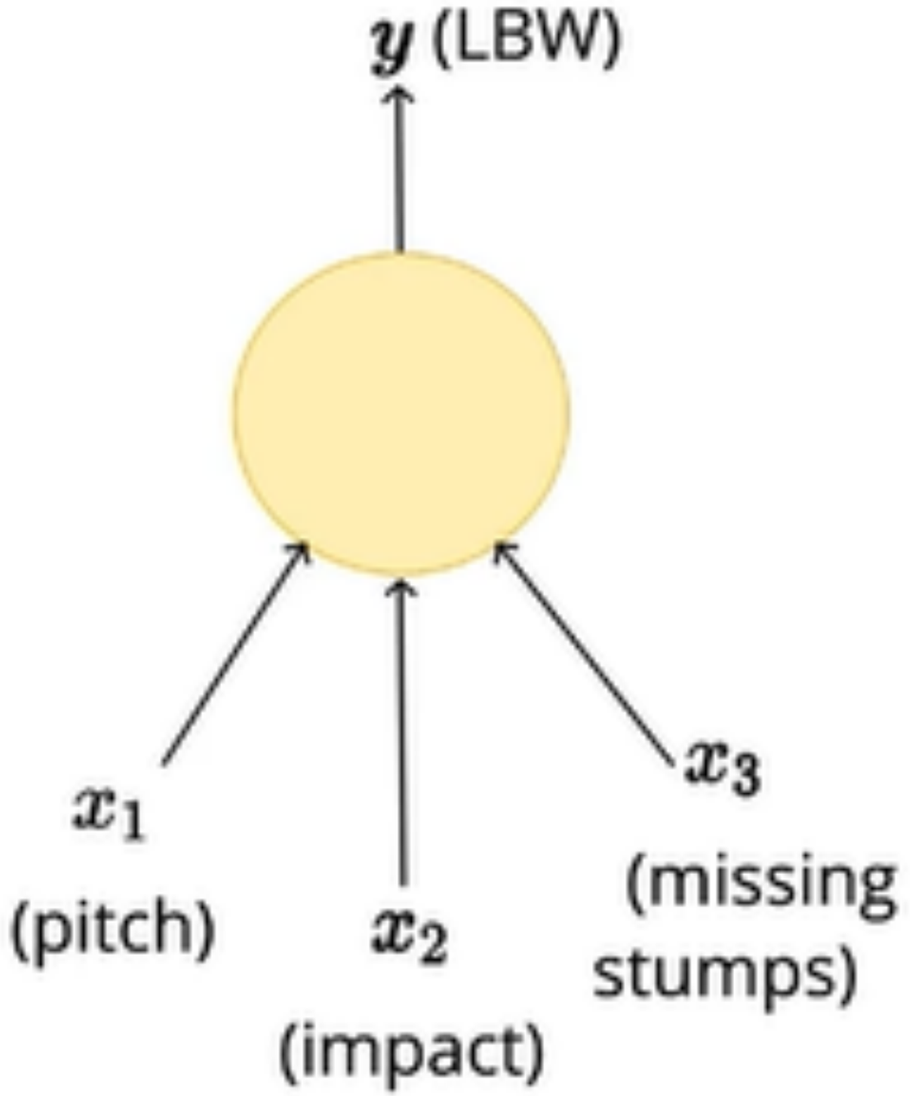


**Bias:**
The baseline threshold adjustment.

Gemini Notebook

# McCulloch Pitts Neuron

- $\hat{y} = \begin{cases} 1, if \sum_{i=1}^{n} x_i \geq b \\ 0, \quad otherwise \end{cases}$
- $loss = \sum_{i} \left( y_i - \hat{y}_i \right)^2$
- Boolean input
- Boolean output
- Fixed slope
- Few possible intercepts (b)



# The Perceptron

- **Mathematical Model**

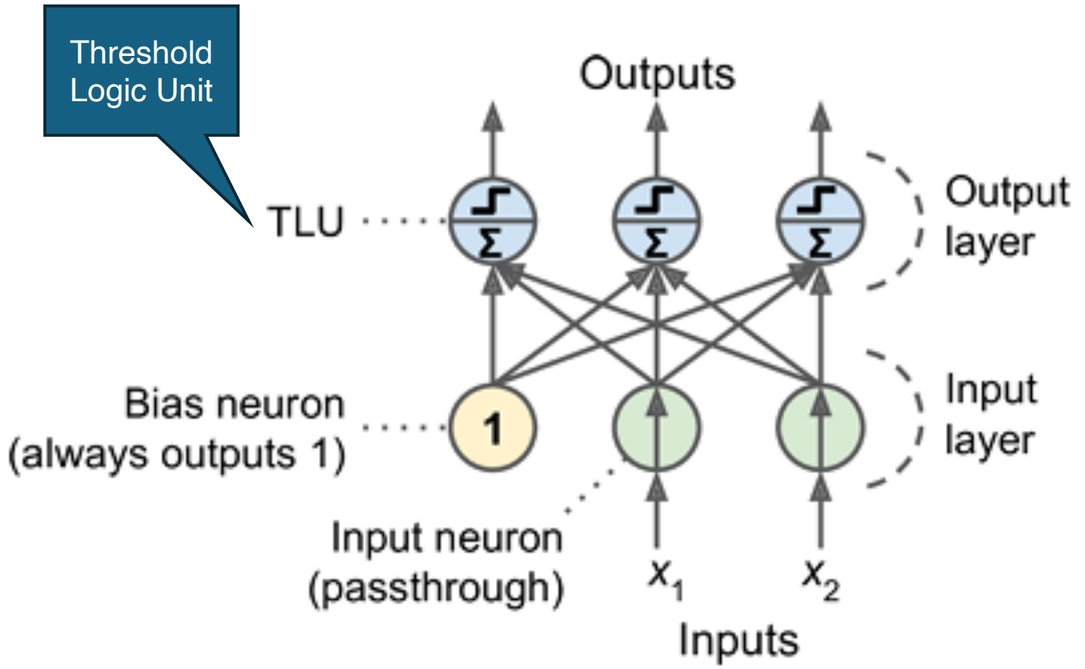
$$z = w \cdot x + b$$

$$y = \phi(z)$$

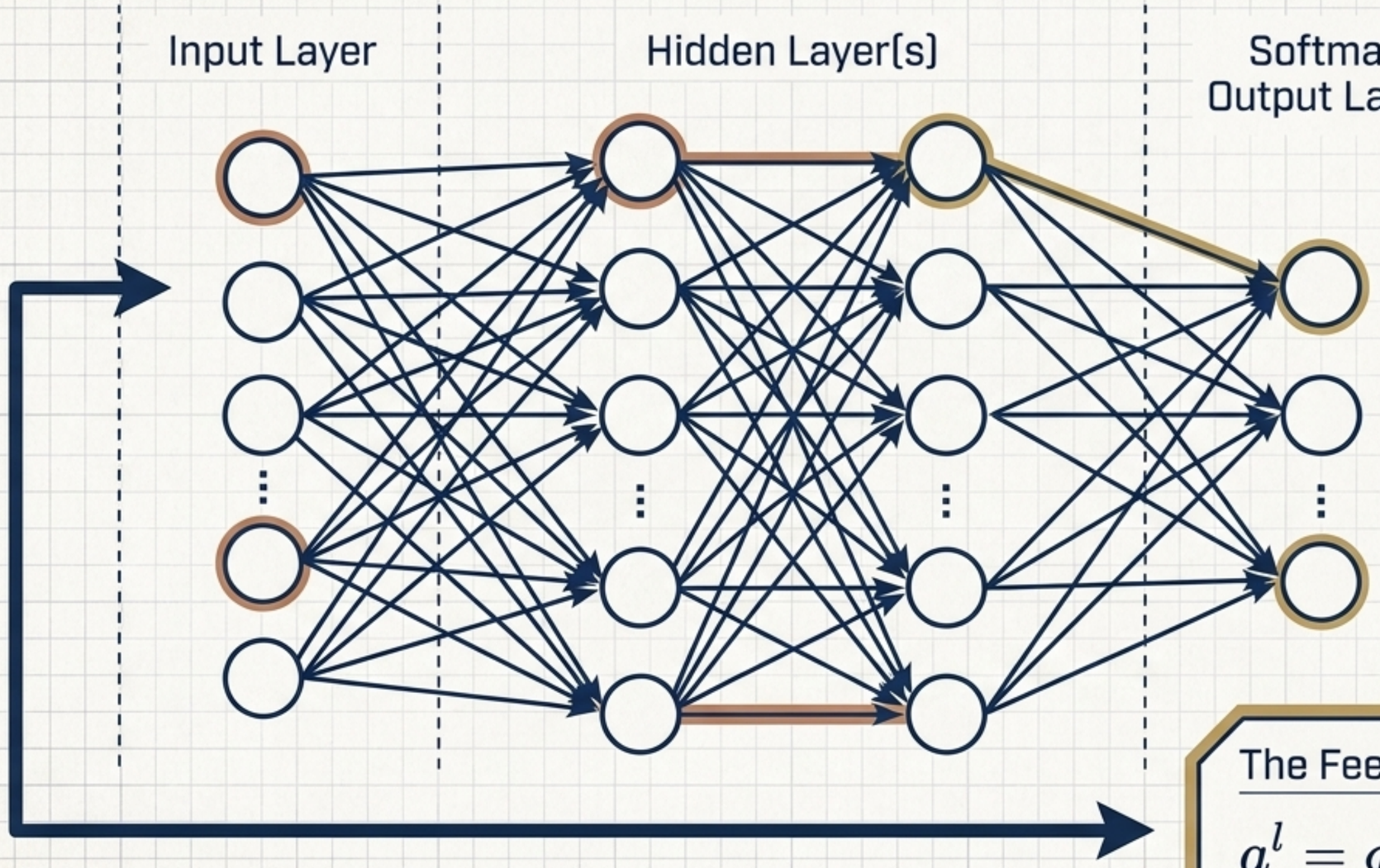
- **Step Function (Heaviside)**

$$\phi(z) = \begin{cases} 1, & if\ z \geq 0 \\ 0, & otherwise \end{cases}$$

- **Learning algorithm**
  - **If $x \in P$ and $\sum_{i=1}^{n} w_i x_i < 0$**
    - $w = w + x$
  - **If $x \in N$ and $\sum_{i=1}^{n} w_i x_i \geq 0$**
    - $w = w - x$



# Assembling the Multilayer Perceptron

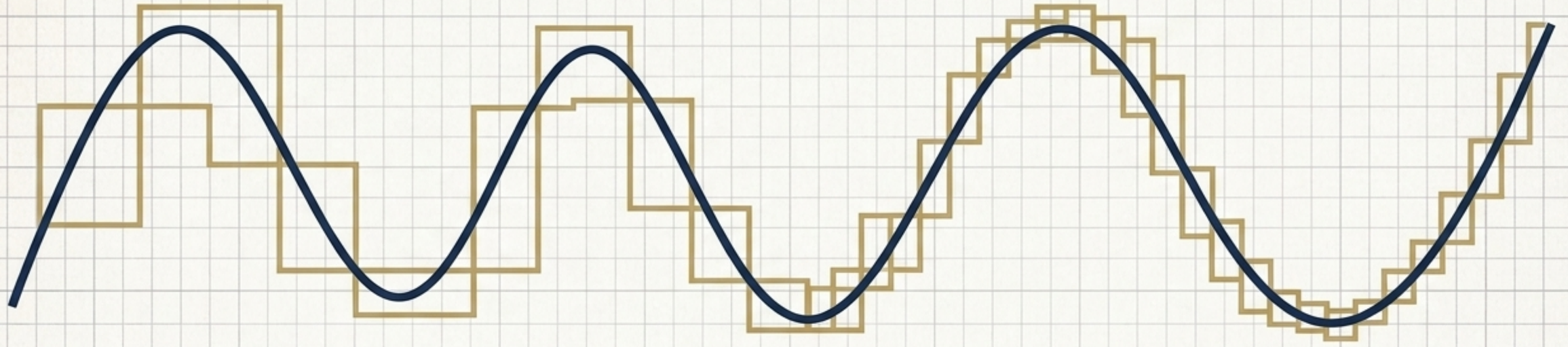



## The Feedforward Process

$$a^l = \phi(W^l a^{l-1} + b^l)$$

Gemini Notebook

# The Universal Approximation Theorem



# An MLP can approximate any continuous function.

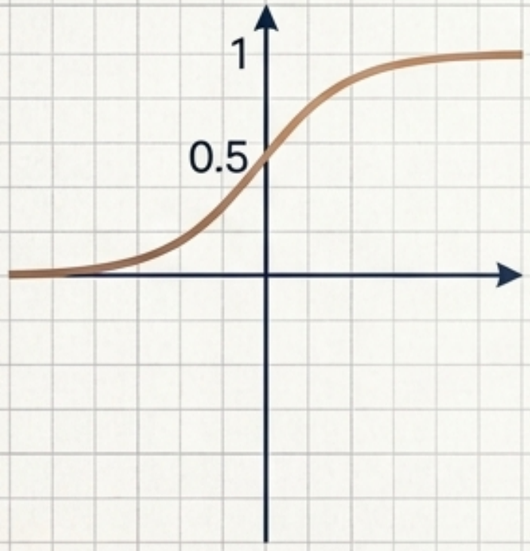
The precise orchestration of weights, biases, and non-linear layers yields infinite flexibility, mathematically capable of mapping any data relationship.

Gemini Notebook

# The Activation Engine Room

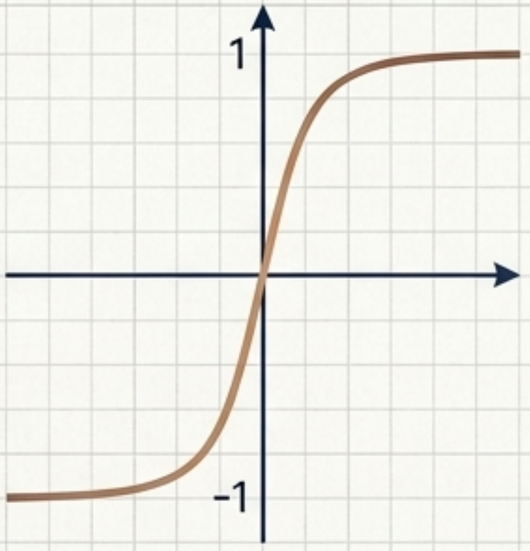
## Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



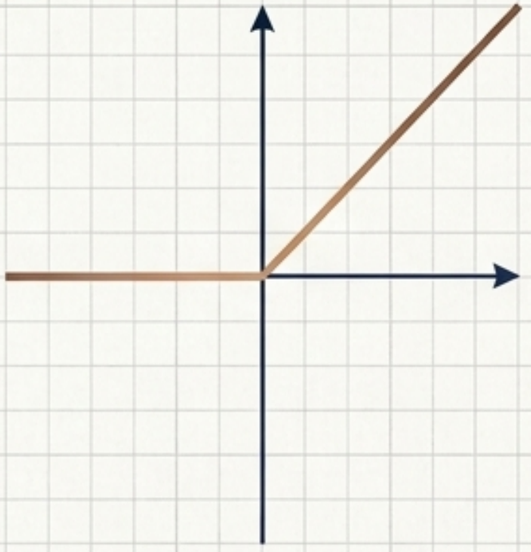

## Hyperbolic Tangent (Tanh)

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



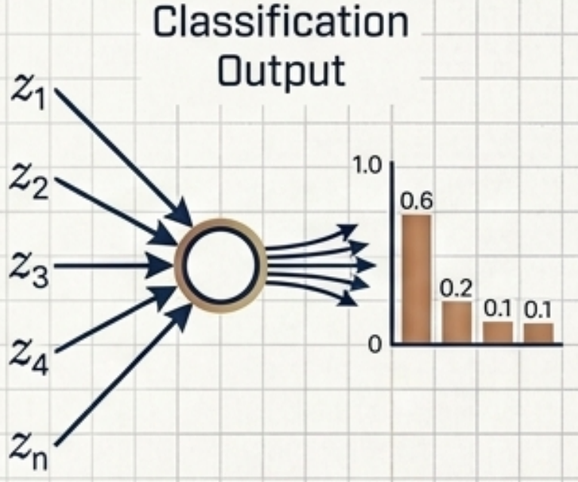

## Rectified Linear Unit (ReLU)

$$\mathrm{ReLU}(z) = \max(0, z)$$



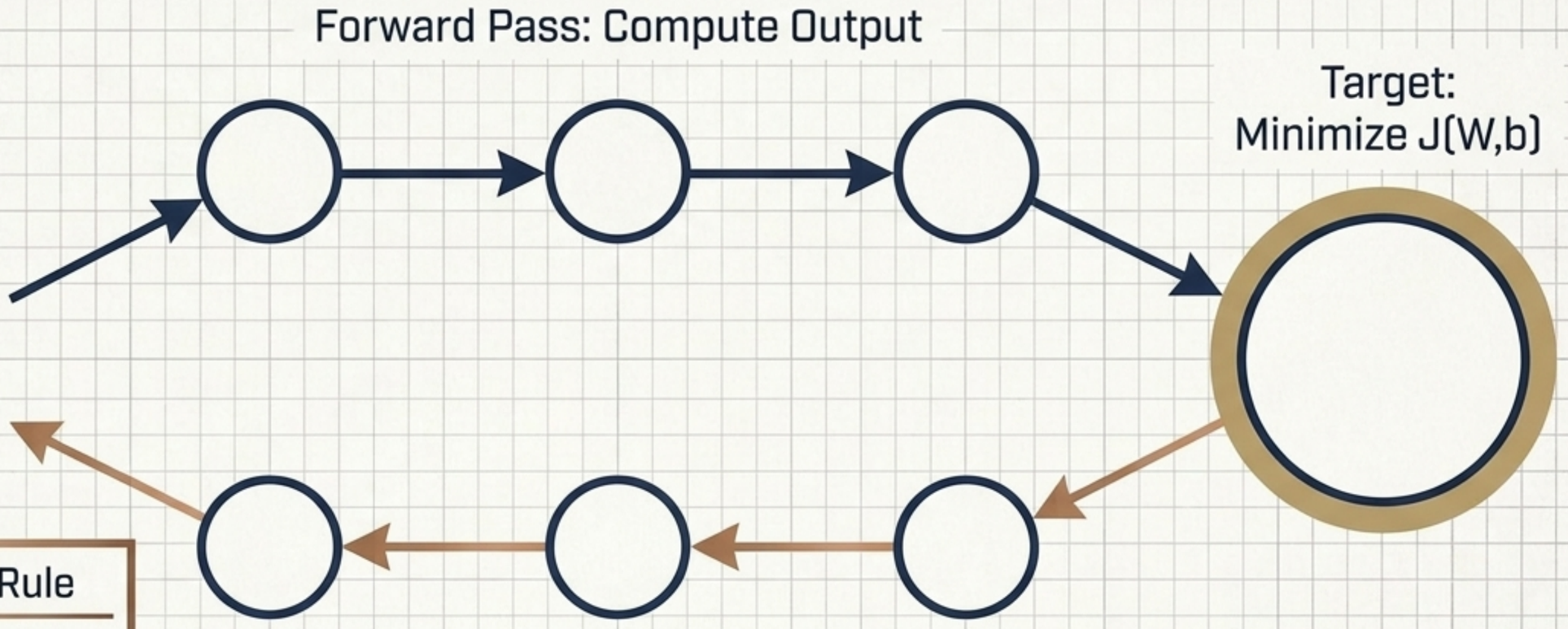
## Softmax

$$\frac{e^{z_i}}{\sum e^{z_j}}$$




Gemini Notebook

# The Training Mechanism



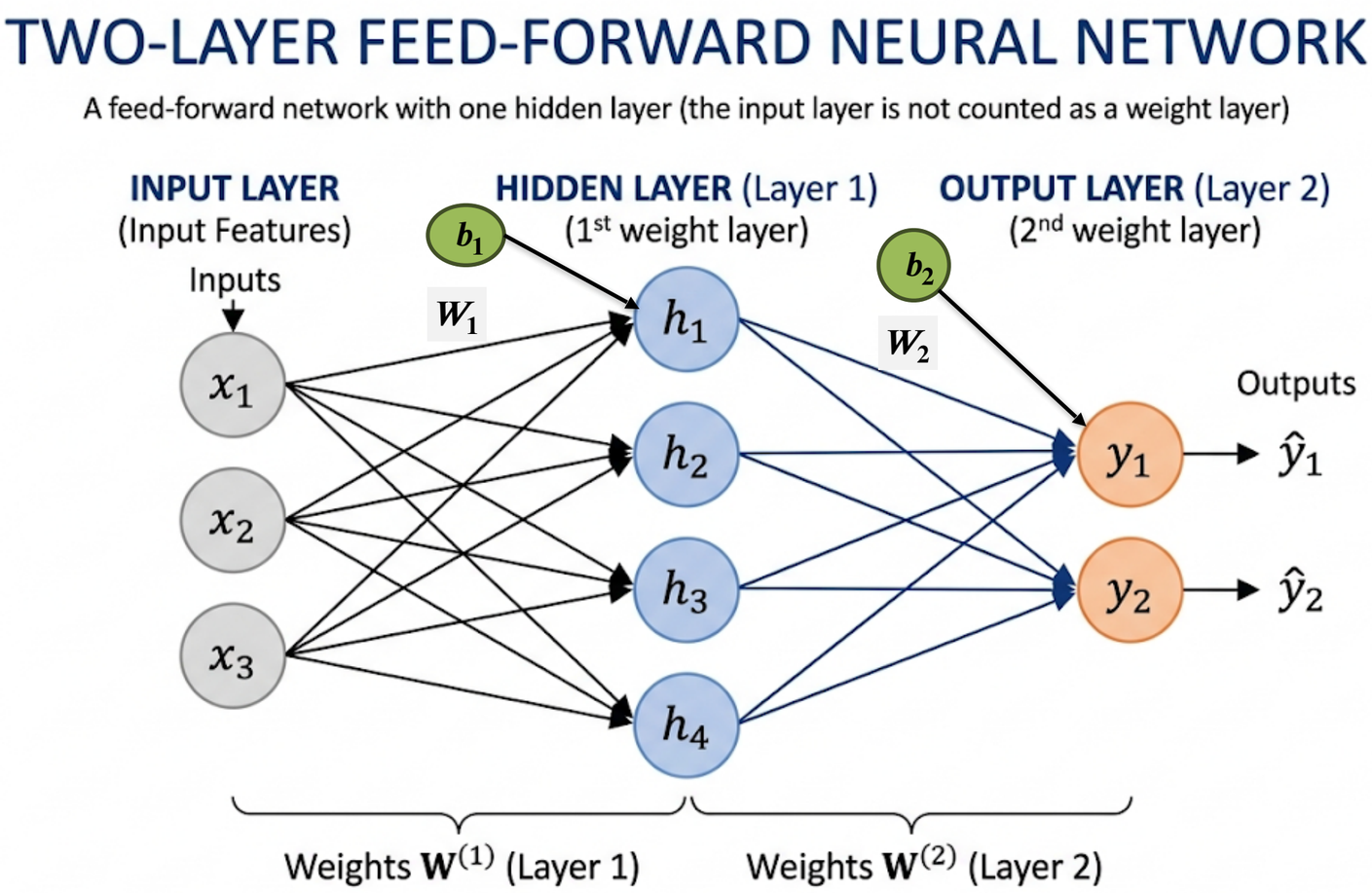

**Gradient Update Rule**

$$W \leftarrow W - \eta \frac{\partial J}{\partial W}$$

$$b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

Gemini Notebook

# Unlocking The Chain Rule




**A "two-layer" neural network consists of exactly 1 Hidden Layer and 1 Output Layer (excluding the Input Layer).**

# Chain Rule

- Let's take a simple **2-layer feedforward network**:

$$z_1 = W_1 x + b_1$$
$$h_1 = f(z_1)$$
$$y_1 = W_2 h_1 + b_2$$
$$\hat{y} = g(y_1)$$

- $Loss\ (L) = Loss(\hat{y}, y)$
- We need

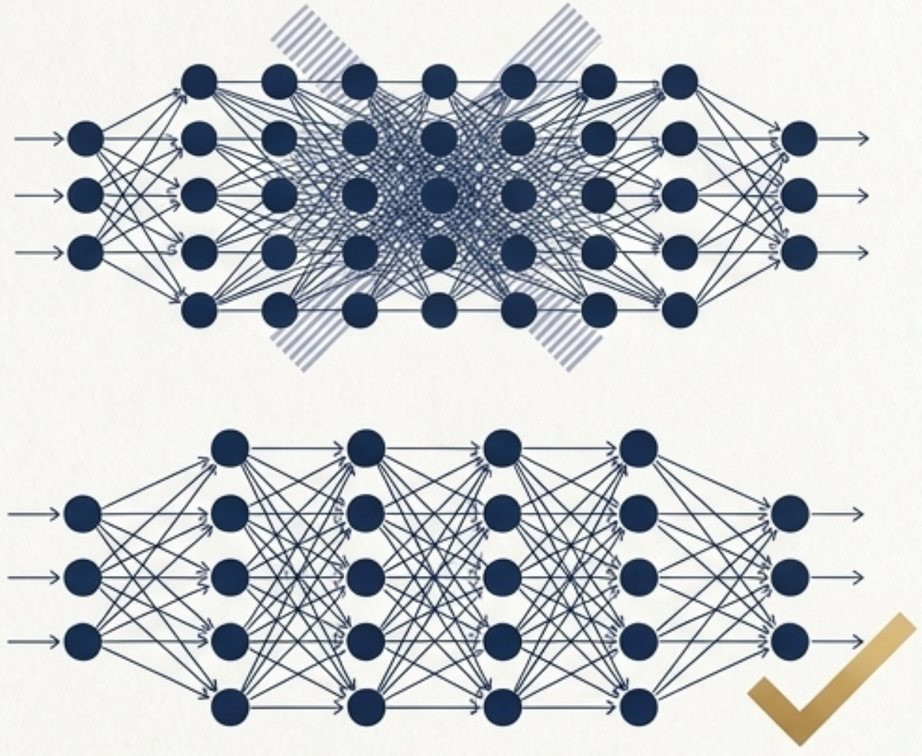
$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}$$

# Backpropagation Using Chain Rule

- **Step 1: Output layer gradient**
  - $\delta_2 = \frac{\partial L}{\partial y_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial y_1}$
  - $\frac{\partial L}{\partial W_2} = \delta_2 h_1^T$
  - $\frac{\partial L}{\partial b_2} = \delta_2$
- **Step 2: Hidden layer gradient**
  - $\delta_1 = \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial y_1} \cdot \frac{\partial y_1}{\partial h_1} \bullet \frac{\partial h_1}{\partial z_1}$
  - $\frac{\partial L}{\partial W_1} = \delta_1 x^T$
  - $\frac{\partial L}{\partial b_1} = \delta_1$

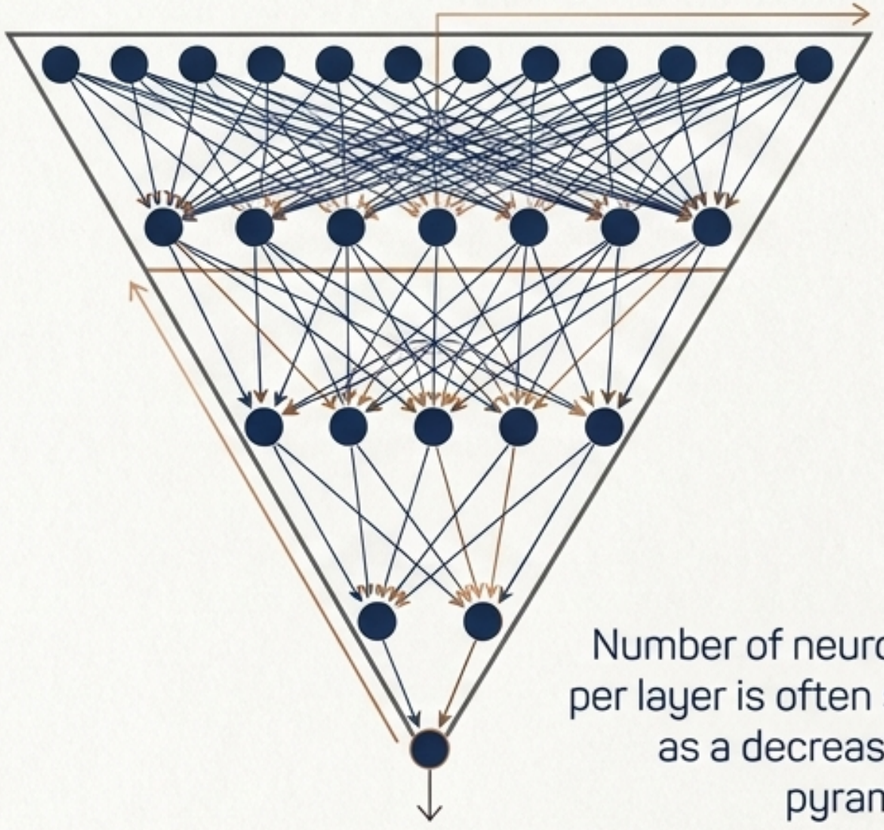
# Architectural Design Principles

## Depth over Breadth



Better outcome by increasing the number of layers instead of the number of neurons per layer.

## Network Shape

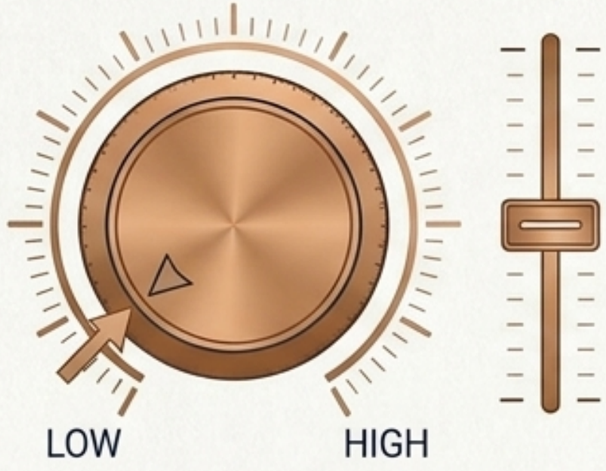


Number of neurons per layer is often set as a decreasing pyramid.

Gemini Notebook

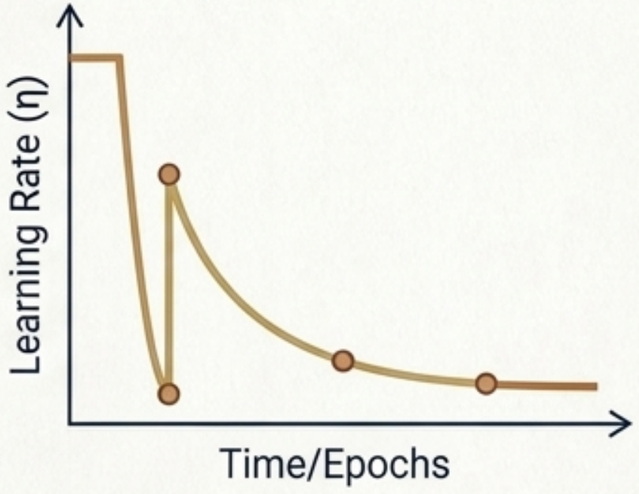
# Hyperparameter Control Panel

## Batch Size Control



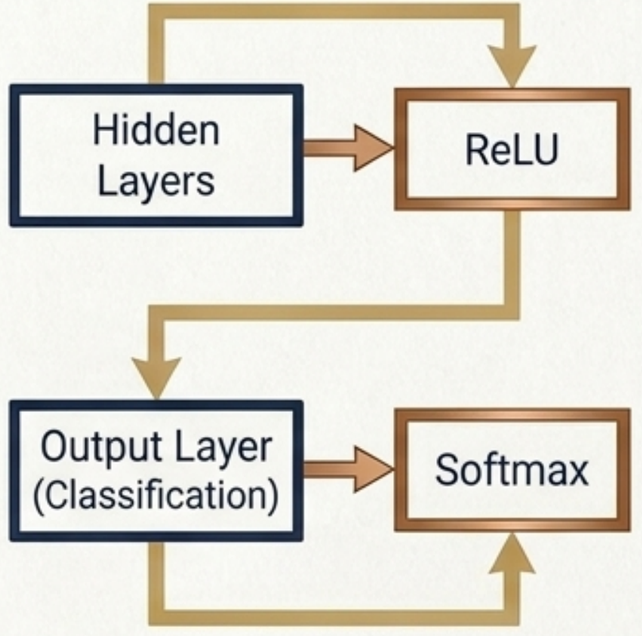

Smaller batches → noisier updates, better generalization of the model.

## Learning Rate Calibration

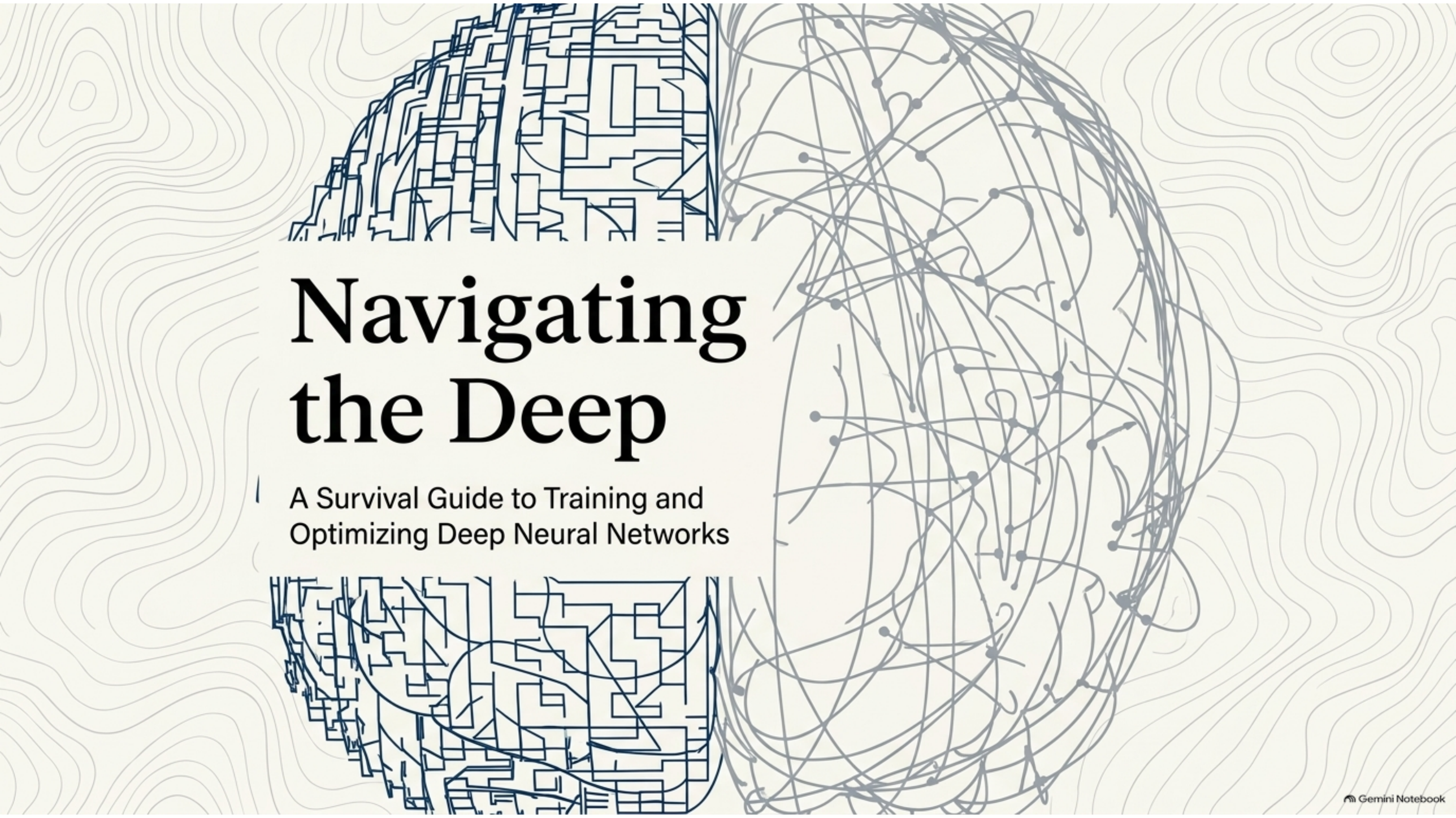



Use learning rate scheduling or adaptive optimizers (e.g., Adam) to control step size.

## Activation Assignment




Gemini Notebook



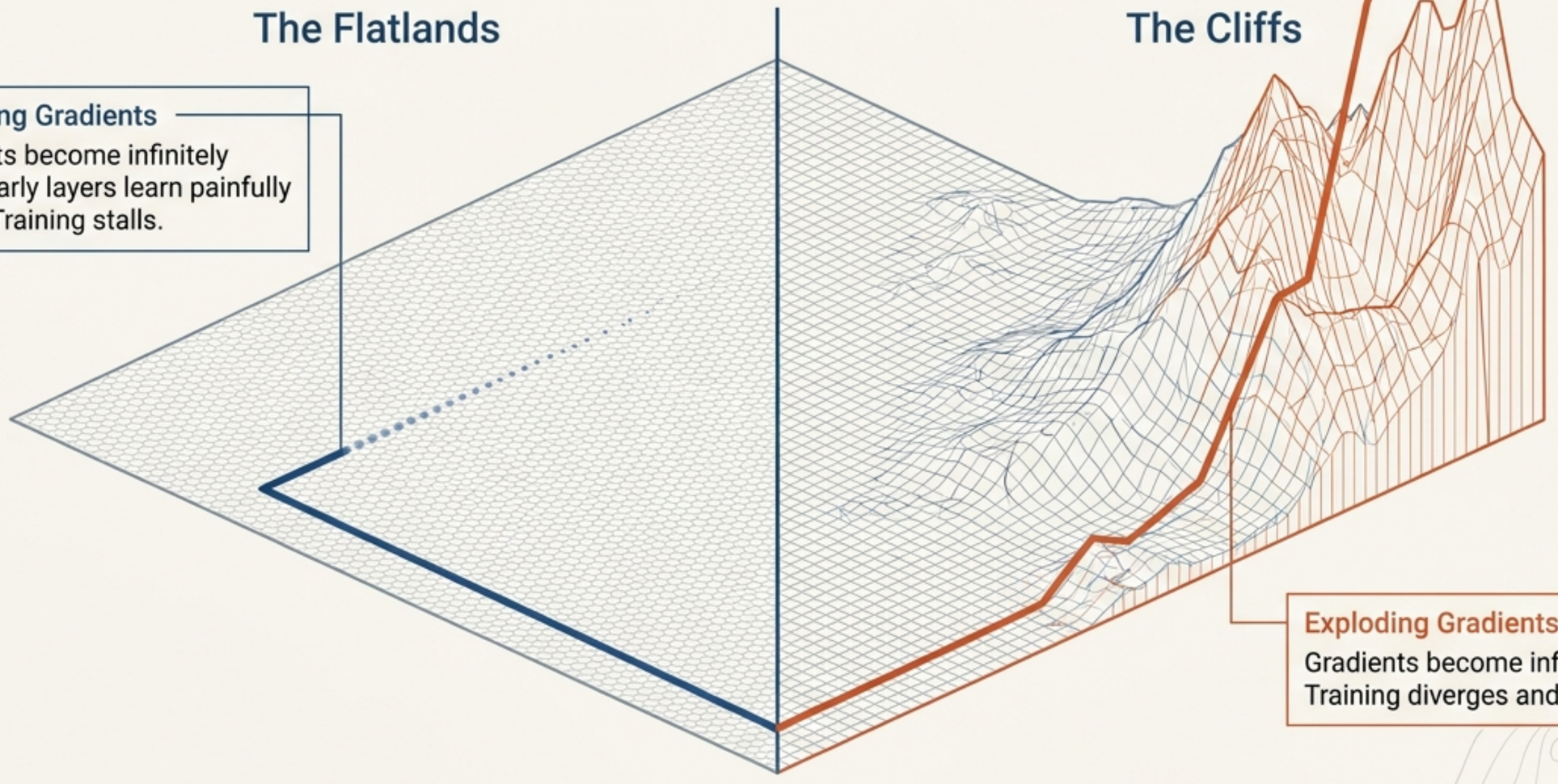
# Navigating the Deep

## A Survival Guide to Training and Optimizing Deep Neural Networks

Gemini Notebook

# The Unstable Topography of Deep Networks

**Key Concept:** Repeated multiplication of gradients during backpropagation destabilizes the terrain.




### Vanishing Gradients

Gradients become infinitely small. Early layers learn painfully slowly. Training stalls.

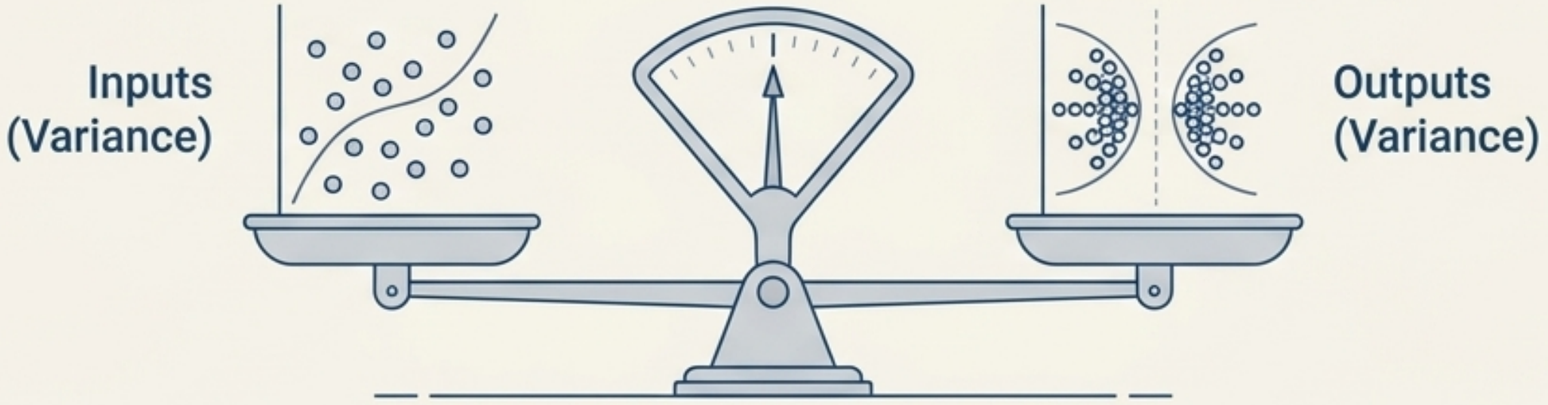
### Exploding Gradients

Gradients become infinitely large. Training diverges and fails.

Gemini Notebook

# Packing the Gear: Proper Initialization

**Core Rule**: Output variance must equal input variance to avoid acting as a single, weak neuron.




## Glorot (Xavier) Initialization

**Best for:** Tanh and Logistic functions.

Variance = $\frac{2}{n_{in} + n_{out}}$

## He Initialization

**Best for:** ReLU and its variants.

Variance = $\frac{2}{n_{in}}$

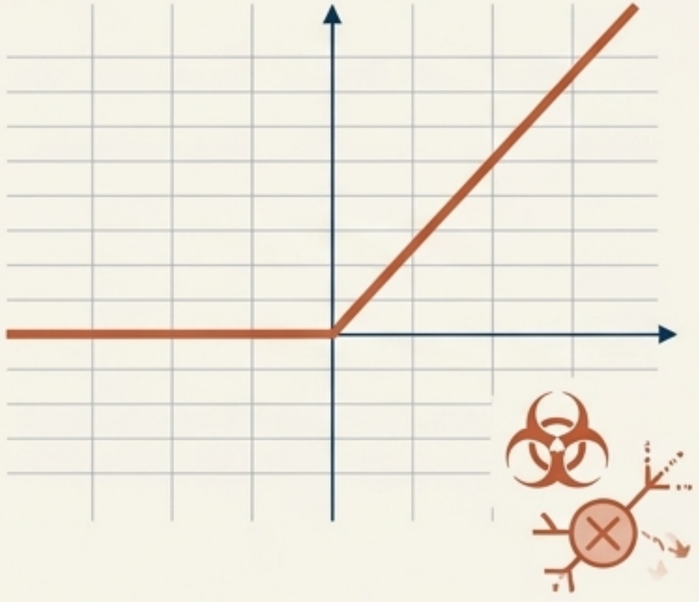
**Benefit:** Maintains stable gradients through deep layers.

Gemini Notebook

# Nonsaturating Activations & The ‘Coma’ Problem

Saturating functions cause gradients to vanish. ReLU avoids this, but suffers from “Dying ReLUs” (neurons immune to gradient descent due to zero output for negative inputs).

## Standard ReLU



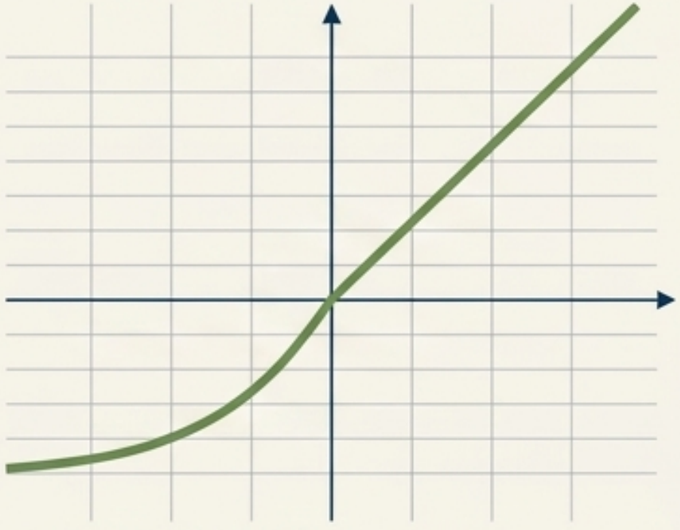
The baseline. Neurons can permanently die during training.

## Leaky ReLU

$$\max(\alpha z, z)$$

Hyperparameter α (typically 0.01) creates a small slope. The neuron never dies; it goes into a coma but can wake up.

## ELU (Exponential Linear Unit)



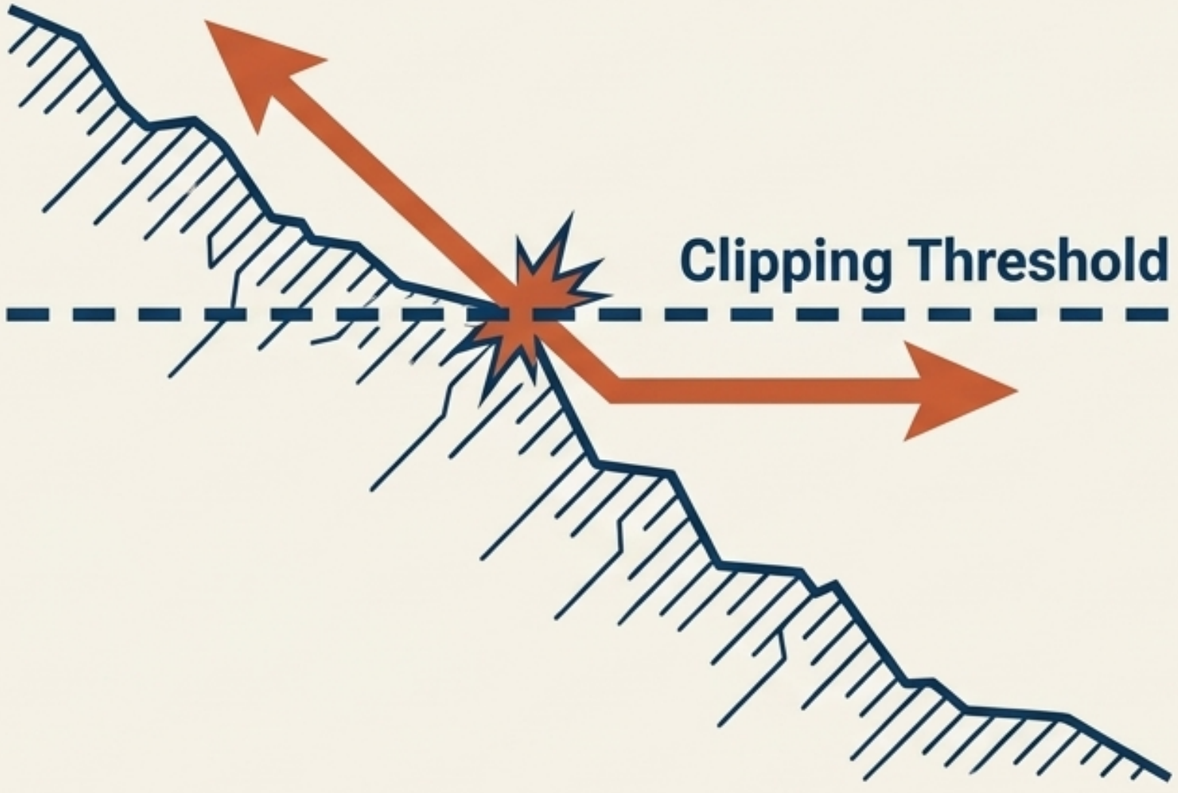
$$\alpha(e^z - 1) \text{ for } z < 0$$

Smooth curve entirely avoids dead neurons.

Gemini Notebook

# Guardrails for the Descent: Gradient Clipping

The primary defense against Exploding Gradients, common in RNNs and deep networks. Intercepts gradients during backpropagation so they never exceed a predefined threshold.




### Value Clipping

Hard caps the specific mathematical value of the gradient.

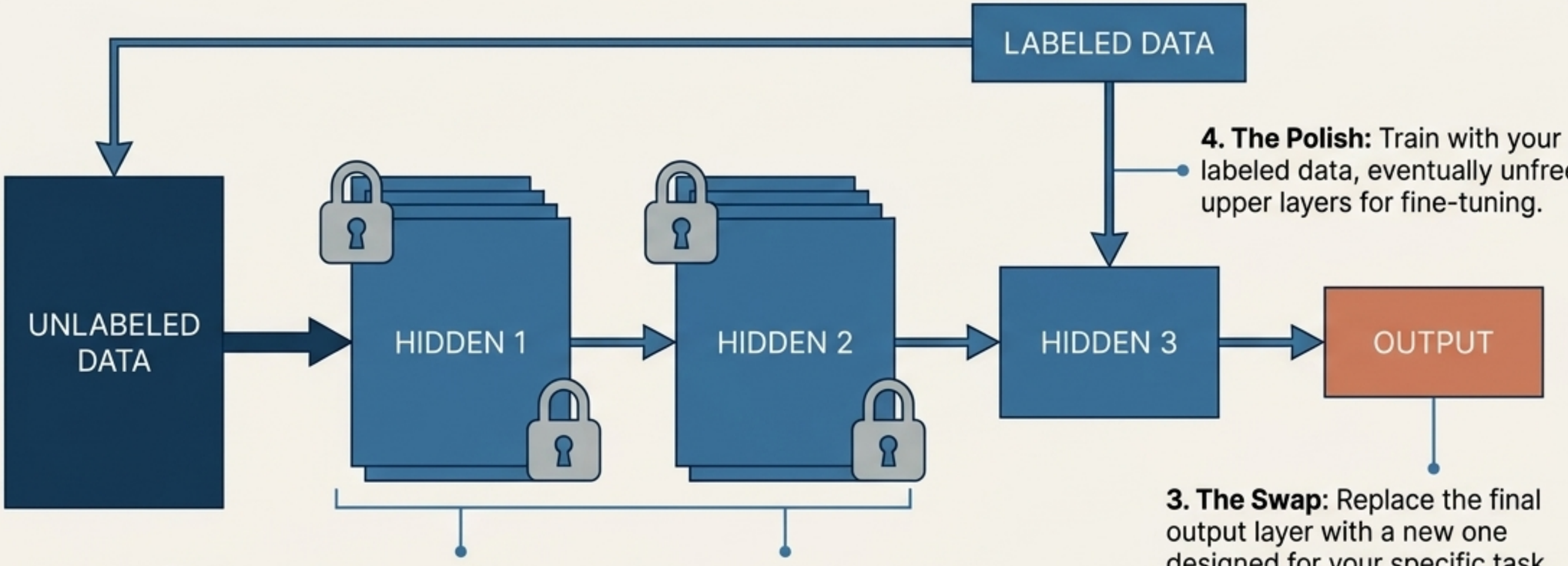
### Norm Clipping

Scales the whole vector down if its norm exceeds the threshold, preserving the gradient's original direction.

Gemini Notebook

# Taking Shortcuts: Reusing Pretrained Layers

**Transfer Learning** allows you to use layers from a pre-trained model as a starting map, saving massive computational time when labeled data is scarce.




**4. The Polish:** Train with your labeled data, eventually unfreezing upper layers for fine-tuning.

**3. The Swap**: Replace the final output layer with a new one designed for your specific task.

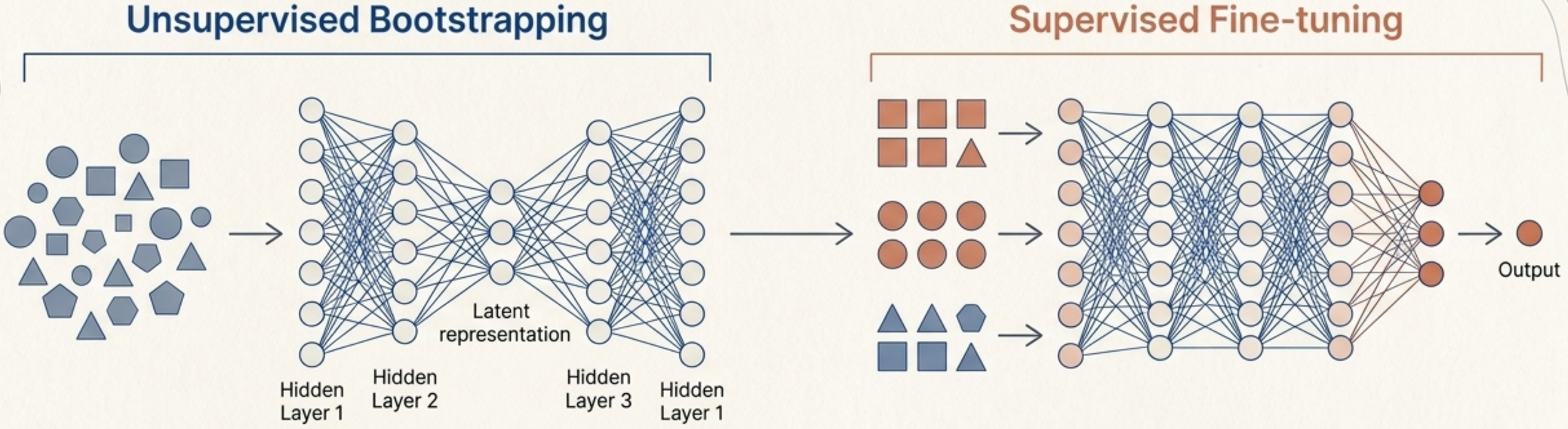
**1. The Foundation**: Import lower hidden layers (detecting basic features like edges/textures).

**2. The Lock**: Freeze these early layers so their weights aren't destroyed during initial training.

Gemini Notebook

# Bootstrapping the Map: Unsupervised Pretraining

Used when you have vast amounts of complex data but very few labels.

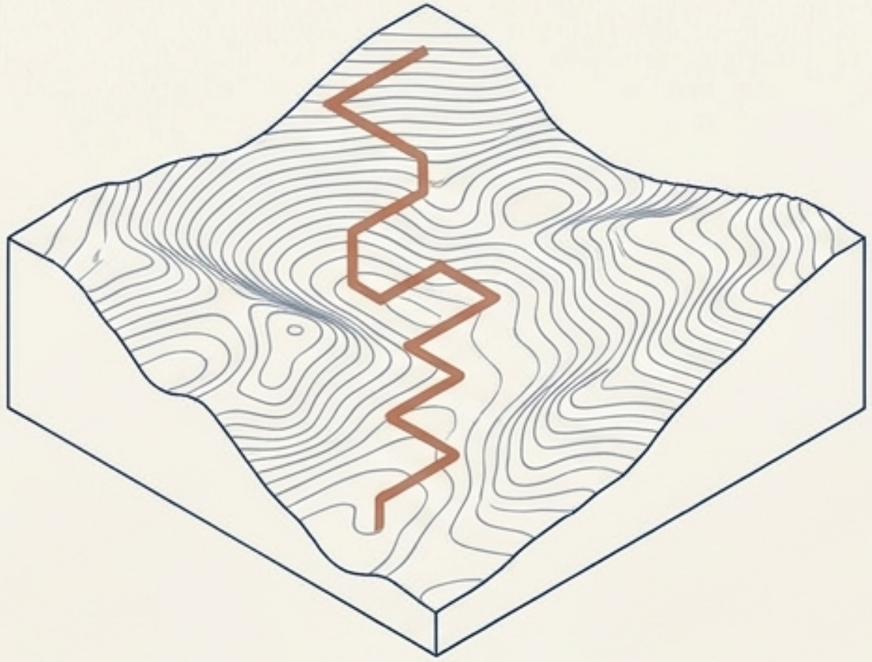



Train individual layers sequentially using unlabeled data via unsupervised techniques (like autoencoders) to learn underlying structures.

Add the final output layer and fine-tune the entire network using the limited labeled data.

Gemini Notebook

# The Hiker's Evolution: Moving Beyond Blind Steps

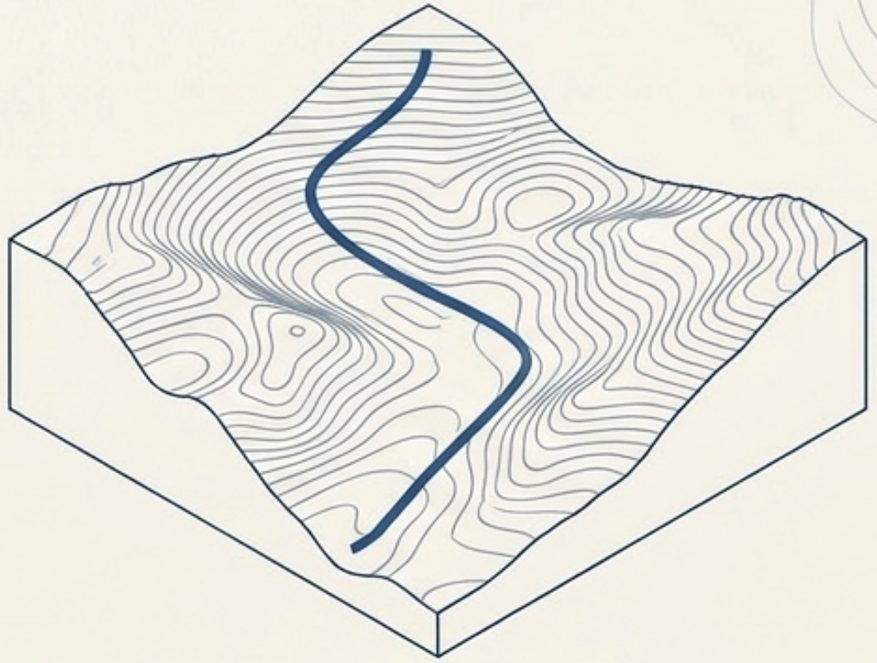


## Vanilla Gradient Descent

Analogy: A hiker looking only at their feet. They take a step in the steepest direction, stop, check again, and step.

Result: Extremely slow on flat plains and gets trapped in noisy paths.

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_\theta J(\theta_t)$$



## Momentum Optimization

Analogy: A heavy ball rolling down a hill. Accelerates by considering past gradients, building speed in a consistent direction.

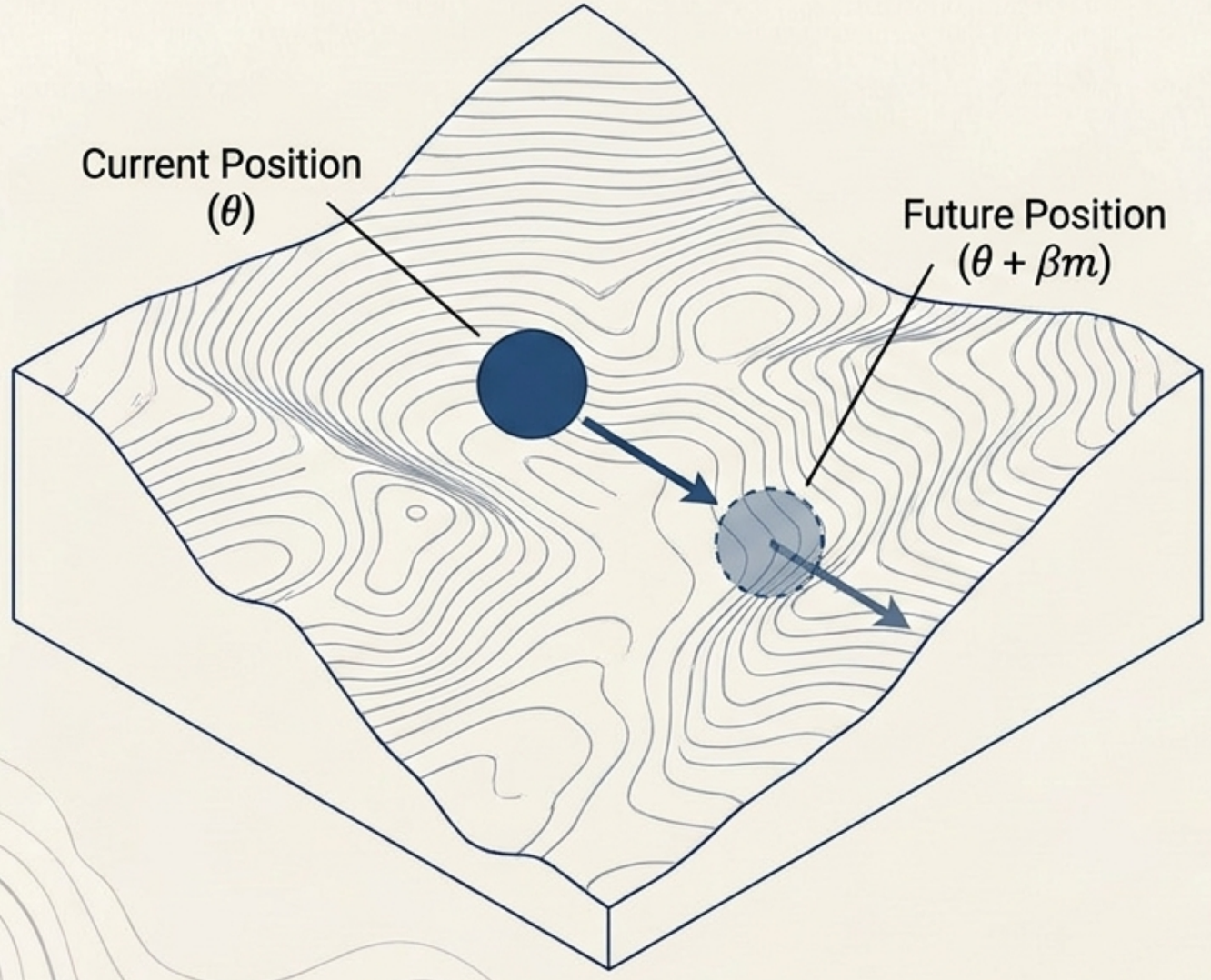
Mechanic: Introduces momentum hyperparameter β (0 to 1). Might overshoot and oscillate before settling.

Exponentially decaying average weights

$$m_t \leftarrow \beta m_{t-1} + \eta \nabla_\theta J(\theta_t)$$
$$\theta_{t+1} \leftarrow \theta_t - m_t$$

Gemini Notebook

# Nesterov Accelerated Gradient (NAG): The Look-Ahead




**Analogy:** A smarter hiker with momentum. Based on their current speed, they calculate where they will be in a moment. They check the slope at that future location and adjust their direction now.

**Key Mechanic:** Measures the gradient of the cost function slightly ahead in the direction of momentum.

**Benefit:** Applies the brakes early when approaching a valley or turn, preventing the severe overshooting that plagues standard Momentum.

**Formula Element:**

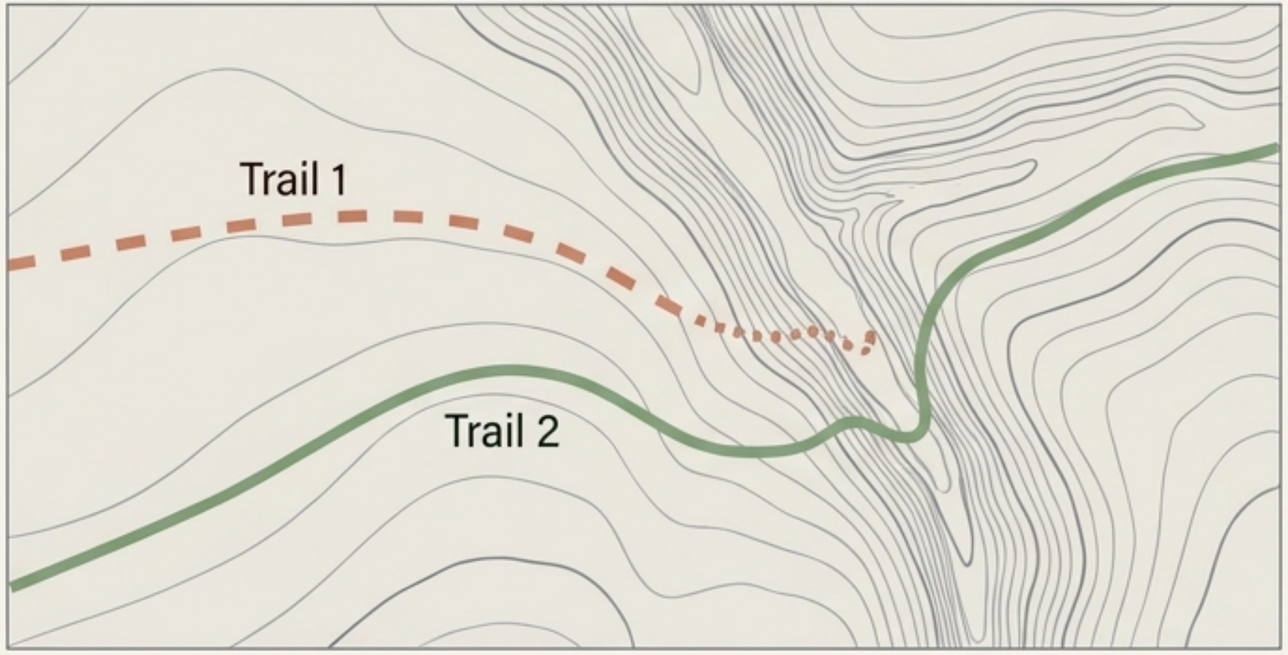
$$m_t \leftarrow \beta m_{t-1} + \eta \nabla_\theta J(\theta_t + \beta m_{t-1})$$

$$\theta_{t+1} \leftarrow \theta_t - m_t$$

# Adaptive Footing: AdaGrad vs. RMSProp

$$s_t \leftarrow s_{t-1} + \nabla_\theta J(\theta_t) \otimes \nabla_\theta J(\theta_t)$$

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta \nabla_\theta J(\theta_t)}{\sqrt{s_t + \epsilon}}$$




$$s_t \leftarrow \beta s_{t-1} + (1-\beta) \nabla_\theta J(\theta_t) \otimes \nabla_\theta J(\theta_t)$$

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta \nabla_\theta J(\theta_t)}{\sqrt{s_t + \epsilon}}$$

## AdaGrad (The Flawed Adapter)

**Analogy:** Personalized step size. Large steps for gentle slopes, small steps for steep ones.

**Drawback:** The “brakes” are too strong. Learning rate scales down so aggressively that it stops entirely before reaching the global optimum.

## RMSProp (The Moving Average)

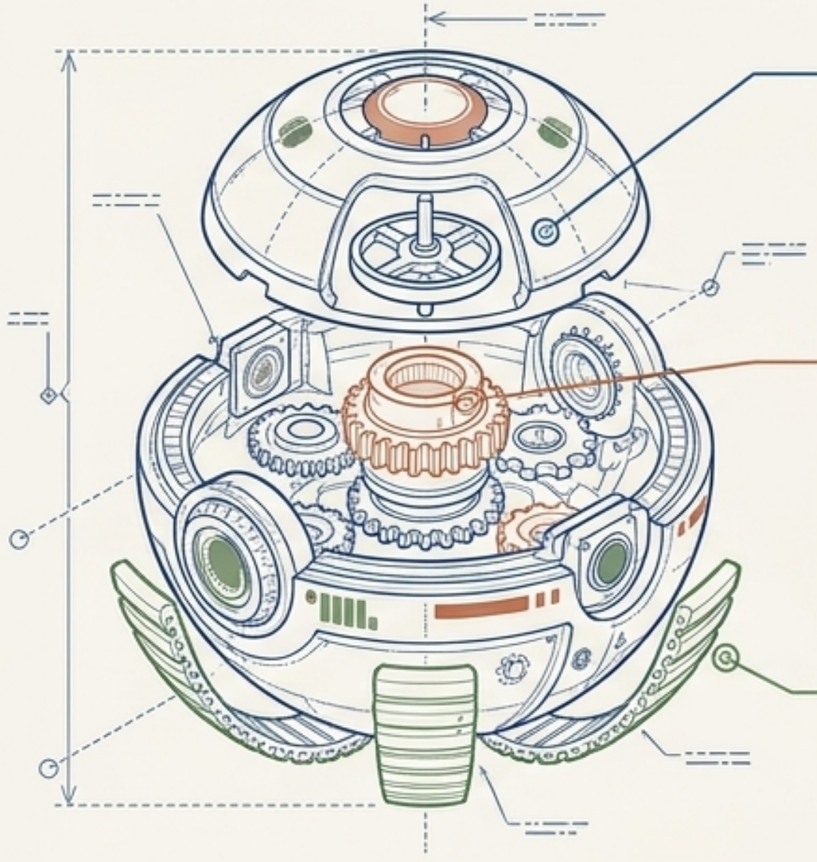
**Analogy:** Fixes AdaGrad by accumulating only the most recent steepness to decide step size.

**Mechanic:** Uses exponential decay (decay rate β typically 0.9). Sudden steep slopes cause temporary small steps that fade over time, preventing the learning rate from vanishing.

Gemini Notebook

# Adam: The Master Explorer

Adaptive Moment Estimation combines the heavy-ball mechanics of Momentum with the personalized adaptive step sizes of RMSProp.



## 1. Internal Gyroscope (Momentum $m_t$)

Keeps the ball rolling consistently through flat areas and noise.

$$m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta J(\theta_t)$$

$$s_t \leftarrow \beta_2 s_{t-1} + (1-\beta_2)\nabla_\theta J(\theta_t) \otimes \nabla_\theta J(\theta_t)$$

## 3. Warm-up Thruster (Bias Correction)

A critical stabilizing period (1 - $\beta^t$) that prevents the algorithm from taking overly large, reckless steps at the very beginning of the journey.

$$\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t} \qquad \hat{s}_t \leftarrow \frac{s_t}{1-\beta_2^t}$$

## 2. Adaptive Treads (RMSProp $s_t$)

Scales individual steps based on recent gradient variance.

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{s}_t + \epsilon}}$$

Gemini Notebook

# iAdam: Improved Adam Optimizer

Khaire, U.M., Dhanalakshmi, R. High-dimensional microarray dataset classification using an improved adam optimizer (iAdam). *J Ambient Intell Human Comput* **11**, 5187–5204 (2020).
https://doi.org/10.1007/s12652-020-01832-3

**1. Look-Ahead Point Computation**

Before evaluating the momentum gradient, a temporary look-ahead point $\theta_{temp}$ is calculated by advancing in the direction of the previous velocity vector $m_{t-1}$:

$$\theta_{temp} = \theta_t - \beta_1 m_{t-1}$$

**2. Gradient Calculations**

**Look-Ahead Gradient ($\hat{h}_t$):** Evaluated at the temporary look-ahead parameter position $\theta_{temp}$ to direct the updated velocity:

$$\hat{h}_t = \nabla_{\theta_{\text{temp}}} \sum_i \mathscr{L}\left(f\left(x_i; \theta_{\text{temp}}\right), y_i\right)$$

**Standard Gradient ($\hat{k}_t$):** Evaluated at the current parameter position $\theta_{t-1}$ for the second moment (variance) scaling:

$$\hat{k}_t = \nabla_{\theta_t} \sum_i \mathscr{L}\left(f\left(x_i; \theta_t\right), y_i\right)$$

**3. Gradient Moment Updates**

**First-Order Moment (Velocity $m_t$):** Uses the look-ahead gradient $\hat{h}_t$to update the momentum:

$$m_t = \beta_1 m_{t-1} + \eta \hat{h}_t$$

**Second-Order Moment (History $s_t$):** Maintains the running exponentially decaying average of the squared standard gradients $\hat{k}_t$:

$$s_t = \beta_2 s_{t-1} + \left(1 - \beta_2\right)\left(\hat{k}_t\right)^2$$

**4. Bias Corrections**

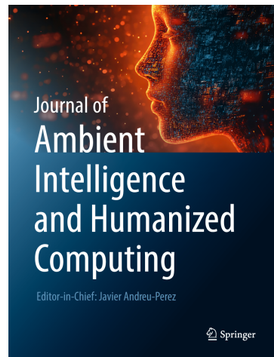
To prevent initialization bias towards zero during the early iterations:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

**5. Parameter Update**

The final parameter update step is computed using the adaptive learning rate term and the bias-corrected velocity:

$$\theta_{t+1} = \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{s}_t + \epsilon}}$$



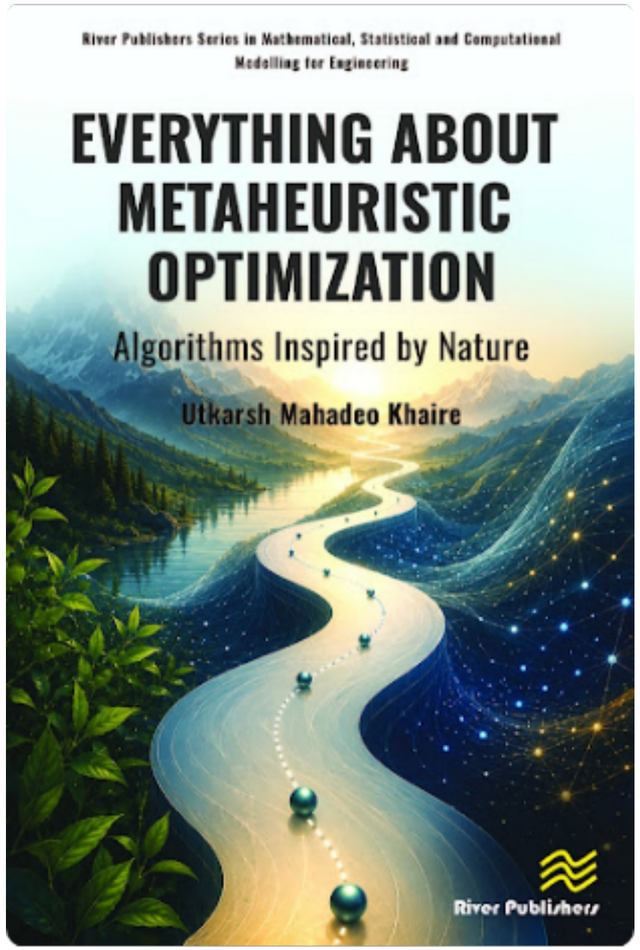
# The Optimizer Toolkit: Synthesis Matrix

| Algorithm | The Hiker Analogy | Key Mechanic | Drawback / Note |
|---|---|---|---|
| Vanilla GD | Looks at feet, steps downhill | Direct gradient update | Extremely slow on plains |
| Momentum | Heavy ball rolling | Considers past gradients | Can overshoot minimums |
| Nesterov (NAG) | Looks ahead | Gradient calculated at future position | Requires momentum tuning |
| AdaGrad | Personalized step size | Decays LR based on slope | Brakes too hard, stops entirely |
| RMSProp | Moving average steps | Exponential decay of past gradients | Hyperparameter tuning needed |
| Adam | The Master Explorer | Combines Momentum, RMSProp, Bias | The modern industry standard |
| iAdam | Take decisions looking at future | Combines NAG and RMSProp | Complex gradient computation |

Gemini Notebook

GRADIENT-FREE PARADIGM

# Beyond Gradients: Metaheuristic Optimization




### Zero Derivative Dependency

Operates effortlessly on non-differentiable, discontinuous, and discrete search spaces.

### Global Exploration Power

Escapes deceptive local optima and saddle points where gradient updates stall.

### Nature-Inspired Collective Search

Leverages swarm intelligence, evolutionary rules, and physics dynamics for global solutions.

# Pacing the Expedition: Learning Rate Schedules

Adjusting the learning rate mid-journey ensures rapid progress initially, transitioning to slow precision near the destination.

Start with a high learning rate then reduce it

## Power Scheduling

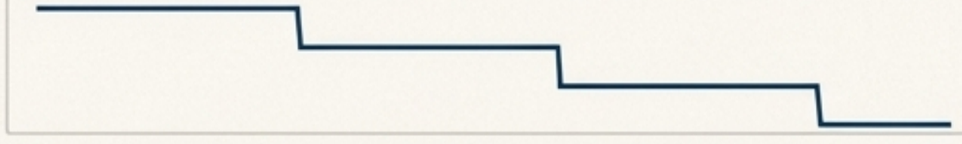
Gradually decreases LR. Formula: $\eta_0 / (1 + t/s)^c$

## Exponential Scheduling

Drops LR exponentially. Formula: $\eta_0 \times 0.1^{t/s}$

## Piecewise Constant

Drops at fixed intervals.
($\eta_0 = 0.1$ for 5 epochs)



## Performance Scheduling

Reduces LR by factor of λ when validation error stalls.

## 1Cycle Scheduling

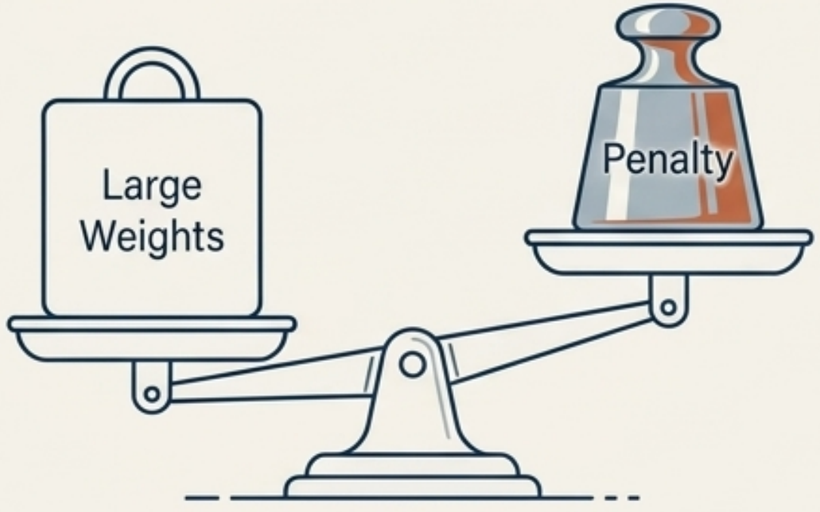
Cyclic LR for faster convergence.

Gemini Notebook

# Traveling Light: Regularization Techniques

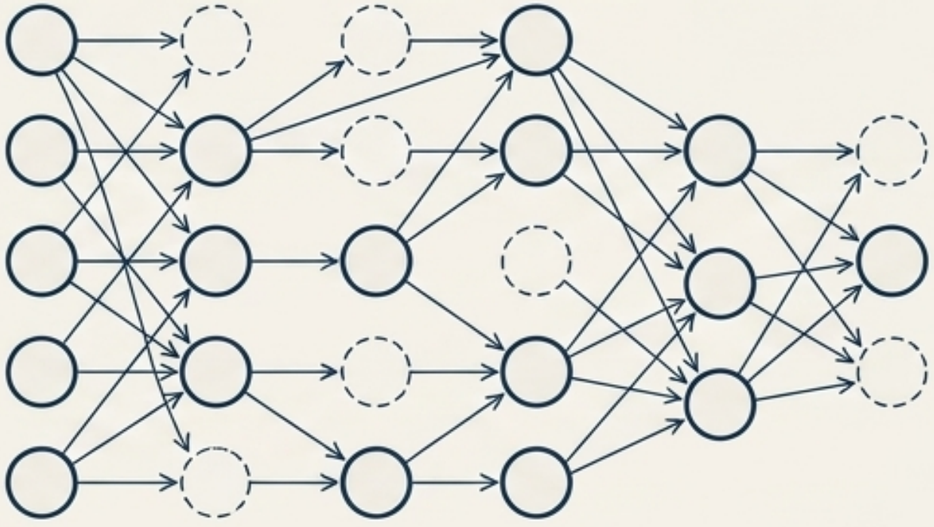
Deep networks are prone to memorizing the training terrain (overfitting). Regularization keeps the model generalized.

## L1 & L2 Regularization




Penalizes aggressively large weights. Forces the network to use a wider, balanced set of parameters rather than relying on massive outliers.

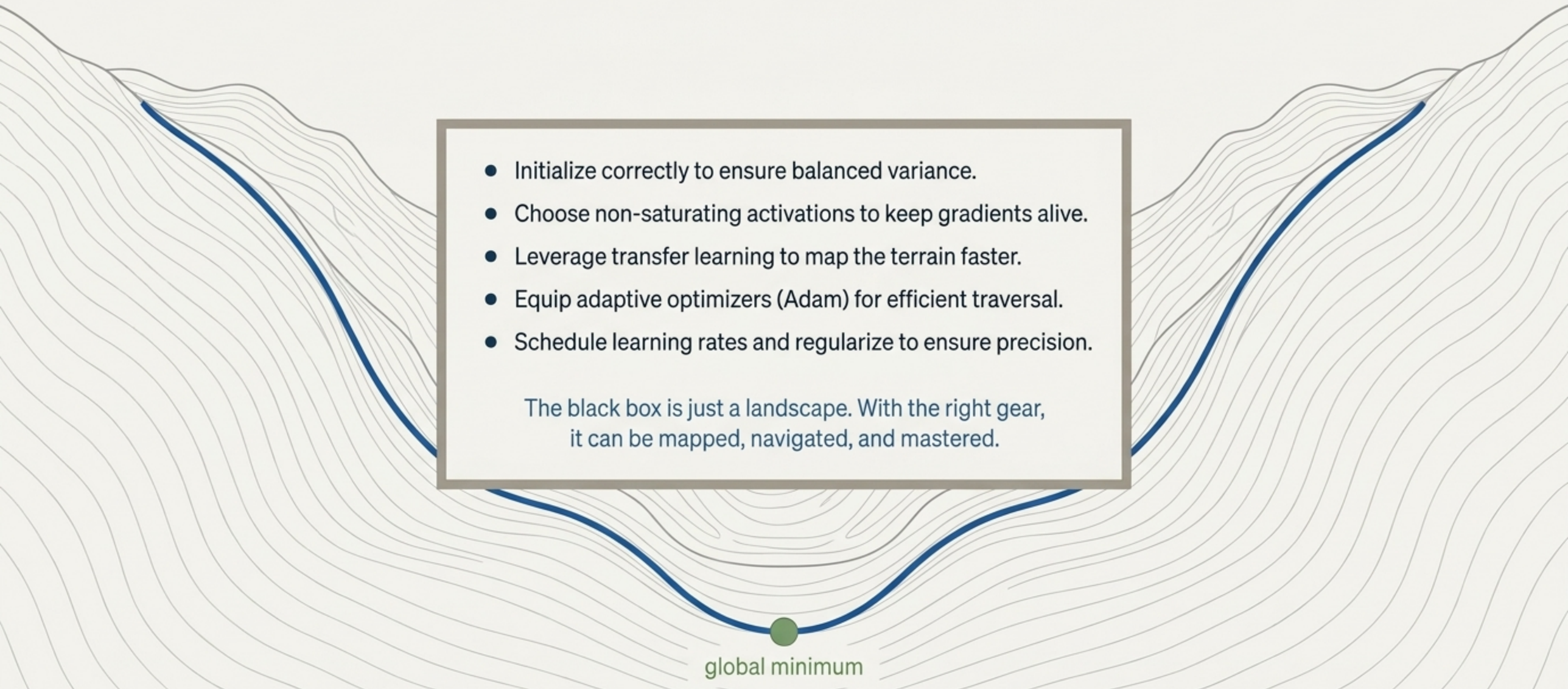
## Dropout



Randomly turns off neurons (typically 10-50%) during training iterations. The network acts like a resilient team where every member must be capable of stepping up.

Gemini Notebook

# Arriving at the Minimum




- Initialize correctly to ensure balanced variance.
- Choose non-saturating activations to keep gradients alive.
- Leverage transfer learning to map the terrain faster.
- Equip adaptive optimizers (Adam) for efficient traversal.
- Schedule learning rates and regularize to ensure precision.

The black box is just a landscape. With the right gear, it can be mapped, navigated, and mastered.

Gemini Notebook